YAPP Router

SystemVerilog & UVM course

The DUT, the UVM environment, the labs 1 to 11C, the test plan and the results — the course site as one document.

https://tamirgabgab.github.io/Qcom-Mentoring/

Table of Contents

- Home
- Getting started
- The DUT
  - Specification
  - Register map
  - RTL walkthrough
- UVM
  - The big picture
  - Project architecture
  - Phases and objections
  - Factory and configuration
  - TLM connections
  - Functional coverage
- Component guides
  - Overview
  - Packet (sequence item)
  - Driver
  - Sequencer and sequences
  - Monitor
  - Agent and env
  - Scoreboard
  - Reference model and module UVC
  - Virtual sequencer
  - Functional coverage
  - Register model (RAL)
  - Random values
  - Random tests
- Test plan
- Glossary
- FAQ and troubleshooting
- Results

- Coverage results
- Regression results
- Bug hunt

Labs

- Overview
- Lab 1 — Stimulus model
- Lab 2 — Test and testbench
- Lab 3 — A simple UVC
- Lab 4 — Factories
- Lab 5 — Sequences
- Lab 6 — Virtual interfaces and the DUT
- Lab 7 — Integrating UVCs
- Lab 8 — Multichannel sequences
- Lab 9A — Scoreboard (TLM)
- Lab 9B — Router module UVC
- Lab 9C — TLM exports
- Lab 9D — Analysis FIFOs
- Lab 10 — Functional coverage
- Lab 11A — Register model generation
- Lab 11B — Register model integration
- Lab 11C — Register tests

Mentor's guide

- Overview
- Session 1 — Data items, components, phases
- Session 2 — A UVC, the factory, configuration
- Session 3 — Sequences
- Session 4 — Interfaces and the DUT
- Session 5 — Integrating UVCs, the virtual sequencer
- Session 6 — TLM, scoreboard, reference model
- Session 7 — Functional coverage
- Session 8 — The register model

Appendix

- Files, classes, tests, sequences
- UVM concept → lab map

Pitfalls
xrun options
Simulating with Verilator
Logs, timelines and waves
Verification status

YAPP Router — SystemVerilog & UVM Course

The interactive project map

Open the project map — one picture of the whole testbench and DUT you can drill into (five views: hierarchy, TLM data flow, UML classes, the environment and the test plan), generated from the source, with the regression and coverage results on every card. Also available as a single offline HTML file.

This site is the companion to the **YAPP packet-router verification project**: a complete UVM environment built from scratch over eleven labs, exactly the way a verification engineer would build one at work — one component at a time, each one tested before the next is added.

Everything here is a **reference solution**: the DUT, every UVM verification component (UVC), every lab in its finished state, the register model, the coverage model and the tests. The code is written to be read: short files, one idea per class, comments that say *why*.

The DUT

The YAPP router: one packet input, three output channels, a host bus and a handful of registers. Small enough to understand completely, rich enough to need a real environment.

→ Specification

UVM, the big picture

Tests, environments, agents, sequences, TLM, the factory and the configuration database — what each piece is for and how they fit.

→ Architecture

Component guides

One page per building block: packet, driver, sequencer, monitor, agent, scoreboard, reference model, virtual sequencer, coverage, register model.

→ Guides

The labs

Labs 1 to 11C, each with objective, concepts, diagrams, the full solution, the tests to run, checkpoint questions with answers and the expected result.

→ Labs

How to use this site

Students

1. Open the repository in a GitHub Codespace (nothing to install; Verilator runs every lab for free) and run `make sim` in `test_install` — or use Xcelium if you have it. Getting started has both.
2. Read the DUT specification: every lab assumes you know the packet format and the three port protocols.
3. Work through the labs in order, **in the lab's `start/` directory**: it is the lab with the bodies of its methods emptied, `start/TASKS.md` lists the steps. Each lab page tells you what to build, why, and what the simulation must show when you are done; `make check LAB=<lab> START=1` tells you whether your log shows it. The finished solution is next to it (`sv/`, `tb/`) when you get stuck or want to compare.
4. Use the component guides when a piece of UVM does not click, the glossary for a word, and the FAQ when the simulator says something you did not expect.

Mentors

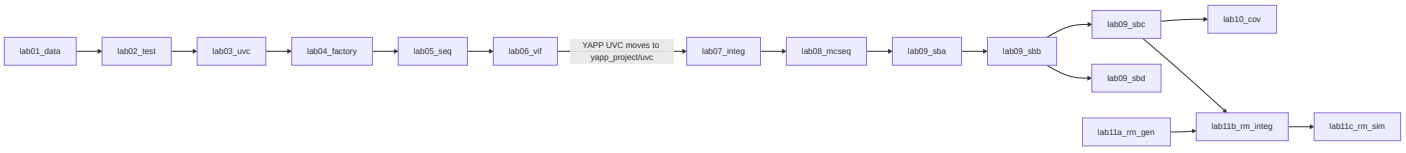
- The labs overview has the session plan, the dependency chain and what each lab adds.
- Every lab page ends with **checkpoint questions**; the answers are in collapsible boxes so you can use them for reviews.
- `labs/<lab>/` is a complete snapshot of the code at the end of that lab. `diff -r labs/lab04_factory labs/lab05_seq` shows exactly what a lab adds.
- The test plan maps DUT features to tests and checkers, the regression results and coverage results show what the last regression did, and the verification status page lists what the Verilator runs proved and what still needs Xcelium.

Course roadmap

Lab	Title	What you build	Key UVM / SV concepts
1	Creating a stimulus model	`yapp_packet`	`uvm_sequence_item`, `do_print` / `do_copy` / `do_compare` constraints, `post_randomize`
2	Test and testbench components	`router_tb`, `base_test`	`uvm_env`, `uvm_test`, `build_phase`, `run_test()`, verbosity

Lab	Title	What you build	Key UVM / SV concepts
3	A simple UVC	driver, sequencer, monitor, agent, env	`uvm_driver`, `seq_item_port`, `is_active`, default sequence
4	Factories	`short_yapp_packet`, config tests	`type_id::create`, type overrides, `uvm_config_int`, `check_config_usage`
5	Sequences	the YAPP sequence library	`uvm_sequence`, `start_item` / `finish_item`, nesting, objections, randomization debug
6	Virtual interfaces and the DUT	`yapp_if`, `hw_top`, `tb_top`	`interface`, `virtual interface` `uvm_config_db`, drain time
7	Integrating UVCs	HBUS, Channel, Clock & Reset in `router_tb`	reuse, configuration, multiple interfaces
8	Multichannel sequences	`router_mcsequencer`, `router_simple_mcseq`	virtual sequencer, `` `uvm_declare_p_sequencer ``
9A	Scoreboard	`router_scoreboard`	analysis ports / imps, `clone()`
9B	Router module UVC	`router_reference`, `router_module_env`	reference model, module UVC
9C	TLM exports *(optional)*	exports on the env	`uvm_analysis_export`
9D	Analysis FIFOs	`router_fifo_scoreboard`	`uvm_tlm_analysis_fifo`, blocking `get`

Lab	Title	What you build	Key UVM / SV concepts
	(optional)		
10	Functional coverage *(optional)*	`yapp_pkt_coverage`, a covergroup on the monitor's packets	coverpoints, bins, cross, `uvm_subscriber`
11A	Register model generation	`yapp_router_reg_pkg`	`uvm_reg`, `uvm_reg_block`, maps
11B	Register model integration	adapter, `set_sequencer`, built-in sequences	front door, backdoor, auto-predict
11C	Register tests	`reg_access_test`, `reg_function_test`	`write/read/peek/poke`, `predict`, introspection



What is in the repository

```
yapp_project/                the complete project, the state after Lab 11C (source of truth)
├── rtl/                     the DUT, one module per file (yapp_router.sv + yapp_router.f)
├── uvc/yapp/                YAPP input UVC   -- built in Labs 1-6, final state
├── uvc/hbus/                HBUS UVC         -- "provided"
├── uvc/channel/             Channel UVC      -- "provided"
├── uvc/clock_and_reset/     Clock & Reset UVC-- "provided"
├── uvc/router/              router module UVC (scoreboard, reference model, coverage collectors)
-- Labs 9A-9D
├── tb/                      the final testbench: tests/, mcseqs/, reg/, run.f, Makefile, the DUT
tracer
└── regression/              the committed report of the last regression: logs, timelines, waves,
coverage
labs/<lab>/                  one snapshot per lab (sv/ in Labs 1-6 and 9A-9D, tb/, run.f,
Makefile);
                             Labs 7+ compile the YAPP UVC and the DUT from yapp_project/
common/                      what every bench shares: rand_util_pkg (rnd::), lab.mk, the
Verilator glue
test_install/                does the simulator find UVM?
scripts/                     lint harness (slang), Verilator flow (sim.py, regress.yaml), the
regression
                             and coverage reports, the project-map generator, the waveform
generator
docs/                        this site
```

About the original material

The project follows the structure of the Cadence *SystemVerilog Accelerated Verification Using UVM* training. No Cadence files are used: the DUT, the "provided" UVCs, the interfaces and the register model were all written for this repository from the public specification of the exercise.

Getting started

What you need

Two ways to run the labs; pick one.

	Verilator (free)	**Cadence Xcelium**
Get it	nothing to install: open the repository in a GitHub Codespace — or `bash scripts/setup_sim.sh` on Ubuntu / WSL2 (builds Verilator 5.052, ~15 min once)	a licence; any release that ships a UVM library (17.04 or later), `-uvmhome CDNS-1.1d` in every `run.f` (`CDNS-1.2` works too, see UVM versions)
Run a test	`make sim TEST=<test>` in the lab's `tb/`	`make run TEST=<test>` in the lab's `tb/`
Waves	`make sim ... WAVES=1`, then VaporView in Codespaces / Surfer / GTKWave	SimVision (`make gui`)
Details	Simulating with Verilator: the same labs, the same tests, the few differences	xrun options

Either way, `pip install pyslang mkdocs-material` adds the lint without a simulator (`make lint`) and lets you build this site; both are optional.

Check the installation

```
cd test_install
make sim                # Verilator
make run                # Xcelium:  xrun -f run.f
```

The log must show the UVM version banner and

```
UVM_INFO ... [INSTALL] UVM TEST INSTALL PASSED!
```

(`make sim` prints `PASS: default seed 1` under it.)

The complete project: `yapp_project/`

`yapp_project/` is the source of truth: the finished verification environment, the state the course reaches at the end of Lab 11C. Everything else in the repository is derived from it.

```
yapp_project/
├── rtl/                the DUT, one module per file
│   ├── yapp_router.sv       top level: wiring only
│   ├── yapp_input_fsm.sv    receives a packet, checks it, routes it
│   ├── yapp_output_channel.sv   FIFO + output handshake, one instance per channel
│   ├── yapp_fifo.sv
│   ├── yapp_hbus_regs.sv    every register and memory, the HBUS slave
│   ├── yapp_error_timer.sv
│   └── yapp_router.f        file list, used from run.f as -F ../rtl/yapp_router.f
├── uvc/
│   ├── yapp/            YAPP input UVC (seqs/ holds the sequence classes; pkt / gap coverage
collectors)
│   ├── hbus/            HBUS UVC, including hbus_reg_adapter
│   ├── channel/         Channel UVC
│   ├── clock_and_reset/ Clock & Reset UVC and clkgen
│   └── router/          router module UVC: reference model, scoreboards, coverage collectors,
packet_compare
├── tb/                  the final testbench
│   ├── tb_top.sv, hw_top.sv, router_tb.sv, router_mcsequencer.sv, yapp_router_tracer.sv (the DUT
tracer)
│   ├── tests/           one file per test class, included by router_test_lib.sv
│   ├── mcseqs/          the multichannel sequences, included by router_mcseqs_lib.sv
│   ├── reg/             the register model classes, included by yapp_router_reg_pkg.sv
│   ├── run.f            xrun command file (default test: reg_function_test)
│   └── Makefile         make run / gui / lint / clean (Xcelium), compile / sim / waves
(Verilator)
└── regression/          the committed report of the last regression: logs, timelines, waves,
coverage
```

```
cd yapp_project/tb
make run TEST=reg_function_test       # any test of tests/, see run.f for the list
make run TEST=uvm_mem_walk_test XRUN_OPTS="+UVM_VERBOSITY=UVM_HIGH"
make lint                             # slang elaboration check, no simulator
```

The labs are snapshots along the way to this project. Labs 1–6 carry their own copy of the YAPP UVC in `labs/<lab>/sv`; from Lab 7 on, every lab's `run.f` compiles the YAPP UVC and the provided UVCs from `yapp_project/uvc/` and the DUT from `yapp_project/rtl/yapp_router.f`, so the lab directory holds the testbench of that lab — plus, in Labs 9A–9D, the router module UVC as it grows (`labs/lab09_*/sv`), whose finished form is `yapp_project/uvc/router`.

Running a lab

Every lab directory has the same shape:

```
labs/lab05_seq/
├── sv/        the UVC source files of that lab (Labs 1-6: the YAPP UVC; 9A-9D: the router module
UVC)
│   └── seqs/      one file per sequence class, included by yapp_tx_seqs.sv
├── tb/        testbench, tests, top modules
│   ├── tests/     one file per test class, included by router_test_lib.sv
│   ├── run.f      xrun command file: what to compile + default plusargs
│   └── Makefile   make run / gui / lint / clean (Xcelium); compile / sim / waves (Verilator)
└── start/     your copy to work in: the same sv/ and tb/, with the bodies of the lab's methods
emptied
    └── TASKS.md   the steps of the lab, which files to write, how to run and check
```

Working on a lab: `start/` and `make check`

`sv/` and `tb/` are the finished lab. **`start/` is where you work**: it holds the finished state of the lab before this one, plus the files this lab adds or changes with the bodies of their methods and constraints emptied to a `// TODO (Lab N)` comment. Every class keeps its prototypes (the table of contents of the class in this repository's style), `new()` and the `super.<phase>()` calls stay so that everything compiles from the first minute, and the comments of the finished bodies are left in place as hints. `start/TASKS.md` lists the steps of the lab page, the files to write and the files that are given. Modules, interfaces, packages and `run.f` are given even when the lab changes them: the work of a lab is in its classes.

```
cd labs/lab05_seq/start/tb
make sim TEST=incr_payload_test      # compiles the start point as it is: the test runs, does
little
# ... write the bodies in start/sv and start/tb, following start/TASKS.md and the lab page ...
make sim TEST=incr_payload_test      # again, until the log shows what the lab page promises
```

`make check` is the lab page's "Expected" paragraph as a program. From the repository root it runs every test of the lab in Verilator and looks in each log for the lines the finished lab prints (`scripts/lab_checks.yaml`): the sequences that must run, the packets that must be collected, the reports that must say `0 mismatched`, and of course `0 UVM_ERROR`. One line per check, `ok` or `MISSING`, and the path of the log when something is missing:

```
make check LAB=lab05_seq START=1     # your work in labs/lab05_seq/start/
make check LAB=lab05_seq             # the finished lab (every line ok)
make check LAB=lab05_seq START=1 TEST=incr_payload_test
```

```
labs/lab05_seq/start/tb:
  FAIL  incr_payload_test          sim      0s    0.3s
        ok       ends with 0 UVM_ERROR / 0 UVM_FATAL
        MISSING  yapp_incr_payload_seq runs as the default sequence
        MISSING  the payload counts up: payload[1] = 1
        log: build/sim/labs_lab05_seq_start_tb/logs/incr_payload_test_s1.log
```

A fresh `start/` is one command away when you want to begin a lab again: `make start` regenerates all of them from the finished labs (`git checkout labs/lab05_seq/start` restores just one).

One class per file, bodies outside the class. The course's library files (`yapp_tx_seqs.sv`, `router_test_lib.sv`, `router_mcseqs_lib.sv`, `yapp_router_reg_pkg.sv`) are kept as the files the packages include, but each of them only `include`s the class files of its sub-directory (`seqs/`, `tests/`, `mcseqs/`, `reg/`). `short_yapp_packet` has its own file next to `yapp_packet.sv`. Inside a file the class body is a table of contents: the properties, the `utils` macro, the `extern constraint` lines and the `extern` method prototypes. After `endclass` come the constraint bodies under a `// <class> -- constraints` banner (`constraint yapp_packet::c_length { ... }`), then the method bodies under `// <class> -- method implementations`, as `function yapp_packet::set_parity();` or `task yapp_012_seq::body();`. Constraint and method bodies look alike: each is preceded by a `//-----` delimiter line (the first one after a banner excepted), opens on its own line and holds one statement per line, with one blank line between bodies. Read the prototypes to learn what a class does, scroll down to see how.

The names follow fixed rules. Every class property starts with `m_` (`m_addr`, `m_length`, `m_monitor`, `m_vif`); arguments and locals never do, so inside a method `length` is the argument and `m_length` the field. The UVM runtime names stay plain: `create("monitor", this)`, `new("item_collected_port", this)`, the `uvm_config_db` keys, so the hierarchy in the logs reads `uvm_test_top.tb.yapp.agent.monitor`. Packed widths are written without a space (`bit[7:0]`). The typedefs and enums of a package live in its `<pkg>_defines.sv` (`yapp_defines.sv`, `hbus_defines.sv`, ...), included first by the package file; an enum type ends with `_e`, its values are upper case with the type name as prefix and the first one is always `<NAME>_UNDEFINED`, a value no constraint or assignment ever produces (`PARITY_TYPE_UNDEFINED, PARITY_TYPE_GOOD, PARITY_TYPE_BAD`). Three more conventions hold everywhere: a function declares its locals at its top (never in a bare `begin … end` in the middle of the body), the body of an `if` / `else` / `for` / `foreach` / `while` / `repeat` is always wrapped in `begin … end`, even a single statement, and random values that are not fields of a randomized object come from the common library `rnd::` (Random values), not from `$urandom`. `make style-check` (`scripts/sv_style.py`) enforces all of this in CI, reporting each violation as `file:line`, and `make style` reformats a file you wrote.

The classes also do without the shortcut macros of the course material: no `uvm_do*` (a sequence writes `create` → `start_item` → `randomize` → `finish_item` itself, and starts a sub-sequence with `seq.start(m_sequencer, this)`), and no `uvm_field_*` automation (`do_print`, `do_copy`, `do_compare`, `do_pack`, `do_unpack` and `do_record` are written by hand, and a component reads its configuration with `uvm_config_int::get` in `build_phase`). What the macros would hide is on the page, where you can read it.

```
cd labs/lab05_seq/tb
make run                                  # +UVM_TESTNAME from run.f
make run TEST=incr_payload_test           # pick another test, no recompilation needed
make run TEST=exhaustive_seq_test XRUN_OPTS="+UVM_VERBOSITY=UVM_FULL +SVSEED=random"
make gui TEST=exhaustive_seq_test         # SimVision, -access +rwc
```

Or from the repository root: `make run LAB=lab05_seq TEST=exhaustive_seq_test`.

The plusargs you will use all the time

Plusarg	Effect
`+UVM_TESTNAME=<class>`	which `uvm_test` subclass `run_test()` creates
`+UVM_VERBOSITY=UVM_LOW|UVM_MEDIUM|UVM_HIGH|UVM_FULL`	filters `uvm_info` messages
`+SVSEED=random`	new random seed every run (the seed is printed in the log)
`+SVSEED=1234`	reproduce a run

The full list of `xrun` options used in the course is in the xrun appendix.

Reading the log

The lines you look for first:

```
UVM_INFO ... Running test base_test...            <- which test runs
[TEST] === base_test  seed 1 ===                  <- the banner of the project's tests (and the
seed)
[TEST] STAGE: ...                                 <- what the test is doing now
[TEST] === base_test  seed 1: PASSED (0 UVM_ERROR, 0 UVM_FATAL) ===
--- UVM Report Summary ---                        <- counts of errors / warnings
UVM_ERROR :    0
```

Every component of the environment prints a short report at the end of the simulation (packets sent, collected, matched...). The lab pages tell you which numbers to expect.

Simulating without Xcelium

Every lab directory also has a free flow with Verilator, from the same `run.f`:

```
cd labs/lab07_integ/tb
make sim TEST=simple_test            # compile (first time ~2.5 min) and run: PASS / FAIL
make sim TEST=simple_test WAVES=1    # with waves in build/sim/.../waves/simple_test.fst
make regress                         # at the root: every lab and every test
```

Installation, waves, the regression in GitHub Actions and the few differences from Xcelium: Simulating with Verilator.

Linting without a simulator

`scripts/lint.py` elaborates a `run.f` with slang together with the real Accellera UVM source. It catches syntax errors, unknown classes and methods, type mismatches and port/interface problems — everything except run-time behaviour.

```
pip install pyslang
make lint                  # every lab + test_install
make lint LAB=lab08_mcseq  # one lab
```

The same check runs in GitHub Actions on every push (`.github/workflows/lint.yml`).

Building this site

```
pip install mkdocs-material
make serve      # http://127.0.0.1:8000, live reload
make docs       # static site in site/
```

Diagrams need network access

The block diagrams are Mermaid sources rendered in the browser; Material for MkDocs fetches the Mermaid library from `unpkg.com` on page load. On a network that blocks CDNs the diagrams show as text. The protocol waveforms are plain SVG files and always render.

Regenerating the project map

The project map is generated from the code. After changing a class, a port or a `connect()` call:

```
pip install pyslang pyyaml jinja2
make map            # rewrites model.json, regmap.js, the standalone HTML and every generated
page:
                    # test-plan.md, coverage-results.md, dut/register-map.md, regression/, the
index tables
make map-check      # what CI runs: fails if the committed map is stale; also runs the
                    # consistency checks (test_model.py) and, when node is installed,
                    # the simulator checks (test_sim.mjs: RouterModel vs. the documented RTL
rules)
make map-export     # optional: SVG/PNG/PDF of every view (needs node + playwright)
```

Descriptions, lab numbers and layout hints live in `scripts/project_map/annotations.yaml` .

UVM versions

The code targets **UVM 1.1d and 1.2** (Cadence libraries). The only API that differs between the two and matters here is how a sequence reaches its starting phase (`starting_phase` member vs `get_starting_phase()`). The shim `common/uvm_version_compat.svh` hides it behind one macro:

//-- // uvm_version_compat.svh -- tiny shim so the course code compiles unchanged on // UVM 1.1d (recommended by the training), UVM 1.2 and IEEE 1800.2 libraries. // // Only ONE difference actually matters for this project: // * UVM 1.1d : a sequence sees its phase through the public member // `starting_phase`. // * UVM 1.2+ : the member is deprecated; use `get_starting_phase()`. // // Use `YAPP_STARTING_PHASE wherever a sequence needs its starting phase:
`//     uvm_phase phase =` YAPP_STARTING_PHASE; // if (phase != null) phase.raise_objection(this); //-- `ifndef UVM_VERSION_COMPAT_SVH` define UVM_VERSION_COMPAT_SVH

`ifdef UVM_VERSION_1_1` define YAPP_STARTING_PHASE starting_phase `else` define YAPP_STARTING_PHASE get_starting_phase() `endif

`endif // UVM_VERSION_COMPAT_SVH

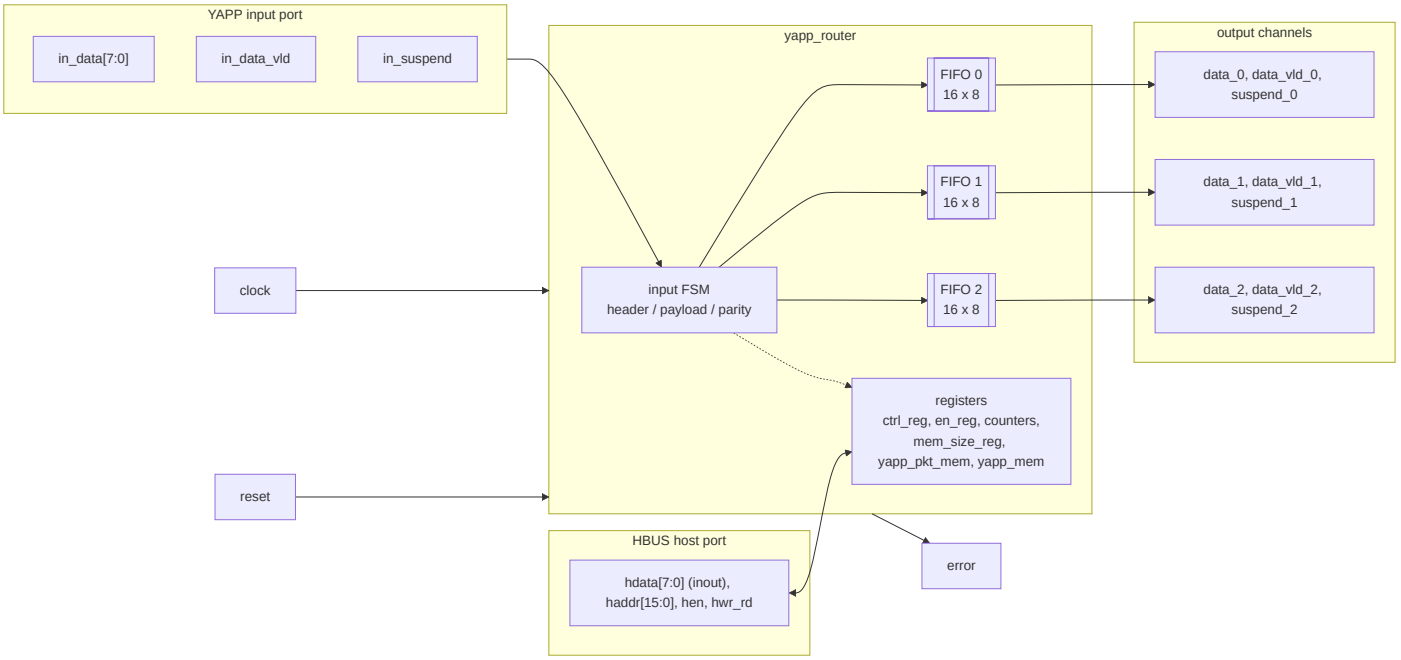
Every `run.f` adds `-incdir ../../../common` so the packages can include it.

The DUT

The YAPP router — specification

Open on the project map →

YAPP stands for *Yet Another Packet Protocol*. The router accepts packets on **one input port** and forwards each one to **one of three output channels**, chosen by the address in the packet header. A **host bus (HBUS)** programs the router's registers and reads its counters and memories.



Ports

Port	Dir	Width	Description
`clock`, `reset`	in	1	reset is active high
`in_data`	in	8	packet bytes
`in_data_vld`	in	1	high for header and payload bytes, low for the parity byte

Port	Dir	Width	Description
`in_suspend`	out	1	DUT asks the sender to hold the current byte (target FIFO full)
`data_x`	out	8	channel *x* data (x = 0, 1, 2)
`data_vld_x`	out	1	a byte is presented on `data_x`
`suspend_x`	in	1	receiver not ready; low = read the byte
`hdata`	inout	8	host data bus (tri-state)
`haddr`	in	16	host address
`hen`	in	1	host access enable
`hwr_rd`	in	1	1 = write, 0 = read
`error`	out	1	pulses after a packet with bad parity

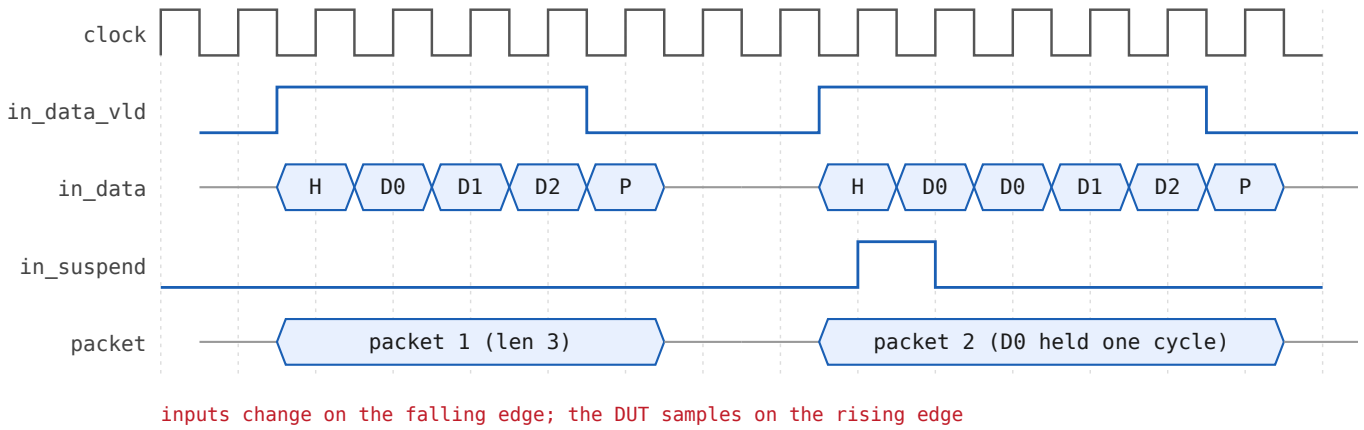
Packet format

Byte	Bits	Content
0 (header)	`[7:2]`	`length` — number of payload bytes, 1 … 63
	`[1:0]`	`addr` — output channel 0, 1 or 2; **3 is illegal**
1 … length	`[7:0]`	payload

Byte	Bits	Content
length + 1	`[7:0]`	`parity` — bitwise XOR of the header and all payload bytes (even parity)

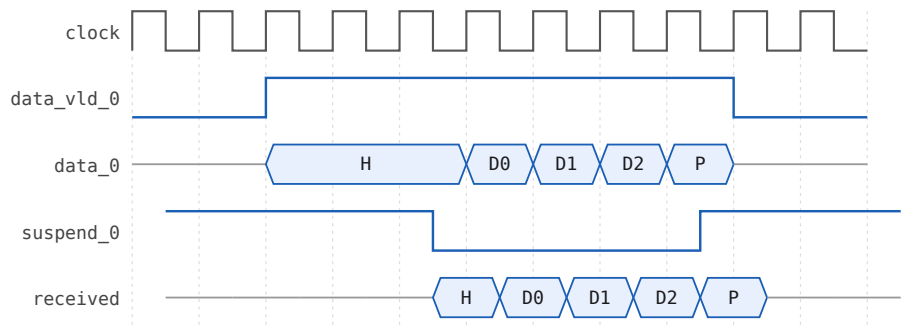
A packet is therefore `length + 2` bytes long, from 3 to 65 bytes.

Input port protocol



- All inputs are active high and **driven on the falling edge** of the clock; the router samples them on the rising edge.
- `in_data_vld` goes high together with the **header** byte. Each following falling edge carries the next payload byte.
- On the falling edge after the last payload byte, `in_data_vld` goes **low** and the **parity** byte is driven.
- `in_suspend` high means "the FIFO is full": the input data must **hold its value** until it drops. A byte is accepted at a rising edge only when `in_suspend` is low.
- `error` pulses high for one cycle, **1 to 10 cycles** after a packet with bad parity has been received.

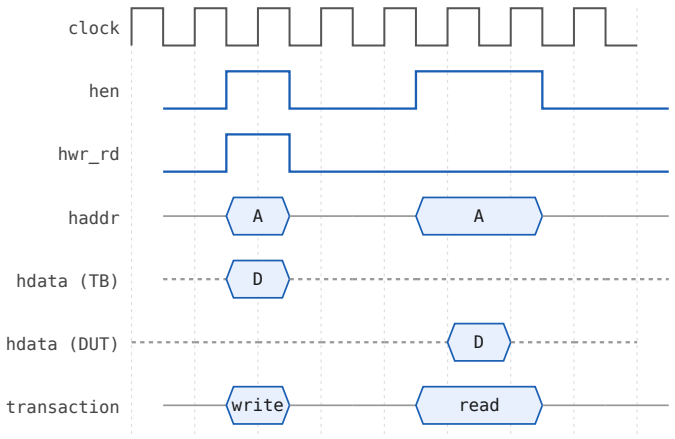
Output channel protocol



receiver drops suspend on the falling edge where it reads the header; one new byte per rising edge while suspend is low

- Each channel has an internal **16-byte FIFO**.
- The router raises `data_vld_x` when a byte is on `data_x`. The receiver reads the byte on a **falling edge** and de-asserts `suspend_x` on that same edge.
- While `suspend_x` stays low, the router presents a **new byte on every rising edge**. The receiver re-asserts `suspend_x` on the falling edge where it reads the parity byte.
- The receiver learns how many bytes to read from the header (`length + 2`).

HBUS host protocol



write: data stored on the rising edge inside the hen cycle; read: address sampled in cycle 1, data driven by the DUT in cycle 2

- Inputs are driven on the falling edge, sampled on the rising edge.
- **Write — 1 cycle.** `hen = 1`, `hwr_rd = 1`, `haddr` and `hdata` valid. The register is written on the rising edge; `hen` returns to 0 in the next cycle.
- **Read — 2 cycles.** `hen = 1`, `hwr_rd = 0`. The DUT samples `haddr` on the first rising edge and drives `hdata` during the second cycle. When `hen` drops, the DUT tri-states `hdata`.
- Because `hdata` is bidirectional, the testbench must drive it only during a write and must connect the DUT to a **wire** (`hbus_if.hdata_w`), not to a logic variable.

Registers

Address	Register	Reset	Bits	Field
`0x1000`	`ctrl_reg`	`0x3f`	`[5:0]`	`maxpktsize` (RAL name `plen`)
			`[7:6]`	unused
`0x1001`	`en_reg`	`0x01`	`[0]`	`router_en`
			`[1]`	`parity_err_cnt_en`
			`[2]`	`oversized_pkt_cnt_en`
			`[3]`	reserved
			`[4]` … `[7]`	`addr0_cnt_en` … `addr3_cnt_en`
`0x1004`	`parity_err_cnt_reg`	`0x00`	`[7:0]`	

Address	Register	Reset	Bits	Field
0x1005	oversized_pkt_cnt_reg	0x00	[7:0]	
0x1006	addr3_cnt_reg	0x00	[7:0]	
0x1009	addr0_cnt_reg	0x00	[7:0]	
0x100a	addr1_cnt_reg	0x00	[7:0]	
0x100b	addr2_cnt_reg	0x00	[7:0]	
0x100d	mem_size_reg	0x00	[7:0]	

Memories

Start	Name	Size	Access	Meaning
0x1010	yapp_pkt_mem	64 × 8	RO	bytes of the last packet received
0x1100	yapp_mem	256 × 8	RW	scratch memory

Addresses `0x1002`, `0x1003`, `0x1007`, `0x1008`, `0x100c`, `0x100e` … `0x100f` and `0x1050` … `0x10ff` are not mapped: they read as `0x00` and ignore writes. Every address holds one byte: the 64 packet-memory bytes occupy `0x1010` … `0x104f`, the 256 scratch bytes `0x1100` … `0x11ff`.

Try it: the register file

A model of `yapp_hbus_regs` + the end-of-packet report of `yapp_input_fsm`. Pick an address, read or write it, and watch the policy work; **Send a packet** moves the counters, fills `yapp_pkt_mem` and updates `mem_size_reg` exactly as the RTL does. The same widget sits on the project map (click the registers block inside the DUT).

HBUS ACCESS

address: 0x1000 ctrl_reg (RW) — or type it: 0x1000 — RW ctrl_reg

operation: Read | Write — data (hex): 0x20 — Execute — Reset

data to write to ctrl_reg -- click a bit to flip it

MSB 7	6	5	4	3	2	1	0 LSB
0	0	**1**	0	0	0	0	0

`[5:0]` **maxpktsize** = `32 (0x20)` -- maximum payload length (RAL name plen)

`[7:6]` **unused** = `0 (0x00)` -- unused, RW

A write to a **RO** register or to an unmapped address is ignored by the router; a read of an unmapped address returns 0x00.

REGISTERS

addr	register		value	fields
0x1000	ctrl_reg	RW	0x3f	maxpktsize = 63

MSB 7	6	5	4	3	2	1	0 LSB
0	0	**1**	**1**	**1**	**1**	**1**	**1**

`[5:0]` **maxpktsize** = `63 (0x3f)` -- maximum payload length (RAL name plen)

`[7:6]` **unused** = `0 (0x00)` -- unused, RW

addr	register		value	fields
0x1001	en_reg	RW	0x01	router_en

MSB 7	6	5	4	3	2	1	0 LSB
0	0	0	0	0	0	0	**1**

`[0]` **router_en** = `1` -- router enable: 0 drops every packet

`[1]` **parity_err_cnt_en** = `0` -- count packets with bad parity

`[2]` **oversized_pkt_cnt_en** = `0` -- count packets longer than maxpktsize

`[3]` **reserved** = `0` -- not implemented

[4] **addr0_cnt_en** = 0 -- count packets to address 0

[5] **addr1_cnt_en** = 0 -- count packets to address 1

[6] **addr2_cnt_en** = 0 -- count packets to address 2

[7] **addr3_cnt_en** = 0 -- count packets to the illegal address 3

0x1004	parity_err_cnt_reg	RO	0x00	0 (dec)
0x1005	oversized_pkt_cnt_reg	RO	0x00	0 (dec)
0x1006	addr3_cnt_reg	RO	0x00	0 (dec)
0x1009	addr0_cnt_reg	RO	0x00	0 (dec)
0x100a	addr1_cnt_reg	RO	0x00	0 (dec)
0x100b	addr2_cnt_reg	RO	0x00	0 (dec)
0x100d	mem_size_reg	RO	0x00	0 (dec)

MEMORIES

yapp_pkt_mem 0x1010 .. 0x104f (64 × 8, RO) -- bytes of the last packet received: [0] header, [1..63] payload

0x1010	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x1020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x1030	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x1040	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

yapp_mem 0x1100 .. 0x11ff (256 × 8, RW) -- scratch memory

0x1100	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x1110	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

SEND A PACKET

addr: 1 | length: 4 | parity: ☐ PARITY_TYPE_BAD | Send

Random payload; the header, the payload and the parity byte are stored in yapp_pkt_mem, the counters move according to en_reg, mem_size_reg takes the length. Oversized (> maxpktsize) and address-3 packets are counted and dropped.

TRANSACTION LOG

```
power-up: registers at reset values, memories cleared
```

Behaviour rules

- `length > maxpktsize` → the **whole packet is dropped**.
- `router_en = 0` → **all packets are dropped**. Changing it in the middle of a packet is undefined.
- Changing the counter-enable bits in the middle of a packet is undefined.

- A counter only counts while its enable bit is set.

Decisions where the specification is silent

The exercise text leaves a few corner cases open. The DUT, the reference model and the register tests in this repository all follow the same choices:

Question	Decision
Does a dropped packet update `yapp_pkt_mem` / `mem_size_reg`?	**Yes** — the packet was *received*, just not forwarded.
Is parity checked / counted and `error` raised for a dropped packet?	**Yes** (oversized or address 3).
Two bad packets end less than 10 cycles apart?	**Each gets its own pulse**, 1..10 cycles after its parity byte; two pulses may be on adjacent cycles (every cycle with `error` high is one pulse).
What happens when `router_en = 0`?	The input is "deaf": the FSM tracks the bytes to stay in sync, but nothing is forwarded, counted or stored.
An oversized packet to address 3?	Counts in **both** `oversized_pkt_cnt_reg` and `addr3_cnt_reg` (each condition is evaluated independently).
Do the address counters count dropped packets?	**Yes** — `addrN_cnt_reg` counts every packet received *with* address N while the router is enabled.
A counter at 255 counts once more?	It **wraps to 0** (8-bit `+ 1`, no saturation); `counter_wrap_test` checks it.
Writing to a read-only register over the HBUS?	Ignored. Writes to `ctrl_reg[7:6]` and `en_reg[3]` are stored and read back (they are declared RW).

YAPP router register map

The router's host port (HBUS) has a 16-bit address and 8-bit data: every address holds one byte. The RTL is `yapp_project/rtl/yapp_hbus_regs.sv`; the full specification with the protocols and timing diagrams is spec.md.

Registers

Address	Name	Access	Reset	Description
`0x1000`	`ctrl_reg`	RW	`0x3f`	Control: maximum payload length
`0x1001`	`en_reg`	RW	`0x01`	Enables: router and the six counters
`0x1004`	`parity_err_cnt_reg`	RO	`0x00`	packets received with bad parity
`0x1005`	`oversized_pkt_cnt_reg`	RO	`0x00`	packets longer than maxpktsize
`0x1006`	`addr3_cnt_reg`	RO	`0x00`	packets to the illegal address 3
`0x1009`	`addr0_cnt_reg`	RO	`0x00`	packets to address 0
`0x100a`	`addr1_cnt_reg`	RO	`0x00`	packets to address 1

Address	Name	Access	Reset	Description
`0x100b`	`addr2_cnt_reg`	RO	`0x00`	packets to address 2
`0x100d`	`mem_size_reg`	RO	`0x00`	payload length of the last packet received

Fields of the read-write registers

`ctrl_reg` (0x1000)

Bits	Field	Description
[5:0]	`maxpktsize`	maximum payload length (RAL name plen)
[7:6]	`unused`	unused, RW

`en_reg` (0x1001)

Bits	Field	Description
[0]	`router_en`	router enable: 0 drops every packet
[1]	`parity_err_cnt_en`	count packets with bad parity
[2]	`oversized_pkt_cnt_en`	count packets longer than maxpktsize
[3]	`reserved`	not implemented
[4]	`addr0_cnt_en`	count packets to address 0

Bits	Field	Description
[5]	`addr1_cnt_en`	count packets to address 1
[6]	`addr2_cnt_en`	count packets to address 2
[7]	`addr3_cnt_en`	count packets to the illegal address 3

Memories

Addresses	Name	Size	Access	Description
`0x1010` .. `0x104f`	`yapp_pkt_mem`	64 bytes	RO	bytes of the last packet received: [0] header, [1..63] payload
`0x1100` .. `0x11ff`	`yapp_mem`	256 bytes	RW	scratch memory

Unmapped addresses

A read returns `0x00`; a write is ignored. Inside the window:

- `0x1002` .. `0x1003`
- `0x1007` .. `0x1008`
- `0x100c`
- `0x100e` .. `0x100f`
- `0x1050` .. `0x10ff`

What a read and a write do

Access	RW register / `yapp_mem`	RO register / `yapp_pkt_mem`	Unmapped
Write	stores the byte	ignored	ignored
Read	the stored byte	the current value	`0x00`

The counters count only while `router_en` (`en_reg[0]`) is 1 and their own enable bit is set, and wrap from 255 to 0. `mem_size_reg` and `yapp_pkt_mem` describe the last packet received while the router was enabled, dropped packets included.

To try it interactively: the register simulator on the site page Specification → Try it: the register file, or in the offline map `docs/downloads/yapp_project_map.html` (click the registers block inside the DUT).

RTL walkthrough — `yapp_project/rtl/`

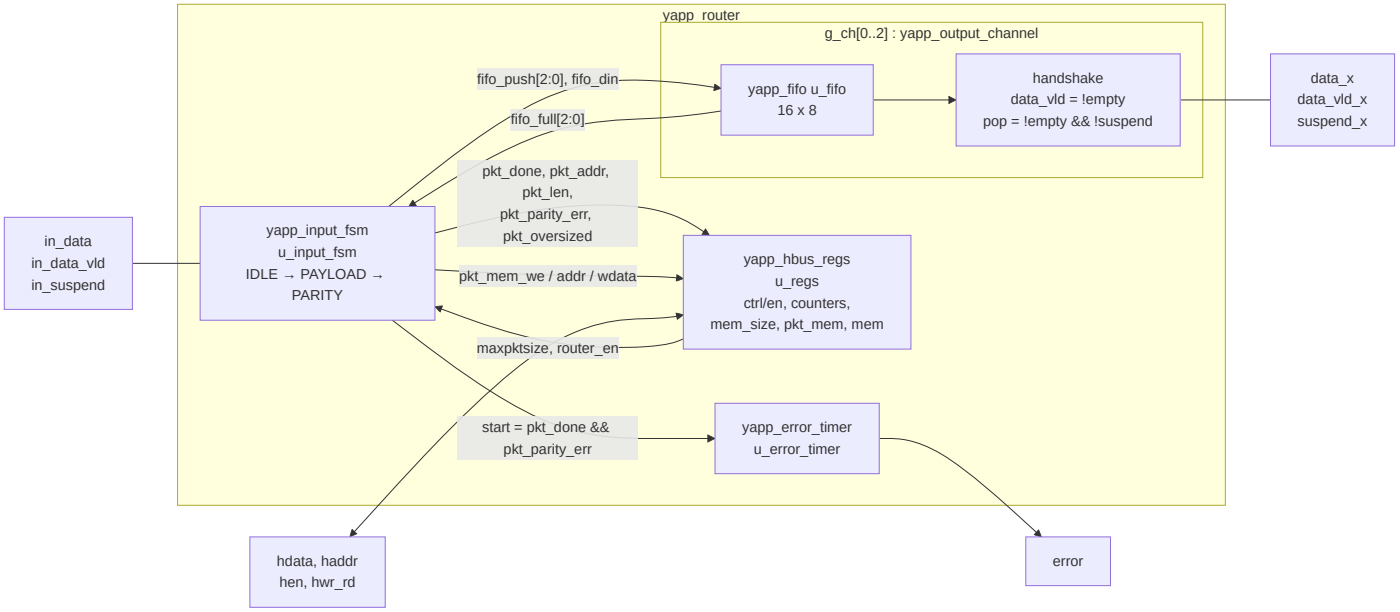
Open on the project map →

The DUT is written to be readable from top to bottom in one sitting: no vendor features, and **one module per file**, cut along the functional blocks of the router. `yapp_router.sv` is the top level and contains nothing but wiring; the behaviour lives in four sub-modules (plus the FIFO each output channel instantiates). This page follows the data: input port → output channels, then the register side.

File	Module	Instance in `yapp_router`	Job
`yapp_router.sv`	`yapp_router`	— (top)	ports of wiring o
`yapp_input_fsm.sv`	`yapp_input_fsm`	`u_input_fsm`	IDLE/PA FSM, dr `in_susp` push, pa
`yapp_output_channel.sv`	`yapp_output_channel`	`g_ch[0..2].u_ch`	one cha output h
`yapp_fifo.sv`	`yapp_fifo`	`g_ch[x].u_ch.u_fifo`	16-byte FIFO
`yapp_hbus_regs.sv`	`yapp_hbus_regs`	`u_regs`	every re memory read/wri
`yapp_error_timer.sv`	`yapp_error_timer`	`u_error_timer`	`error` p cycles a

The file list `yapp_project/rtl/yapp_router.f` compiles them in dependency order (`-F ../rtl/yapp_router.f` from the testbench directory).

Structure



Two things to notice in the wiring:

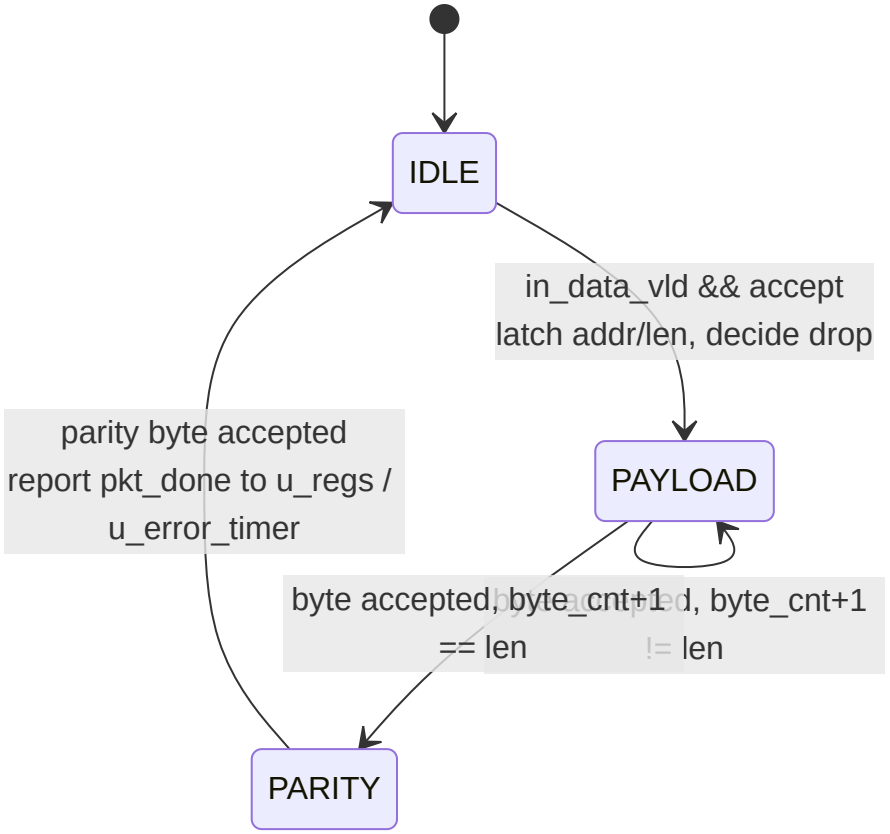
- the register block and the FSM are a loop: `u_regs` publishes the live `maxpktsize` / `router_en` fields, the FSM reports back what happened to the packet;
- everything the FSM sends to `u_regs` and `u_error_timer` is **combinational** (`pkt_done` is a decoded condition, not a flop), so the counters and the error timer update at the very same clock edge as in the original single-module design — the split moved code, not timing.

`yapp_router` — the top

```
yapp_input_fsm      u_input_fsm   (...);
generate for (ch = 0; ch < 3; ch++) begin : g_ch
  yapp_output_channel u_ch (.push(fifo_push[ch]), .din(fifo_din), .suspend(suspend[ch]),
                            .data(ch_data[ch]), .data_vld(ch_data_vld[ch]), .full(fifo_full[ch]),
...);
end endgenerate
yapp_hbus_regs      u_regs        (...);
yapp_error_timer    u_error_timer (.start(pkt_done && pkt_parity_err), .error(error), ...);
```

The port list is the one the testbench has always seen (`clock`, `reset`, the YAPP input port, `data_x/data_vld_x/suspend_x` for `x = 0..2`, the HBUS port, `error`). Only the `suspend_x` inputs are packed into a 3-bit vector and the channel outputs unpacked again; there is no logic here.

`yapp_input_fsm` — the input port



The decision to **drop** is taken when the header arrives, from the live register fields that `u_regs` exports:

```
wire hdr_drop = !router_en || (hdr_len > maxpktsize) || (hdr_addr == 2'd3);
```

`in_suspend` is asserted only for packets that will be forwarded and whose target FIFO is full; a dropped packet is swallowed without using FIFO space, so it can never stall the input.

```
always_comb begin
  case (state)
    IDLE:    in_suspend = in_data_vld && !hdr_drop && fifo_full[hdr_addr];
    default: in_suspend = !cur_drop && fifo_full[cur_addr];
  endcase
end
wire accept = !in_suspend;
```

Why this matters for the testbench

The driver (`yapp_if.send_to_dut`) holds a byte while `in_suspend` is high and the monitor (`yapp_if.collect_packets`) counts a byte only when `in_data_vld && !in_suspend` on a rising edge. DUT, driver and monitor agree on **one** rule: *a byte is accepted at a rising edge where `in_suspend` is low*.

The FSM pushes accepted bytes of a forwarded packet into the FIFO of `cur_addr` (`fifo_push[cur_addr] = 1` with `fifo_din = in_data`), accumulates the XOR of header and payload in `parity_acc`, and at the parity byte reports the packet to the register block:

```
assign pkt_done       = (state == PARITY) && accept && cur_enabled;
assign pkt_addr       = cur_addr;
assign pkt_len        = cur_len;
assign pkt_parity_err = (in_data != parity_acc);
assign pkt_oversized  = cur_oversized;
```

`cur_enabled` is `router_en` as it was when the header was accepted: a packet received while the router is disabled is tracked (so the FSM stays in sync) but never reported, forwarded or stored. The two `$display("... ROUTER DROPS PACKET ...")` lines that Lab 9A looks for in the log are printed here, in the `PARITY` state.

The bytes of the packet also leave the FSM through a small write port for the packet memory — the header to address 0 (if `router_en`), payload byte *n* to address *n* (if `cur_enabled`):

```
IDLE:    if (in_data_vld && accept && router_en)   begin pkt_mem_we = 1; pkt_mem_addr = 6'd0;
end
PAYLOAD: if (in_data_vld && accept && cur_enabled) begin pkt_mem_we = 1; pkt_mem_addr = byte_cnt
+ 6'd1; end
```

`yapp_output_channel` and `yapp_fifo` — the output side

One channel is one FIFO plus three lines of handshake:

```
yapp_fifo #(.DEPTH(16), .WIDTH(8)) u_fifo (.push, .din, .pop, .dout(data), .empty, .full, ...);
assign data_vld = !empty;
assign pop      = !empty && !suspend;
```

Nothing is registered on the output side: the FIFO head is on the bus as long as the FIFO is not empty, and the receiver's `suspend_x` directly gates the pop. That gives exactly the specified behaviour: new byte on every rising edge while `suspend_x` is low. `full` goes back to the FSM as `fifo_full[x]` and is what makes `in_suspend` rise.

`yapp_fifo` is a plain synchronous FIFO: `count` tells `empty` / `full`, `dout` always shows the head (`mem[rd_ptr]`), `push` / `pop` move the pointers. The three channels are created in a `generate` loop `g_ch[0..2]` in the top.

`yapp_hbus_regs` — registers, memories and HBUS

All registers and memories live here, under the names the register model's backdoor paths use (`hw_top.dut.u_regs.ctrl_reg`, `...u_regs.yapp_mem`, ...): `ctrl_reg`, `en_reg`, the six counters, `mem_size_reg`, `yapp_pkt_mem[0:63]` and `yapp_mem[0:255]`. The decoded fields go back to the FSM:

```
assign maxpktsize = ctrl_reg[5:0];
assign router_en  = en_reg[0];
```

Counters and `mem_size_reg` are updated from the FSM's end-of-packet report, i.e. in the cycle the parity byte is accepted and only for packets received with `router_en` set (`pkt_done` already includes that):

```
if (pkt_mem_we) begin
  yapp_pkt_mem[pkt_mem_addr] <= pkt_mem_wdata;
end
if (pkt_done) begin
  mem_size_reg <= {2'b00, pkt_len};
  if (pkt_parity_err && parity_err_cnt_en) begin
    parity_err_cnt_reg <= parity_err_cnt_reg + 8'd1;
  end
  if (pkt_oversized && oversized_pkt_cnt_en) begin
    oversized_pkt_cnt_reg <= oversized_pkt_cnt_reg + 8'd1;
  end
  case (pkt_addr)  // one of addr0..3_cnt_reg++, each behind its en_reg bit
```

Dropped packets are still fully received, so an oversized packet to address 3 increments **both** `oversized_pkt_cnt_reg` and `addr3_cnt_reg`.

The HBUS side is a one-cycle write and a two-cycle read with a tri-stated data bus:

```
assign hdata = hdata_oe ? hdata_out : 8'bz;

always_ff @(posedge clock) begin
  hdata_oe <= hen && !hwr_rd;         // drive during the second read cycle
  if (hen && !hwr_rd) begin
    hdata_out <= rd_mux;
  end
  if (hen && hwr_rd) begin
    ...                               // single-cycle write
  end
end
```

`rd_mux` is a combinational decoder of `haddr` over the registers, the packet memory (`0x1010..0x104f`) and the scratch memory (`0x1100..0x11ff`). Writes reach only `ctrl_reg`, `en_reg` and `yapp_mem`.

```
`ifdef INJECT_ERROR
  if (haddr[7:0] == 8'h2a) begin
    rd_mux[3] = ~rd_mux[3];   // Lab 11B: make the memory test fail
  end
`endif
```

`yapp_error_timer` — the `error` pulse

Every packet with bad parity received while the router is enabled (`start = pkt_done && pkt_parity_err`) gets its own one-cycle `error` pulse, 1..10 cycles after its parity byte. A free-running 4-bit counter picks the wanted delay; the pulses still owed are a schedule, `sched[k] = 1` meaning "a pulse k cycles from now". Every cycle the schedule moves one step and `error` follows `sched[1]`; a new bad packet takes the first free slot at or after its wanted delay (wrapping inside 1..10).

```
wire [3:0]  err_delay  = (rnd_cnt % 4'd10) + 4'd1;
wire [10:1] sched_next = sched >> 1;              // the schedule after this edge

error <= sched[1];
if (start) begin
  sched <= sched_next | (10'b1 << (free_slot(sched_next, err_delay) - 4'd1));
end else begin
  sched <= sched_next;
end
```

A packet is at least 3 cycles long, so at most 3 earlier pulses are owed when a new one ends and a free slot always exists. Two pulses may land on adjacent cycles; every cycle with `error` high is one pulse. For an isolated packet the timing is that of a single down-counter loaded with the delay.

Why a schedule

The first version had one down-counter, re-loaded by every bad packet. The storms of `random_parity_test` (1..3-byte bad packets back to back) showed that a second bad packet ending before the first pulse took that pulse away: one pulse for two packets. See Random tests.

The files

- **yapp_project/rtl/yapp_router.sv**
- **yapp_project/rtl/yapp_input_fsm.sv**
- **yapp_project/rtl/yapp_output_channel.sv**
- **yapp_project/rtl/yapp_fifo.sv**
- **yapp_project/rtl/yapp_hbus_regs.sv**
- **yapp_project/rtl/yapp_error_timer.sv**

yapp_project/rtl/yapp_router.f

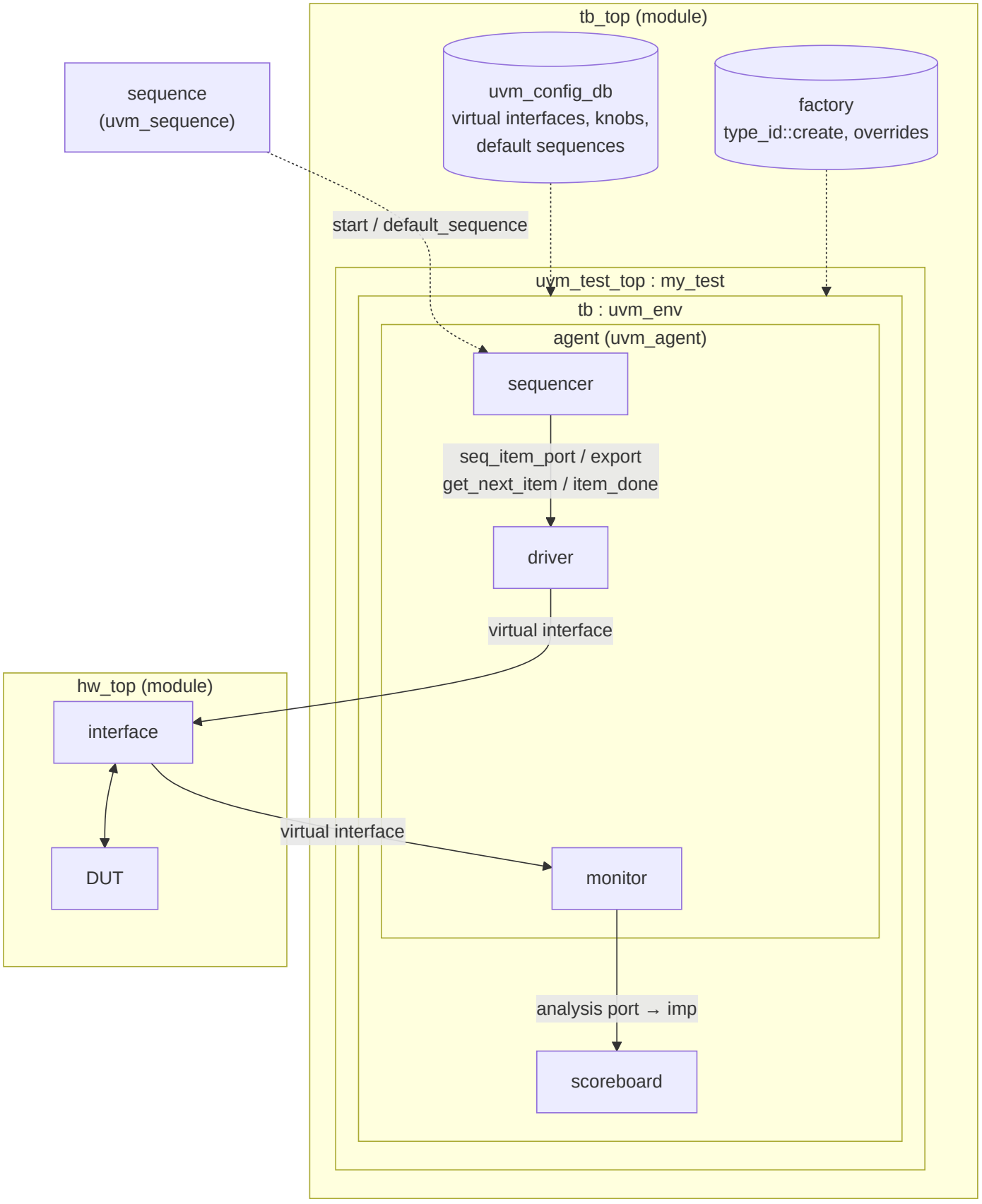
UVM

UVM — the big picture

UVM is a class library plus a set of conventions for building **reusable, layered testbenches**. The point of the layers is separation of concerns:

- **tests** say *what* to do (which sequences, which configuration),
- **sequences** say *which transactions* to generate,
- **drivers** say *how a transaction becomes pin wiggles*,
- **monitors** turn pin wiggles back into transactions,
- **scoreboards and reference models** decide whether the DUT did the right thing,
- **the environment** wires all of it together and is reused unchanged by every test.

A generic UVM testbench

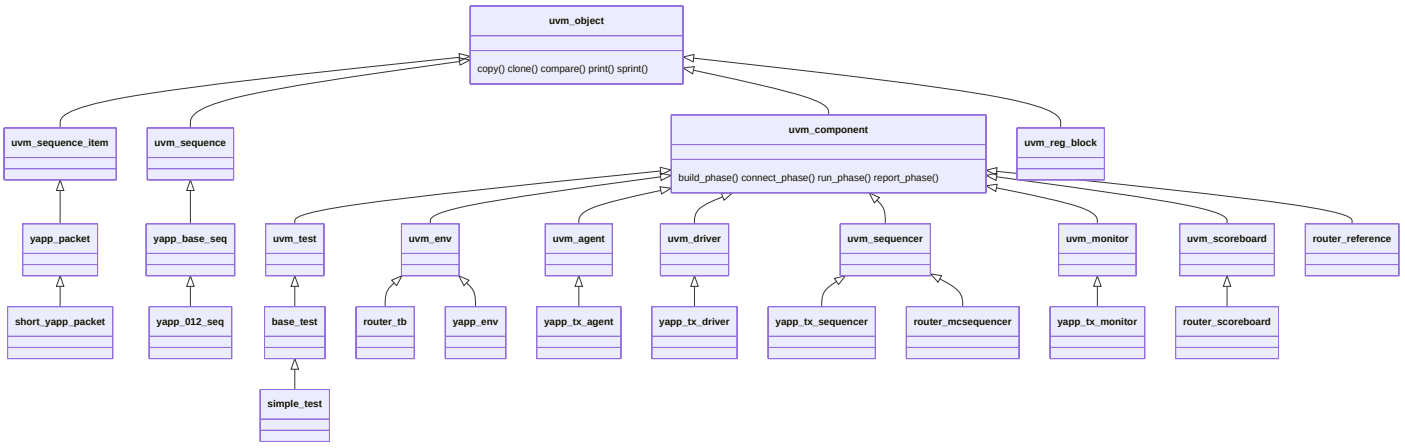


Reading the diagram:

- Everything inside `uvm_test_top` is a **component**: it has a name, a parent and goes through the **phases** (`build`, `connect`, `run`, `report`, ...).

- A **sequence** is *not* a component. It is an object that is started on a sequencer and produces **sequence items** (transactions).
- The two halves of the testbench — the class-based UVM side and the module-based hardware side — meet at the **interface**: the UVM side holds a **virtual interface** handle that was published in `uvm_config_db`.

The class hierarchy you will use



Base class	Use it for	Has a parent?	Goes through phases?
`uvm_sequence_item`	a transaction (packet, bus access)	no	no
`uvm_sequence #(ITEM)`	a generator of transactions	no	no (runs inside `run_phase` of its sequencer)
`uvm_driver #(ITEM)`	transaction → pins	yes	yes
`uvm_monitor`	pins → transaction	yes	yes
`uvm_sequencer #(ITEM)`	arbitration between sequences, hands items to the driver	yes	yes

Base class	Use it for	Has a parent?	Goes through phases?
`uvm_agent`	driver + sequencer + monitor for **one** interface	yes	yes
`uvm_env`	a group of agents and analysis components	yes	yes
`uvm_test`	the top of the component tree; configures the env	yes	yes
`uvm_scoreboard`	checking	yes	yes

Concepts and where the labs introduce them

Concept	One-line summary
Data item methods	`do_print` / `do_copy` / `do_compare` / `do_pack` behind `print()`, `copy()`, `clone()`, `compare()`, `pack()` (the course uses `` `uvm_field_* `` macros for the same thing)
Phases	`build` (top-down) → `connect` → `end_of_elaboration` → `start_of_simulation` → **`run`** (time passes) → `extract` → `check` → `report`
`run_test()`	creates the test named by `+UVM_TESTNAME` and starts phasing
Agent, active/passive	`is_active` decides whether a driver and sequencer are built

Concept	One-line summary
Default sequence	`uvm_config_wrapper::set(..., "run_phase", "default_sequence", type)`
Factory	`type_id::create()` + `set_type_override_by_type()`
Configuration	`uvm_config_int::set/get`, `check_config_usage()`
Sequences	`body()`, `start_item` / `randomize() with` / `finish_item`, `seq.start(sequencer, this)` for nesting
Objections	`raise_objection` / `drop_objection` keep `run_phase` alive; drain time
Virtual interface	`uvm_config_db #(virtual yapp_if)`
Virtual sequencer	a sequencer that only holds handles to other sequencers
TLM analysis	`uvm_analysis_port` → `uvm_analysis_imp` / `export` / `tlm_analysis_fifo`
Coverage	a covergroup in a `uvm_subscriber` fed by the monitor's analysis port (Functional coverage)
Register model	`uvm_reg`, maps, adapter, front/backdoor

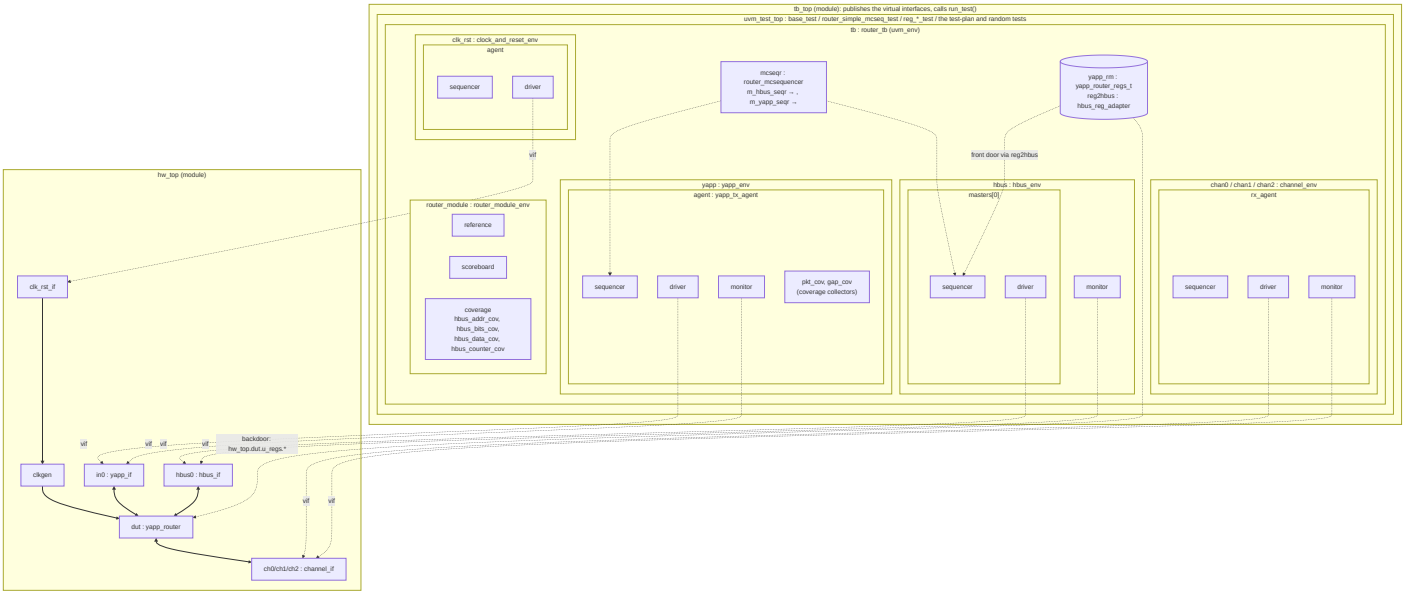
Go deeper: Phases and objections · Factory and configuration · TLM connections · Functional coverage.

Project architecture

Open on the project map →

This is what the testbench looks like once every lab is done (`yapp_project/tb`, the source of truth; `labs/lab11c_rm_sim` is the last snapshot of the course). Each lab page has the sub-diagram for its own stage.

Component tree



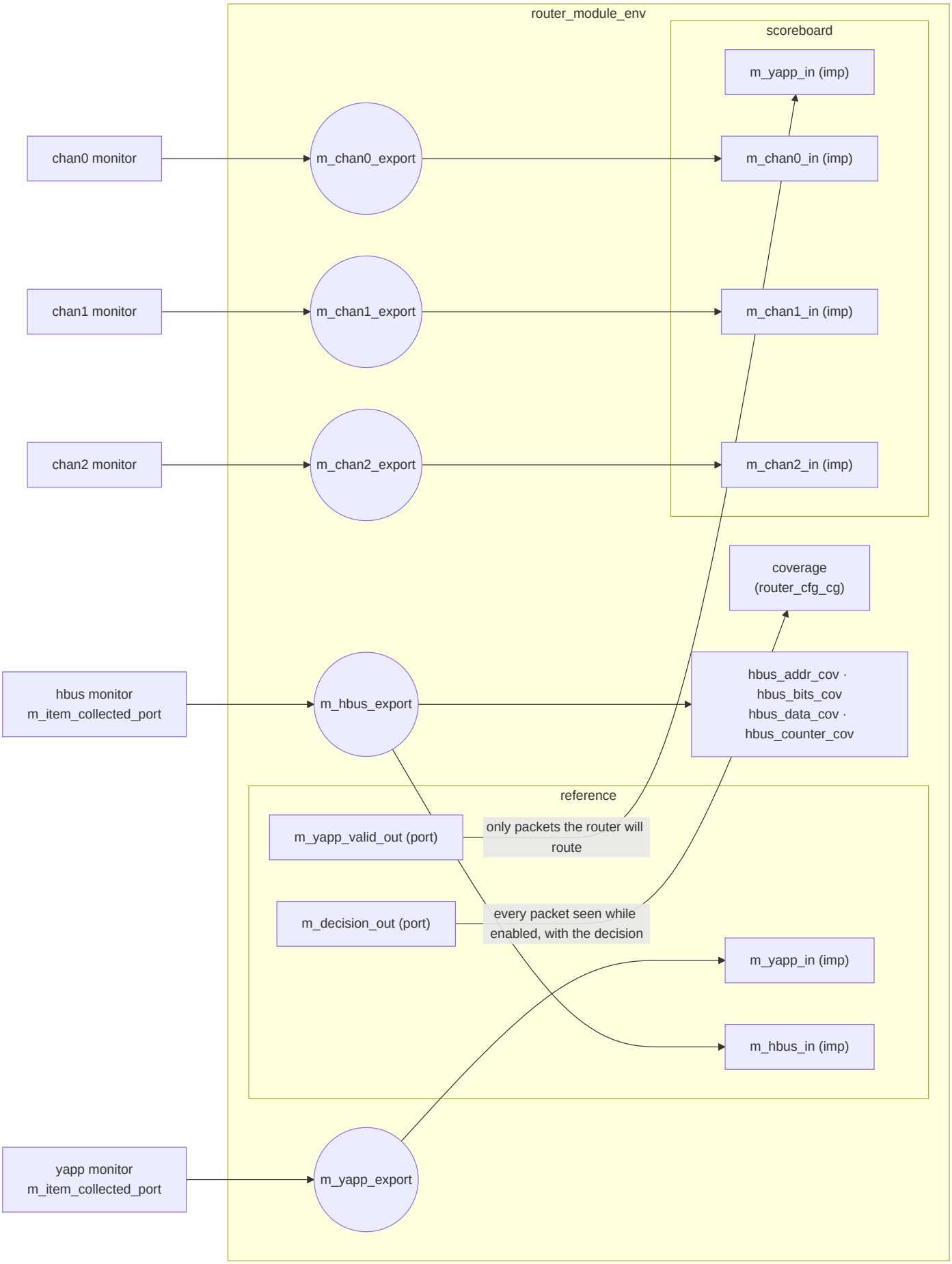
Instance paths

These are the names you use in `uvm_config_*::set` calls and hierarchical references. They come straight from `uvm_top.print_topology()`.

Path	Type	Used for
`tb.yapp.agent.sequencer`	`yapp_tx_sequencer`	YAPP default sequence; `m_mcseqr.m_yapp_seqr`
`tb.yapp.agent.monitor`	`yapp_tx_monitor`	`m_item_collected_por` (→ scoreboard, reference, `pkt_cov`,

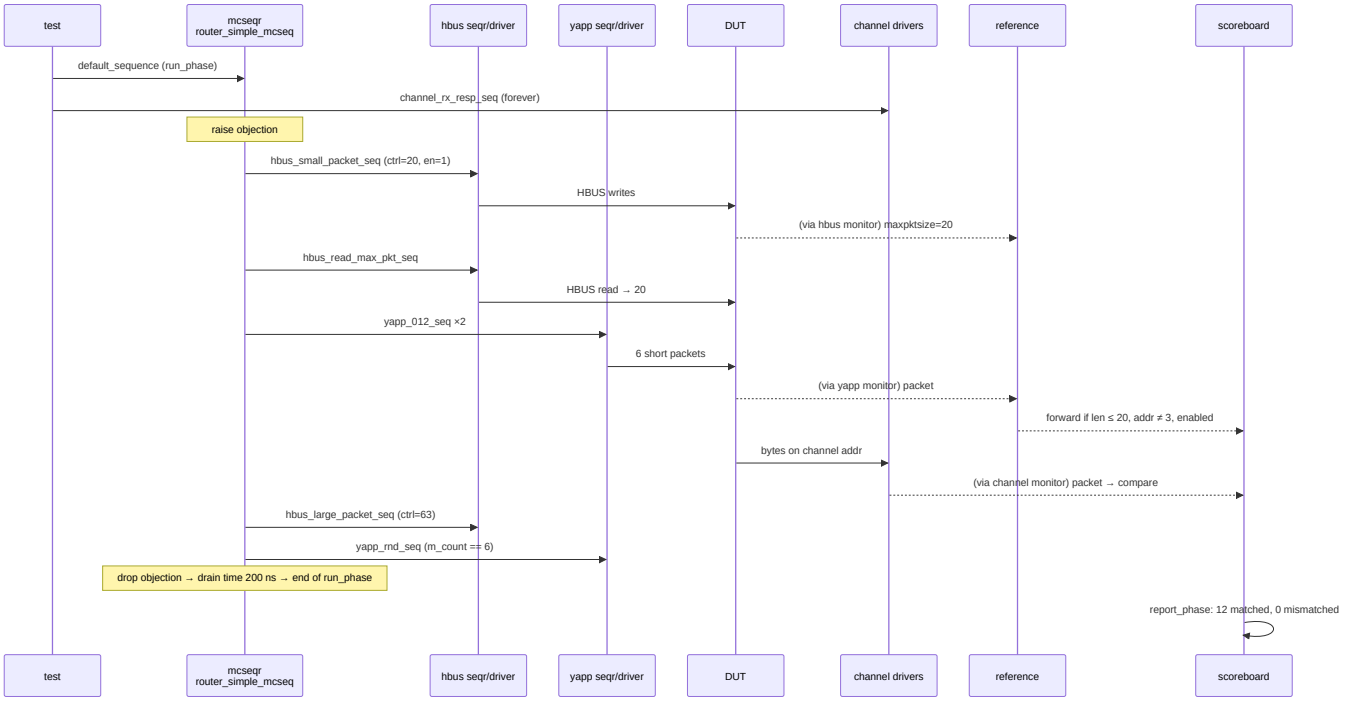
Path	Type	Used for
		`gap_cov`), `m_num_bad_parity`
`tb.yapp.agent.pkt_cov`, `gap_cov`	`yapp_pkt_coverage`, `yapp_gap_coverage`	the covergroups of the YAPP packets and the gaps between them
`tb.router_module.coverage`, `hbus_*_cov`	`router_coverage`, `router_hbus_*_coverage`	the covergroups of the router configuration and of the HBUS traffi
`tb.chan*.rx_agent.sequencer`	`channel_rx_sequencer`	`channel_rx_resp_seq` (one wildcard for all three)
`tb.hbus.masters[0].sequencer`	`hbus_master_sequencer`	`m_mcseqr.m_hbus_seqr` register map sequence
`tb.hbus.monitor`	`hbus_monitor`	`m_item_collected_por` → reference model
`tb.clk_rst.agent.sequencer`	`clock_and_reset_sequencer`	`clk10_rst5_seq`
`tb.mcseqr`	`router_mcsequencer`	`router_simple_mcseq`
`tb.router_module`	`router_module_env`	exports `yapp_export`, `hbus_export`, `chan0..2_export`
`tb.yapp_rm.router_yapp_regs`	`yapp_regs_c`	register handles in the Lab 11 tests

TLM connections


router_module_env
scoreboard
m_yapp_in (imp)
chan0 monitor
m_chan0_export
m_chan0_in (imp)
chan1 monitor
m_chan1_export
m_chan1_in (imp)
chan2 monitor
m_chan2_export
m_chan2_in (imp)
coverage
(router_cfg_cg)
hbus monitor
m_item_collected_port
m_hbus_export
hbus_addr_cov ·
hbus_bits_cov
hbus_data_cov ·
hbus_counter_cov
reference
m_yapp_valid_out (port)
only packets the router will route
m_decision_out (port)
every packet seen while enabled, with the decision
m_yapp_in (imp)
m_hbus_in (imp)
yapp monitor
m_item_collected_port
m_yapp_export

- **ports** (`uvm_analysis_port`) are owned by producers (monitors). `write()` on a port calls `write()` on every connected imp — synchronously, in the producer's thread.
- **imps** (`uvm_analysis_imp_<suffix>`) are owned by consumers and bound to a `write_<suffix>()` method.
- **exports** forward: they let an env expose an imp that lives deeper inside.

Data flow of one test: `router_simple_mcseq_test`



Files of the final environment

The final environment is `yapp_project/` (`rtl/` , `uvc/` , `tb/`); the lab column says where each file first appears. Every class has its own file, with `extern` prototypes in the class and the method bodies after `endclass` ; sequences spell out `start_item` / `randomize` / `finish_item` instead of `uvm_do*` , and data items implement their `do_*` methods instead of using `uvm_field_*` macros (see Component guides).

File	Class / module	Lab
`yapp_project/uvc/yapp/yapp_packet.sv`	`yapp_packet` , `short_yapp_packet` , `parity_type_e`	1, 4
`yapp_project/uvc/yapp/yapp_tx_driver.sv`	`yapp_tx_driver`	3, 6

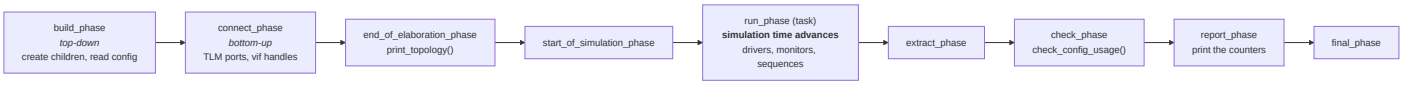
File	Class / module	Lab
`yapp_project/uvc/yapp/yapp_tx_sequencer.sv`	`yapp_tx_sequencer`	3
`yapp_project/uvc/yapp/yapp_tx_monitor.sv`	`yapp_tx_monitor` (+ analysis port)	3, 6, 9A, 10
`yapp_project/uvc/yapp/yapp_pkt_coverage.sv`, `yapp_gap_coverage.sv`	the coverage collectors of the YAPP packets (subscribers of the monitor)	10, test plan
`yapp_project/uvc/yapp/yapp_tx_agent.sv`, `yapp_env.sv`	`yapp_tx_agent`, `yapp_env`	3, 4
`yapp_project/uvc/yapp/yapp_tx_seqs.sv`	the sequence library	3, 5, 7, 10
`yapp_project/uvc/yapp/yapp_if.sv`	`yapp_if`	6
`yapp_project/uvc/hbus/*`, `yapp_project/uvc/channel/*`, `yapp_project/uvc/clock_and_reset/*`	the provided UVCs	7
`yapp_project/tb/router_tb.sv`	`router_tb`	2 → 11B
`yapp_project/tb/router_test_lib.sv` + `tests/`	the tests, one class per file	2 → 11C
`yapp_project/tb/hw_top.sv`, `tb_top.sv`	top modules	6, 7
`yapp_project/tb/router_mcsequencer.sv`, `router_mcseqs_lib.sv` + `mcseqs/`	virtual sequencer and sequences	8

File	Class / module	Lab
`yapp_project/uvc/router/router_scoreboard.sv`, `router_reference.sv`, `router_decision.sv`, `router_module_env.sv`, `router_fifo_scoreboard.sv`	router module UVC	9A–9D
`yapp_project/uvc/router/router_coverage.sv`, `router_hbus_addr_coverage.sv`, `router_hbus_bits_coverage.sv`, `router_hbus_data_coverage.sv`, `router_hbus_counter_coverage.sv`	the coverage collectors of the router (configuration, HBUS address map, register bits, `yapp_mem` patterns, counter values)	test plan
`yapp_project/tb/yapp_router_tracer.sv`	the DUT tracer bound into the router: `[DUT]` log lines and the per-cycle signal trace	test plan
`yapp_project/tb/yapp_router_reg_pkg.sv` + `reg/`	register model	11A
`yapp_project/rtl/yapp_router.sv`, `yapp_input_fsm.sv`, `yapp_output_channel.sv`, `yapp_fifo.sv`, `yapp_hbus_regs.sv`, `yapp_error_timer.sv` (`yapp_router.f`)	the DUT, one module per file	—

Phases and objections

The phases

Every `uvm_component` runs through the same ordered list of phases. The **function** phases take no simulation time; `run_phase` is a **task** and is where the clock ticks.



Phase	Direction	What the course puts there
`build_phase`	top-down (test first, leaves last)	`super.build_phase(phase)`, configuratior `type_id::create()` of the children
`connect_phase`	bottom-up	`m_driver.seq_item_port.connect(m_seque` analysis port → imp, `uvm_config_db::get` hierarchical handles of the virtual sequen
`end_of_elaboration_phase`	bottom-up	`uvm_top.print_topology()` in the test
`start_of_simulation_phase`	bottom-up	the optional "who runs first" experiment c
`run_phase`	parallel for all components	driver loop, monitor loop, default sequenc
`check_phase`	bottom-up	`check_config_usage()`, "are the FIFOs en
`report_phase`	bottom-up	every component prints its statistics

Lab 3: which `start_of_simulation_phase` runs first, which last — and why?

The bottom-up phases are executed from the leaves to the root: the driver / monitor / sequencer first, then the agent, the YAPP env, the testbench, and the test last. `build_phase` is the only top-down one because a parent must exist before it can create its children.

Why `run_phase` ends when it ends: objections

`run_phase` finishes as soon as **nobody objects** to it ending. A component or sequence that still has work to do raises an objection and drops it when done:

```
task run_phase(uvm_phase phase);
  phase.raise_objection(this);
  ...                                 // do the work
  phase.drop_objection(this);
endtask
```

In this project the **sequences** own the objections. `yapp_base_seq` raises one in `pre_body()` and drops it in `post_body()`, so every sequence of the library keeps the simulation alive exactly as long as it runs:

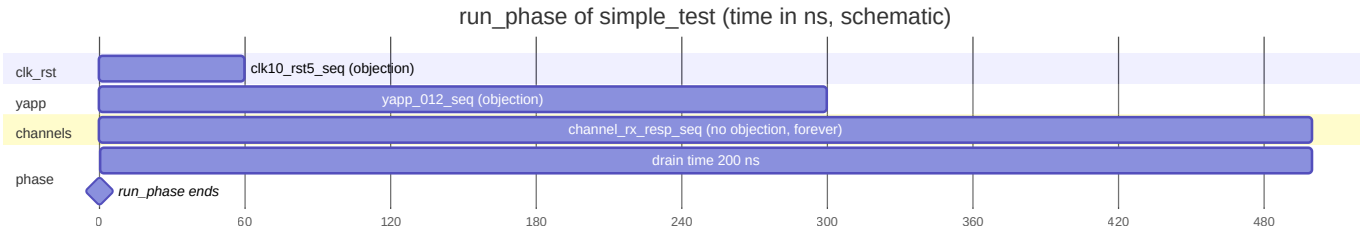
```
task pre_body();
  uvm_phase phase = `YAPP_STARTING_PHASE;      // starting_phase / get_starting_phase()
  if (phase != null) begin
    phase.raise_objection(this, get_type_name());
  end
endtask
```

`starting_phase` is only set for a sequence that the sequencer starts as its **default sequence**; for a sub-sequence (`m_seq_1.start(m_sequencer, this)`) it is `null`, hence the test. The multichannel sequence (Lab 8) and the HBUS/Clock sequences use the same pattern. The channel response sequence deliberately does **not** raise one: a receiver must never keep the simulation running on its own.

Drain time

When the last objection drops, packets may still be inside the router. The test adds a **drain time**: the phase waits that long after the last drop before it ends.

```
task run_phase(uvm_phase phase);
  uvm_objection obj = phase.get_objection();
  obj.set_drain_time(this, 200ns);
endtask
```



Phase-related pitfalls seen in the labs

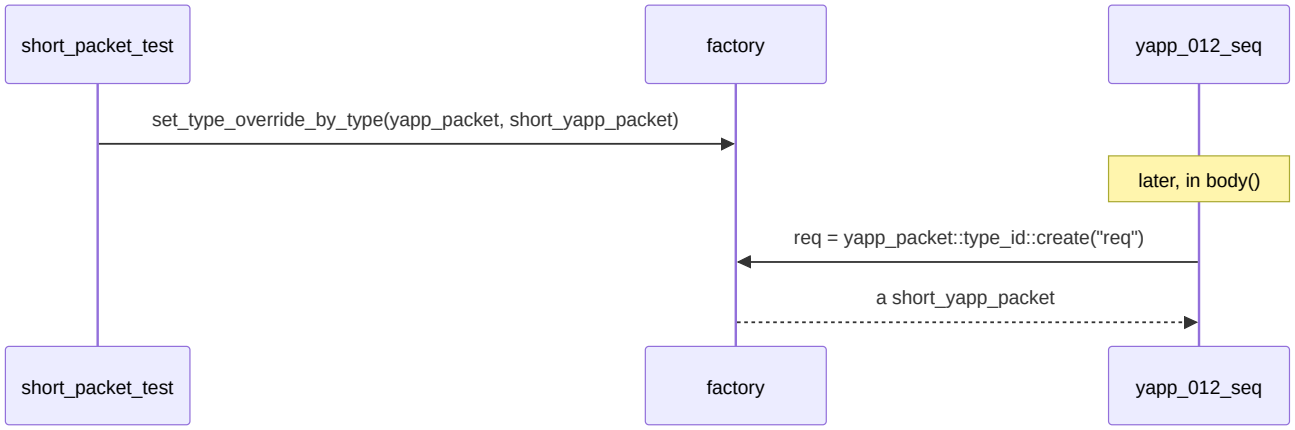
- Reading the configuration (`uvm_config_int::get(this, "", "is_active", …)`) **after** creating the children — the value arrives too late to decide whether a driver exists. In this repository the `get()` calls come right after `super.build_phase(phase)` , before any `create` .
- Calling `uvm_config_*::set` **after** creating the component that reads it (Lab 4 `set_config_test`).
- No drain time → the last packet never leaves the router and the scoreboard reports packets left in its queues (Lab 6).
- A `forever` loop in `run_phase` without an objection is fine (drivers, monitors); a `forever` loop **with** an objection never lets the test end.

Factory and configuration

Both mechanisms exist for one reason: a **test** must be able to change what the **environment** does *without editing the environment*.

The factory

`type_id::create()` asks the factory for an object of the requested type. The factory may answer with a **different, derived** type if an override is in place.



Rules of the game:

1. **Register** the class with the factory: `` `uvm_object_utils(T) `` / `` `uvm_component_utils(T) `` (Lab 1, 2).
2. **Create** through the factory, never with `new()` directly, for anything a test might want to replace: `T::type_id::create("name", this)` for components (Lab 4), `req = yapp_packet::type_id::create("req")` in a sequence for items (the `` `uvm_do `` macros of the course do the same call internally).
3. **Override** from the test, before the object is created:

```
function void build_phase(uvm_phase phase);
  set_type_override_by_type(yapp_packet::get_type(), short_yapp_packet::get_type());
  super.build_phase(phase);     // the testbench is built after the override
endfunction
```

The override is global for the whole simulation: every `yapp_packet` the sequences generate becomes a `short_yapp_packet` — the sequences never know. That is how `exhaustive_seq_test` sends short packets with a library that was written for full-size ones.

Why a derived class can replace the base class everywhere

`short_yapp_packet` only *adds* constraints. Everything that works with a `yapp_packet` handle (the driver, the monitor, the scoreboard) keeps working, because a derived object **is a** base object.

Configuration

`uvm_config_db #(T)::set(context, instance_path, field, value)` stores a value under a hierarchical name; `get()` retrieves it. The course uses three typed shorthands:

Typedef	T	Used for
`uvm_config_int`	`int`	`is_active`, `channel_id`, `num_masters`, `recording_detail`
`uvm_config_wrapper`	`uvm_object_wrapper`	the **default sequence** type of a sequencer phase
`yapp_vif_config` (and friends)	`virtual yapp_if`	virtual interfaces

```
// in a test, BEFORE the env is built
uvm_config_int::set(this, "tb.yapp.agent", "is_active", UVM_PASSIVE);
uvm_config_int::set(this, "tb.chan0", "channel_id", 0);
uvm_config_wrapper::set(this, "tb.chan*.rx_agent.sequencer.run_phase",
                        "default_sequence", channel_rx_resp_seq::get_type());

// in tb_top (module, context = null): wildcards reach driver AND monitor
yapp_vif_config::set(null, "*.tb.yapp.*", "vif", hw_top.in0);

// in the consumer
if (!yapp_vif_config::get(this, "", "vif", m_vif))
  `uvm_error("NOVIF", "vif not set")
```

How a component receives a setting: with `get()`, in its `build_phase`, right after `super.build_phase(phase)` and before it creates its children:

```
function void yapp_tx_agent::build_phase(uvm_phase phase);
  super.build_phase(phase);
  begin
    uvm_bitstream_t cfg_is_active;
    if (uvm_config_int::get(this, "", "is_active", cfg_is_active))
      is_active = uvm_active_passive_enum'(cfg_is_active);
  end
  m_monitor = yapp_tx_monitor::type_id::create("monitor", this);
  ...
```

That is how `is_active`, `channel_id`, `num_masters` and the virtual interfaces arrive. The course declares such fields with `` `uvm_field_* `` macros and lets `super.build_phase()` look them up; the result is the same, but with the explicit `get()` you can see which settings a component reads and when, and `check_config_usage()` below reports the ones nobody read.

`check_config_usage()`

Called in `check_phase` of the test, it lists every setting that nobody read. In Lab 4 `set_config_test` made the agent passive, so the sequencer was never built and the `default_sequence` setting of `base_test` became unused. The fix in this repository: `base_test` puts its sequence configuration in a virtual function `configure_sequences()` that the passive test overrides with an empty body — no stray settings, no warnings.

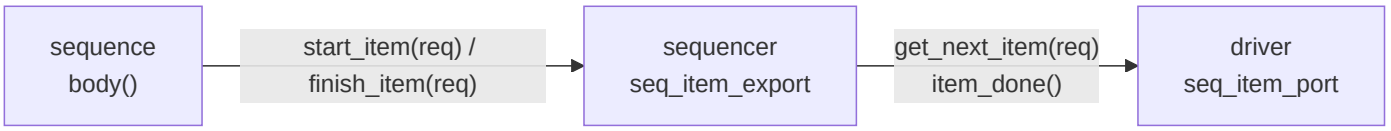
Factory vs configuration — which one?

Want to change…	Use
the **type** of something (packet, driver, scoreboard)	factory override
a **value** (a knob, a mode, a handle, a sequence to run)	configuration
the **structure** (is there a driver?)	configuration of `is_active`, read in `build_phase`

TLM connections

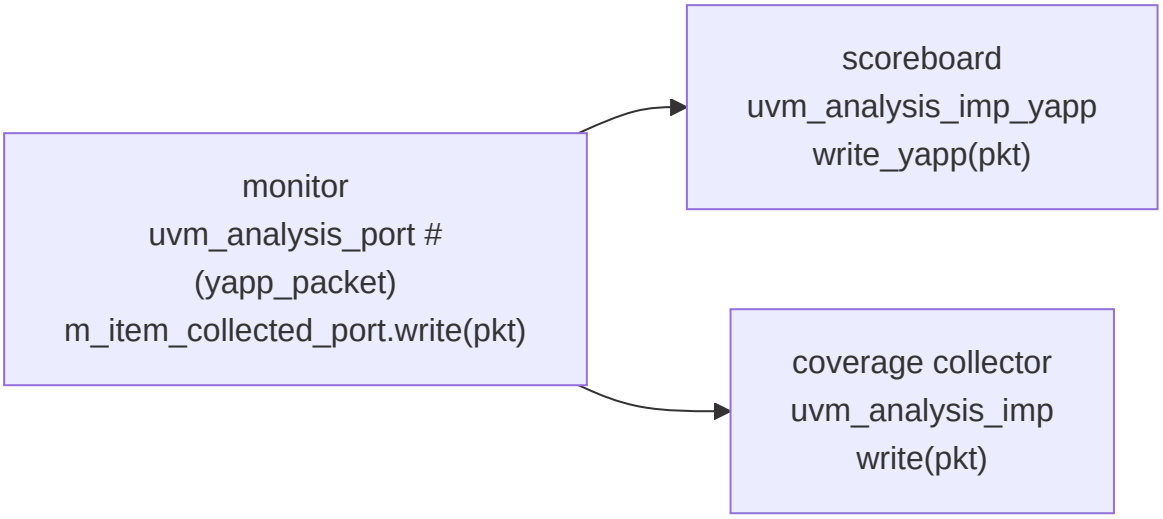
Transaction-Level Modelling ports let components exchange *objects* instead of signals. UVM has many flavours; the course uses four.

Driver ↔ sequencer: `seq_item_port`



- `uvm_driver` already has `seq_item_port`; `uvm_sequencer` already has `seq_item_export`. The agent connects them (Lab 3).
- `get_next_item()` **blocks** until a sequence offers an item; `item_done()` unblocks the sequence's `finish_item()`. One item at a time, in order.

Monitor → analysis components: port / imp



- A **port** can be connected to any number of imps (0 included). `write()` is a function: it calls every imp **synchronously**.
- An **imp** must implement `write()`. When one component needs several imps (the scoreboard listens to YAPP *and* three channels) the `` `uvm_analysis_imp_decl(_suffix) `` macro creates a flavour whose method is `write_suffix()`:

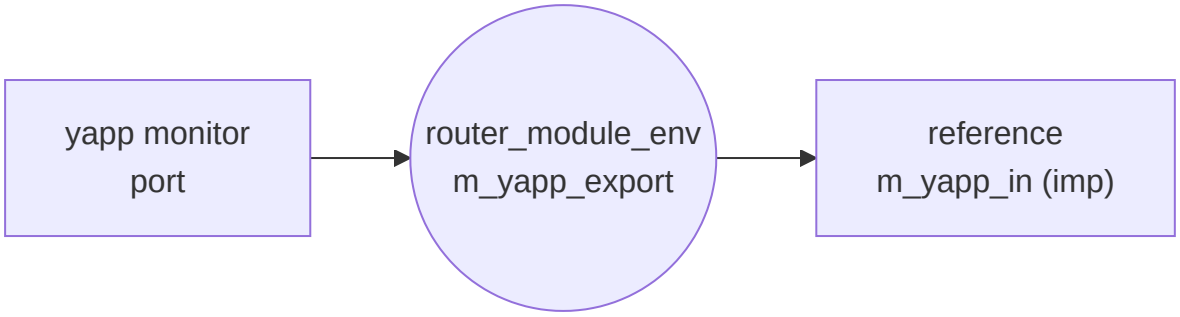
```
`uvm_analysis_imp_decl(_yapp)                    // outside the class
`uvm_analysis_imp_decl(_chan0)

class router_scoreboard extends uvm_scoreboard;
  uvm_analysis_imp_yapp  #(yapp_packet,    router_scoreboard) m_yapp_in;
  uvm_analysis_imp_chan0 #(channel_packet, router_scoreboard) m_chan0_in;
  function void write_yapp (yapp_packet p);    ... endfunction
  function void write_chan0(channel_packet p); ... endfunction
```

The handle is shared

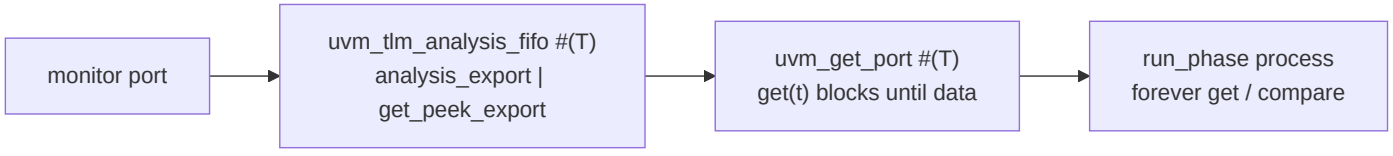
`write()` passes a **handle**. If the consumer stores it, it must `clone()` first, or create a new object per transaction on the producer side — otherwise the next packet overwrites the stored one. The scoreboard clones (Lab 9A); every monitor in this repository also creates a fresh object per transaction, which is what Lab 9D requires.

Export: forwarding a connection point



An **export** is a pass-through. It lets `router_module_env` show one set of connection points at its boundary while the imps stay inside the reference model and the scoreboard (Lab 9C). The testbench then connects `monitor.port → env.export` and never looks inside the env.

Analysis FIFO: decoupling with a buffer



Instead of reacting inside a `write()` callback, the consumer **pulls** in its own process (Lab 9D). This makes it natural to wait for *two* things in order ("first the YAPP packet, then the matching channel packet") and to use time and blocking calls, which a function cannot do.

Connecting the pieces

Producer side	Consumer side	Connect call (in the
`uvm_analysis_port #(T)`	`uvm_analysis_imp_x #(T, C)`	`port.connect(imp)`
`uvm_analysis_port #(T)`	`uvm_analysis_export #(T)`	`port.connect(expor`
`uvm_analysis_port #(T)`	`uvm_tlm_analysis_fifo #(T)`	`port.connect(fifo.`
`uvm_get_port #(T)`	`uvm_tlm_analysis_fifo #(T)`	`get_port.connect(f`
`uvm_driver::seq_item_port`	`uvm_sequencer::seq_item_export`	`m_driver.seq_item_`

Always connect **from the port to the export/imp** (`port.connect(target)`), and always in a `connect_phase` — the components must exist first.

Functional coverage

Functional coverage answers one question the simulator cannot answer by itself: **did the tests exercise everything the plan asked for?** A scoreboard tells you the DUT was *right* for the traffic it saw; coverage tells you *which* traffic it saw. Code coverage (lines, branches, toggles, collected by the simulator) says which RTL ran; functional coverage is written by the verification engineer and says which *scenarios* happened: every packet length, every address, a bad parity to every legal channel, a register written while traffic was flowing. This page is the general picture; Functional coverage in the project shows the seven collectors of the router and Lab 10 builds the first one.

The vocabulary

Term	What it is	In the router
covergroup	a named set of things to count, with a `sample()` function that takes the values	`yapp_pkt_cg`, `hbus_addr_cg`, …
coverpoint	one value to watch, sorted into bins	`length_cp : coverpoint length`
bin	a bucket of values; a bin is *hit* when a sampled value falls into it	`bins SMALL = {[2:10]};`
cross	every combination of the bins of two or more coverpoints	length × address × parity
ignore_bins	combinations that are not part of the requirement (dropped from the denominator)	good parity in the cross

Term	What it is	In the router
illegal_bins	combinations that must never happen (an error when hit)	not used here: address 3 *is* exercised on purpose
hole	a bin no run hit	what the regression report lists first
merge	adding up the hits of every run and seed into one database	`make sim-report`

The percentage of a covergroup is *bins hit / bins*, so the bins decide what 100 % means: five length bins say "we sent short, medium and long packets", not "we sent all 63 lengths". Choose them from the plan, not from the value range.

Where it samples: a subscriber on the monitor

Coverage lives where the truth is. The **monitor** reports what really crossed the interface, whatever the sequence intended and whatever the driver did; a `uvm_subscriber` hangs on the monitor's analysis port and samples in its `write()`:

```
class yapp_pkt_coverage extends uvm_subscriber #(yapp_packet);

  int m_num_sampled;

  `uvm_component_utils(yapp_pkt_coverage)

  covergroup yapp_pkt_cg with function sample(bit[5:0] length, bit[1:0] addr, parity_type_e
parity_type);
    option.per_instance = 1;
    length_cp : coverpoint length { bins MIN = {1}; bins SMALL = {[2:10]}; bins MEDIUM =
{[11:40]};
                                    bins LARGE = {[41:62]}; bins MAX = {63}; }
    addr_cp   : coverpoint addr   { bins legal[] = {[0:2]}; bins illegal_addr = {3}; }
    parity_cp : coverpoint parity_type { bins good = {PARITY_TYPE_GOOD}; bins bad =
{PARITY_TYPE_BAD}; }
    len_x_addr_x_parity : cross length_cp, addr_cp, parity_cp {
      ignore_bins good_parity  = binsof(parity_cp) intersect {PARITY_TYPE_GOOD};
      ignore_bins illegal_addr = binsof(addr_cp.illegal_addr);
    }
  endgroup

  extern function new(string name, uvm_component parent);
  extern function void write(yapp_packet t);

endclass : yapp_pkt_coverage

//------------------------------------------------------------------------------
function void yapp_pkt_coverage::write(yapp_packet t);
  yapp_pkt_cg.sample(t.m_length, t.m_addr, t.m_parity_type);
  m_num_sampled++;
endfunction : write
```

The agent connects it like any other analysis consumer: `m_monitor.m_item_collected_port.connect(m_pkt_cov.analysis_export)`. Nothing in the monitor knows the collector exists, so the coverage model can be swapped, extended or left out of a passive agent without touching the UVC. The course puts the covergroup *inside* the monitor; the bins and the numbers are the same, only the file differs (see where the covergroups live).

> **Why not the sequence or the driver?**
>
> A sequence only knows what it *asked* for; a driver may drop or corrupt what it was given. The monitor sees the packet the DUT received, including traffic from other sources, and it exists in a passive agent that drives nothing. Sampling there means the percentage describes the DUT's day, not the testbench's intentions.

The coverage-driven loop

1. **Plan.** Every requirement of the test plan names its coverage: which covergroup, which bins, prove it happened.
2. **Run.** Each run writes its own coverage database next to its log.

3. **Merge.** The regression merges every run and seed into one picture (`make regress`, then `make sim-report`): one hit anywhere is enough for a bin.
4. **Read the holes.** The report lists the bins no run hit, first.
5. **Close them with stimulus.** A hole is a scenario the tests never produced: write a sequence that reaches it (`yapp_coverage_seq` sends every length bucket to every address with both parities), or a constraint that makes random traffic reach it more often, then run again.
6. **Review the bins** when 100 % arrives: a model that is easy to fill was asking too little.

Coverage is additive across tests, so a directed test for one bin and a random test that hits the rest together make 100 %; the coverage results page shows which tests contributed to each bin, and the Coverage view of the project map draws the same collectors as cards.

Reading the numbers

- `option.per_instance = 1` reports every collector instance on its own; with one agent it changes nothing, with three channel monitors it tells the channels apart.
- A cross with `ignore_bins` counts only the combinations that matter: 5 lengths × 3 legal addresses × bad parity = 15 bins, not 5 × 4 × 2 = 40.
- A bin hit once by one seed is as covered as a bin hit a thousand times. Hit counts are for judging the stimulus mix, not the percentage.
- Verilator, xrun and the other simulators keep their own database formats; the report generator reads Verilator's (`.dat`) and merges with `verilator_coverage`, xrun uses IMC (Seeing the results).

Where to look next

- Functional coverage in the project: the seven collectors, their requirements and how they are fed.
- Lab 10: write the first covergroup, find the holes, close them.
- Coverage results: every covergroup of the last regression, merged, with the tests behind each bin, and the annotated source.
- Regression results: the runs that produced those numbers.

Component guides

Component guides

Open on the project map →

The lab pages follow the course order. These guides explain the same code by **role**: open the one for the component you are writing, read what it must do, look at the reference implementation and the mistakes people make.

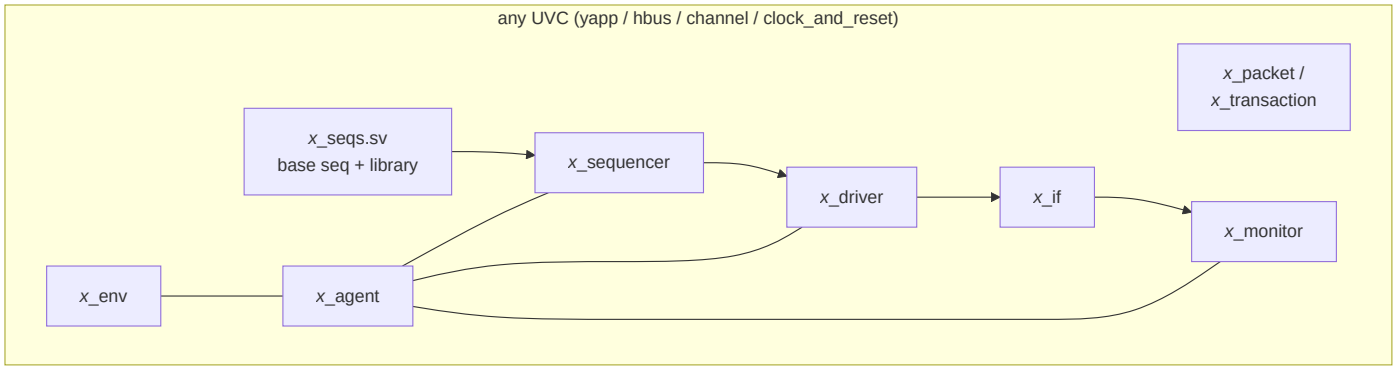
Guide	Reference implementation	Role in one sentence
Packet (sequence item)	`yapp_project/uvc/yapp/yapp_packet.sv`	the data: fields, constraints, parity
Driver	`yapp_project/uvc/yapp/yapp_tx_driver.sv` + `yapp_if.sv`	transaction → pins
Sequencer and sequences	`yapp_project/uvc/yapp/yapp_tx_seqs.sv`	which transactions, in which order
Monitor	`yapp_project/uvc/yapp/yapp_tx_monitor.sv`	pins → transaction, publish
Agent and env	`yapp_project/uvc/yapp/yapp_tx_agent.sv`, `yapp_env.sv`, `labs/*/tb/router_tb.sv`	structure, configuration, connections
Scoreboard	`yapp_project/uvc/router/router_scoreboard.sv`	did the right packet come out of the right channel?
Reference model and	`yapp_project/uvc/router/router_reference.sv`, `router_module_env.sv`	which packets *should* come out

Guide	Reference implementation	Role in one sentence
module UVC		
Virtual sequencer	`yapp_project/tb/router_mcsequencer.sv`, `mcseqs/router_simple_mcseq.sv`	coordinate several interfaces
Functional coverage	`yapp_project/uvc/yapp/yapp_pkt_coverage.sv` and the six other `*_coverage.sv` collectors	did we test everything we meant to?
Random values	`common/rand_util_pkg.sv`	every random value outside a randomized object: `rnd::get_int(...)`
Random tests	`yapp_project/tb/tests/random_*_test.sv`, `mcseqs/router_random_mcseq.sv`, `router_reference.sv`	random bursts and configurations, every HBUS read checked against the full reference model
Register model (RAL)	`yapp_project/tb/yapp_router_reg_pkg.sv` + `reg/`, `yapp_project/uvc/hbus/hbus_reg_adapter.sv`	registers as objects: front door, backdoor, prediction

The reference implementations live in `yapp_project/` (the finished project); the lab pages point at the snapshot where each one first appears.

How the code is written. Every class has its own file. The class body holds the properties (every one prefixed `m_`: `m_addr`, `m_monitor`, `m_vif`), the `utils` macro, the `extern constraint` and `extern` method prototypes; after `endclass` come the constraint bodies (`constraint yapp_packet::c_length {`, one statement per line) and then the method bodies (`function yapp_tx_agent::build_phase(...)`), each under a `//-----` delimiter line. The typedefs and enums of a package live in its `<pkg>_defines.sv`, every enum starts with `<NAME>_UNDEFINED` and its values carry the type name (`PARITY_TYPE_GOOD`), and packed widths are written `bit[7:0]`. There are no `uvm_do*` macros — a sequence creates its item, calls `start_item`, randomizes, calls `finish_item`, and starts a sub-sequence with `seq.start(sequencer, this)` — and no `uvm_field_*` automation: transactions implement `do_print`, `do_copy`, `do_compare`, `do_pack`, `do_unpack` and `do_record`; components implement `do_print` and read their knobs with `uvm_config_int::get` in `build_phase`. The guides below explain the code as it is written here.

All four UVCs in this repository have the same shape, so once the YAPP UVC makes sense the others read like variations:

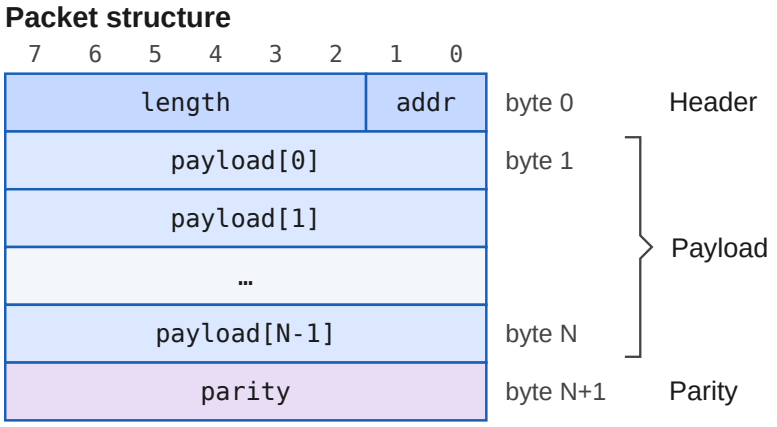


Packet — the sequence item

Open on the project map →

A sequence item is the **unit of stimulus**: one object that describes one transaction. For YAPP that is one packet; for HBUS one read or write; for the channel one "receive a packet with this response delay".

The packet on the wire



1 ≤ N ≤ 63 payload bytes · header = {length[5:0], addr[1:0]} · addr 3 is illegal
parity = even bitwise parity (XOR) over the header and all payload bytes · packet = N + 2 bytes

Byte	Bits	Content
0 (header)	`[7:2]`	`m_length` — number of payload bytes, 1 … 63
	`[1:0]`	`m_addr` — output channel 0, 1 or 2; **3 is illegal**
1 … length	`[7:0]`	`payload[0]` … `payload[length-1]`
length + 1	`[7:0]`	`m_parity` — even bitwise parity: the XOR of the header and every payload byte

A packet is `length + 2` bytes long (3 to 65). `yapp_packet` holds exactly these fields plus two knobs that never reach the wire: `m_parity_type` (GOOD or BAD, what `set_parity()` writes) and `m_packet_delay`. The same figure lives next to the code, in `yapp_project/uvc/yapp/README.md`.

Try it: build or check a packet

Set the fields and watch the byte stream, the computed parity and the checks (legal address, length vs. payload size, parity, `maxpktsize`); flip **PARITY_TYPE_BAD** to see what the error injection of the course does. **Parse bytes** decodes a packet copied from a log. **Send to router** pushes it through the same router model as the register simulator on the DUT page.

Build a packet | Parse bytes

FIELDS (YAPP_PACKET: M_ADDR, M_LENGTH, M_PAYLOAD[], M_PARITY, M_PARITY_TYPE)

addr (2 bits): 1

length (6 bits): 4

Random payload | Counting payload | Clear

payload bytes (hex)

```
cc 5f 24 85
```

m_parity_type

☐ PARITY_TYPE_BAD: flip bit 0

set_parity(): GOOD = XOR of header + payload; BAD = GOOD with one bit flipped

ON THE WIRE

11	cc	5f	24	85	23
header len=4 addr=1	payload[0]	payload[1]	payload[2]	payload[3]	parity

header 0x11 = 00010001 -> length[7:2] = 4, addr[1:0] = 1

parity 0x23 (expected 0x23 = 11 ^ cc ^ 5f ^ 24 ^ 85)

total 6 bytes

CHECKS: A VALID PACKET

- ✔ address 1 → channel 1
- ✔ length 4 (payload bytes)
  6 bits, 1 .. 63
- ✔ 6 bytes = length + 2 (header + payload + parity)
- ✔ parity 0x23 is correct
  even bitwise parity: XOR of the header and every payload byte
- ✔ length 4 <= maxpktsize 63

Router state used by the checks: maxpktsize = 63, router_en = 1

SEND IT TO THE ROUTER

Send to router | Reset router

the same byte stream through yapp_input_fsm: FIFO push or drop, counters, yapp_pkt_mem, mem_size_reg, error

router registers

What a sequence item must have

Element	`yapp_packet`	Why
base class `uvm_sequence_item`	✔	sequencers and drivers are parameterized with it
`` `uvm_object_utils ``	✔	factory registration (`type_id::create` , overrides)
`do_print` , `do_copy` , `do_compare` , `do_pack` / `do_unpack` , `do_record`	one line per field: `m_addr` , `m_length` , `m_payload` , `m_parity` , `m_parity_type` , `m_packet_delay`	what `print()` , `copy()` , `clone()` , `compare()` , `pack()` and transaction recording do with the fields — written by hand, no `uvm_field_*` automation
constructor `new(string name = "...")`	✔	objects have a name, no parent
`rand` fields + constraints	legal address, length/payload size, parity distribution, delay	random stimulus with the rules of the protocol

Element	`yapp_packet`	Why
derived values	`m_parity` via `set_parity()` in `post_randomize()`	computed from the random fields
control knobs	`m_parity_type`, `m_packet_delay`	not on the wire; steer the driver / the error injection

The reference implementation

```
//------------------------------------------------------------------------------
// yapp_packet.sv -- the YAPP packet as a UVM sequence item (Lab 1)
//
//          7   6   5   4   3   2   1   0
//        +-----------------------+-------+
// byte 0 |      length[5:0]      | addr  |  header
//        +-----------------------+-------+
// byte 1 |          payload[0]           |  \
//        |             ...               |   > N == length, 1..63 bytes
// byte N |         payload[N-1]          |  /
//        +-------------------------------+
// byte N+1 |          parity             |  even bitwise parity (XOR) of header and payload
//        +-------------------------------+
//   figure: docs/assets/packet_structure.svg (see README.md in this directory)
//
// Control knobs (not part of the packet on the wire):
//   parity_type  -- PARITY_TYPE_GOOD / PARITY_TYPE_BAD, decides what set_parity() writes
//   packet_delay -- idle clock cycles the driver inserts before the packet; on
//                   a packet the monitor collected, the gap it measured (-1
//                   for the first packet after reset)
//------------------------------------------------------------------------------

class yapp_packet extends uvm_sequence_item;

  // Packet fields
  rand bit[1:0] m_addr;
  rand bit[5:0] m_length;
  rand bit[7:0] m_payload[];
       bit[7:0] m_parity;          // computed, never randomized directly

  // Control knobs
  rand parity_type_e m_parity_type;
  rand int           m_packet_delay;

  `uvm_object_utils(yapp_packet)

  // Constraints (named so sequences can switch them off)
  extern constraint c_addr_legal;
  extern constraint c_length;
  extern constraint c_parity_defined;
  extern constraint c_parity_dist;
  extern constraint c_delay;

  // Printing, copying, comparing, packing and recording are written by hand
  // (do_print / do_copy / do_compare / do_pack / do_unpack / do_record below):
  // no uvm_field_* automation.
  extern virtual function void do_copy(uvm_object rhs);
  extern virtual function bit  do_compare(uvm_object rhs, uvm_comparer comparer);
  extern virtual function void do_print(uvm_printer printer);
  extern virtual function void do_pack(uvm_packer packer);
  extern virtual function void do_unpack(uvm_packer packer);
  extern virtual function void do_record(uvm_recorder recorder);

  extern function new(string name = "yapp_packet");

  // Correct parity: XOR of header and every payload byte
  extern function bit[7:0] calc_parity();
```

```
  // "good" / "bad" for the one-line log messages
  extern function string parity_text();

  // Write the parity byte according to the m_parity_type knob.
  // A bad parity flips one random bit of the correct value.
  extern function void set_parity();

  // Called automatically after every successful randomize(): sizes the
  // payload to `m_length` if the solver did not, then sets the parity
  extern function void post_randomize();

endclass : yapp_packet

//------------------------------------------------------------------------------
// yapp_packet -- constraints
//------------------------------------------------------------------------------

constraint yapp_packet::c_addr_legal {
  m_addr != 2'd3;
}

//------------------------------------------------------------------------------
constraint yapp_packet::c_length {
  m_length inside {[1:63]};
  m_payload.size() == m_length;
}

//------------------------------------------------------------------------------
constraint yapp_packet::c_parity_defined {
  m_parity_type != PARITY_TYPE_UNDEFINED;
}

//------------------------------------------------------------------------------
constraint yapp_packet::c_parity_dist {
  m_parity_type dist { PARITY_TYPE_GOOD := 5, PARITY_TYPE_BAD := 1 };
}

//------------------------------------------------------------------------------
constraint yapp_packet::c_delay {
  m_packet_delay inside {[1:20]};
}

//------------------------------------------------------------------------------
// yapp_packet -- method implementations
//------------------------------------------------------------------------------

function yapp_packet::new(string name = "yapp_packet");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
function bit[7:0] yapp_packet::calc_parity();
  calc_parity = {m_length, m_addr};
  foreach (m_payload[i]) begin
    calc_parity ^= m_payload[i];
  end
endfunction : calc_parity

//------------------------------------------------------------------------------
```

```
function string yapp_packet::parity_text();
  if (m_parity_type == PARITY_TYPE_GOOD) begin
    return "good";
  end
  return "bad";
endfunction : parity_text

//------------------------------------------------------------------------------
function void yapp_packet::set_parity();
  int unsigned bad_bit;
  m_parity = calc_parity();
  if (m_parity_type == PARITY_TYPE_BAD) begin
    // draw the bit once and flip it: two draws would pick two different bits
    // and leave the parity correct about half of the time
    bad_bit = rnd::get_index(8, {get_full_name(), ".bad_parity_bit"});
    m_parity[bad_bit] = ~m_parity[bad_bit];
  end
endfunction : set_parity

//------------------------------------------------------------------------------
function void yapp_packet::post_randomize();
  // Some solvers (Verilator 5.052) do not resize payload when randomize() is
  // called with an inline `with {...}`. Size it here: no sequence constrains
  // the payload bytes, so nothing is lost.
  if (m_payload.size() != m_length) begin
    m_payload = rnd::get_bytes(m_length, {get_full_name(), ".payload"});
  end
  set_parity();
endfunction : post_randomize

//------------------------------------------------------------------------------
function void yapp_packet::do_copy(uvm_object rhs);
  yapp_packet rhs_;
  if (!$cast(rhs_, rhs)) begin
    `uvm_fatal(get_type_name(), "do_copy: rhs is not a yapp_packet")
  end
  super.do_copy(rhs);
  m_addr         = rhs_.m_addr;
  m_length       = rhs_.m_length;
  m_payload      = rhs_.m_payload;
  m_parity       = rhs_.m_parity;
  m_parity_type  = rhs_.m_parity_type;
  m_packet_delay = rhs_.m_packet_delay;
endfunction : do_copy

//------------------------------------------------------------------------------
function bit yapp_packet::do_compare(uvm_object rhs, uvm_comparer comparer);
  yapp_packet rhs_;
  if (!$cast(rhs_, rhs)) begin
    return 0;
  end
  do_compare = super.do_compare(rhs, comparer);
  do_compare &= comparer.compare_field_int("addr", m_addr, rhs_.m_addr, $bits(m_addr), UVM_HEX);
  do_compare &= comparer.compare_field_int("length", m_length, rhs_.m_length, $bits(m_length),
UVM_DEC);
  do_compare &= comparer.compare_field_int("payload.size()", m_payload.size(),
rhs_.m_payload.size(), 32, UVM_DEC);
  foreach (m_payload[i]) begin
    if (i < rhs_.m_payload.size()) begin
```

```
      do_compare &= comparer.compare_field_int($sformatf("payload[%0d]", i), m_payload[i],
rhs_.m_payload[i], $bits(m_payload[i]), UVM_HEX);
    end
  end
  do_compare &= comparer.compare_field_int("parity", m_parity, rhs_.m_parity, $bits(m_parity),
UVM_HEX);
  do_compare &= comparer.compare_field_int("parity_type", m_parity_type, rhs_.m_parity_type,
$bits(m_parity_type));
endfunction : do_compare

//------------------------------------------------------------------------------
function void yapp_packet::do_print(uvm_printer printer);
  super.do_print(printer);
  printer.print_field("addr", m_addr, $bits(m_addr), UVM_HEX);
  printer.print_field("length", m_length, $bits(m_length), UVM_DEC);
  printer.print_array_header("payload", m_payload.size());
  foreach (m_payload[i]) begin
    printer.print_field($sformatf("[%0d]", i), m_payload[i], $bits(m_payload[i]), UVM_HEX, "[");
  end
  printer.print_array_footer(m_payload.size());
  printer.print_field("parity", m_parity, $bits(m_parity), UVM_HEX);
  printer.print_generic("parity_type", "parity_type_e", $bits(m_parity_type),
m_parity_type.name());
  printer.print_field("packet_delay", m_packet_delay, $bits(m_packet_delay), UVM_DEC);
endfunction : do_print

//------------------------------------------------------------------------------
function void yapp_packet::do_pack(uvm_packer packer);
  super.do_pack(packer);
  packer.pack_field_int(m_addr, $bits(m_addr));
  packer.pack_field_int(m_length, $bits(m_length));
  packer.pack_field_int(m_payload.size(), 32);
  foreach (m_payload[i]) begin
    packer.pack_field_int(m_payload[i], $bits(m_payload[i]));
  end
  packer.pack_field_int(m_parity, $bits(m_parity));
  packer.pack_field_int(m_parity_type, $bits(m_parity_type));
  packer.pack_field_int(m_packet_delay, $bits(m_packet_delay));
endfunction : do_pack

//------------------------------------------------------------------------------
function void yapp_packet::do_unpack(uvm_packer packer);
  super.do_unpack(packer);
  m_addr = packer.unpack_field_int($bits(m_addr));
  m_length = packer.unpack_field_int($bits(m_length));
  m_payload = new[packer.unpack_field_int(32)];
  foreach (m_payload[i]) begin
    m_payload[i] = packer.unpack_field_int($bits(m_payload[i]));
  end
  m_parity = packer.unpack_field_int($bits(m_parity));
  m_parity_type = parity_type_e'(packer.unpack_field_int($bits(m_parity_type)));
  m_packet_delay = packer.unpack_field_int($bits(m_packet_delay));
endfunction : do_unpack

//------------------------------------------------------------------------------
function void yapp_packet::do_record(uvm_recorder recorder);
  super.do_record(recorder);
  recorder.record_field("addr", m_addr, $bits(m_addr), UVM_HEX);
  recorder.record_field("length", m_length, $bits(m_length), UVM_DEC);
```

```
  foreach (m_payload[i]) begin
    recorder.record_field($sformatf("payload[%0d]", i), m_payload[i], $bits(m_payload[i]),
UVM_HEX);
  end
  recorder.record_field("parity", m_parity, $bits(m_parity), UVM_HEX);
  recorder.record_string("parity_type", m_parity_type.name());
  recorder.record_field("packet_delay", m_packet_delay, $bits(m_packet_delay), UVM_DEC);
endfunction : do_record
```

Design notes

- **Named constraints** (`c_addr_legal`, ...) can be switched off: `req.c_addr_legal.constraint_mode(0)` is how `yapp_88_packets_seq` sends to the illegal address 3.
- `m_parity` is **not** `rand`: it is a function of the other fields. It is recomputed in `post_randomize()`, and `set_parity()` is public so a sequence that edits the payload afterwards (`yapp_incr_payload_seq`) can call it. A bad parity flips one bit whose index comes from the common random library (`rnd::get_index(8, ...)`, see Random values).
- `do_compare` leaves `m_packet_delay` out (the course marks the field `UVM_NOCOMPARE`): two packets with the same contents are equal even if they were sent with different gaps. `do_copy` and `do_print` do include it.
- The `do_*` methods are the one place where every field is listed. Add a field and forget one of them, and `print()` still works but `compare()` is silently blind to it — the kind of bug the field macros hide and these methods make visible.
- `short_yapp_packet` adds a constraint, nothing else. The factory override in the tests turns every packet into a short one.

Try it (Lab 1)

```
pkt.print();                          // table of all fields
$cast(clone_pkt, pkt.clone());        // new object with the same contents
copy_pkt.copy(pkt);                   // into an existing object
pkt.compare(clone_pkt);               // 1 if equal (reports the first difference)
pkt.print(uvm_default_tree_printer);  // same data, other layout
```

Common mistakes

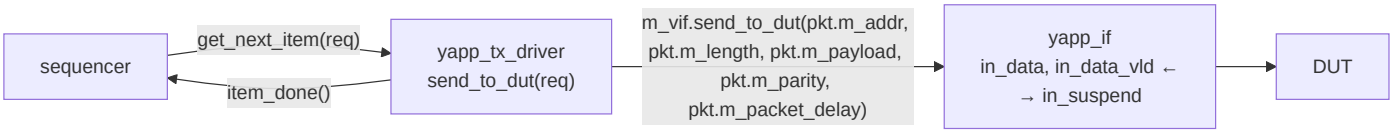
- Forgetting `m_payload.size() == m_length` in the constraint → random array size.
- Making `m_parity` `rand` → the parity is wrong *unless* you add a constraint, and then you cannot inject errors.
- Declaring `parity_type_e` inside the class → sequences cannot name `PARITY_TYPE_GOOD` / `PARITY_TYPE_BAD` (the inline `randomize() with { … }` constraints reference them).
- `$cast` missing in `do_copy` / `do_compare`: `rhs` arrives as a `uvm_object`; without the cast you cannot reach the fields.

- Using `new()` instead of `type_id::create()` in sequences → the factory override of Lab 4 never applies.

Driver

Open on the project map →

The driver pulls transactions from the sequencer and turns them into **pin activity** through a virtual interface. It is the only component that *drives* DUT inputs.



The loop every driver has

```
task run_phase(uvm_phase phase);
  reset_signals();                      // idle values while reset is active
  forever begin
    seq_item_port.get_next_item(req);   // blocks until a sequence has an item
    send_to_dut(req);                   // takes time
    seq_item_port.item_done();          // tells the sequence it can continue
  end
endtask
```

Three details that matter:

1. `req` is declared by `uvm_driver #(yapp_packet)` — you do not declare it.
2. `get_next_item` / `item_done` come in pairs. Forgetting `item_done()` hangs the sequence forever.
3. The driver **never raises an objection**: it is a slave of the sequences.

Where the protocol lives: the interface

The course keeps the timing in `yapp_if` tasks and has the driver call them. That separates *what* (a packet) from *how* (edges and handshake), and the same task is reusable from a non-UVM testbench.

```
//------------------------------------------------------------------------------
// yapp_if.sv -- YAPP input-port interface
//
// Holds the three DUT input-port signals and the protocol knowledge:
//   * send_to_dut()    -- used by the driver to transmit one packet
//   * collect_packets()-- used by the monitor to observe one packet
//   * yapp_reset()     -- used by the driver while reset is active
//
// Protocol recap (see yapp_router.sv for the DUT view):
//   * The driver changes in_data / in_data_vld on the FALLING edge.
//   * The DUT accepts the byte on the next RISING edge unless it is asserting
//     in_suspend (target FIFO full). While in_suspend is high the byte must be
//     held unchanged.
//   * in_data_vld is high for the header and payload bytes and low for the
//     parity byte, which is driven on the falling edge right after the last
//     payload byte.
//------------------------------------------------------------------------------
`timescale 1ns/1ns

interface yapp_if (input logic clock, input logic reset);

  logic [7:0] in_data;
  logic       in_data_vld;
  logic       in_suspend;

  // When the last packet ended (its parity byte accepted): a packet that the
  // driver gets in that same time step may follow with no idle cycle at all
  time last_end;
  bit  last_end_valid;

  //------------------------------------------------------------------------
  // Driver side
  //------------------------------------------------------------------------
  task automatic yapp_reset();
    @(negedge clock);
    in_data     <= 8'h00;
    in_data_vld <= 1'b0;
  endtask : yapp_reset

  // Hold the current byte until the DUT has accepted it: a byte is taken at a
  // rising edge where in_suspend is low, so the decision is sampled on the
  // rising edge (exactly as the DUT and the monitor do). The next byte is then
  // driven on the following falling edge. Sampling in_suspend on the falling
  // edge instead would lose a header whose target FIFO was full at the rising
  // edge and freed up right after it (in_suspend falls before the falling edge).
  task automatic wait_accept();
    do @(posedge clock); while (in_suspend);
    @(negedge clock);
  endtask : wait_accept

  task automatic send_to_dut(input bit [1:0] addr,
                             input bit [5:0] length,
                             input bit [7:0] payload[],
                             input bit [7:0] parity,
                             input int       packet_delay);
    // Inter-packet gap: packet_delay idle cycles (rising edges without
    // in_data_vld). Right after a packet the driver already stands on the
    // falling edge where the next byte may go out, so packet_delay == 0 sends
```

```
    // the header in the very next cycle (the DUT accepts it there).
    if (!(last_end_valid && $time == last_end)) begin
      @(negedge clock);
    end
    repeat (packet_delay) begin
      @(negedge clock);
    end

    // Header: {length, addr} together with in_data_vld
    in_data_vld <= 1'b1;
    in_data     <= {length, addr};

    // Payload, one byte per accepted cycle
    foreach (payload[i]) begin
      wait_accept();
      in_data <= payload[i];
    end

    // Parity with in_data_vld low
    wait_accept();
    in_data_vld <= 1'b0;
    in_data     <= parity;

    // Make sure the parity byte was accepted before returning to idle
    wait_accept();
    in_data <= 8'h00;
    last_end       = $time;
    last_end_valid = 1'b1;
  endtask : send_to_dut

  //-------------------------------------------------------------------------
  // Monitor side: sample on the RISING edge. A byte is accepted by the DUT
  // when it is not suspending the input at that edge.
  //-------------------------------------------------------------------------
  task automatic collect_packets(output bit [1:0] addr,
                                 output bit [5:0] length,
                                 output bit [7:0] payload[],
                                 output bit [7:0] parity,
                                 output int       idle_cycles);
    // Wait for an accepted header, counting the idle cycles (rising edges
    // without in_data_vld) since the call: right after a packet this is the
    // gap between the two packets
    idle_cycles = 0;
    do begin
      @(posedge clock);
      if (!in_data_vld) begin
        idle_cycles++;
      end
    end while (!(in_data_vld && !in_suspend));
    length = in_data[7:2];
    addr   = in_data[1:0];
    payload = new[length];

    // Payload bytes
    for (int i = 0; i < length; i++) begin
      do @(posedge clock); while (!(in_data_vld && !in_suspend));
      payload[i] = in_data;
    end
```

```
    // Parity byte: first accepted cycle after the payload
    do @(posedge clock); while (in_suspend);
    parity = in_data;
  endtask : collect_packets

endinterface : yapp_if
```

`wait_accept()` is the heart of the handshake: a byte is taken at a **rising** edge where `in_suspend` is low, so the task waits for such a rising edge and only then, on the next falling edge, lets the driver change the data. Sampling `in_suspend` on the falling edge would be wrong: a header offered to a full FIFO is refused at the rising edge, and if the receiver frees a slot at that same edge `in_suspend` is already low again at the falling edge, so the driver would move on and the header would be lost. The `backpressure_test` of the test plan exercises exactly this path.

The gap between packets. `m_packet_delay` is the number of idle cycles (rising edges without `in_data_vld`) before the header. When the driver gets the next packet in the same time step in which the previous one ended (`last_end`), it already stands on the falling edge where a byte may go out, so `m_packet_delay == 0` puts the header in the cycle right after the parity byte -- the shortest gap the DUT accepts. The random packets keep `c_delay` (1..20); `yapp_gap_seq` switches it off to send gaps of 0..3, and `back_to_back_test` checks them against the monitor's measurement (ROUTE-04).

The reference implementation

```
//------------------------------------------------------------------------------
// yapp_tx_driver.sv -- pulls packets from the sequencer and drives the
// YAPP input port through the virtual interface (Lab 3 skeleton, Lab 6 protocol)
//------------------------------------------------------------------------------
class yapp_tx_driver extends uvm_driver #(yapp_packet);

  // Virtual interface, set by tb_top through yapp_vif_config
  virtual interface yapp_if m_vif;

  // Statistics
  int m_num_sent;

  `uvm_component_utils(yapp_tx_driver)

  // The fields are printed by do_print() below:
  // no uvm_field_* automation.
  extern virtual function void do_print(uvm_printer printer);
  extern function new(string name, uvm_component parent);

  extern function void connect_phase(uvm_phase phase);
  extern task run_phase(uvm_phase phase);

  // Drive idle values and stay quiet while reset is active
  extern task reset_signals();

  // The classic driver loop
  extern task get_and_drive();
  extern task send_to_dut(yapp_packet pkt);

  extern function void report_phase(uvm_phase phase);

endclass : yapp_tx_driver

//------------------------------------------------------------------------------
// yapp_tx_driver -- method implementations
//------------------------------------------------------------------------------

function yapp_tx_driver::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//------------------------------------------------------------------------------
function void yapp_tx_driver::connect_phase(uvm_phase phase);
  if (!yapp_vif_config::get(this, "", "vif", m_vif)) begin
    `uvm_error("NOVIF", {"virtual interface must be set for: ", get_full_name(), ".vif"})
  end
endfunction : connect_phase

//------------------------------------------------------------------------------
task yapp_tx_driver::run_phase(uvm_phase phase);
  reset_signals();
  get_and_drive();
endtask : run_phase

//------------------------------------------------------------------------------
task yapp_tx_driver::reset_signals();
  m_vif.yapp_reset();
  wait (m_vif.reset === 1'b0);
```

```
endtask : reset_signals

//------------------------------------------------------------------------------
task yapp_tx_driver::get_and_drive();
  forever begin
    seq_item_port.get_next_item(req);     // blocks until a sequence sends one
    send_to_dut(req);
    seq_item_port.item_done();            // unblocks the sequence
  end
endtask : get_and_drive

//------------------------------------------------------------------------------
task yapp_tx_driver::send_to_dut(yapp_packet pkt);
  void'(begin_tr(pkt, "Driver_YAPP_Packet"));   // transaction recording
  // One line per packet (UVM_MEDIUM); the full table only at UVM_HIGH
  `uvm_info(get_type_name(),
            $sformatf("TX pkt #%0d: addr=%0d len=%0d parity=%s delay=%0d", m_num_sent + 1,
pkt.m_addr, pkt.m_length,
                      pkt.parity_text(), pkt.m_packet_delay), UVM_MEDIUM)
  `uvm_info(get_type_name(), $sformatf("Packet is \n%s", pkt.sprint()), UVM_HIGH)
  m_vif.send_to_dut(pkt.m_addr, pkt.m_length, pkt.m_payload, pkt.m_parity, pkt.m_packet_delay);
  m_num_sent++;
  `uvm_info(get_type_name(), $sformatf("TX pkt #%0d accepted by the DUT", m_num_sent),
UVM_MEDIUM)
  end_tr(pkt);
endtask : send_to_dut

//------------------------------------------------------------------------------
function void yapp_tx_driver::report_phase(uvm_phase phase);
  `uvm_info(get_type_name(), $sformatf("YAPP driver report: %0d packets sent", m_num_sent),
UVM_LOW)
endfunction : report_phase

//------------------------------------------------------------------------------
function void yapp_tx_driver::do_print(uvm_printer printer);
  super.do_print(printer);
  printer.print_field("num_sent", m_num_sent, $bits(m_num_sent), UVM_DEC);
endfunction : do_print
```

Reset handling

`reset_signals()` drives idle values on the first falling edge and then waits for `reset` to be low. With the Clock & Reset UVC reset is asserted at time 0 by `clk10_rst5_seq`, so `wait (m_vif.reset === 1'b0)` holds the driver for the first five cycles. The same pattern is used by every driver and monitor in the repository (`@(posedge m_vif.clock); wait (m_vif.reset === 1'b0);` in monitors).

Transaction recording

`begin_tr(req, "Driver_YAPP_Packet")` / `end_tr(req)` give the packet a named stream in the waveform viewer (`recording_detail` is enabled by `base_test`). In SimVision, add `Driver_YAPP_Packet` and `Monitor_YAPP_Packet` from the driver / monitor instances to see packets as bars with their fields.

Other drivers in the project

Driver	Transaction → pins
`hbus_master_driver`	`hbus_write(addr, data)` or `hbus_read(addr, data)`; the read value is written back into `req.m_hdata` so the sequence (and the register adapter) can use it
`channel_rx_driver`	plays the **receiver**: `receive_packet(m_delay)` waits for `data_vld`, drops `suspend`, reads `length + 2` bytes
`clock_and_reset_driver`	`start_clock_and_reset(period, cycles)`

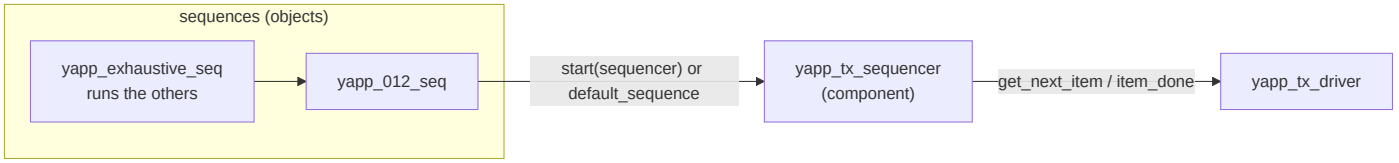
Common mistakes

- Driving on the rising edge: the DUT samples there — race. Drive on the falling edge (all `yapp_if` tasks do).
- Changing `in_data` while `in_suspend` is high.
- Not waiting for the parity byte to be accepted before returning → the next packet's header overwrites it when the FIFO is full.
- Fetching the virtual interface in `build_phase` of the driver: the test's `set()` happens in `tb_top` *before* `run_test()`, so `build_phase` would work too, but `connect_phase` is the convention (all handles exist by then).

Sequencer and sequences

Open on the project map →

The **sequencer** is the traffic controller of an agent: sequences run *on* it, it arbitrates between them and hands one item at a time to the driver. The **sequences** are where the stimulus is actually described.



The sequencer

Nothing to write beyond the boilerplate — the base class does all the work:

```
//------------------------------------------------------------------------------
// yapp_tx_sequencer.sv -- arbitrates yapp_packet items towards the driver (Lab 3)
//
// Nothing to add: uvm_sequencer #(T) already contains the whole machinery.
//------------------------------------------------------------------------------
class yapp_tx_sequencer extends uvm_sequencer #(yapp_packet);

  `uvm_component_utils(yapp_tx_sequencer)

  extern function new(string name, uvm_component parent);

endclass : yapp_tx_sequencer

//------------------------------------------------------------------------------
// yapp_tx_sequencer -- method implementations
//------------------------------------------------------------------------------

function yapp_tx_sequencer::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new
```

Anatomy of a sequence

```
class yapp_1_seq extends yapp_base_seq;        // 1. base class with objections
  `uvm_object_utils(yapp_1_seq)                // 2. factory registration (object, not
component!)
  extern function new(string name = "yapp_1_seq");   // 3. object constructor: name only
  extern task body();                                 // 4. the stimulus
endclass : yapp_1_seq

function yapp_1_seq::new(string name = "yapp_1_seq");
  super.new(name);
endfunction : new

task yapp_1_seq::body();
  `uvm_info(get_type_name(), "Executing yapp_1_seq sequence", UVM_LOW)
  req = yapp_packet::type_id::create("req");           // create through the factory
  start_item(req);                                     // wait for the sequencer's grant
  if (!req.randomize() with { req.m_addr == 2'd1; })     // randomize, with constraints
    `uvm_error(get_type_name(), "req.randomize() failed")
  finish_item(req);                                    // to the driver; returns after item_done
endtask : body
```

`req` is declared by `uvm_sequence #(yapp_packet)`. Every item goes through the same four steps; the course material hides them behind a macro, this repository writes them out so you see what the macro does:

Step	Here	Course macro
create through the factory	`req = yapp_packet::type_id::create("req")`	`` `uvm_create( ``
wait for the sequencer's grant	`start_item(req)`	
randomize (with constraints), check the result	`if (!req.randomize() with { ... }) `uvm_error(...)`	
send to the driver, wait for `item_done`	`finish_item(req)`	`` `uvm_send(re ``

Step	Here	Course macro
all four	the four lines above	`` `uvm_do(req) `` `` `uvm_do_with( ``

Between `start_item` and `finish_item` the sequence holds the sequencer, so keep that window short: no delays, no waiting for the DUT. `yapp_incr_payload_seq` moves the randomization *before* `start_item`, because it needs to edit the payload and recompute the parity before the driver sees the packet.

A **nested sequence** is created the same way and started on the current sequencer, with the parent as the second argument so the sequencer sees the relationship (`` `uvm_do(seq_1) `` in the course):

```
m_seq_1 = yapp_1_seq::type_id::create("seq_1");
if (!m_seq_1.randomize())
  `uvm_error(get_type_name(), "seq_1.randomize() failed")
m_seq_1.start(m_sequencer, this);       // blocks until seq_1.body() returns
```

The library

```
//------------------------------------------------------------------------------
// yapp_tx_seqs.sv -- YAPP sequence library (Lab 3 base, Lab 5 library, Lab 7 opt.)
//
// Every sequence:
//   * extends yapp_base_seq (objection handling),
//   * has an object utils macro and a constructor,
//   * prints its name with `uvm_info at UVM_LOW at the start of body().
//
// One class per file: the classes live in seqs/<class>.sv and are included below.
//------------------------------------------------------------------------------

`include "seqs/yapp_base_seq.sv"
`include "seqs/yapp_5_packets.sv"

//------------------------------------------------------------------------------
// Lab 5 sequences
//------------------------------------------------------------------------------

`include "seqs/yapp_1_seq.sv"
`include "seqs/yapp_012_seq.sv"
`include "seqs/yapp_111_seq.sv"
`include "seqs/yapp_repeat_addr_seq.sv"
`include "seqs/yapp_incr_payload_seq.sv"
`include "seqs/yapp_rnd_seq.sv"
`include "seqs/six_yapp_seq.sv"
`include "seqs/yapp_exhaustive_seq.sv"
`include "seqs/yapp_coverage_seq.sv"
`include "seqs/yapp_88_packets_seq.sv"

//------------------------------------------------------------------------------
// Test-plan sequences (used by the tests of yapp_project/tb/tests)
//------------------------------------------------------------------------------

`include "seqs/yapp_pkt_seq.sv"
`include "seqs/yapp_boundary_seq.sv"
`include "seqs/yapp_gap_seq.sv"
`include "seqs/yapp_random_burst_seq.sv"

//------------------------------------------------------------------------------
// yapp_base_seq.sv -- YAPP sequence library (Lab 3 base, Lab 5 library, Lab 7 opt.)
// Split out of yapp_tx_seqs.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// yapp_base_seq: raises an objection while the sequence runs, so run_phase
// does not end before the last packet was delivered.
//------------------------------------------------------------------------------
class yapp_base_seq extends uvm_sequence #(yapp_packet);

  `uvm_object_utils(yapp_base_seq)

  extern function new(string name = "yapp_base_seq");
  extern task pre_body();

  extern task post_body();

endclass : yapp_base_seq
```

```
//------------------------------------------------------------------------------
// yapp_base_seq -- method implementations
//------------------------------------------------------------------------------

function yapp_base_seq::new(string name = "yapp_base_seq");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task yapp_base_seq::pre_body();
  uvm_phase phase = `YAPP_STARTING_PHASE;
  if (phase != null) begin
    phase.raise_objection(this, get_type_name());
  end
endtask : pre_body

//------------------------------------------------------------------------------
task yapp_base_seq::post_body();
  uvm_phase phase = `YAPP_STARTING_PHASE;
  if (phase != null) begin
    phase.drop_objection(this, get_type_name());
  end
endtask : post_body

//------------------------------------------------------------------------------
// yapp_5_packets.sv -- YAPP sequence library (Lab 3 base, Lab 5 library, Lab 7 opt.)
// Split out of yapp_tx_seqs.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// yapp_5_packets: five random packets (provided in Lab 3)
//   Test class configuration template:
//     uvm_config_wrapper::set(this, "tb.yapp.agent.sequencer.run_phase",
//                             "default_sequence", yapp_5_packets::get_type());
//------------------------------------------------------------------------------
class yapp_5_packets extends yapp_base_seq;

  `uvm_object_utils(yapp_5_packets)

  extern function new(string name = "yapp_5_packets");
  extern task body();

endclass : yapp_5_packets

//------------------------------------------------------------------------------
// yapp_5_packets -- method implementations
//------------------------------------------------------------------------------

function yapp_5_packets::new(string name = "yapp_5_packets");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task yapp_5_packets::body();
  `uvm_info(get_type_name(), "Executing yapp_5_packets sequence", UVM_LOW)
  repeat (5) begin
    req = yapp_packet::type_id::create("req");
    start_item(req);
    if (!req.randomize()) begin
```

```
      `uvm_error(get_type_name(), "req.randomize() failed")
    end
    finish_item(req);
  end
endtask : body

//------------------------------------------------------------------------------
// yapp_1_seq.sv -- YAPP sequence library (Lab 3 base, Lab 5 library, Lab 7 opt.)
// Split out of yapp_tx_seqs.sv: one class per file.
//------------------------------------------------------------------------------

// yapp_1_seq: one packet to address 1
class yapp_1_seq extends yapp_base_seq;

  `uvm_object_utils(yapp_1_seq)

  extern function new(string name = "yapp_1_seq");
  extern task body();

endclass : yapp_1_seq

//------------------------------------------------------------------------------
// yapp_1_seq -- method implementations
//------------------------------------------------------------------------------

function yapp_1_seq::new(string name = "yapp_1_seq");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task yapp_1_seq::body();
  `uvm_info(get_type_name(), "Executing yapp_1_seq sequence", UVM_LOW)
  req = yapp_packet::type_id::create("req");
  start_item(req);
  if (!req.randomize() with { req.m_addr == 2'd1; }) begin
    `uvm_error(get_type_name(), "req.randomize() failed")
  end
  finish_item(req);
endtask : body

//------------------------------------------------------------------------------
// yapp_012_seq.sv -- YAPP sequence library (Lab 3 base, Lab 5 library, Lab 7 opt.)
// Split out of yapp_tx_seqs.sv: one class per file.
//------------------------------------------------------------------------------

// yapp_012_seq: three packets, to address 0, then 1, then 2
class yapp_012_seq extends yapp_base_seq;

  `uvm_object_utils(yapp_012_seq)

  extern function new(string name = "yapp_012_seq");
  extern task body();

endclass : yapp_012_seq

//------------------------------------------------------------------------------
// yapp_012_seq -- method implementations
//------------------------------------------------------------------------------
```

```
function yapp_012_seq::new(string name = "yapp_012_seq");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task yapp_012_seq::body();
  `uvm_info(get_type_name(), "Executing yapp_012_seq sequence", UVM_LOW)
  req = yapp_packet::type_id::create("req");
  start_item(req);
  if (!req.randomize() with { req.m_addr == 2'd0; }) begin
    `uvm_error(get_type_name(), "req.randomize() failed")
  end
  finish_item(req);

  req = yapp_packet::type_id::create("req");
  start_item(req);
  if (!req.randomize() with { req.m_addr == 2'd1; }) begin
    `uvm_error(get_type_name(), "req.randomize() failed")
  end
  finish_item(req);

  req = yapp_packet::type_id::create("req");
  start_item(req);
  if (!req.randomize() with { req.m_addr == 2'd2; }) begin
    `uvm_error(get_type_name(), "req.randomize() failed")
  end
  finish_item(req);
endtask : body

//------------------------------------------------------------------------------
// yapp_111_seq.sv -- YAPP sequence library (Lab 3 base, Lab 5 library, Lab 7 opt.)
// Split out of yapp_tx_seqs.sv: one class per file.
//------------------------------------------------------------------------------

// yapp_111_seq: nested sequence -- runs yapp_1_seq three times
class yapp_111_seq extends yapp_base_seq;

  yapp_1_seq m_seq_1;

  `uvm_object_utils(yapp_111_seq)

  extern function new(string name = "yapp_111_seq");
  extern task body();

endclass : yapp_111_seq

//------------------------------------------------------------------------------
// yapp_111_seq -- method implementations
//------------------------------------------------------------------------------

function yapp_111_seq::new(string name = "yapp_111_seq");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task yapp_111_seq::body();
  `uvm_info(get_type_name(), "Executing yapp_111_seq sequence", UVM_LOW)
  repeat (3) begin
    m_seq_1 = yapp_1_seq::type_id::create("seq_1");
```

```
    if (!m_seq_1.randomize()) begin
      `uvm_error(get_type_name(), "seq_1.randomize() failed")
    end
    m_seq_1.start(m_sequencer, this);
  end
endtask : body

//------------------------------------------------------------------------------
// yapp_repeat_addr_seq.sv -- YAPP sequence library (Lab 3 base, Lab 5 library, Lab 7 opt.)
// Split out of yapp_tx_seqs.sv: one class per file.
//------------------------------------------------------------------------------

// yapp_repeat_addr_seq: two packets to the same random (legal) address
class yapp_repeat_addr_seq extends yapp_base_seq;

  // A random SEQUENCE property, randomized when the sequence is randomized
  rand bit[1:0] m_seq_addr;

  `uvm_object_utils(yapp_repeat_addr_seq)

  extern constraint c_seq_addr;

  extern function new(string name = "yapp_repeat_addr_seq");
  extern task body();

endclass : yapp_repeat_addr_seq

//------------------------------------------------------------------------------
// yapp_repeat_addr_seq -- constraints
//------------------------------------------------------------------------------

constraint yapp_repeat_addr_seq::c_seq_addr {
  m_seq_addr != 2'd3;
}

//------------------------------------------------------------------------------
// yapp_repeat_addr_seq -- method implementations
//------------------------------------------------------------------------------

function yapp_repeat_addr_seq::new(string name = "yapp_repeat_addr_seq");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task yapp_repeat_addr_seq::body();
  `uvm_info(get_type_name(),
            $sformatf("Executing yapp_repeat_addr_seq sequence (addr %0d)", m_seq_addr), UVM_LOW)
  req = yapp_packet::type_id::create("req");
  start_item(req);
  if (!req.randomize() with { req.m_addr == m_seq_addr; }) begin
    `uvm_error(get_type_name(), "req.randomize() failed")
  end
  finish_item(req);

  req = yapp_packet::type_id::create("req");
  start_item(req);
  if (!req.randomize() with { req.m_addr == m_seq_addr; }) begin
    `uvm_error(get_type_name(), "req.randomize() failed")
  end
```

```
  finish_item(req);
endtask : body

//------------------------------------------------------------------------------
// yapp_incr_payload_seq.sv -- YAPP sequence library (Lab 3 base, Lab 5 library, Lab 7 opt.)
// Split out of yapp_tx_seqs.sv: one class per file.
//------------------------------------------------------------------------------

// yapp_incr_payload_seq: one packet whose payload is 0, 1, 2, ... length-1
class yapp_incr_payload_seq extends yapp_base_seq;

  `uvm_object_utils(yapp_incr_payload_seq)

  extern function new(string name = "yapp_incr_payload_seq");
  extern task body();

endclass : yapp_incr_payload_seq

//------------------------------------------------------------------------------
// yapp_incr_payload_seq -- method implementations
//------------------------------------------------------------------------------

function yapp_incr_payload_seq::new(string name = "yapp_incr_payload_seq");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task yapp_incr_payload_seq::body();
  `uvm_info(get_type_name(), "Executing yapp_incr_payload_seq sequence", UVM_LOW)
  req = yapp_packet::type_id::create("req");   // build through the factory
  if (!req.randomize()) begin
    `uvm_error(get_type_name(), "Randomization failed")
  end
  foreach (req.m_payload[i]) begin
    req.m_payload[i] = i;
  end
  req.set_parity();                        // payload changed -> recompute parity
  start_item(req);   // hand it to the driver
  finish_item(req);
endtask : body

//------------------------------------------------------------------------------
// yapp_rnd_seq.sv -- YAPP sequence library (Lab 3 base, Lab 5 library, Lab 7 opt.)
// Split out of yapp_tx_seqs.sv: one class per file.
//------------------------------------------------------------------------------

// yapp_rnd_seq (optional): a random number (1..10) of random packets
class yapp_rnd_seq extends yapp_base_seq;

  rand int m_count;

  `uvm_object_utils(yapp_rnd_seq)

  extern constraint c_count;

  extern function new(string name = "yapp_rnd_seq");
  extern task body();

endclass : yapp_rnd_seq
```

```
//------------------------------------------------------------------------------
// yapp_rnd_seq -- constraints
//------------------------------------------------------------------------------

constraint yapp_rnd_seq::c_count {
  m_count inside {[1:10]};
}

//------------------------------------------------------------------------------
// yapp_rnd_seq -- method implementations
//------------------------------------------------------------------------------

function yapp_rnd_seq::new(string name = "yapp_rnd_seq");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task yapp_rnd_seq::body();
  `uvm_info(get_type_name(),
            $sformatf("Executing yapp_rnd_seq sequence (%0d packets)", m_count), UVM_LOW)
  repeat (m_count) begin
    req = yapp_packet::type_id::create("req");
    start_item(req);
    if (!req.randomize()) begin
      `uvm_error(get_type_name(), "req.randomize() failed")
    end
    finish_item(req);
  end
endtask : body

//------------------------------------------------------------------------------
// six_yapp_seq.sv -- YAPP sequence library (Lab 3 base, Lab 5 library, Lab 7 opt.)
// Split out of yapp_tx_seqs.sv: one class per file.
//------------------------------------------------------------------------------

// six_yapp_seq (optional): yapp_rnd_seq constrained to exactly six packets
class six_yapp_seq extends yapp_base_seq;

  yapp_rnd_seq m_rnd_seq;

  `uvm_object_utils(six_yapp_seq)

  extern function new(string name = "six_yapp_seq");
  extern task body();

endclass : six_yapp_seq

//------------------------------------------------------------------------------
// six_yapp_seq -- method implementations
//------------------------------------------------------------------------------

function six_yapp_seq::new(string name = "six_yapp_seq");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task six_yapp_seq::body();
  `uvm_info(get_type_name(), "Executing six_yapp_seq sequence", UVM_LOW)
```

```
  m_rnd_seq = yapp_rnd_seq::type_id::create("rnd_seq");
  if (!m_rnd_seq.randomize() with { m_rnd_seq.m_count == 6; }) begin
    `uvm_error(get_type_name(), "rnd_seq.randomize() failed")
  end
  m_rnd_seq.start(m_sequencer, this);
endtask : body

//------------------------------------------------------------------------------
// yapp_exhaustive_seq.sv -- YAPP sequence library (Lab 3 base, Lab 5 library, Lab 7 opt.)
// Split out of yapp_tx_seqs.sv: one class per file.
//------------------------------------------------------------------------------

// yapp_exhaustive_seq: runs every sequence above once
class yapp_exhaustive_seq extends yapp_base_seq;

  yapp_1_seq            m_seq_1;
  yapp_012_seq          m_seq_012;
  yapp_111_seq          m_seq_111;
  yapp_repeat_addr_seq  m_seq_repeat_addr;
  yapp_incr_payload_seq m_seq_incr_payload;
  yapp_rnd_seq          m_seq_rnd;
  six_yapp_seq          m_seq_six;

  `uvm_object_utils(yapp_exhaustive_seq)

  extern function new(string name = "yapp_exhaustive_seq");
  extern task body();

endclass : yapp_exhaustive_seq

//------------------------------------------------------------------------------
// yapp_exhaustive_seq -- method implementations
//------------------------------------------------------------------------------

function yapp_exhaustive_seq::new(string name = "yapp_exhaustive_seq");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task yapp_exhaustive_seq::body();
  `uvm_info(get_type_name(), "Executing yapp_exhaustive_seq sequence", UVM_LOW)
  m_seq_1 = yapp_1_seq::type_id::create("seq_1");
  if (!m_seq_1.randomize()) begin
    `uvm_error(get_type_name(), "seq_1.randomize() failed")
  end
  m_seq_1.start(m_sequencer, this);

  m_seq_012 = yapp_012_seq::type_id::create("seq_012");
  if (!m_seq_012.randomize()) begin
    `uvm_error(get_type_name(), "seq_012.randomize() failed")
  end
  m_seq_012.start(m_sequencer, this);

  m_seq_111 = yapp_111_seq::type_id::create("seq_111");
  if (!m_seq_111.randomize()) begin
    `uvm_error(get_type_name(), "seq_111.randomize() failed")
  end
  m_seq_111.start(m_sequencer, this);
```

```
  m_seq_repeat_addr = yapp_repeat_addr_seq::type_id::create("seq_repeat_addr");
  if (!m_seq_repeat_addr.randomize()) begin
    `uvm_error(get_type_name(), "seq_repeat_addr.randomize() failed")
  end
  m_seq_repeat_addr.start(m_sequencer, this);

  m_seq_incr_payload = yapp_incr_payload_seq::type_id::create("seq_incr_payload");
  if (!m_seq_incr_payload.randomize()) begin
    `uvm_error(get_type_name(), "seq_incr_payload.randomize() failed")
  end
  m_seq_incr_payload.start(m_sequencer, this);

  m_seq_rnd = yapp_rnd_seq::type_id::create("seq_rnd");
  if (!m_seq_rnd.randomize()) begin
    `uvm_error(get_type_name(), "seq_rnd.randomize() failed")
  end
  m_seq_rnd.start(m_sequencer, this);

  m_seq_six = six_yapp_seq::type_id::create("seq_six");
  if (!m_seq_six.randomize()) begin
    `uvm_error(get_type_name(), "seq_six.randomize() failed")
  end
  m_seq_six.start(m_sequencer, this);
endtask : body

//------------------------------------------------------------------------------
// yapp_coverage_seq.sv -- YAPP sequence library (Lab 3 base, Lab 5 library, Lab 7 opt.)
// Split out of yapp_tx_seqs.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// yapp_coverage_seq (Lab 10): close the coverage model in one run.
// Every length bucket (MIN/SMALL/MEDIUM/LARGE/MAX) to every address 0..3,
// once with good and once with bad parity -> 5 x 4 x 2 = 40 packets.
//------------------------------------------------------------------------------
class yapp_coverage_seq extends yapp_base_seq;

  `uvm_object_utils(yapp_coverage_seq)

  extern function new(string name = "yapp_coverage_seq");
  extern task body();

endclass : yapp_coverage_seq

//------------------------------------------------------------------------------
// yapp_coverage_seq -- method implementations
//------------------------------------------------------------------------------

function yapp_coverage_seq::new(string name = "yapp_coverage_seq");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task yapp_coverage_seq::body();
  int lengths[5] = '{1, 5, 20, 50, 63};   // one value inside each bin
  `uvm_info(get_type_name(), "Executing yapp_coverage_seq sequence", UVM_LOW)
  for (int a = 0; a < 4; a++) begin
    foreach (lengths[i]) begin
      for (int bad = 0; bad < 2; bad++) begin
```

```
        req = yapp_packet::type_id::create("req");
        req.c_addr_legal.constraint_mode(0);
        req.c_parity_dist.constraint_mode(0);   // the sequence chooses the parity
        start_item(req);
        if (!req.randomize() with { req.m_addr == a;
                                    req.m_length == lengths[i];
                                    req.m_parity_type == (bad ? PARITY_TYPE_BAD :
PARITY_TYPE_GOOD); }) begin
          `uvm_error(get_type_name(), "req.randomize() failed")
        end
        finish_item(req);
      end
    end
  end
endtask : body

//------------------------------------------------------------------------------
// yapp_88_packets_seq.sv -- YAPP sequence library (Lab 3 base, Lab 5 library, Lab 7 opt.)
// Split out of yapp_tx_seqs.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// yapp_88_packets_seq (Lab 7 optional integration test):
// every address 0..3 (3 is illegal!) with payload lengths 1..22, 20% bad
// parity -> 4 x 22 = 88 packets. The legal-address constraint of the packet
// has to be switched off to reach address 3.
//------------------------------------------------------------------------------
class yapp_88_packets_seq extends yapp_base_seq;

  `uvm_object_utils(yapp_88_packets_seq)

  extern function new(string name = "yapp_88_packets_seq");
  extern task body();

endclass : yapp_88_packets_seq

//------------------------------------------------------------------------------
// yapp_88_packets_seq -- method implementations
//------------------------------------------------------------------------------

function yapp_88_packets_seq::new(string name = "yapp_88_packets_seq");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task yapp_88_packets_seq::body();
  `uvm_info(get_type_name(), "Executing yapp_88_packets_seq sequence", UVM_LOW)
  for (int a = 0; a < 4; a++) begin
    for (int l = 1; l <= 22; l++) begin
      req = yapp_packet::type_id::create("req");
      req.c_addr_legal.constraint_mode(0);
      req.c_parity_dist.constraint_mode(0);   // the inline dist below replaces it
      start_item(req);
      if (!req.randomize() with { req.m_addr == a;
                                  req.m_length == l;
                                  req.m_parity_type dist { PARITY_TYPE_GOOD := 4, PARITY_TYPE_BAD
:= 1 }; }) begin
        `uvm_error(get_type_name(), "req.randomize() failed")
      end
```

```
      finish_item(req);
    end
  end
endtask : body

//------------------------------------------------------------------------------
// yapp_pkt_seq.sv -- YAPP sequence library (test-plan sequences)
// Split out of yapp_tx_seqs.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// yapp_pkt_seq: one packet whose address, length and parity the test chooses by
// randomizing the sequence with an inline constraint, for example
//   seq.randomize() with { pkt_addr == 3; pkt_len == 20; }
// The soft constraints give a legal, good packet when the test says nothing.
// The packet that went out stays in `req`, so the test can compare the DUT's
// memories and counters against exactly what was sent.
//------------------------------------------------------------------------------
class yapp_pkt_seq extends yapp_base_seq;

  rand bit[1:0]     m_pkt_addr;
  rand bit[5:0]     m_pkt_len;
  rand parity_type_e m_pkt_parity;

  `uvm_object_utils(yapp_pkt_seq)

  extern constraint c_len;
  extern constraint c_legal;
  extern constraint c_parity;

  extern function new(string name = "yapp_pkt_seq");
  extern task body();

endclass : yapp_pkt_seq

//------------------------------------------------------------------------------
// yapp_pkt_seq -- constraints
//------------------------------------------------------------------------------

constraint yapp_pkt_seq::c_len {
  m_pkt_len inside {[1:63]};
}

//------------------------------------------------------------------------------
constraint yapp_pkt_seq::c_legal {
  soft m_pkt_addr != 2'd3;
}

//------------------------------------------------------------------------------
constraint yapp_pkt_seq::c_parity {
  m_pkt_parity != PARITY_TYPE_UNDEFINED;
  soft m_pkt_parity == PARITY_TYPE_GOOD;
}

//------------------------------------------------------------------------------
// yapp_pkt_seq -- method implementations
//------------------------------------------------------------------------------

function yapp_pkt_seq::new(string name = "yapp_pkt_seq");
```

```
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task yapp_pkt_seq::body();
  `uvm_info(get_type_name(), $sformatf("Executing yapp_pkt_seq (addr %0d, length %0d, %s)",
                                       m_pkt_addr, m_pkt_len, m_pkt_parity.name()), UVM_LOW)
  req = yapp_packet::type_id::create("req");
  req.c_addr_legal.constraint_mode(0);      // the test may ask for address 3
  req.c_parity_dist.constraint_mode(0);     // the sequence chooses the parity
  start_item(req);
  if (!req.randomize() with { req.m_addr        == m_pkt_addr;
                              req.m_length      == m_pkt_len;
                              req.m_parity_type == m_pkt_parity; }) begin
    `uvm_error(get_type_name(), "req.randomize() failed")
  end
  finish_item(req);
endtask : body

//------------------------------------------------------------------------------
// yapp_boundary_seq.sv -- YAPP sequence library (test-plan sequences)
// Split out of yapp_tx_seqs.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// yapp_boundary_seq: packets around the maxpktsize limit, to every address
// including the illegal 3. For each address the lengths maxpktsize - 1,
// maxpktsize, maxpktsize + 1 and 63 are sent (those that exist in 1..63), all
// with good parity. The sequence counts what it sent, so the test can predict
// the counters and the scoreboard without re-deriving the drop rules.
//------------------------------------------------------------------------------
class yapp_boundary_seq extends yapp_base_seq;

  rand bit[5:0] m_maxpktsize;               // the value programmed into ctrl_reg

  // Bookkeeping for the test
  int m_sent_total;
  int m_sent_oversized;                      // length > maxpktsize
  int m_sent_per_addr[4];                    // per address, dropped or not
  int m_sent_forwarded;                      // legal address and not oversized

  `uvm_object_utils(yapp_boundary_seq)

  extern constraint c_max;

  extern function new(string name = "yapp_boundary_seq");
  extern task body();

endclass : yapp_boundary_seq

//------------------------------------------------------------------------------
// yapp_boundary_seq -- constraints
//------------------------------------------------------------------------------

constraint yapp_boundary_seq::c_max {
  m_maxpktsize inside {[2:62]};
}

//------------------------------------------------------------------------------
```

```
// yapp_boundary_seq -- method implementations
//------------------------------------------------------------------------------

function yapp_boundary_seq::new(string name = "yapp_boundary_seq");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task yapp_boundary_seq::body();
  int lens[$];
  int cand[4];
  cand = '{m_maxpktsize - 1, m_maxpktsize, m_maxpktsize + 1, 63};
  foreach (cand[i]) begin
    if (cand[i] >= 1 && cand[i] <= 63 && !(cand[i] inside {lens})) begin
      lens.push_back(cand[i]);
    end
  end
  `uvm_info(get_type_name(), $sformatf("Executing yapp_boundary_seq (maxpktsize %0d, lengths
%p)",
                                       m_maxpktsize, lens), UVM_LOW)
  m_sent_total = 0;
  m_sent_oversized = 0;
  m_sent_forwarded = 0;
  foreach (m_sent_per_addr[i]) begin
    m_sent_per_addr[i] = 0;
  end
  for (int a = 0; a < 4; a++) begin
    foreach (lens[i]) begin
      req = yapp_packet::type_id::create("req");
      req.c_addr_legal.constraint_mode(0);
      req.c_parity_dist.constraint_mode(0);   // the sequence chooses the parity
      start_item(req);
      if (!req.randomize() with { req.m_addr        == a;
                                  req.m_length      == lens[i];
                                  req.m_parity_type == PARITY_TYPE_GOOD; }) begin
        `uvm_error(get_type_name(), "req.randomize() failed")
      end
      finish_item(req);
      m_sent_total++;
      m_sent_per_addr[a]++;
      if (lens[i] > m_maxpktsize) begin
        m_sent_oversized++;
      end
      if (a != 3 && lens[i] <= m_maxpktsize) begin
        m_sent_forwarded++;
      end
    end
  end
endtask : body

//------------------------------------------------------------------------------
// yapp_gap_seq.sv -- YAPP sequence library (test-plan sequences)
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// yapp_gap_seq: `count` short good packets to the legal addresses with chosen
// gaps (packet_delay = idle cycles before the packet). The first packet waits
// 3 cycles; after it the gaps cycle 1, 2, 3, 0, 1, 2, 3, 0, ... so every gap
// from back to back (0) to "longer" (3) appears. The sequence counts the gaps
```

```
// it asked for between its packets, in the same four classes as the monitor
// (0, 1, 2, 3 or more), so the test can compare planned and measured gaps. The
// first packet is not counted: its gap depends on what the bus did before.
//------------------------------------------------------------------------------
class yapp_gap_seq extends yapp_base_seq;

  rand int unsigned m_count;

  // Bookkeeping for the test: gaps asked for after the first packet, per class 0, 1, 2, 3+
  int m_planned_gap[4];

  `uvm_object_utils(yapp_gap_seq)

  extern constraint c_count;

  extern function new(string name = "yapp_gap_seq");
  extern task body();

endclass : yapp_gap_seq

//------------------------------------------------------------------------------
// yapp_gap_seq -- constraints
//------------------------------------------------------------------------------

constraint yapp_gap_seq::c_count {
  m_count inside {[8:40]};
}

//------------------------------------------------------------------------------
// yapp_gap_seq -- method implementations
//------------------------------------------------------------------------------

function yapp_gap_seq::new(string name = "yapp_gap_seq");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task yapp_gap_seq::body();
  int gap;
  `uvm_info(get_type_name(), $sformatf("Executing yapp_gap_seq (%0d packets)", m_count), UVM_LOW)
  foreach (m_planned_gap[i]) begin
    m_planned_gap[i] = 0;
  end
  for (int i = 0; i < m_count; i++) begin
    gap = (i == 0) ? 3 : i % 4;
    req = yapp_packet::type_id::create("req");
    req.c_parity_dist.constraint_mode(0);   // the sequence chooses the parity
    req.c_delay.constraint_mode(0);         // the sequence chooses the gap (0 included)
    start_item(req);
    if (!req.randomize() with { req.m_length       inside {[1:8]};
                                req.m_parity_type  == PARITY_TYPE_GOOD;
                                req.m_packet_delay == gap; }) begin
      `uvm_error(get_type_name(), "req.randomize() failed")
    end
    finish_item(req);
    if (i > 0) begin
      m_planned_gap[gap]++;
    end
  end
```

```
endtask : body

//------------------------------------------------------------------------------
// yapp_random_burst_seq.sv -- YAPP sequence library (random tests)
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// yapp_random_burst_seq: `count` packets whose address, length, parity and gap
// are drawn per packet from weights the caller sets (the knobs below), so one
// burst can mix legal and illegal addresses, lengths around the maxpktsize in
// force, bad parity and back-to-back packets.
//
// Length of each packet, in this order of the percent knobs:
//   short_pct     1..3 bytes (the shortest packets: bad-parity storms)
//   long_pct      40..63 bytes (longer than a channel FIFO: back-pressure)
//   boundary_pct  one of 1, maxpktsize-1, maxpktsize, maxpktsize+1, 63
//                 (clipped to 1..63)
//   otherwise     1..63
// Gap before each packet (idle cycles): gap0_pct back to back, gap_short_pct
// 1..3, otherwise 4..20.
//
// Every choice comes from rnd:: and is pinned with `==` in randomize() with
// the packet's own address / parity / delay constraints switched off (dist and
// == on the same field do not mix in Verilator); the payload stays random.
//------------------------------------------------------------------------------
class yapp_random_burst_seq extends yapp_base_seq;

  rand int unsigned m_count;

  // Knobs (percent unless said otherwise)
  bit[5:0] m_maxpktsize   = 6'd63;          // the value in force, for the boundary lengths
  int       m_addr3_pct    = 5;              // packets to the illegal address 3
  int       m_bad_pct      = 15;             // packets with bad parity
  int       m_short_pct    = 0;
  int       m_long_pct     = 0;
  int       m_boundary_pct = 40;
  int       m_gap0_pct     = 25;
  int       m_gap_short_pct = 35;

  // What was sent, for the [TEST] stage lines
  int m_sent_per_addr[4];
  int m_sent_bad;
  int m_sent_gap0;
  int m_sent_len_min;
  int m_sent_len_max;

  `uvm_object_utils(yapp_random_burst_seq)

  extern constraint c_count;

  extern function new(string name = "yapp_random_burst_seq");
  extern task body();

  // One draw of each field, from the knobs
  extern function bit[1:0] pick_addr();
  extern function bit[5:0] pick_length();
  extern function int       pick_gap();

  // "addr0..3 a/b/c/d, bad n, gap0 n, len lo..hi"
```

```
  extern function string summary();

endclass : yapp_random_burst_seq

//------------------------------------------------------------------------------
// yapp_random_burst_seq -- constraints
//------------------------------------------------------------------------------

constraint yapp_random_burst_seq::c_count {
  m_count inside {[1:100]};
}

//------------------------------------------------------------------------------
// yapp_random_burst_seq -- method implementations
//------------------------------------------------------------------------------

function yapp_random_burst_seq::new(string name = "yapp_random_burst_seq");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task yapp_random_burst_seq::body();
  bit[1:0]     a;
  bit[5:0]     len;
  parity_type_e par;
  int           gap;
  `uvm_info(get_type_name(), $sformatf("Executing yapp_random_burst_seq (%0d packets, maxpktsize
%0d)",
                                       m_count, m_maxpktsize), UVM_LOW)
  foreach (m_sent_per_addr[i]) begin
    m_sent_per_addr[i] = 0;
  end
  m_sent_bad     = 0;
  m_sent_gap0    = 0;
  m_sent_len_min = 64;
  m_sent_len_max = 0;
  for (int i = 0; i < m_count; i++) begin
    a   = pick_addr();
    len = pick_length();
    par = (rnd::get_int(0, 99, "yapp_random_burst_seq.parity") < m_bad_pct) ? PARITY_TYPE_BAD :
PARITY_TYPE_GOOD;
    gap = pick_gap();
    req = yapp_packet::type_id::create("req");
    req.c_addr_legal.constraint_mode(0);    // the sequence chooses the address (3 included)
    req.c_parity_dist.constraint_mode(0);   // ... the parity
    req.c_delay.constraint_mode(0);         // ... and the gap (0 included)
    start_item(req);
    if (!req.randomize() with { req.m_addr         == a;
                                req.m_length       == len;
                                req.m_parity_type  == par;
                                req.m_packet_delay == gap; }) begin
      `uvm_error(get_type_name(), "req.randomize() failed")
    end
    finish_item(req);
    m_sent_per_addr[a]++;
    if (par == PARITY_TYPE_BAD) begin
      m_sent_bad++;
    end
    if (gap == 0) begin
```

```
      m_sent_gap0++;
    end
    if (len < m_sent_len_min) begin
      m_sent_len_min = len;
    end
    if (len > m_sent_len_max) begin
      m_sent_len_max = len;
    end
  end
endtask : body

//------------------------------------------------------------------------------
function bit[1:0] yapp_random_burst_seq::pick_addr();
  if (rnd::get_int(0, 99, "yapp_random_burst_seq.addr3") < m_addr3_pct) begin
    return 2'd3;
  end
  return 2'(rnd::get_int(0, 2, "yapp_random_burst_seq.addr"));
endfunction : pick_addr

//------------------------------------------------------------------------------
function bit[5:0] yapp_random_burst_seq::pick_length();
  int r;
  int len;
  r = rnd::get_int(0, 99, "yapp_random_burst_seq.length_class");
  if (r < m_short_pct) begin
    len = rnd::get_int(1, 3, "yapp_random_burst_seq.short_length");
  end else if (r < m_short_pct + m_long_pct) begin
    len = rnd::get_int(40, 63, "yapp_random_burst_seq.long_length");
  end else if (r < m_short_pct + m_long_pct + m_boundary_pct) begin
    case (rnd::get_int(0, 4, "yapp_random_burst_seq.boundary"))
      0:       len = 1;
      1:       len = int'(m_maxpktsize) - 1;
      2:       len = int'(m_maxpktsize);
      3:       len = int'(m_maxpktsize) + 1;
      default: len = 63;
    endcase
  end else begin
    len = rnd::get_int(1, 63, "yapp_random_burst_seq.length");
  end
  if (len < 1) begin
    len = 1;
  end
  if (len > 63) begin
    len = 63;
  end
  return 6'(len);
endfunction : pick_length

//------------------------------------------------------------------------------
function int yapp_random_burst_seq::pick_gap();
  int r;
  r = rnd::get_int(0, 99, "yapp_random_burst_seq.gap_class");
  if (r < m_gap0_pct) begin
    return 0;
  end
  if (r < m_gap0_pct + m_gap_short_pct) begin
    return rnd::get_int(1, 3, "yapp_random_burst_seq.short_gap");
  end
  return rnd::get_int(4, 20, "yapp_random_burst_seq.gap");
```

```
endfunction : pick_gap

//------------------------------------------------------------------------------
function string yapp_random_burst_seq::summary();
  return $sformatf("addr0..3 %0d/%0d/%0d/%0d, bad parity %0d, gap 0 %0d, length %0d..%0d",
                   m_sent_per_addr[0], m_sent_per_addr[1], m_sent_per_addr[2],
m_sent_per_addr[3],
                   m_sent_bad, m_sent_gap0, m_sent_len_min, m_sent_len_max);
endfunction : summary
```

The last four are not in the course: `yapp_pkt_seq` (one packet the test chooses), `yapp_boundary_seq` (lengths around `maxpktsize`), `yapp_gap_seq` (exact idle cycles between packets) and `yapp_random_burst_seq` (the knobs of the random tests) were written for the test plan.

Patterns worth copying

Pattern	Where	Note
objection in `pre_body` / `post_body`	`yapp_base_seq`	only root sequences have a starting phase; su see `null`
inline constraint	`yapp_012_seq`	`req.randomize() with { req.m_addr == 2'd0;`
nested sequence	`yapp_111_seq`	`m_seq_1.start(m_sequencer, this)` three time
random **sequence** property	`yapp_repeat_addr_seq`	`rand bit[1:0] m_seq_addr;` randomized wher sequence is — two items share it
create / modify / send	`yapp_incr_payload_seq`	`randomize()`, edit, `set_parity()`, then `start_` `finish_item`
constrained nesting	`six_yapp_seq`	`m_rnd_seq.randomize() with { m_rnd_seq.m_c` before `rnd_seq.start(...)`

Pattern	Where	Note
switching a constraint off	`yapp_88_packets_seq`, `yapp_coverage_seq`	`req.c_addr_legal.constraint_mode(0)` to reac
run everything	`yapp_exhaustive_seq`	the cheapest regression of a library

Starting a sequence

Default sequence (from a test)

```
uvm_config_wrapper::set(this, "tb.yapp.agent.sequencer.run_phase",
                        "default_sequence", yapp_012_seq::get_type());
```

The sequencer creates and starts it when `run_phase` begins. Set it in the test's `build_phase`, before the testbench is built.

Explicitly (from a test or another sequence)

```
yapp_012_seq seq = yapp_012_seq::type_id::create("seq");
seq.start(m_tb.m_yapp.m_agent.m_sequencer);          // blocks until body() returns
```

Used by `reg_function_test` (Lab 11C). Remember to raise an objection in the test around it.

From a virtual sequence

```
m_yapp_012 = yapp_012_seq::type_id::create("yapp_012");
if (!m_yapp_012.randomize()) `uvm_error(get_type_name(), "yapp_012.randomize() failed")
m_yapp_012.start(p_sequencer.m_yapp_seqr, this);   // `uvm_do_on in the course
```

The target is a handle of the virtual sequencer, not `m_sequencer`. See the virtual sequencer guide.

Randomization failures (Lab 5)

An inline constraint that contradicts a constraint of the item makes `randomize()` **fail**: it returns 0, the sequence reports `req.randomize()` `failed`, the item keeps its old values and `finish_item` still sends it. In batch mode the simulation does not stop. That is what happened in Lab 5 when `short_yapp_packet` forbade address 2 while `yapp_012_seq` asked for it. Debug with `-gui -access rwc` (SimVision stops on the failure and opens the constraint manager) or simply read the warning: it names the conflicting constraints.

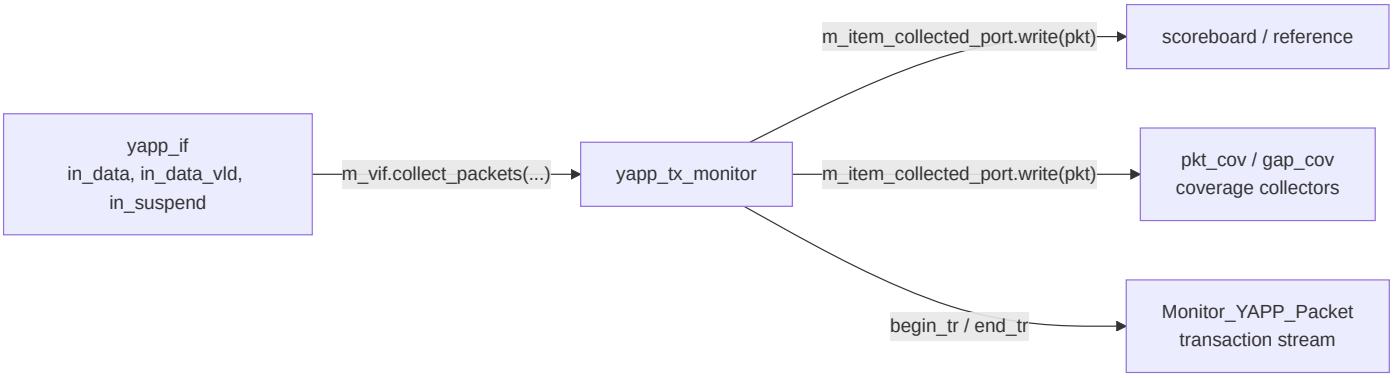
Common mistakes

- `` `uvm_component_utils `` on a sequence (it is an object).
- A constructor with a `parent` argument.
- Forgetting `super.new(name)`.
- Ignoring the return value of `randomize()`: the item is sent with stale values and nobody notices.
- Calling `finish_item` without `start_item` (or `start_item` on a different handle than `finish_item`).
- Raising an objection in a sub-sequence without the `null` check (`starting_phase` is `null` there).
- A `forever` sequence (`channel_rx_resp_seq`) **with** an objection: the test never ends.

Monitor

Open on the project map →

The monitor is the **passive** half of an agent: it watches the interface, rebuilds transactions and publishes them. It never drives anything, it is always built (active *and* passive agents), and everything that checks or measures — scoreboard, reference model, coverage collectors — hangs off it.



The three jobs

Job	Code
collect	`m_vif.collect_packets(pkt.m_addr, pkt.m_length, pkt.m_payload, pkt.m_pari` on the rising edge; a byte counts when `in_data_vld && !in_suspend`; `idle_cyc` before the header
publish	`uvm_analysis_port #(yapp_packet) m_item_collected_port;` constructed in `ne` after each packet
cover	not in the monitor: `yapp_pkt_coverage` (`yapp_pkt_cg`) and `yapp_gap_coverage` (subscribe to `m_item_collected_port`; the monitor writes the gap it measured in `m_packet_delay` (-1 for the first packet) so the gap collector can sample it — se coverage

The reference implementation

```
//------------------------------------------------------------------------------
// yapp_tx_monitor.sv -- observes packets on the YAPP input port (Lab 3/6/9A/10)
//
//   * watches the interface through a virtual interface (Lab 6)
//   * publishes every packet on item_collected_port (Lab 9A)
// The functional coverage of the input traffic (Lab 10) is not here: it lives
// in yapp_pkt_coverage and yapp_gap_coverage, subscribers of
// item_collected_port (see yapp_tx_agent).
//------------------------------------------------------------------------------
class yapp_tx_monitor extends uvm_monitor;

  // Virtual interface, set by tb_top through yapp_vif_config
  virtual interface yapp_if m_vif;

  // Analysis port: the scoreboard / reference model subscribe here
  uvm_analysis_port #(yapp_packet) m_item_collected_port;

  // Statistics
  int m_num_pkt_col;
  int m_num_bad_parity;   // packets seen with a wrong parity byte (Lab 11C uses it)

  // Gap before each packet: idle cycles since the previous packet's parity
  // byte (test plan ROUTE-04), also written into the collected packet's
  // packet_delay (-1 for the first packet after reset). num_gap[0..2] count
  // gaps of 0, 1 and 2 cycles, num_gap[3] the longer ones; the first packet
  // after reset is not counted.
  int m_last_gap = -1;
  int m_num_gap[4];

  `uvm_component_utils(yapp_tx_monitor)

  // The fields are printed by do_print() below:
  // no uvm_field_* automation.
  extern virtual function void do_print(uvm_printer printer);
  extern function new(string name, uvm_component parent);

  extern function void connect_phase(uvm_phase phase);
  extern task run_phase(uvm_phase phase);

  extern task collect_packets();
  extern function void report_phase(uvm_phase phase);

endclass : yapp_tx_monitor

//------------------------------------------------------------------------------
// yapp_tx_monitor -- method implementations
//------------------------------------------------------------------------------

function yapp_tx_monitor::new(string name, uvm_component parent);
  super.new(name, parent);
  m_item_collected_port = new("item_collected_port", this);
endfunction : new

//------------------------------------------------------------------------------
function void yapp_tx_monitor::connect_phase(uvm_phase phase);
  if (!yapp_vif_config::get(this, "", "vif", m_vif)) begin
    `uvm_error("NOVIF", {"virtual interface must be set for: ", get_full_name(), ".vif"})
  end
```

```
endfunction : connect_phase

//------------------------------------------------------------------------------
task yapp_tx_monitor::run_phase(uvm_phase phase);
  `uvm_info(get_type_name(), "YAPP monitor running", UVM_LOW)
  collect_packets();
endtask : run_phase

//------------------------------------------------------------------------------
task yapp_tx_monitor::collect_packets();
  yapp_packet pkt;
  int idle_cycles;
  bit first = 1'b1;
  // Nothing to observe while reset is active
  @(posedge m_vif.clock);
  wait (m_vif.reset === 1'b0);
  forever begin
    // A NEW object per packet: subscribers keep the handle (analysis FIFOs
    // do not clone), so re-using one object would corrupt earlier packets.
    pkt = yapp_packet::type_id::create("pkt", this);
    m_vif.collect_packets(pkt.m_addr, pkt.m_length, pkt.m_payload, pkt.m_parity, idle_cycles);
    if (first) begin
      pkt.m_packet_delay = -1;               // no previous packet: no gap
    end else begin
      pkt.m_packet_delay = idle_cycles;
      m_last_gap = idle_cycles;
      m_num_gap[(idle_cycles > 3) ? 3 : idle_cycles]++;
    end
    first = 1'b0;
    void'(begin_tr(pkt, "Monitor_YAPP_Packet"));
    pkt.m_parity_type = (pkt.m_parity == pkt.calc_parity()) ? PARITY_TYPE_GOOD : PARITY_TYPE_BAD;
    m_num_pkt_col++;
    if (pkt.m_parity_type == PARITY_TYPE_BAD) begin
      m_num_bad_parity++;
    end
    `uvm_info(get_type_name(),
              $sformatf("RX pkt #%0d on the input: addr=%0d len=%0d parity=%s gap=%0d",
m_num_pkt_col, pkt.m_addr,
                        pkt.m_length, pkt.parity_text(), idle_cycles), UVM_MEDIUM)
    `uvm_info(get_type_name(), $sformatf("Packet collected:\n%s", pkt.sprint()), UVM_HIGH)
    m_item_collected_port.write(pkt);
    end_tr(pkt);
  end
endtask : collect_packets

//------------------------------------------------------------------------------
function void yapp_tx_monitor::report_phase(uvm_phase phase);
  `uvm_info(get_type_name(),
            $sformatf("YAPP monitor report: %0d packets collected, %0d with bad parity",
                      m_num_pkt_col, m_num_bad_parity), UVM_LOW)
  `uvm_info(get_type_name(),
            $sformatf("YAPP gaps: %0d back to back, %0d of 1 cycle, %0d of 2, %0d longer",
                      m_num_gap[0], m_num_gap[1], m_num_gap[2], m_num_gap[3]), UVM_LOW)
endfunction : report_phase

//------------------------------------------------------------------------------
function void yapp_tx_monitor::do_print(uvm_printer printer);
  super.do_print(printer);
```

```
  printer.print_field("num_pkt_col", m_num_pkt_col, $bits(m_num_pkt_col), UVM_DEC);
endfunction : do_print
```

Design notes

- **One new object per packet.** `pkt = yapp_packet::type_id::create("pkt", this)` is inside the loop. Subscribers receive a handle; if the monitor re-used one object, every stored packet would silently change into the latest one. The scoreboard clones anyway (belt and braces); analysis FIFOs (Lab 9D) do not.
- **Sampling edge.** The monitor samples on the **rising** edge, where the DUT samples. On that edge `in_suspend` still has its pre-edge value (the DUT updates it with a non-blocking assignment), so `in_data_vld && !in_suspend` is exactly the DUT's own acceptance condition.
- **parity_type is derived**: the monitor recomputes the parity and marks the packet `PARITY_TYPE_GOOD` / `PARITY_TYPE_BAD`. The channel monitor does the same.
- `m_num_pkt_col` and `m_num_bad_parity` are plain counters printed in `report_phase`; `reg_function_test` uses `m_num_bad_parity` to know what the DUT's parity counter should read.
- The channel monitor additionally checks that a packet's address matches the `channel_id` of the channel it arrived on — a cheap built-in checker.

The other monitors

Monitor	Collects	Publishes
`channel_rx_monitor`	a packet on `data_x` (byte consumed when `data_vld && !suspend` on the rising edge)	`channel_packet`
`hbus_monitor`	one read or write (`hen` high; reads take the data on the second rising edge)	`hbus_transaction`

Common mistakes

- A type parameter on `uvm_monitor` (there is none).
- Constructing the analysis port in `build_phase` and forgetting it in `new()` — either works, but it must exist before `connect_phase`.

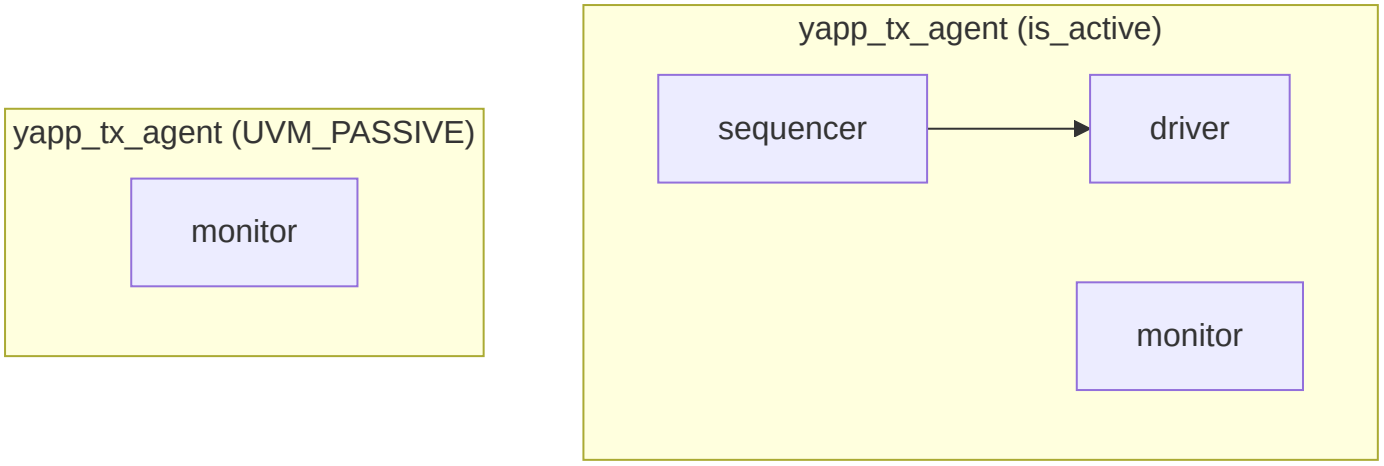
- Sampling on the falling edge in the same timestep as the driver's non-blocking assignments: you read the *old* values. Sample on the opposite edge from the one the signals are driven on.
- Counting the parity byte with `in_data_vld` high — it is driven with `in_data_vld` **low**.

Agent and env

Open on the project map →

Agent: one interface, three components

An agent packages everything one interface needs. `is_active` decides whether it **generates** traffic (driver + sequencer) or only **observes** it.



```
//------------------------------------------------------------------------------
// yapp_tx_agent.sv -- the YAPP transmit agent (Lab 3, factory in Lab 4)
//
//   is_active == UVM_ACTIVE  : monitor + driver + sequencer (generates traffic)
//   is_active == UVM_PASSIVE : monitor only (just watches the bus)
// Both modes: the coverage subscribers pkt_cov (yapp_pkt_cg, Lab 10) and
// gap_cov (yapp_gap_cg) on the monitor's item_collected_port.
//------------------------------------------------------------------------------
class yapp_tx_agent extends uvm_agent;

  yapp_tx_monitor   m_monitor;
  yapp_pkt_coverage m_pkt_cov;
  yapp_gap_coverage m_gap_cov;
  yapp_tx_driver    m_driver;
  yapp_tx_sequencer m_sequencer;

  // is_active is inherited from uvm_agent (default UVM_ACTIVE); build_phase
  // picks up an override set with uvm_config_int::set(..., "is_active", ...)
  `uvm_component_utils(yapp_tx_agent)

  // The fields are printed by do_print() below and read from uvm_config_db in build_phase():
  // no uvm_field_* automation.
  extern virtual function void do_print(uvm_printer printer);
  extern function new(string name, uvm_component parent);

  extern function void build_phase(uvm_phase phase);
  extern function void connect_phase(uvm_phase phase);

endclass : yapp_tx_agent

//------------------------------------------------------------------------------
// yapp_tx_agent -- method implementations
//------------------------------------------------------------------------------

function yapp_tx_agent::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//------------------------------------------------------------------------------
function void yapp_tx_agent::build_phase(uvm_phase phase);
  uvm_bitstream_t cfg_is_active;
  super.build_phase(phase);
  // overrides set with uvm_config_int::set(...)
  if (uvm_config_int::get(this, "", "is_active", cfg_is_active)) begin
    is_active = uvm_active_passive_enum'(cfg_is_active);
  end
  m_monitor = yapp_tx_monitor::type_id::create("monitor", this);
  m_pkt_cov = yapp_pkt_coverage::type_id::create("pkt_cov", this);
  m_gap_cov = yapp_gap_coverage::type_id::create("gap_cov", this);
  if (is_active == UVM_ACTIVE) begin
    m_driver    = yapp_tx_driver::type_id::create("driver", this);
    m_sequencer = yapp_tx_sequencer::type_id::create("sequencer", this);
  end
endfunction : build_phase

//------------------------------------------------------------------------------
function void yapp_tx_agent::connect_phase(uvm_phase phase);
  m_monitor.m_item_collected_port.connect(m_pkt_cov.analysis_export);
```

```
  m_monitor.m_item_collected_port.connect(m_gap_cov.analysis_export);
  if (is_active == UVM_ACTIVE) begin
    m_driver.seq_item_port.connect(m_sequencer.seq_item_export);
  end
endfunction : connect_phase

//------------------------------------------------------------------------------
function void yapp_tx_agent::do_print(uvm_printer printer);
  super.do_print(printer);
  printer.print_generic("is_active", "uvm_active_passive_enum", $bits(is_active),
is_active.name());
endfunction : do_print
```

- `is_active` is inherited from `uvm_agent`. The first thing `build_phase` does after `super.build_phase(phase)` is ask the configuration database for it: `uvm_config_int::get(this, "", "is_active", cfg_is_active)`. That is what lets `uvm_config_int::set(this, "tb.yapp.agent", "is_active", UVM_PASSIVE)` in a test change the agent (Lab 4 `set_config_test`). The course gets the same effect from the `` `uvm_field_enum `` macro; here the `get()` is on the page, so you can see *when* the value arrives — before the children are created, which is why it can decide whether the driver exists.
- `do_print` adds `is_active` to the topology print; without it the agent would show up with no fields of its own.
- The monitor is created unconditionally, the driver and the sequencer only when active, and the connection only when both exist.

Env: a group of agents

The UVC's env holds its agents (one here; `hbus_env` has a `masters[]` array and no slave agent, because the router is the only slave on the bus). The testbench env holds the UVC envs plus the analysis components.

```
//------------------------------------------------------------------------------
// yapp_env.sv -- YAPP UVC top level (Lab 3)
//------------------------------------------------------------------------------
class yapp_env extends uvm_env;

  yapp_tx_agent m_agent;

  `uvm_component_utils(yapp_env)

  extern function new(string name, uvm_component parent);
  extern function void build_phase(uvm_phase phase);

endclass : yapp_env

//------------------------------------------------------------------------------
// yapp_env -- method implementations
//------------------------------------------------------------------------------

function yapp_env::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//------------------------------------------------------------------------------
function void yapp_env::build_phase(uvm_phase phase);
  super.build_phase(phase);
  m_agent = yapp_tx_agent::type_id::create("agent", this);
endfunction : build_phase
```

The testbench env: `router_tb`

The final version (Lab 11B) shows every responsibility of a top-level env:

```
//------------------------------------------------------------------------------
// router_tb.sv -- the testbench environment (Lab 11B: + register model)
//
//   yapp_rm  : the register model (yapp_router_regs_t)
//   reg2hbus : adapter translating register operations to HBUS transactions
//
// Front-door accesses go  model -> default_map -> reg2hbus -> HBUS sequencer.
// Backdoor accesses use the HDL paths rooted at "hw_top.dut.u_regs" (the register block of the
router).
//------------------------------------------------------------------------------
class router_tb extends uvm_env;

  yapp_env            m_yapp;
  channel_env         m_chan0;
  channel_env         m_chan1;
  channel_env         m_chan2;
  hbus_env            m_hbus;
  clock_and_reset_env m_clk_rst;
  router_mcsequencer  m_mcseqr;
  router_module_env   m_router_module;

  yapp_router_regs_t  m_yapp_rm;
  hbus_reg_adapter    m_reg2hbus;

  `uvm_component_utils(router_tb)

  // The fields are printed by do_print() below:
  // no uvm_field_* automation.
  extern virtual function void do_print(uvm_printer printer);
  extern function new(string name, uvm_component parent);

  extern function void build_phase(uvm_phase phase);
  extern function void connect_phase(uvm_phase phase);

endclass : router_tb

//------------------------------------------------------------------------------
// router_tb -- method implementations
//------------------------------------------------------------------------------

function router_tb::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//------------------------------------------------------------------------------
function void router_tb::build_phase(uvm_phase phase);
  super.build_phase(phase);
  `uvm_info(get_type_name(), "Executing the build phase of the testbench", UVM_HIGH)

  uvm_config_int::set(this, "chan0", "channel_id", 0);
  uvm_config_int::set(this, "chan1", "channel_id", 1);
  uvm_config_int::set(this, "chan2", "channel_id", 2);
  uvm_config_int::set(this, "hbus",  "num_masters", 1);

  m_yapp          = yapp_env::type_id::create("yapp", this);
  m_chan0         = channel_env::type_id::create("chan0", this);
  m_chan1         = channel_env::type_id::create("chan1", this);
  m_chan2         = channel_env::type_id::create("chan2", this);
```

```
  m_hbus          = hbus_env::type_id::create("hbus", this);
  m_clk_rst       = clock_and_reset_env::type_id::create("clk_rst", this);
  m_mcseqr        = router_mcsequencer::type_id::create("mcseqr", this);
  m_router_module = router_module_env::type_id::create("router_module", this);

  // Register model: build, lock, backdoor root, implicit prediction
  m_yapp_rm = yapp_router_regs_t::type_id::create("yapp_rm");
  m_yapp_rm.build();
  m_yapp_rm.lock_model();
  m_yapp_rm.set_hdl_path_root("hw_top.dut.u_regs");
  m_yapp_rm.default_map.set_auto_predict(1);

  m_reg2hbus = hbus_reg_adapter::type_id::create("reg2hbus");
endfunction : build_phase

//------------------------------------------------------------------------------
function void router_tb::connect_phase(uvm_phase phase);
  m_mcseqr.m_hbus_seqr = m_hbus.m_masters[0].m_sequencer;
  m_mcseqr.m_yapp_seqr = m_yapp.m_agent.m_sequencer;

  m_yapp.m_agent.m_monitor.m_item_collected_port.connect(m_router_module.m_yapp_export);
  m_hbus.m_monitor.m_item_collected_port.connect(m_router_module.m_hbus_export);
  m_chan0.m_rx_agent.m_monitor.m_item_collected_port.connect(m_router_module.m_chan0_export);
  m_chan1.m_rx_agent.m_monitor.m_item_collected_port.connect(m_router_module.m_chan1_export);
  m_chan2.m_rx_agent.m_monitor.m_item_collected_port.connect(m_router_module.m_chan2_export);

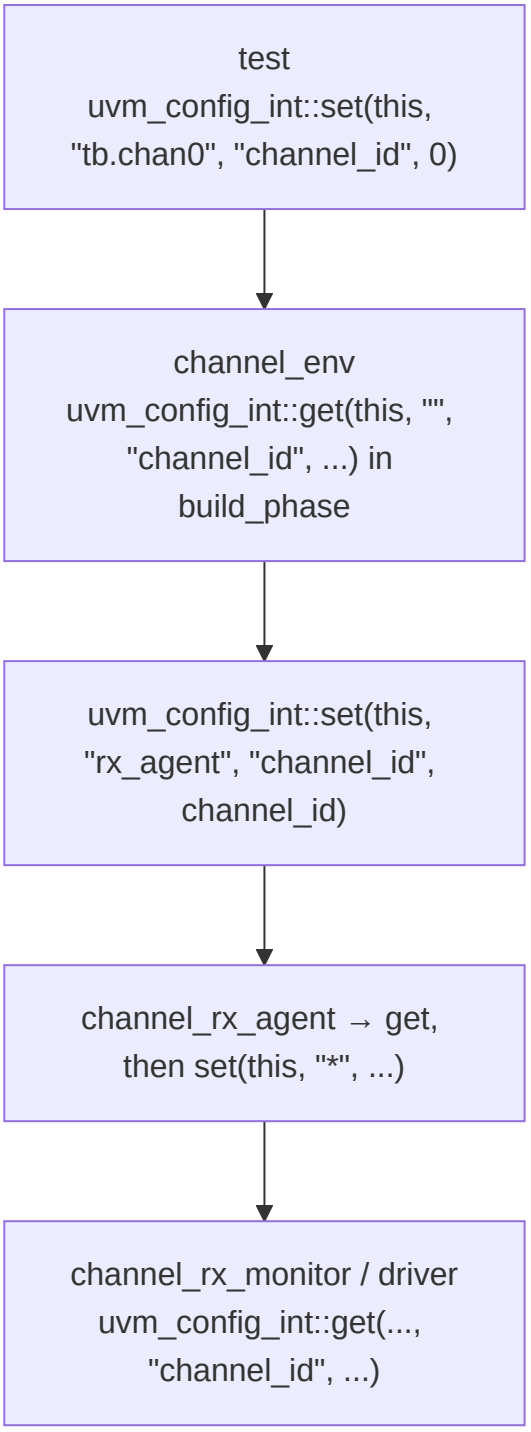
  // Front door: the model's map drives the HBUS master sequencer via the adapter
  m_yapp_rm.default_map.set_sequencer(m_hbus.m_masters[0].m_sequencer, m_reg2hbus);
endfunction : connect_phase

//------------------------------------------------------------------------------
function void router_tb::do_print(uvm_printer printer);
  super.do_print(printer);
  printer.print_object("yapp_rm", m_yapp_rm);
endfunction : do_print
```

Responsibility	Where
configure children **before** creating them	`uvm_config_int::set(this, "chan0", "channel_id", 0)` …
create everything through the factory	`type_id::create("name", this)`
TLM connections	`connect_phase`: monitor ports → module UVC exports
hierarchical handles for the	`m_mcseqr.m_hbus_seqr = m_hbus.m_masters[0].m_sequencer`

Responsibility	Where
virtual sequencer	
register model plumbing	`build()`, `lock_model()`, `set_hdl_path_root`, `set_auto_predict`, `set_sequencer`

How configuration flows down



Each level reads the value in its own `build_phase` and re-publishes it for its children. That keeps every component independent of the names above it. The same pattern gives `hbus_env` its `num_masters`.

Common mistakes

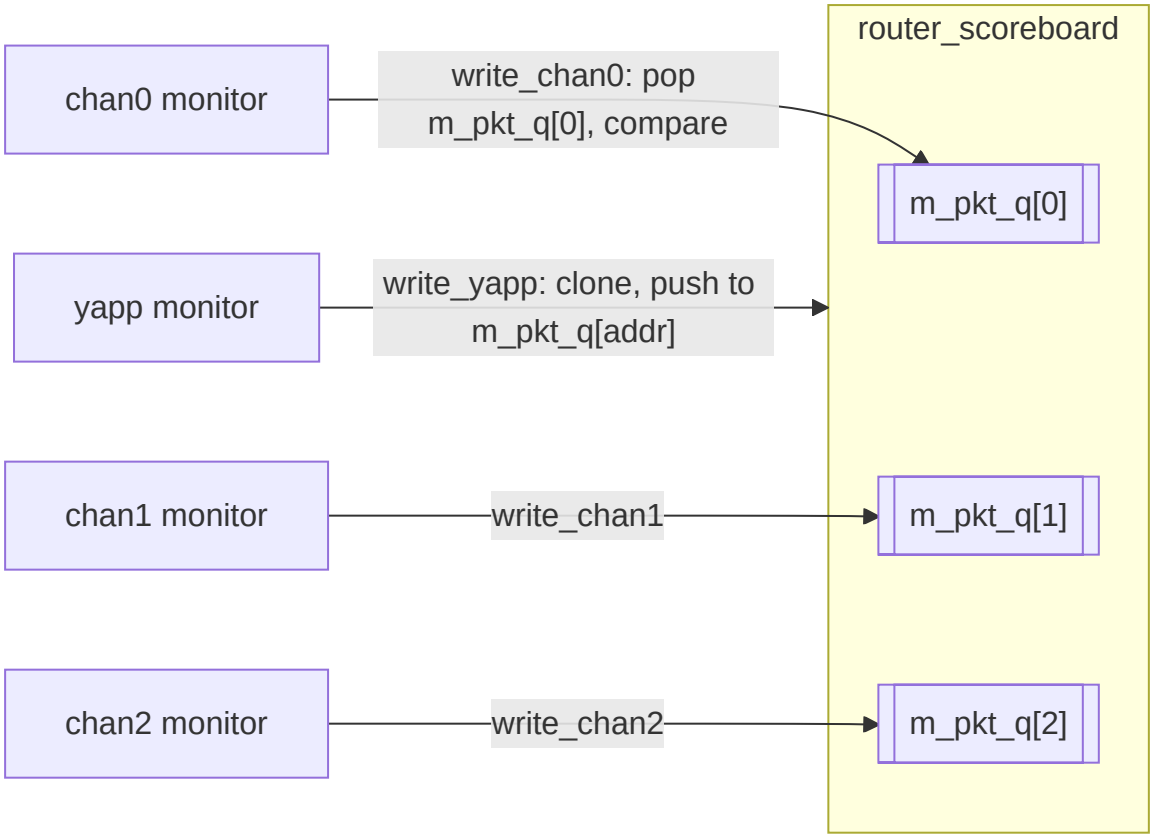
- Creating a child before setting its configuration.
- `new("name", this)` instead of `type_id::create` → overrides do not apply.

- Connecting in `build_phase` — the children do not exist yet when the parent's `build_phase` starts; use `connect_phase`.
- Wildcards in hierarchical references: `m_mcseqr.m_yapp_seqr = m_yapp.m_agent.m_sequencer` is SystemVerilog, not a config string.

Scoreboard

Open on the project map →

The scoreboard answers one question: **did the right packet come out of the right channel, in order?** It receives what went *in* (YAPP monitor) and what came *out* (three channel monitors) and compares.



Design

Decision	Reason
one **queue per channel**	packets to different channels may overtake each other; within a channel the order is preserved
clone on the YAPP side	the monitor hands over a handle and moves on

Decision	Reason
pop and compare on the channel side	FIFO semantics: the oldest expected packet is the one that must appear
a custom compare function	`yapp_packet` and `channel_packet` are different classes, so `uvm_object::compare()` does not apply
counters + `report_phase`	received = matched + mismatched + left in queues (+ address 3): if the equation does not hold, something was lost

The reference implementation

```
//------------------------------------------------------------------------------
// router_scoreboard.sv -- compares what went into the router with what came
// out of each channel (Lab 9A)
//
//   YAPP monitor ──port──► yapp_in  (imp)  : clone and queue per address
//   ChanN monitor──port──► chanN_in (imp)  : pop queue N and compare
//
// An analysis imp needs a write_<suffix>() method per suffix; the suffixes are
// created by the `uvm_analysis_imp_decl macros below (outside the class).
//------------------------------------------------------------------------------
`uvm_analysis_imp_decl(_yapp)
`uvm_analysis_imp_decl(_chan0)
`uvm_analysis_imp_decl(_chan1)
`uvm_analysis_imp_decl(_chan2)

class router_scoreboard extends uvm_scoreboard;

  // TLM connection points
  uvm_analysis_imp_yapp  #(yapp_packet,    router_scoreboard) m_yapp_in;
  uvm_analysis_imp_chan0 #(channel_packet, router_scoreboard) m_chan0_in;
  uvm_analysis_imp_chan1 #(channel_packet, router_scoreboard) m_chan1_in;
  uvm_analysis_imp_chan2 #(channel_packet, router_scoreboard) m_chan2_in;

  // Expected packets, one queue per output channel
  yapp_packet m_pkt_q[3][$];

  // Statistics
  int m_packets_in;         // received from the YAPP monitor
  int m_packets_matched;    // compared OK
  int m_packets_mismatched; // compare failed
  int m_packets_unexpected; // channel packet with an empty queue
  int m_packets_dropped_addr3;

  `uvm_component_utils(router_scoreboard)

  extern function new(string name, uvm_component parent);

  // The comparison function supplied with the lab
  `include "packet_compare.sv"

  //--------------------------------------------------------------------------
  // YAPP side: the monitor passes a HANDLE, so clone before storing it
  //--------------------------------------------------------------------------
  extern function void write_yapp(yapp_packet packet);

  //--------------------------------------------------------------------------
  // Channel side
  //--------------------------------------------------------------------------
  extern function void write_chan0(channel_packet packet);
  extern function void write_chan1(channel_packet packet);

  extern function void write_chan2(channel_packet packet);
  extern function void check_channel(int ch, channel_packet cp);

  // Knob (uvm_config_db #(bit) "check_empty"): an expected packet still queued
  // at the end of the test is an error. The project tests set it; the lab tests
  // keep the course's 200 ns drain time, in which the last packet may still be
  // on its way out, so they leave it off.
```

```
  bit m_check_empty;

  extern function void build_phase(uvm_phase phase);

  //------------------------------------------------------------------------
  // End of test: every expected packet must have come out of its channel
  //------------------------------------------------------------------------
  extern function void check_phase(uvm_phase phase);

  //------------------------------------------------------------------------
  // End of test: the numbers must add up
  //   packets_in == matched + mismatched + left in queues (+ addr 3)
  //------------------------------------------------------------------------
  extern function void report_phase(uvm_phase phase);

endclass : router_scoreboard

//------------------------------------------------------------------------------
// router_scoreboard -- method implementations
//------------------------------------------------------------------------------

function router_scoreboard::new(string name, uvm_component parent);
  super.new(name, parent);
  m_yapp_in  = new("yapp_in",  this);
  m_chan0_in = new("chan0_in", this);
  m_chan1_in = new("chan1_in", this);
  m_chan2_in = new("chan2_in", this);
endfunction : new

//------------------------------------------------------------------------------
function void router_scoreboard::write_yapp(yapp_packet packet);
  yapp_packet pkt;
  $cast(pkt, packet.clone());
  m_packets_in++;
  if (pkt.m_addr == 2'd3) begin
    // Lab 9A assumes legal traffic only; keep the count honest anyway
    m_packets_dropped_addr3++;
    `uvm_warning(get_type_name(), "Packet to illegal address 3 is not expected on any channel")
    return;
  end
  m_pkt_q[pkt.m_addr].push_back(pkt);
  `uvm_info(get_type_name(),
            $sformatf("Expecting packet on channel %0d: addr=%0d len=%0d (%0d waiting)",
pkt.m_addr, pkt.m_addr,
                      pkt.m_length, m_pkt_q[pkt.m_addr].size()), UVM_HIGH)
endfunction : write_yapp

//------------------------------------------------------------------------------
function void router_scoreboard::write_chan0(channel_packet packet);
  check_channel(0, packet);
endfunction : write_chan0

//------------------------------------------------------------------------------
function void router_scoreboard::write_chan1(channel_packet packet);
  check_channel(1, packet);
endfunction : write_chan1

//------------------------------------------------------------------------------
function void router_scoreboard::write_chan2(channel_packet packet);
```

```
  check_channel(2, packet);
endfunction : write_chan2

//------------------------------------------------------------------------------
function void router_scoreboard::check_channel(int ch, channel_packet cp);
  yapp_packet expected;
  if (m_pkt_q[ch].size() == 0) begin
    m_packets_unexpected++;
    `uvm_error(get_type_name(),
               $sformatf("Channel %0d packet received but nothing was expected:\n%s", ch,
cp.sprint()))
    return;
  end
  expected = m_pkt_q[ch].pop_front();
  if (comp_equal(expected, cp)) begin
    m_packets_matched++;
    `uvm_info(get_type_name(),
              $sformatf("ch%0d pkt matched: addr=%0d len=%0d (%0d matched so far)", ch,
cp.m_addr, cp.m_length,
                        m_packets_matched), UVM_MEDIUM)
  end else begin
    m_packets_mismatched++;
    `uvm_error(get_type_name(),
               $sformatf("Channel %0d packet MISMATCH\nexpected:\n%s\nreceived:\n%s",
                         ch, expected.sprint(), cp.sprint()))
  end
endfunction : check_channel

//------------------------------------------------------------------------------
function void router_scoreboard::build_phase(uvm_phase phase);
  super.build_phase(phase);
  void'(uvm_config_db #(bit)::get(this, "", "check_empty", m_check_empty));
endfunction : build_phase

//------------------------------------------------------------------------------
function void router_scoreboard::check_phase(uvm_phase phase);
  if (!m_check_empty) begin
    return;
  end
  for (int ch = 0; ch < 3; ch++) begin
    if (m_pkt_q[ch].size() > 0) begin
      `uvm_error(get_type_name(), $sformatf("%0d packet(s) expected on channel %0d never came out
(first: addr=%0d len=%0d)",
                                            m_pkt_q[ch].size(), ch, m_pkt_q[ch][0].m_addr,
m_pkt_q[ch][0].m_length))
    end
  end
endfunction : check_phase

//------------------------------------------------------------------------------
function void router_scoreboard::report_phase(uvm_phase phase);
  `uvm_info(get_type_name(), $sformatf({"\n--- Scoreboard report ---\n",
    "  packets received  : %0d\n",
    "  packets matched   : %0d\n",
    "  packets mismatched: %0d\n",
    "  packets unexpected: %0d\n",
    "  packets to addr 3 : %0d\n",
    "  left in queue 0/1/2: %0d / %0d / %0d"},
    m_packets_in, m_packets_matched, m_packets_mismatched, m_packets_unexpected,
```

```
m_packets_dropped_addr3,
    m_pkt_q[0].size(), m_pkt_q[1].size(), m_pkt_q[2].size()), UVM_LOW)
endfunction : report_phase
```

The comparison functions (`` `include ``d into the class body):

```
//------------------------------------------------------------------------------
// packet_compare.sv -- supplied for Lab 9A: compare a YAPP packet (what went
// into the router) with a Channel packet (what came out).
//
// The two are different classes, so the built-in compare() cannot be used
// directly. This file is `included INSIDE the scoreboard class body.
//------------------------------------------------------------------------------

  // Plain SystemVerilog comparison, one `uvm_error per difference
  function bit comp_equal(input yapp_packet yp, input channel_packet cp);
    bit ok = 1;
    if (yp.m_addr != cp.m_addr) begin
      `uvm_error("PKT_COMPARE", $sformatf("Address mismatch: YAPP %0d, Channel %0d", yp.m_addr,
cp.m_addr))
      ok = 0;
    end
    if (yp.m_length != cp.m_length) begin
      `uvm_error("PKT_COMPARE", $sformatf("Length mismatch: YAPP %0d, Channel %0d", yp.m_length,
cp.m_length))
      ok = 0;
    end
    foreach (yp.m_payload[i]) begin
      if (i >= cp.m_payload.size()) begin
        break;
      end
      if (yp.m_payload[i] != cp.m_payload[i]) begin
        `uvm_error("PKT_COMPARE", $sformatf("Payload[%0d] mismatch: YAPP 0x%02h, Channel 0x%02h",
                                            i, yp.m_payload[i], cp.m_payload[i]))
        ok = 0;
      end
    end
    if (yp.m_parity != cp.m_parity) begin
      `uvm_error("PKT_COMPARE", $sformatf("Parity mismatch: YAPP 0x%02h, Channel 0x%02h",
yp.m_parity, cp.m_parity))
      ok = 0;
    end
    return ok;
  endfunction : comp_equal

  // Optional: the same check with the uvm_comparer policy object
  function bit comp_equal_uvm(input yapp_packet yp, input channel_packet cp);
    uvm_comparer comparer = new();
    bit ok = 1;
    ok &= comparer.compare_field_int("addr",   yp.m_addr,   cp.m_addr,   2);
    ok &= comparer.compare_field_int("length", yp.m_length, cp.m_length, 6);
    ok &= comparer.compare_field_int("parity", yp.m_parity, cp.m_parity, 8);
    if (yp.m_payload.size() == cp.m_payload.size()) begin
      foreach (yp.m_payload[i]) begin
        ok &= comparer.compare_field_int($sformatf("payload[%0d]", i), yp.m_payload[i],
cp.m_payload[i], 8);
      end
    end else begin
      ok = 0;
    end
    if (!ok) begin
      `uvm_error("PKT_COMPARE", "uvm_comparer found differences between the YAPP and Channel
packets")
    end
```

```
    return ok;
  endfunction : comp_equal_uvm
```

What a failure looks like

Run `scoreboard_drop_test` (Lab 9A, no short-packet override, `maxpktsize` = 20). The RTL prints `ROUTER DROPS PACKET`, the scoreboard raises `UVM_ERROR … MISMATCH` for the next packet on that channel, and the report shows packets left in a queue. Then run `router_simple_mcseq_test`: 12 received, 12 matched, 0 left.

Alternatives in this repository

- **Lab 9B** puts a reference model in front of the scoreboard so that dropped packets are never expected.
- **Lab 9D** rewrites the scoreboard around analysis FIFOs and blocking `get()` calls (`router_fifo_scoreboard`).

Common mistakes

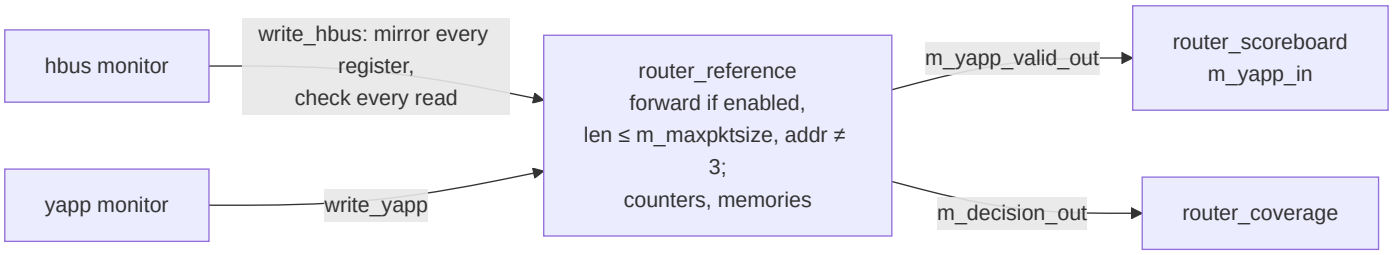
- Storing the handle instead of a clone.
- Comparing `m_packet_delay` or other non-wire fields.
- Declaring the `` `uvm_analysis_imp_decl `` macros inside the class (they define classes; they belong at package scope, once per suffix).
- Reporting only mismatches: a scoreboard that never saw a packet is "green" too — always print the counts.

Reference model and the router module UVC

Open on the project map →

Why a reference model

The scoreboard of Lab 9A expects **every** packet to come out. The router drops packets — when disabled, when oversized, when addressed to 3 — and the rules depend on registers written over the HBUS. Something has to know those rules and the current register values: the **reference model**.



```
//------------------------------------------------------------------------------
// router_reference.sv -- reference model of the router (Lab 9B, grown into a
// full model of the registers for the random tests)
//
//   HBUS monitor ──port──► hbus_in (imp) : writes update the register mirror
//                                          and the scratch memory; reads are
//                                          checked against the model
//                                          (`check_reads`, off by default)
//   YAPP monitor ──port──► yapp_in (imp) : forward the packet on
//                                          yapp_valid_out only if the router
//                                          will route it; update the counters,
//                                          mem_size_reg and yapp_pkt_mem;
//                                          publish the decision on
//                                          decision_out (router_coverage)
//
// The model follows docs/dut/spec.md ("Behaviour rules" and "Decisions"):
//   * only packets received while router_en = 1 count, set mem_size_reg and
//     are stored in yapp_pkt_mem (dropped ones too); router_en = 0 is "deaf"
//   * each counter uses its enable bit in en_reg at the end of the packet and
//     wraps at 255
//   * yapp_pkt_mem[0] = header {length, addr}, [1..length] = payload; a shorter
//     packet leaves the old tail; the memories are not reset, so a byte is
//     only checked after it has been written
//   * unmapped addresses read 0; writes to read-only locations are ignored
// The decision is taken when the monitor publishes the packet (after its
// parity byte); the DUT takes it at the header. They agree as long as the
// configuration does not change in the middle of a packet, which the spec
// leaves undefined and the tests never do.
// A backdoor poke() is not seen here, so tests that poke keep check_reads off.
//
// `_yapp` was declared by router_scoreboard.sv (same package); only `_hbus`
// is new here.
//------------------------------------------------------------------------------
`uvm_analysis_imp_decl(_hbus)

class router_reference extends uvm_component;

  // Inputs
  uvm_analysis_imp_yapp #(yapp_packet,      router_reference) m_yapp_in;
  uvm_analysis_imp_hbus #(hbus_transaction, router_reference) m_hbus_in;
  // Output: packets the router is expected to deliver
  uvm_analysis_port #(yapp_packet) m_yapp_valid_out;
  // Output: every packet received while enabled, with the decision and the
  // configuration it met (router_coverage subscribes)
  uvm_analysis_port #(router_decision) m_decision_out;

  // Register mirror (reset values of the DUT)
  bit[7:0] m_ctrl_reg = 8'h3f;
  bit[7:0] m_en_reg   = 8'h01;
  bit[5:0] m_maxpktsize = 6'h3f;            // = ctrl_reg[5:0]
  bit      m_router_en  = 1'b1;              // = en_reg[0]
  bit[7:0] m_parity_err_cnt;
  bit[7:0] m_oversized_cnt;
  bit[7:0] m_addr_cnt[4];                   // addr0..addr3 counters
  bit[7:0] m_mem_size;
  bit[7:0] m_pkt_mem[64];
  bit      m_pkt_mem_known[64];             // written since time 0
  bit[7:0] m_scratch_mem[256];              // yapp_mem
```

```
  bit       m_scratch_known[256];

  // Knob (uvm_config_db #(bit) "check_reads"): compare every HBUS read with the model
  bit m_check_reads;

  // Statistics
  int m_packets_forwarded;
  int m_dropped_disabled;
  int m_dropped_size;
  int m_dropped_addr;
  int m_reads_checked;
  int m_reads_unknown;                      // memory bytes never written: not checked
  int m_read_mismatches;

  `uvm_component_utils(router_reference)

  extern function new(string name, uvm_component parent);
  extern function void build_phase(uvm_phase phase);

  // Keep the model up to date from the bus traffic; check the reads
  extern function void write_hbus(hbus_transaction tr);

  // Same decision the RTL makes when the header arrives, then the statistics
  extern function void write_yapp(yapp_packet pkt);

  // What a read of `addr` must return; known = 0 for a memory byte never written
  extern function bit[7:0] expected_read(bit[15:0] addr, output bit known);

  extern function void report_phase(uvm_phase phase);

endclass : router_reference

//------------------------------------------------------------------------------
// router_reference -- method implementations
//------------------------------------------------------------------------------

function router_reference::new(string name, uvm_component parent);
  super.new(name, parent);
  m_yapp_in        = new("yapp_in", this);
  m_hbus_in        = new("hbus_in", this);
  m_yapp_valid_out = new("yapp_valid_out", this);
  m_decision_out   = new("decision_out", this);
endfunction : new

//------------------------------------------------------------------------------
function void router_reference::build_phase(uvm_phase phase);
  super.build_phase(phase);
  void'(uvm_config_db #(bit)::get(this, "", "check_reads", m_check_reads));
endfunction : build_phase

//------------------------------------------------------------------------------
function void router_reference::write_hbus(hbus_transaction tr);
  bit[7:0] exp;
  bit       known;
  if (tr.m_hwr_rd == HBUS_OP_READ) begin
    if (!m_check_reads) begin
      return;
    end
    exp = expected_read(tr.m_haddr, known);
```

```
    if (!known) begin
      m_reads_unknown++;
    end else begin
      m_reads_checked++;
      if (tr.m_hdata !== exp) begin
        m_read_mismatches++;
        `uvm_error("REF_READ", $sformatf("HBUS read 0x%04h returned 0x%02h, the model expects
0x%02h",
                                         tr.m_haddr, tr.m_hdata, exp))
      end else begin
        `uvm_info(get_type_name(), $sformatf("HBUS read 0x%04h = 0x%02h as expected", tr.m_haddr,
tr.m_hdata), UVM_HIGH)
      end
    end
    return;
  end
  if (tr.m_haddr >= 16'h1100 && tr.m_haddr <= 16'h11ff) begin
    m_scratch_mem[tr.m_haddr[7:0]]   = tr.m_hdata;
    m_scratch_known[tr.m_haddr[7:0]] = 1'b1;
    return;
  end
  case (tr.m_haddr)
    16'h1000: begin
      m_ctrl_reg   = tr.m_hdata;
      m_maxpktsize = tr.m_hdata[5:0];
      `uvm_info(get_type_name(), $sformatf("maxpktsize is now %0d", m_maxpktsize), UVM_LOW)
    end
    16'h1001: begin
      m_en_reg    = tr.m_hdata;
      m_router_en = tr.m_hdata[0];
      `uvm_info(get_type_name(), $sformatf("router_en is now %0d (en_reg 0x%02h)", m_router_en,
m_en_reg), UVM_LOW)
    end
    default: ;   // read-only or unmapped: the write is ignored
  endcase
endfunction : write_hbus

//------------------------------------------------------------------------------
function void router_reference::write_yapp(yapp_packet pkt);
  bit             bad;
  router_reason_e reason;
  router_decision decision;
  bad = (pkt.m_parity_type == PARITY_TYPE_BAD);
  if (!m_router_en) begin
    m_dropped_disabled++;
    `uvm_info(get_type_name(),
              $sformatf("Packet dropped: router disabled (addr=%0d len=%0d)", pkt.m_addr,
pkt.m_length), UVM_LOW)
    return;                                  // deaf: no count, no memory, no pulse
  end
  if (pkt.m_addr == 2'd3) begin
    reason = ROUTER_REASON_ADDR3;
    m_dropped_addr++;
    `uvm_info(get_type_name(), $sformatf("Packet dropped: illegal address 3 (len=%0d)",
pkt.m_length), UVM_LOW)
  end else if (pkt.m_length > m_maxpktsize) begin
    reason = ROUTER_REASON_OVERSIZED;
    m_dropped_size++;
    `uvm_info(get_type_name(),
```

```
              $sformatf("Packet dropped: length %0d > maxpktsize %0d (addr=%0d)", pkt.m_length,
m_maxpktsize, pkt.m_addr),
              UVM_LOW)
  end else begin
    reason = ROUTER_REASON_FORWARDED;
    m_packets_forwarded++;
    `uvm_info(get_type_name(),
              $sformatf("Packet forwarded: addr=%0d len=%0d -> channel %0d", pkt.m_addr,
pkt.m_length, pkt.m_addr),
              UVM_MEDIUM)
    m_yapp_valid_out.write(pkt);
  end

  // Statistics of a packet received while enabled (forwarded or dropped)
  if (bad && m_en_reg[1]) begin
    m_parity_err_cnt++;
  end
  if (pkt.m_length > m_maxpktsize && m_en_reg[2]) begin
    m_oversized_cnt++;
  end
  if (m_en_reg[4 + pkt.m_addr]) begin
    m_addr_cnt[pkt.m_addr]++;
  end
  m_mem_size = {2'b00, pkt.m_length};
  m_pkt_mem[0]       = {pkt.m_length, pkt.m_addr};
  m_pkt_mem_known[0] = 1'b1;
  for (int i = 0; i < pkt.m_length; i++) begin
    m_pkt_mem[i + 1]       = pkt.m_payload[i];
    m_pkt_mem_known[i + 1] = 1'b1;
  end

  decision            = router_decision::type_id::create("decision");
  decision.m_pkt        = pkt;
  decision.m_maxpktsize = m_maxpktsize;
  decision.m_en_reg     = m_en_reg;
  decision.m_reason     = reason;
  m_decision_out.write(decision);
endfunction : write_yapp

//------------------------------------------------------------------------------
function bit[7:0] router_reference::expected_read(bit[15:0] addr, output bit known);
  known = 1'b1;
  if (addr >= 16'h1100 && addr <= 16'h11ff) begin
    known = m_scratch_known[addr[7:0]];
    return m_scratch_mem[addr[7:0]];
  end
  if (addr >= 16'h1010 && addr <= 16'h104f) begin
    known = m_pkt_mem_known[addr - 16'h1010];
    return m_pkt_mem[addr - 16'h1010];
  end
  case (addr)
    16'h1000: return m_ctrl_reg;
    16'h1001: return m_en_reg;
    16'h1004: return m_parity_err_cnt;
    16'h1005: return m_oversized_cnt;
    16'h1006: return m_addr_cnt[3];
    16'h1009: return m_addr_cnt[0];
    16'h100a: return m_addr_cnt[1];
    16'h100b: return m_addr_cnt[2];
```

```
    16'h100d: return m_mem_size;
    default:  return 8'h00;                // not mapped
  endcase
endfunction : expected_read

//------------------------------------------------------------------------------
function void router_reference::report_phase(uvm_phase phase);
  `uvm_info(get_type_name(), $sformatf({"\n--- Reference model report ---\n",
    "  forwarded to scoreboard   : %0d\n",
    "  dropped (router disabled) : %0d\n",
    "  dropped (oversized)       : %0d\n",
    "  dropped (address 3)       : %0d\n",
    "  counters parity/oversized/addr0..3 : %0d / %0d / %0d %0d %0d %0d"},
    m_packets_forwarded, m_dropped_disabled, m_dropped_size, m_dropped_addr,
    m_parity_err_cnt, m_oversized_cnt, m_addr_cnt[0], m_addr_cnt[1], m_addr_cnt[2],
m_addr_cnt[3]), UVM_LOW)
  if (m_check_reads) begin
    `uvm_info(get_type_name(), $sformatf("HBUS reads checked against the model: %0d (%0d
mismatches, %0d of memory bytes never written)",
                                         m_reads_checked, m_read_mismatches, m_reads_unknown),
UVM_LOW)
  end
endfunction : report_phase
```

In the course the model is tiny: it mirrors two register fields from the bus traffic and re-implements the one decision the RTL makes when a header arrives (Lab 9B). The project's version grew with the test plan and the random tests: it keeps every register, the six counters with their enables and wrap-around, `mem_size`, the packet memory and `yapp_mem`, and, when `check_reads` is set, compares **every HBUS read** with what the model expects (`REF_READ` errors otherwise). The routing decision is the same one; the model must agree with the DUT on the corner cases listed in the specification.

Interface UVC vs module UVC

	Interface UVC (yapp, hbus, channel, clock)	**Module UVC (router)**
talks to	pins, through an interface	transactions, through TLM
contains	driver, sequencer, monitor	reference model, scoreboard, coverage
generates stimulus?	yes	no

	Interface UVC (yapp, hbus, channel, clock)	Module UVC (router)
reusable across	any design with that interface	any testbench of **this** design

`router_module_env` is the module UVC: an `uvm_env` that builds the reference, the scoreboard and the five coverage collectors of the router, connects them, and (Lab 9C) exposes **exports** so the testbench never reaches inside. The reference also publishes every packet it received while enabled, with its decision, on `decision_out`; the `router_coverage` collector subscribes (Functional coverage).

```
//------------------------------------------------------------------------------
// router_module_env.sv -- the router MODULE UVC with exports (Lab 9C)
//
// Every external connection point is now an analysis EXPORT on the env, so
// the testbench never has to know where the imps live inside the module UVC:
//
//   yapp_export ──► reference.yapp_in       chan0_export ──► scoreboard.chan0_in
//   hbus_export ──► reference.hbus_in       chan1_export ──► scoreboard.chan1_in
//                                           chan2_export ──► scoreboard.chan2_in
//   reference.yapp_valid_out ──► scoreboard.yapp_in   (internal, unchanged)
//   reference.decision_out   ──► coverage             (router_coverage)
//   hbus_export              ──► hbus_addr_cov, hbus_bits_cov, hbus_data_cov,
//                                hbus_counter_cov   (the HBUS coverage collectors)
//------------------------------------------------------------------------------
class router_module_env extends uvm_env;

  router_reference  m_reference;
  router_scoreboard m_scoreboard;
  router_coverage   m_coverage;
  // HBUS coverage: one collector per covergroup
  router_hbus_addr_coverage    m_hbus_addr_cov;
  router_hbus_bits_coverage    m_hbus_bits_cov;
  router_hbus_data_coverage    m_hbus_data_cov;
  router_hbus_counter_coverage m_hbus_counter_cov;

  // Top-level connection points
  uvm_analysis_export #(yapp_packet)      m_yapp_export;
  uvm_analysis_export #(hbus_transaction) m_hbus_export;
  uvm_analysis_export #(channel_packet)   m_chan0_export;
  uvm_analysis_export #(channel_packet)   m_chan1_export;
  uvm_analysis_export #(channel_packet)   m_chan2_export;

  `uvm_component_utils(router_module_env)

  extern function new(string name, uvm_component parent);
  extern function void build_phase(uvm_phase phase);

  extern function void connect_phase(uvm_phase phase);

endclass : router_module_env

//------------------------------------------------------------------------------
// router_module_env -- method implementations
//------------------------------------------------------------------------------

function router_module_env::new(string name, uvm_component parent);
  super.new(name, parent);
  m_yapp_export  = new("yapp_export",  this);
  m_hbus_export  = new("hbus_export",  this);
  m_chan0_export = new("chan0_export", this);
  m_chan1_export = new("chan1_export", this);
  m_chan2_export = new("chan2_export", this);
endfunction : new

//------------------------------------------------------------------------------
function void router_module_env::build_phase(uvm_phase phase);
  super.build_phase(phase);
  m_reference  = router_reference::type_id::create("reference", this);
```

```
  m_scoreboard = router_scoreboard::type_id::create("scoreboard", this);
  m_coverage   = router_coverage::type_id::create("coverage", this);
  m_hbus_addr_cov    = router_hbus_addr_coverage::type_id::create("hbus_addr_cov", this);
  m_hbus_bits_cov    = router_hbus_bits_coverage::type_id::create("hbus_bits_cov", this);
  m_hbus_data_cov    = router_hbus_data_coverage::type_id::create("hbus_data_cov", this);
  m_hbus_counter_cov = router_hbus_counter_coverage::type_id::create("hbus_counter_cov", this);
endfunction : build_phase

//------------------------------------------------------------------------------
function void router_module_env::connect_phase(uvm_phase phase);
  // exports -> internal imps
  m_yapp_export.connect(m_reference.m_yapp_in);
  m_hbus_export.connect(m_reference.m_hbus_in);
  m_chan0_export.connect(m_scoreboard.m_chan0_in);
  m_chan1_export.connect(m_scoreboard.m_chan1_in);
  m_chan2_export.connect(m_scoreboard.m_chan2_in);
  // internal link stays as in Lab 9B
  m_reference.m_yapp_valid_out.connect(m_scoreboard.m_yapp_in);
  m_reference.m_decision_out.connect(m_coverage.analysis_export);
  // the HBUS coverage collectors
  m_hbus_export.connect(m_hbus_addr_cov.analysis_export);
  m_hbus_export.connect(m_hbus_bits_cov.analysis_export);
  m_hbus_export.connect(m_hbus_data_cov.analysis_export);
  m_hbus_export.connect(m_hbus_counter_cov.analysis_export);
endfunction : connect_phase
```

```
//------------------------------------------------------------------------------
// router_module_pkg.sv -- router module UVC package (final: Labs 9A-9D)
//
//   router_scoreboard       imp-based scoreboard (9A)
//   router_decision         what the reference decided for one packet
//   router_reference        register-aware packet filter (9B), full model
//   router_coverage         coverage of the configuration the packets met
//   router_hbus_*_coverage  coverage of the HBUS accesses: address map, register
//                           bits, yapp_mem data, counter values
//   router_module_env       env with top-level exports (9B/9C)
//   router_fifo_scoreboard  alternative scoreboard on analysis FIFOs (9D)
//
// Needs the packet types of the interface UVCs, hence the imports.
// Include order: scoreboard first (declares the _yapp/_chanN imps), then the
// decision object, the reference model (declares _hbus), its coverage
// subscriber, then the env that holds them.
//------------------------------------------------------------------------------
package router_module_pkg;

  import uvm_pkg::*;
  import rand_util_pkg::*;   // rnd:: random values (common/rand_util_pkg.sv)
  `include "uvm_macros.svh"
  import yapp_pkg::*;
  import channel_pkg::*;
  import hbus_pkg::*;

  `include "router_defines.sv"   // typedefs and enums of the package

  `include "router_scoreboard.sv"
  `include "router_decision.sv"
  `include "router_reference.sv"
  `include "router_coverage.sv"
  `include "router_hbus_addr_coverage.sv"
  `include "router_hbus_bits_coverage.sv"
  `include "router_hbus_data_coverage.sv"
  `include "router_hbus_counter_coverage.sv"
  `include "router_module_env.sv"
  `include "router_fifo_scoreboard.sv"

endpackage : router_module_pkg
```

The analysis-FIFO variant (Lab 9D)

The same checking written as a **process** instead of callbacks: `router_fifo_scoreboard` pulls packets from `uvm_tlm_analysis_fifo`s with blocking `get()` calls, keeps the register mirror in a parallel thread and checks in `check_phase` that every FIFO is empty.

```
//------------------------------------------------------------------------------
// router_fifo_scoreboard.sv -- scoreboard + reference model built on TLM
// analysis FIFOs and blocking get() calls (Lab 9D)
//
// Instead of reacting inside write() callbacks, the checking runs as a
// PROCESS in run_phase:
//
//   yapp_export ──► yapp_fifo ──get──► [drop?] ──► chan_fifo[addr] ──get──► compare
//   hbus_export ──► hbus_fifo ──get──► mirror maxpktsize / router_en   (parallel thread)
//
// Analysis FIFOs do NOT clone: every monitor must create a new object per
// transaction (ours do), otherwise the FIFO contents get overwritten.
//------------------------------------------------------------------------------
class router_fifo_scoreboard extends uvm_scoreboard;

  // External connection points
  uvm_analysis_export #(yapp_packet)      m_yapp_export;
  uvm_analysis_export #(hbus_transaction) m_hbus_export;
  uvm_analysis_export #(channel_packet)   m_chan_export[3];

  // Storage
  uvm_tlm_analysis_fifo #(yapp_packet)      m_yapp_fifo;
  uvm_tlm_analysis_fifo #(hbus_transaction) m_hbus_fifo;
  uvm_tlm_analysis_fifo #(channel_packet)   m_chan_fifo[3];

  // Blocking readers
  uvm_get_port #(yapp_packet)      m_yapp_get;
  uvm_get_port #(hbus_transaction) m_hbus_get;
  uvm_get_port #(channel_packet)   m_chan_get[3];

  // Register mirror (DUT reset values)
  bit[5:0] m_maxpktsize = 6'h3f;
  bit      m_router_en  = 1'b1;

  // Statistics
  int m_packets_in, m_packets_matched, m_packets_mismatched;
  int m_dropped_disabled, m_dropped_size, m_dropped_addr;

  `uvm_component_utils(router_fifo_scoreboard)

  extern function new(string name, uvm_component parent);

  `include "packet_compare.sv"

  extern function void connect_phase(uvm_phase phase);
  extern task run_phase(uvm_phase phase);

  // Parallel thread: keep the register mirror current
  extern task mirror_registers();

  // Main thread: one YAPP packet at a time, in order
  extern task check_packets();

  // Every FIFO must be empty when the test ends
  extern function void check_phase(uvm_phase phase);
  extern function void report_phase(uvm_phase phase);

endclass : router_fifo_scoreboard
```

```
//------------------------------------------------------------------------------
// router_fifo_scoreboard -- method implementations
//------------------------------------------------------------------------------

function router_fifo_scoreboard::new(string name, uvm_component parent);
  super.new(name, parent);
  m_yapp_export = new("yapp_export", this);
  m_hbus_export = new("hbus_export", this);
  m_yapp_fifo   = new("yapp_fifo", this);
  m_hbus_fifo   = new("hbus_fifo", this);
  m_yapp_get    = new("yapp_get", this);
  m_hbus_get    = new("hbus_get", this);
  foreach (m_chan_fifo[i]) begin
    m_chan_export[i] = new($sformatf("chan%0d_export", i), this);
    m_chan_fifo[i]   = new($sformatf("chan%0d_fifo", i), this);
    m_chan_get[i]    = new($sformatf("chan%0d_get", i), this);
  end
endfunction : new

//------------------------------------------------------------------------------
function void router_fifo_scoreboard::connect_phase(uvm_phase phase);
  // exports -> FIFO analysis exports (monitors write into the FIFOs)
  m_yapp_export.connect(m_yapp_fifo.analysis_export);
  m_hbus_export.connect(m_hbus_fifo.analysis_export);
  // get ports -> FIFO get/peek exports (we read out of the FIFOs)
  m_yapp_get.connect(m_yapp_fifo.get_peek_export);
  m_hbus_get.connect(m_hbus_fifo.get_peek_export);
  foreach (m_chan_fifo[i]) begin
    m_chan_export[i].connect(m_chan_fifo[i].analysis_export);
    m_chan_get[i].connect(m_chan_fifo[i].get_peek_export);
  end
endfunction : connect_phase

//------------------------------------------------------------------------------
task router_fifo_scoreboard::run_phase(uvm_phase phase);
  fork
    mirror_registers();
    check_packets();
  join
endtask : run_phase

//------------------------------------------------------------------------------
task router_fifo_scoreboard::mirror_registers();
  hbus_transaction tr;
  forever begin
    m_hbus_get.get(tr);
    if (tr.m_hwr_rd == HBUS_OP_WRITE) begin
      case (tr.m_haddr)
        16'h1000: m_maxpktsize = tr.m_hdata[5:0];
        16'h1001: m_router_en  = tr.m_hdata[0];
        default: ;
      endcase
    end
  end
endtask : mirror_registers

//------------------------------------------------------------------------------
task router_fifo_scoreboard::check_packets();
```

```
  yapp_packet    yp;
  channel_packet cp;
  forever begin
    m_yapp_get.get(yp);                    // blocks until the monitor delivers one
    m_packets_in++;
    if (!m_router_en) begin
      m_dropped_disabled++;
      `uvm_info(get_type_name(), "Packet dropped: router disabled", UVM_LOW)
    end else if (yp.m_addr == 2'd3) begin
      m_dropped_addr++;
      `uvm_info(get_type_name(), "Packet dropped: illegal address 3", UVM_LOW)
    end else if (yp.m_length > m_maxpktsize) begin
      m_dropped_size++;
      `uvm_info(get_type_name(), $sformatf("Packet dropped: length %0d > maxpktsize %0d",
                                           yp.m_length, m_maxpktsize), UVM_LOW)
    end else begin
      m_chan_get[yp.m_addr].get(cp);         // blocks until the channel delivers it
      if (comp_equal(yp, cp)) begin
        m_packets_matched++;
        `uvm_info(get_type_name(), $sformatf("Channel %0d packet matched", yp.m_addr),
UVM_MEDIUM)
      end else begin
        m_packets_mismatched++;
        `uvm_error(get_type_name(), $sformatf("Channel %0d packet
MISMATCH\nexpected:\n%s\nreceived:\n%s",
                                              yp.m_addr, yp.sprint(), cp.sprint()))
      end
    end
  end
endtask : check_packets

//------------------------------------------------------------------------------
function void router_fifo_scoreboard::check_phase(uvm_phase phase);
  if (m_yapp_fifo.used() != 0) begin
    `uvm_error(get_type_name(), $sformatf("%0d YAPP packets still unchecked",
m_yapp_fifo.used()))
  end
  foreach (m_chan_fifo[i]) begin
    if (m_chan_fifo[i].used() != 0) begin
      `uvm_error(get_type_name(), $sformatf("%0d packets left in channel %0d FIFO",
m_chan_fifo[i].used(), i))
    end
  end
endfunction : check_phase

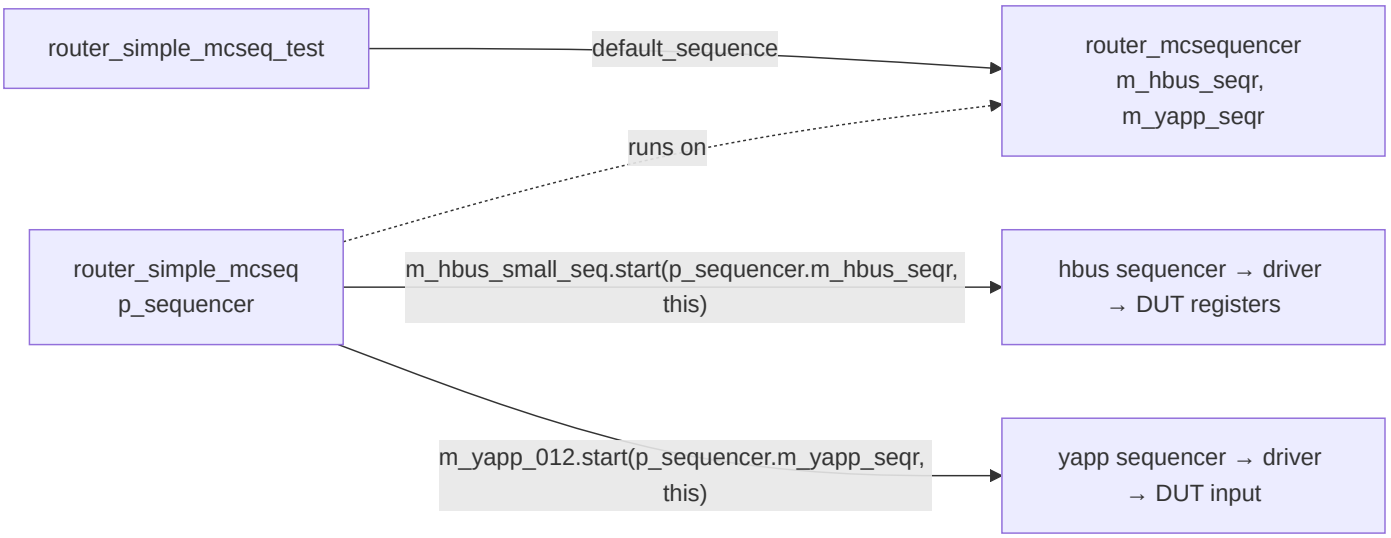
//------------------------------------------------------------------------------
function void router_fifo_scoreboard::report_phase(uvm_phase phase);
  `uvm_info(get_type_name(), $sformatf({"\n--- FIFO scoreboard report ---\n",
    "  packets received  : %0d\n",
    "  packets matched   : %0d\n",
    "  packets mismatched: %0d\n",
    "  dropped (disabled/oversized/addr3): %0d / %0d / %0d"},
    m_packets_in, m_packets_matched, m_packets_mismatched,
    m_dropped_disabled, m_dropped_size, m_dropped_addr), UVM_LOW)
endfunction : report_phase
```

	imp / callback style (9A–9C)	FIFO / process style (9D)
reacts	immediately, in the monitor's thread	whenever the checking process gets to it
can block / wait	no (functions)	yes (tasks)
storage	your own queues	the FIFOs
cloning	you clone	FIFOs store handles: monitors must create new objects
end-of-test check	queue sizes in `report_phase`	`fifo.used()` in `check_phase`

Virtual (multichannel) sequencer

Open on the project map →

A system-level test needs to say "program the router over the HBUS, **then** send YAPP traffic, **then** reprogram it". No single agent can express that ordering. A **virtual sequencer** is a sequencer that drives no interface: it only holds handles to the real sequencers, and a **virtual sequence** running on it starts sub-sequences on those handles.



The sequencer

```
//------------------------------------------------------------------------------
// router_mcsequencer.sv -- multichannel (virtual) sequencer (Lab 8)
//
// It drives no interface itself: it only holds handles to the sequencers it
// coordinates. A multichannel sequence runs on it and starts sub-sequences on
// those handles. The Channel UVCs run a fixed response sequence and the
// Clock & Reset UVC is left out to keep things simple.
//------------------------------------------------------------------------------
class router_mcsequencer extends uvm_sequencer;

  hbus_master_sequencer m_hbus_seqr;   // assigned by router_tb in connect_phase
  yapp_tx_sequencer     m_yapp_seqr;

  `uvm_component_utils(router_mcsequencer)

  extern function new(string name, uvm_component parent);

endclass : router_mcsequencer

//------------------------------------------------------------------------------
// router_mcsequencer -- method implementations
//------------------------------------------------------------------------------

function router_mcsequencer::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new
```

The handles are assigned in `router_tb.connect_phase` with **hierarchical references** (no config strings, no wildcards):

```
m_mcseqr.m_hbus_seqr = m_hbus.m_masters[0].m_sequencer;
m_mcseqr.m_yapp_seqr = m_yapp.m_agent.m_sequencer;
```

The sequence

```
//------------------------------------------------------------------------------
// router_mcseqs_lib.sv -- multichannel sequence library (Lab 8)
//
// `uvm_declare_p_sequencer gives every sequence a typed `p_sequencer` handle,
// so `p_sequencer.hbus_seqr` / `p_sequencer.yapp_seqr` are available.
// seq.start(p_sequencer.<seqr>, this) runs a sub-sequence on another sequencer.
//
// One class per file: the classes live in mcseqs/<class>.sv and are included below.
//------------------------------------------------------------------------------

`include "mcseqs/router_mcseq_base.sv"
`include "mcseqs/router_simple_mcseq.sv"
`include "mcseqs/router_random_mcseq.sv"
`include "mcseqs/router_hbus_sweep_mcseq.sv"

//------------------------------------------------------------------------------
// router_mcseq_base.sv -- router_mcseq_base (multichannel sequence library (Lab 8))
// Split out of router_mcseqs_lib.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// router_mcseq_base: objection on the starting phase + p_sequencer, and the
// helpers the virtual sequences share: one HBUS write or read on the HBUS
// sequencer, and a [TEST] STAGE line
//------------------------------------------------------------------------------
class router_mcseq_base extends uvm_sequence;

  int m_num_reads;                          // HBUS reads done through hbus_read()

  `uvm_object_utils(router_mcseq_base)
  `uvm_declare_p_sequencer(router_mcsequencer)

  extern function new(string name = "router_mcseq_base");
  extern task pre_body();

  extern task post_body();

  extern task hbus_write(bit[15:0] addr, bit[7:0] data);
  extern task hbus_read(bit[15:0] addr, output bit[7:0] data);

  // A [TEST] STAGE line, as base_test::stage() prints it
  extern function void stage(string what);

endclass : router_mcseq_base

//------------------------------------------------------------------------------
// router_mcseq_base -- method implementations
//------------------------------------------------------------------------------

function router_mcseq_base::new(string name = "router_mcseq_base");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task router_mcseq_base::pre_body();
  uvm_phase phase = `YAPP_STARTING_PHASE;
  if (phase != null) begin
    phase.raise_objection(this, get_type_name());
```

```
  end
endtask : pre_body

//------------------------------------------------------------------------------
task router_mcseq_base::post_body();
  uvm_phase phase = `YAPP_STARTING_PHASE;
  if (phase != null) begin
    phase.drop_objection(this, get_type_name());
  end
endtask : post_body

//------------------------------------------------------------------------------
task router_mcseq_base::hbus_write(bit[15:0] addr, bit[7:0] data);
  hbus_write_seq wr;
  wr = hbus_write_seq::type_id::create("wr");
  if (!wr.randomize() with { m_addr == local::addr; m_data == local::data; }) begin
    `uvm_error(get_type_name(), "hbus_write_seq.randomize() failed")
  end
  wr.start(p_sequencer.m_hbus_seqr, this);
endtask : hbus_write

//------------------------------------------------------------------------------
task router_mcseq_base::hbus_read(bit[15:0] addr, output bit[7:0] data);
  hbus_read_seq rd;
  rd = hbus_read_seq::type_id::create("rd");
  if (!rd.randomize() with { m_addr == local::addr; }) begin
    `uvm_error(get_type_name(), "hbus_read_seq.randomize() failed")
  end
  rd.start(p_sequencer.m_hbus_seqr, this);
  data = rd.m_data;
  m_num_reads++;
endtask : hbus_read

//------------------------------------------------------------------------------
function void router_mcseq_base::stage(string what);
  `uvm_info("TEST", {"STAGE: ", what}, UVM_LOW)
endfunction : stage

//------------------------------------------------------------------------------
// router_simple_mcseq.sv -- router_simple_mcseq (multichannel sequence library (Lab 8))
// Split out of router_mcseqs_lib.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// router_simple_mcseq: program the router over HBUS, then send YAPP traffic
//   1. maxpktsize = 20, router enabled        (hbus_small_packet_seq)
//   2. read MAXPKTSIZE back                    (hbus_read_max_pkt_seq)
//   3. 6 packets to addresses 0,1,2            (yapp_012_seq twice)
//   4. maxpktsize = 63                         (hbus_large_packet_seq)
//   5. read MAXPKTSIZE back                    (hbus_read_max_pkt_seq)
//   6. 6 random packets                        (yapp_rnd_seq, count == 6)
//------------------------------------------------------------------------------
class router_simple_mcseq extends router_mcseq_base;

  hbus_small_packet_seq m_hbus_small_seq;
  hbus_large_packet_seq m_hbus_large_seq;
  hbus_read_max_pkt_seq m_hbus_read_seq;
  yapp_012_seq          m_yapp_012;
  yapp_rnd_seq          m_yapp_rnd;
```

```
  `uvm_object_utils(router_simple_mcseq)

  extern function new(string name = "router_simple_mcseq");
  extern task body();

endclass : router_simple_mcseq

//------------------------------------------------------------------------------
// router_simple_mcseq -- method implementations
//------------------------------------------------------------------------------

function router_simple_mcseq::new(string name = "router_simple_mcseq");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task router_simple_mcseq::body();
  `uvm_info(get_type_name(), "Executing router_simple_mcseq sequence", UVM_LOW)

  // Small packets allowed, router enabled -- and check the register
  m_hbus_small_seq = hbus_small_packet_seq::type_id::create("hbus_small_seq");
  if (!m_hbus_small_seq.randomize()) begin
    `uvm_error(get_type_name(), "hbus_small_seq.randomize() failed")
  end
  m_hbus_small_seq.start(p_sequencer.m_hbus_seqr, this);

  m_hbus_read_seq = hbus_read_max_pkt_seq::type_id::create("hbus_read_seq");
  if (!m_hbus_read_seq.randomize()) begin
    `uvm_error(get_type_name(), "hbus_read_seq.randomize() failed")
  end
  m_hbus_read_seq.start(p_sequencer.m_hbus_seqr, this);

  // Six packets to channels 0, 1, 2
  repeat (2) begin
    m_yapp_012 = yapp_012_seq::type_id::create("yapp_012");
    if (!m_yapp_012.randomize()) begin
      `uvm_error(get_type_name(), "yapp_012.randomize() failed")
    end
    m_yapp_012.start(p_sequencer.m_yapp_seqr, this);
  end

  // Large packets allowed -- and check the register
  m_hbus_large_seq = hbus_large_packet_seq::type_id::create("hbus_large_seq");
  if (!m_hbus_large_seq.randomize()) begin
    `uvm_error(get_type_name(), "hbus_large_seq.randomize() failed")
  end
  m_hbus_large_seq.start(p_sequencer.m_hbus_seqr, this);

  m_hbus_read_seq = hbus_read_max_pkt_seq::type_id::create("hbus_read_seq");
  if (!m_hbus_read_seq.randomize()) begin
    `uvm_error(get_type_name(), "hbus_read_seq.randomize() failed")
  end
  m_hbus_read_seq.start(p_sequencer.m_hbus_seqr, this);

  // Six random packets
  m_yapp_rnd = yapp_rnd_seq::type_id::create("yapp_rnd");
  if (!m_yapp_rnd.randomize() with { m_yapp_rnd.m_count == 6; }) begin
    `uvm_error(get_type_name(), "yapp_rnd.randomize() failed")
```

```
  end
  m_yapp_rnd.start(p_sequencer.m_yapp_seqr, this);
endtask : body
```

- `` `uvm_declare_p_sequencer(router_mcsequencer) `` adds a typed `p_sequencer` handle (and a cast check when the sequence starts).
- Each sub-sequence is created through the factory, randomized (the one constraint in this sequence is `m_yapp_rnd.m_count == 6`) and started with `seq.start(p_sequencer.m_hbus_seqr, this)` or `seq.start(p_sequencer.m_yapp_seqr, this)`: the first argument picks the **real** sequencer, the second makes this sequence the parent. The course writes `` `uvm_do_on(seq, seqr) `` / `` `uvm_do_on_with `` for the same thing.
- `start()` blocks until the sub-sequence's `body()` returns, which is what gives the "program, then send, then reprogram" ordering.
- The objection is raised on the **virtual** sequence's starting phase; the sub-sequences see `null` and skip theirs.
- The channels are not controlled from here: they run `channel_rx_resp_seq` forever from the test. The Clock & Reset UVC is left out for simplicity.

The test

```
function void configure_sequences();
  set_clock_and_channel_sequences();
  uvm_config_wrapper::set(this, "tb.mcseqr.run_phase",
                          "default_sequence", router_simple_mcseq::get_type());
  // NO default sequence on the YAPP or HBUS sequencers: the mcseq owns them
endfunction
```

Reading the log

The multichannel sequence is easy to follow in the log: HBUS `WRITE addr=0x1000 data=0x14`, `WRITE addr=0x1001 data=0x01`, `READ addr=0x1000 data=0x14`, six YAPP packets to channels 0/1/2 with their channel monitor counterparts, `WRITE addr=0x1000 data=0x3f`, a read-back, six random packets. If the interfaces overlap confusingly, insert `#100ns;` between the steps of `body()`.

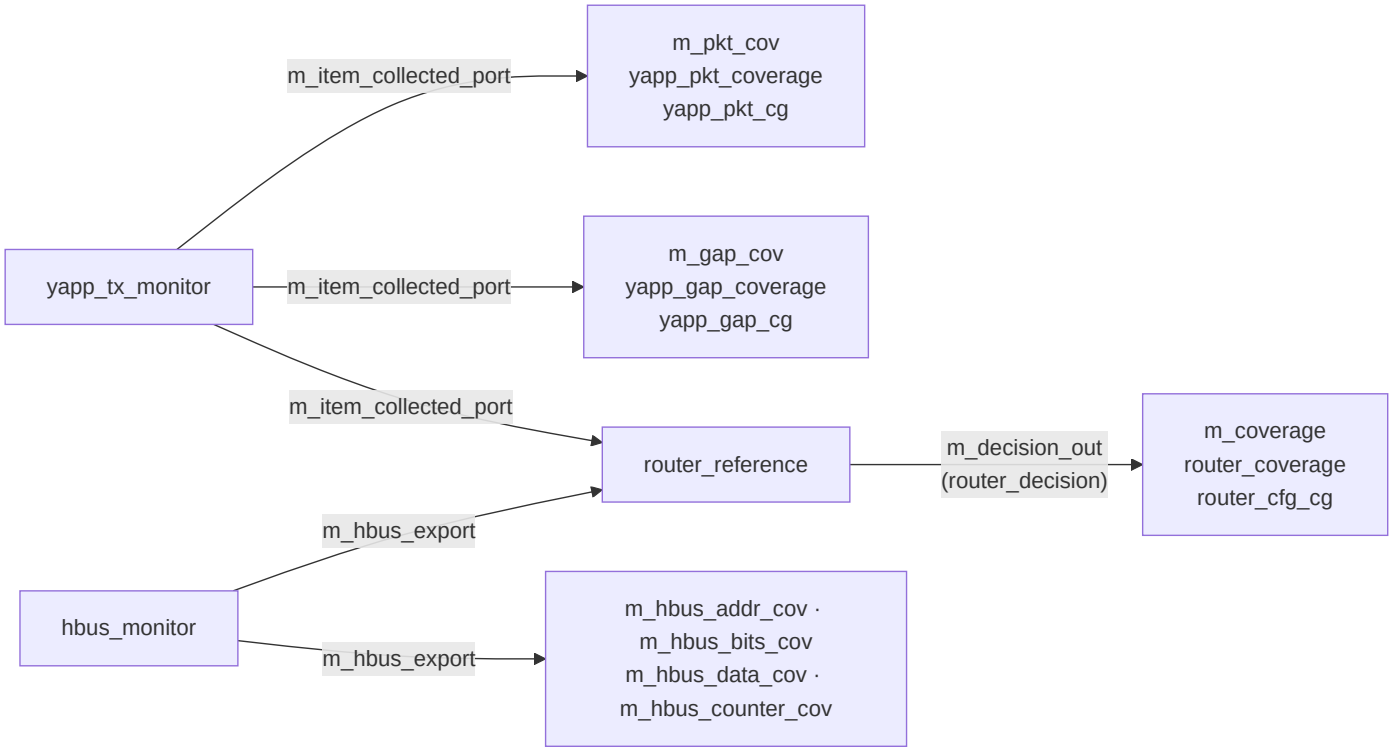
Functional coverage

Open on the project map →

Functional coverage answers "**did we exercise what we meant to?**" — not whether the DUT was right (that is the scoreboard's job). It samples what the **monitors** publish (the YAPP packets, the HBUS cycles) and what the reference model decided, because they see what *actually* happened on the bus, whatever the sequences intended. The concepts (covergroups, bins, crosses, the coverage-driven loop) are on Functional coverage in the UVM chapter; this page is about the collectors of this project.

Where the covergroups live

Every covergroup has **its own file**: a *coverage collector*, a `uvm_subscriber` that holds one covergroup and samples it in `write()`. The monitor and the reference model only publish on an analysis port; the collectors subscribe.



Collector (instance)	File
`tb.yapp.agent.pkt_cov`	`yapp_project/uvc/yapp/yapp_pkt_coverage.sv`

Collector (instance)	File
`tb.yapp.agent.gap_cov`	`yapp_project/uvc/yapp/yapp_gap_coverage.sv`
`tb.router_module.coverage`	`yapp_project/uvc/router/router_coverage.sv`
`tb.router_module.hbus_addr_cov`	`yapp_project/uvc/router/router_hbus_addr_cover`
`tb.router_module.hbus_bits_cov`	`yapp_project/uvc/router/router_hbus_bits_cover`
`tb.router_module.hbus_data_cov`	`yapp_project/uvc/router/router_hbus_data_cover`

Collector (instance)	File
`tb.router_module.hbus_counter_cov`	`yapp_project/uvc/router/router_hbus_counter_co`

Why not inside the monitor, as the Cadence lab writes it? The monitor's job is to observe and publish. A separate collector can be added, replaced (factory) or left out without touching the monitor, a passive agent keeps it, and a collector can sample what *another* component decided (the reference model's drop reason), which the monitor never sees. The bins are the same, so the percentage is the same.

HBUS coverage

The four HBUS collectors subscribe to `router_module_env.m_hbus_export`, next to the reference model. They live in the router module UVC, not in the HBUS UVC: the HBUS UVC is generic, the address map belongs to the router.

Covergroup	Coverpoints	Bins
`hbus_addr_cg`	`addr_cp`: the 9 registers, the 7 holes between them (`reg_hole[]`: 0x1002, 0x1003, 0x1007, 0x1008, 0x100c, 0x100e, 0x100f), first / middle / last byte of `yapp_pkt_mem` (0x1010..0x104f), of the unmapped range 0x1050..0x10ff and of `yapp_mem` (0x1100..0x11ff); `rw_cp`: read, write	25 + 2, cross `addr_x_rw` 50
`hbus_bits_cg`	sampled once per bit of every write to `ctrl_reg` / `en_reg`: `reg_cp` (ctrl, en) × `bit_cp` (`b[0..7]`) × `val_cp` (0, 1)	cross 32
`hbus_data_cg`	accesses to `yapp_mem`: `rw_cp` × `pattern_cp` (0x00, 0xff, 0x55, 0xaa, other)	cross 10
`hbus_counter_cg`	reads of a counter: `counter_cp` (parity, oversized, addr0..addr3) × `value_cp` (0,	cross 18

Covergroup	Coverpoints	Bins
	1..254, 255)	

`hbus_sweep_test` (`router_hbus_sweep_mcseq`) closes them: it writes and reads every address of the window, reads the whole register window back after each write to a register, a hole or a read-only byte (so a write that lands on the wrong register is caught), walks the bits of `ctrl_reg` / `en_reg`, fills `yapp_mem` with the four patterns and a random value, and sends 1020 two-byte bad-parity packets (oversized at `m_maxpktsize = 1`, so dropped) to take every counter to 255. Every read is checked by the reference model (`m_check_reads = 1`), and the test fails unless the four covergroups reach 100 %. Making the RTL write `ctrl_reg` on a write to the hole 0x1002 makes it fail with `HBUS read 0x1000 returned 0x00, the model expects 0xaa`.

The requirements (Lab 10)

Req	Statement	Coverpoint / cross
REQ1	every packet length, bucketed	`length_cp`: `MIN=1`, `SMALL=[2:10]`, `MEDIUM=[11:40]`, `LARGE=[41:62]`, `MAX=63`
REQ2	every address, including the illegal one	`addr_cp`: `legal[] = {[0:2]}`, `illegal_addr = {3}`
REQ3	every length bucket to every legal address **with** a parity error	`len_x_addr_x_parity` cross, ignoring `PARITY_TYPE_GOOD` and `illegal_addr`

The covergroup

```
covergroup yapp_pkt_cg with function sample(bit[5:0] length,
                                            bit[1:0] addr,
                                            parity_type_e parity_type);
  option.per_instance = 1;
  length_cp : coverpoint length {
    bins MIN    = {1};
    bins SMALL  = {[2:10]};
    bins MEDIUM = {[11:40]};
    bins LARGE  = {[41:62]};
    bins MAX    = {63};
  }
  addr_cp : coverpoint addr {
    bins legal[]      = {[0:2]};
    bins illegal_addr = {3};
  }
  parity_cp : coverpoint parity_type {
    bins good = {PARITY_TYPE_GOOD};
    bins bad  = {PARITY_TYPE_BAD};
  }
  len_x_addr_x_parity : cross length_cp, addr_cp, parity_cp {
    ignore_bins good_parity  = binsof(parity_cp) intersect {PARITY_TYPE_GOOD};
    ignore_bins illegal_addr = binsof(addr_cp.illegal_addr);
  }
endgroup
```

- Inside a class a covergroup is **created** in the constructor — `yapp_pkt_cg = new();` — no separate variable.
- `with function sample(...)` lets the collector pass the values explicitly; the alternative is coverpoints on class members sampled with `sample()`.
- `illegal_addr` is a normal bin, not `illegal_bins`: REQ2 wants to *count* packets to address 3, not flag them as errors.
- The cross is **restricted** with `ignore_bins` so that only the combinations REQ3 cares about count towards 100%: 5 lengths × 3 addresses × PARITY_TYPE_BAD = 15 bins.
- `parity_cp` names its two bins. An enum coverpoint without bins gets one bin per value on Xcelium, but Verilator bins it by value range.
- Verilator ignores `ignore_bins` written with `binsof() intersect`, so under `` `ifdef VERILATOR `` the collector builds the same 15-bin cross from two extra coverpoints that hold only the legal addresses and only `PARITY_TYPE_BAD` -- see Simulating with Verilator.

Sampling

`yapp_pkt_coverage::write(t)` calls `yapp_pkt_cg.sample(t.length, t.addr, t.parity_type)` for every packet the monitor publishes:

```
//------------------------------------------------------------------------------
// yapp_pkt_coverage.sv -- functional coverage of the input packets (Lab 10)
//
//   yapp_tx_monitor.item_collected_port ──► analysis_export (write)
//
// The covergroup lives in its own subscriber, not in the monitor: the monitor
// only observes and publishes, and a coverage model can be added, replaced or
// left out without touching it. The Cadence lab writes the same covergroup
// inside the monitor; the bins, and so the percentage, are the same.
//------------------------------------------------------------------------------
class yapp_pkt_coverage extends uvm_subscriber #(yapp_packet);

  int m_num_sampled;

  //----------------------------------------------------------------------------
  // Functional coverage (Lab 10)
  //   REQ1 all packet lengths, bucketed
  //   REQ2 all addresses, including the illegal one
  //   REQ3 every length bucket to every legal address with a parity error
  //----------------------------------------------------------------------------
  covergroup yapp_pkt_cg with function sample(bit[5:0] length,
                                              bit[1:0] addr,
                                              parity_type_e parity_type);
    option.per_instance = 1;
    length_cp : coverpoint length {
      bins MIN    = {1};
      bins SMALL  = {[2:10]};
      bins MEDIUM = {[11:40]};
      bins LARGE  = {[41:62]};
      bins MAX    = {63};
    }
    addr_cp : coverpoint addr {
      bins legal[]      = {[0:2]};
      bins illegal_addr = {3};
    }
    // one bin per enum value, written out (Verilator 5.052 bins an enum by range)
    parity_cp : coverpoint parity_type {
      bins good = {PARITY_TYPE_GOOD};
      bins bad  = {PARITY_TYPE_BAD};
    }
    // REQ3: only legal addresses and only PARITY_TYPE_BAD are interesting
`ifdef VERILATOR
    // (Verilator 5.052 drops ignore_bins with binsof/intersect, so the cross is
    // built from coverpoints that hold only the interesting bins: same 15 bins)
    legal_addr_cp : coverpoint addr { bins legal[] = {[0:2]}; }
    bad_parity_cp : coverpoint parity_type { bins bad = {PARITY_TYPE_BAD}; }
    len_x_addr_x_parity : cross length_cp, legal_addr_cp, bad_parity_cp;
`else
    len_x_addr_x_parity : cross length_cp, addr_cp, parity_cp {
      ignore_bins good_parity  = binsof(parity_cp) intersect {PARITY_TYPE_GOOD};
      ignore_bins illegal_addr = binsof(addr_cp.illegal_addr);
    }
`endif
  endgroup : yapp_pkt_cg

  `uvm_component_utils(yapp_pkt_coverage)

  extern function new(string name, uvm_component parent);
```

```
  // One packet from the monitor
  extern function void write(yapp_packet t);

  extern function void report_phase(uvm_phase phase);

endclass : yapp_pkt_coverage

//------------------------------------------------------------------------------
// yapp_pkt_coverage -- method implementations
//------------------------------------------------------------------------------

function yapp_pkt_coverage::new(string name, uvm_component parent);
  super.new(name, parent);
  yapp_pkt_cg = new();   // a covergroup inside a class is created with new()
endfunction : new

//------------------------------------------------------------------------------
function void yapp_pkt_coverage::write(yapp_packet t);
  m_num_sampled++;
  yapp_pkt_cg.sample(t.m_length, t.m_addr, t.m_parity_type);
endfunction : write

//------------------------------------------------------------------------------
function void yapp_pkt_coverage::report_phase(uvm_phase phase);
  `uvm_info(get_type_name(), $sformatf("YAPP packet coverage: %0d packets sampled, coverage
%.1f%%",
                                       m_num_sampled, yapp_pkt_cg.get_inst_coverage()), UVM_LOW)
endfunction : report_phase
```

The agent builds the collectors and connects them (`yapp_tx_agent`):

```
m_monitor.m_item_collected_port.connect(m_pkt_cov.analysis_export);
m_monitor.m_item_collected_port.connect(m_gap_cov.analysis_export);
```

Closing the holes

Test	Covers	Misses
`router_simple_mcseq_test` (short packets)	MIN, SMALL, some MEDIUM; addresses 0–2	LARGE, MAX, address 3, most of the cross
`test_uvc_integration` (88 packets, lengths 1–22)	all addresses incl. 3	LARGE, MAX

Test	Covers	Misses
`coverage_test` → `yapp_coverage_seq`	**everything**: each length bucket × each address × good/bad parity (40 packets)	—

`yapp_coverage_seq` is the "modify your stimulus to achieve coverage" step of the lab. Run it with several seeds (`+SVSEED=random`) and merge the results in IMC to see 100% on the three requirements.

Seeing the results (Verilator)

Every Verilator run collects functional coverage: `scripts/sim.py` compiles with `--coverage-user` (the covergroup bins only, no line or toggle coverage) and each run writes its bins to `build/sim/<bench>/coverage/<test>_s<seed>.dat`. Three places show them:

1. **The log of one run.** Each collector prints its percentage in `report_phase`:

```
[yapp_pkt_coverage] YAPP packet coverage: 22 packets sampled, coverage 60.0%
[yapp_gap_coverage] YAPP gap coverage: 21 gaps sampled, coverage 100.0%
[router_coverage] Coverage of the router configuration: 65.2%
```

2. **The whole regression, merged.** After `make regress`:

```
make coverage ONLY=yapp_project      # summary and the holes, in the terminal
make sim-report ONLY=yapp_project    # build/sim/report/coverage.md + HTML
```

   `coverage.md` lists, per covergroup, covered / total bins, the **holes** (bins no run hit), every bin with its hits, the number of runs and the tests that hit it, and what each test adds (bins only that test hits). The committed report of the project is `yapp_project/regression/coverage.md`; the same data is the site page Coverage results.

3. **The annotated sources (HTML).** `genhtml` (lcov) draws every collector's source with the hit count next to each bin line: coverage/yapp_project_tb/index.html. *Lines* there means "a bin on this line was hit"; a line with several bins (`legal[]`, a cross) shows each one under *Branches* (`+` hit, `-` hole).

The project map shows the merged bins too: open a coverage collector (`pkt_cov`, `gap_cov`, `coverage`, `hbus_addr_cov`, `hbus_bits_cov`, `hbus_data_cov`, `hbus_counter_cov`) and its panel lists every bin with its hits and the tests that hit it; a hole is red, a click shows its line, and the panel links to the annotated source. The regression results pages say how many bins each test hit.

The percentage is covered bins / bins of the covergroup, as Verilator's `get_inst_coverage()` gives it (the helper coverpoints of the Verilator cross count too); IMC weighs coverpoints differently, see what is not verified.

Running with coverage (Xcelium)

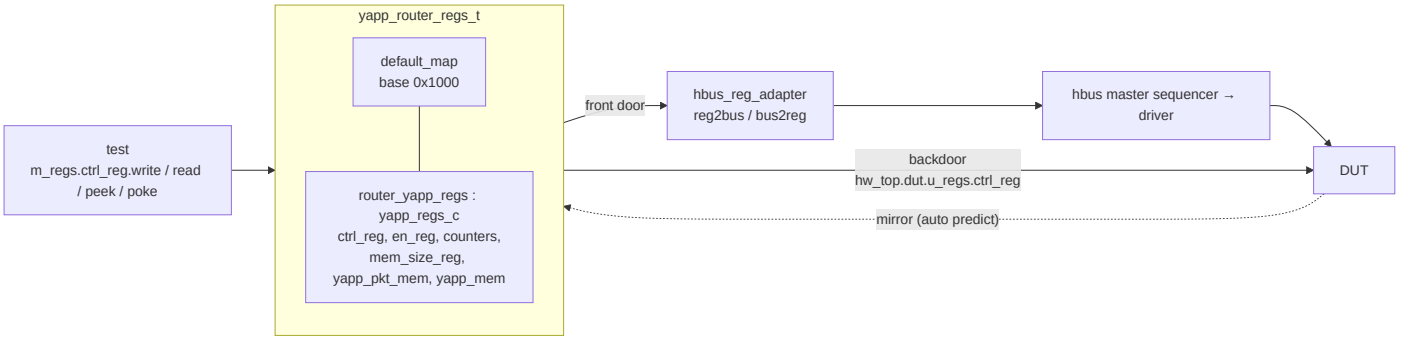
```
cd labs/lab10_cov/tb
make run TEST=coverage_test          # run.f already has -coverage U -covoverwrite
imc -load cov_work/scope/coverage_test
```

In IMC (or SimVision → Windows → Tools → Coverage): *Verification Hierarchy* → *uvm_pkg* → … *yapp_pkt_coverage*, right-click → *Cover Group Analysis*, open the *Items* of `yapp_pkt_cg`.

Register model (RAL)

Open on the project map →

The UVM register layer models the DUT's registers as **objects**: a test writes `regs.ctrl_reg.write(status, 8'h14)` and never spells out an HBUS cycle. The same call can go through the **front door** (real bus transactions, via an adapter and a sequencer) or the **backdoor** (direct access to the RTL variables through their HDL paths).



The model (Lab 11A)

Written by hand here (the course generates it with Cadence `reg_verifier` from an IP-XACT file). Three layers:

Layer	Class	Holds
field	`uvm_reg_field`	bits, access policy (`RW` / `RO`), reset value
register	`ctrl_reg_c` , `en_reg_c` , `*_cnt_reg_c` , `mem_size_reg_c`	fields, `build()`
block	`yapp_regs_c`	registers, memories, a map with offsets and HDL paths
top block	`yapp_router_regs_t`	the sub-block at base `0x1000` , `default_map`

```
//------------------------------------------------------------------------------
// yapp_router_reg_pkg.sv -- UVM register model (RAL) of the YAPP router
//
// In the Cadence flow this file is GENERATED from yapp_router_regs.xml by
// reg_verifier (as yapp_router_regs_rdb.sv). It is written by hand here so the
// course does not depend on that tool; the class names match what the lab
// manual expects:
//
//   yapp_router_regs_t              top block, owns default_map (HBUS, base 0x1000)
//   └── router_yapp_regs            yapp_regs_c
//       ├── ctrl_reg                ctrl_reg_c      0x1000  field plen[5:0] (a.k.a. maxpktsize)
//       ├── en_reg                  en_reg_c        0x1001  8 enable bits
//       ├── parity_err_cnt_reg      ..._c           0x1004  RO
//       ├── oversized_pkt_cnt_reg   ..._c           0x1005  RO
//       ├── addr3_cnt_reg           ..._c           0x1006  RO
//       ├── addr0_cnt_reg           ..._c           0x1009  RO
//       ├── addr1_cnt_reg           ..._c           0x100a  RO
//       ├── addr2_cnt_reg           ..._c           0x100b  RO
//       ├── mem_size_reg            ..._c           0x100d  RO (DUT uses bits [5:0])
//       ├── yapp_pkt_mem            yapp_pkt_mem_c  0x1010  64 x 8  RO
//       └── yapp_mem                yapp_mem_c      0x1100  256 x 8 RW
//
// Backdoor: every register / memory is configured with the HDL name of the
// matching RTL variable, so after
//     yapp_rm.set_hdl_path_root("hw_top.dut.u_regs");
// peek()/poke() resolve to hw_top.dut.u_regs.ctrl_reg, hw_top.dut.u_regs.yapp_mem[i], ...
//------------------------------------------------------------------------------
package yapp_router_reg_pkg;

  import uvm_pkg::*;
  `include "uvm_macros.svh"
  `include "reg/ctrl_reg_c.sv"
  `include "reg/en_reg_c.sv"
  `include "reg/ro_byte_reg_c.sv"
  `include "reg/parity_err_cnt_reg_c.sv"
  `include "reg/oversized_pkt_cnt_reg_c.sv"
  `include "reg/addr3_cnt_reg_c.sv"
  `include "reg/addr0_cnt_reg_c.sv"
  `include "reg/addr1_cnt_reg_c.sv"
  `include "reg/addr2_cnt_reg_c.sv"
  `include "reg/mem_size_reg_c.sv"
  `include "reg/yapp_pkt_mem_c.sv"
  `include "reg/yapp_mem_c.sv"
  `include "reg/yapp_regs_c.sv"
  `include "reg/yapp_router_regs_t.sv"

endpackage : yapp_router_reg_pkg

//------------------------------------------------------------------------------
// ctrl_reg_c.sv -- ctrl_reg_c (register model)
// Split out of yapp_router_reg_pkg.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// ctrl_reg : maximum packet length
//------------------------------------------------------------------------------
class ctrl_reg_c extends uvm_reg;
```

```
  rand uvm_reg_field plen;     // [5:0]  maximum payload length ("maxpktsize")
  rand uvm_reg_field unused;   // [7:6]

  `uvm_object_utils(ctrl_reg_c)

  extern function new(string name = "ctrl_reg_c");
  extern virtual function void build();

endclass : ctrl_reg_c

//------------------------------------------------------------------------------
// ctrl_reg_c -- method implementations
//------------------------------------------------------------------------------

function ctrl_reg_c::new(string name = "ctrl_reg_c");
  super.new(name, 8, UVM_NO_COVERAGE);
endfunction : new

//------------------------------------------------------------------------------
function void ctrl_reg_c::build();
  plen   = uvm_reg_field::type_id::create("plen");
  unused = uvm_reg_field::type_id::create("unused");
  //               parent size lsb access volatile reset  has_reset is_rand
individually_accessible
  plen.configure  (this,  6,   0,  "RW",  0,       6'h3f, 1,        1,      0);
  unused.configure(this,  2,   6,  "RW",  0,       2'h0,  1,        1,      0);
endfunction : build

//------------------------------------------------------------------------------
// en_reg_c.sv -- en_reg_c (register model)
// Split out of yapp_router_reg_pkg.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// en_reg : router enable and counter enables
//------------------------------------------------------------------------------
class en_reg_c extends uvm_reg;

  rand uvm_reg_field router_en;             // [0]
  rand uvm_reg_field parity_err_cnt_en;     // [1]
  rand uvm_reg_field oversized_pkt_cnt_en;  // [2]
  rand uvm_reg_field reserved;              // [3]
  rand uvm_reg_field addr0_cnt_en;          // [4]
  rand uvm_reg_field addr1_cnt_en;          // [5]
  rand uvm_reg_field addr2_cnt_en;          // [6]
  rand uvm_reg_field addr3_cnt_en;          // [7]

  `uvm_object_utils(en_reg_c)

  extern function new(string name = "en_reg_c");
  extern virtual function void build();

endclass : en_reg_c

//------------------------------------------------------------------------------
// en_reg_c -- method implementations
//------------------------------------------------------------------------------

function en_reg_c::new(string name = "en_reg_c");
```

```
  super.new(name, 8, UVM_NO_COVERAGE);
endfunction : new

//------------------------------------------------------------------------------
function void en_reg_c::build();
  router_en            = uvm_reg_field::type_id::create("router_en");
  parity_err_cnt_en    = uvm_reg_field::type_id::create("parity_err_cnt_en");
  oversized_pkt_cnt_en = uvm_reg_field::type_id::create("oversized_pkt_cnt_en");
  reserved             = uvm_reg_field::type_id::create("reserved");
  addr0_cnt_en         = uvm_reg_field::type_id::create("addr0_cnt_en");
  addr1_cnt_en         = uvm_reg_field::type_id::create("addr1_cnt_en");
  addr2_cnt_en         = uvm_reg_field::type_id::create("addr2_cnt_en");
  addr3_cnt_en         = uvm_reg_field::type_id::create("addr3_cnt_en");
  router_en.configure           (this, 1, 0, "RW", 0, 1'b1, 1, 1, 0);
  parity_err_cnt_en.configure   (this, 1, 1, "RW", 0, 1'b0, 1, 1, 0);
  oversized_pkt_cnt_en.configure(this, 1, 2, "RW", 0, 1'b0, 1, 1, 0);
  reserved.configure            (this, 1, 3, "RW", 0, 1'b0, 1, 1, 0);
  addr0_cnt_en.configure        (this, 1, 4, "RW", 0, 1'b0, 1, 1, 0);
  addr1_cnt_en.configure        (this, 1, 5, "RW", 0, 1'b0, 1, 1, 0);
  addr2_cnt_en.configure        (this, 1, 6, "RW", 0, 1'b0, 1, 1, 0);
  addr3_cnt_en.configure        (this, 1, 7, "RW", 0, 1'b0, 1, 1, 0);
endfunction : build

//------------------------------------------------------------------------------
// ro_byte_reg_c.sv -- ro_byte_reg_c (register model)
// Split out of yapp_router_reg_pkg.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------
// Read-only 8-bit counter registers: one small class each, sharing a base
//------------------------------------------------------------------------
virtual class ro_byte_reg_c extends uvm_reg;

  rand uvm_reg_field value;   // [7:0]

  extern function new(string name);
  extern virtual function void build();

endclass : ro_byte_reg_c

//------------------------------------------------------------------------------
// ro_byte_reg_c -- method implementations
//------------------------------------------------------------------------------

function ro_byte_reg_c::new(string name);
  super.new(name, 8, UVM_NO_COVERAGE);
endfunction : new

//------------------------------------------------------------------------------
function void ro_byte_reg_c::build();
  value = uvm_reg_field::type_id::create("value");
  value.configure(this, 8, 0, "RO", 0, 8'h00, 1, 0, 0);
endfunction : build

//------------------------------------------------------------------------------
// parity_err_cnt_reg_c.sv -- parity_err_cnt_reg_c (register model)
// Split out of yapp_router_reg_pkg.sv: one class per file.
//------------------------------------------------------------------------------
```

```
class parity_err_cnt_reg_c extends ro_byte_reg_c;
  `uvm_object_utils(parity_err_cnt_reg_c)
  extern function new(string name = "parity_err_cnt_reg_c");
endclass : parity_err_cnt_reg_c

//------------------------------------------------------------------------------
// parity_err_cnt_reg_c -- method implementations
//------------------------------------------------------------------------------

function parity_err_cnt_reg_c::new(string name = "parity_err_cnt_reg_c"); super.new(name);
endfunction : new

//------------------------------------------------------------------------------
// oversized_pkt_cnt_reg_c.sv -- oversized_pkt_cnt_reg_c (register model)
// Split out of yapp_router_reg_pkg.sv: one class per file.
//------------------------------------------------------------------------------

class oversized_pkt_cnt_reg_c extends ro_byte_reg_c;
  `uvm_object_utils(oversized_pkt_cnt_reg_c)
  extern function new(string name = "oversized_pkt_cnt_reg_c");
endclass : oversized_pkt_cnt_reg_c

//------------------------------------------------------------------------------
// oversized_pkt_cnt_reg_c -- method implementations
//------------------------------------------------------------------------------

function oversized_pkt_cnt_reg_c::new(string name = "oversized_pkt_cnt_reg_c"); super.new(name);
endfunction : new

//------------------------------------------------------------------------------
// addr3_cnt_reg_c.sv -- addr3_cnt_reg_c (register model)
// Split out of yapp_router_reg_pkg.sv: one class per file.
//------------------------------------------------------------------------------

class addr3_cnt_reg_c extends ro_byte_reg_c;
  `uvm_object_utils(addr3_cnt_reg_c)
  extern function new(string name = "addr3_cnt_reg_c");
endclass : addr3_cnt_reg_c

//------------------------------------------------------------------------------
// addr3_cnt_reg_c -- method implementations
//------------------------------------------------------------------------------

function addr3_cnt_reg_c::new(string name = "addr3_cnt_reg_c"); super.new(name);
endfunction : new

//------------------------------------------------------------------------------
// addr0_cnt_reg_c.sv -- addr0_cnt_reg_c (register model)
// Split out of yapp_router_reg_pkg.sv: one class per file.
//------------------------------------------------------------------------------

class addr0_cnt_reg_c extends ro_byte_reg_c;
  `uvm_object_utils(addr0_cnt_reg_c)
  extern function new(string name = "addr0_cnt_reg_c");
endclass : addr0_cnt_reg_c

//------------------------------------------------------------------------------
// addr0_cnt_reg_c -- method implementations
//------------------------------------------------------------------------------
```

```
function addr0_cnt_reg_c::new(string name = "addr0_cnt_reg_c"); super.new(name);
endfunction : new

//------------------------------------------------------------------------------
// addr1_cnt_reg_c.sv -- addr1_cnt_reg_c (register model)
// Split out of yapp_router_reg_pkg.sv: one class per file.
//------------------------------------------------------------------------------

class addr1_cnt_reg_c extends ro_byte_reg_c;
  `uvm_object_utils(addr1_cnt_reg_c)
  extern function new(string name = "addr1_cnt_reg_c");
endclass : addr1_cnt_reg_c

//------------------------------------------------------------------------------
// addr1_cnt_reg_c -- method implementations
//------------------------------------------------------------------------------

function addr1_cnt_reg_c::new(string name = "addr1_cnt_reg_c"); super.new(name);
endfunction : new

//------------------------------------------------------------------------------
// addr2_cnt_reg_c.sv -- addr2_cnt_reg_c (register model)
// Split out of yapp_router_reg_pkg.sv: one class per file.
//------------------------------------------------------------------------------

class addr2_cnt_reg_c extends ro_byte_reg_c;
  `uvm_object_utils(addr2_cnt_reg_c)
  extern function new(string name = "addr2_cnt_reg_c");
endclass : addr2_cnt_reg_c

//------------------------------------------------------------------------------
// addr2_cnt_reg_c -- method implementations
//------------------------------------------------------------------------------

function addr2_cnt_reg_c::new(string name = "addr2_cnt_reg_c"); super.new(name);
endfunction : new

//------------------------------------------------------------------------------
// mem_size_reg_c.sv -- mem_size_reg_c (register model)
// Split out of yapp_router_reg_pkg.sv: one class per file.
//------------------------------------------------------------------------------

// mem_size_reg: length of the last packet. The DUT only implements [5:0];
// the model keeps the full byte like the register map does.
class mem_size_reg_c extends ro_byte_reg_c;
  `uvm_object_utils(mem_size_reg_c)
  extern function new(string name = "mem_size_reg_c");
endclass : mem_size_reg_c

//------------------------------------------------------------------------------
// mem_size_reg_c -- method implementations
//------------------------------------------------------------------------------

function mem_size_reg_c::new(string name = "mem_size_reg_c"); super.new(name);
endfunction : new

//------------------------------------------------------------------------------
// yapp_pkt_mem_c.sv -- yapp_pkt_mem_c (register model)
```

```
// Split out of yapp_router_reg_pkg.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------
// Memories
//------------------------------------------------------------------------
class yapp_pkt_mem_c extends uvm_mem;
  `uvm_object_utils(yapp_pkt_mem_c)
  extern function new(string name = "yapp_pkt_mem_c");
endclass : yapp_pkt_mem_c

//------------------------------------------------------------------------------
// yapp_pkt_mem_c -- method implementations
//------------------------------------------------------------------------------

function yapp_pkt_mem_c::new(string name = "yapp_pkt_mem_c");
  super.new(name, 64, 8, "RO", UVM_NO_COVERAGE);   // bytes of the last packet
endfunction : new

//------------------------------------------------------------------------------
// yapp_mem_c.sv -- yapp_mem_c (register model)
// Split out of yapp_router_reg_pkg.sv: one class per file.
//------------------------------------------------------------------------------

class yapp_mem_c extends uvm_mem;
  `uvm_object_utils(yapp_mem_c)
  extern function new(string name = "yapp_mem_c");
endclass : yapp_mem_c

//------------------------------------------------------------------------------
// yapp_mem_c -- method implementations
//------------------------------------------------------------------------------

function yapp_mem_c::new(string name = "yapp_mem_c");
  super.new(name, 256, 8, "RW", UVM_NO_COVERAGE);  // scratch memory
endfunction : new

//------------------------------------------------------------------------------
// yapp_regs_c.sv -- yapp_regs_c (register model)
// Split out of yapp_router_reg_pkg.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------
// yapp_regs_c : the register block (offsets relative to its base)
//------------------------------------------------------------------------
class yapp_regs_c extends uvm_reg_block;

  rand ctrl_reg_c              ctrl_reg;
  rand en_reg_c                en_reg;
  rand parity_err_cnt_reg_c    parity_err_cnt_reg;
  rand oversized_pkt_cnt_reg_c oversized_pkt_cnt_reg;
  rand addr3_cnt_reg_c         addr3_cnt_reg;
  rand addr0_cnt_reg_c         addr0_cnt_reg;
  rand addr1_cnt_reg_c         addr1_cnt_reg;
  rand addr2_cnt_reg_c         addr2_cnt_reg;
  rand mem_size_reg_c          mem_size_reg;
       yapp_pkt_mem_c          yapp_pkt_mem;
       yapp_mem_c              yapp_mem;
```

```
  `uvm_object_utils(yapp_regs_c)

  extern function new(string name = "yapp_regs_c");
  extern virtual function void build();

endclass : yapp_regs_c

//------------------------------------------------------------------------------
// yapp_regs_c -- method implementations
//------------------------------------------------------------------------------

function yapp_regs_c::new(string name = "yapp_regs_c");
  super.new(name, UVM_NO_COVERAGE);
endfunction : new

//------------------------------------------------------------------------------
function void yapp_regs_c::build();
  // one byte per address, little endian
  default_map = create_map("default_map", 'h0, 1, UVM_LITTLE_ENDIAN, 0);

  ctrl_reg              = ctrl_reg_c::type_id::create("ctrl_reg");
  en_reg                = en_reg_c::type_id::create("en_reg");
  parity_err_cnt_reg    = parity_err_cnt_reg_c::type_id::create("parity_err_cnt_reg");
  oversized_pkt_cnt_reg = oversized_pkt_cnt_reg_c::type_id::create("oversized_pkt_cnt_reg");
  addr3_cnt_reg         = addr3_cnt_reg_c::type_id::create("addr3_cnt_reg");
  addr0_cnt_reg         = addr0_cnt_reg_c::type_id::create("addr0_cnt_reg");
  addr1_cnt_reg         = addr1_cnt_reg_c::type_id::create("addr1_cnt_reg");
  addr2_cnt_reg         = addr2_cnt_reg_c::type_id::create("addr2_cnt_reg");
  mem_size_reg          = mem_size_reg_c::type_id::create("mem_size_reg");
  yapp_pkt_mem          = yapp_pkt_mem_c::type_id::create("yapp_pkt_mem");
  yapp_mem              = yapp_mem_c::type_id::create("yapp_mem");

  // configure(parent block, register file, HDL path of the RTL variable)
  ctrl_reg.configure             (this, null, "ctrl_reg");
  en_reg.configure               (this, null, "en_reg");
  parity_err_cnt_reg.configure   (this, null, "parity_err_cnt_reg");
  oversized_pkt_cnt_reg.configure(this, null, "oversized_pkt_cnt_reg");
  addr3_cnt_reg.configure        (this, null, "addr3_cnt_reg");
  addr0_cnt_reg.configure        (this, null, "addr0_cnt_reg");
  addr1_cnt_reg.configure        (this, null, "addr1_cnt_reg");
  addr2_cnt_reg.configure        (this, null, "addr2_cnt_reg");
  mem_size_reg.configure         (this, null, "mem_size_reg");
  yapp_pkt_mem.configure         (this, "yapp_pkt_mem");
  yapp_mem.configure             (this, "yapp_mem");

  ctrl_reg.build();
  en_reg.build();
  parity_err_cnt_reg.build();
  oversized_pkt_cnt_reg.build();
  addr3_cnt_reg.build();
  addr0_cnt_reg.build();
  addr1_cnt_reg.build();
  addr2_cnt_reg.build();
  mem_size_reg.build();

  default_map.add_reg(ctrl_reg,              'h00, "RW");
  default_map.add_reg(en_reg,                'h01, "RW");
  default_map.add_reg(parity_err_cnt_reg,    'h04, "RO");
  default_map.add_reg(oversized_pkt_cnt_reg, 'h05, "RO");
```

```
  default_map.add_reg(addr3_cnt_reg,          'h06, "RO");
  default_map.add_reg(addr0_cnt_reg,          'h09, "RO");
  default_map.add_reg(addr1_cnt_reg,          'h0a, "RO");
  default_map.add_reg(addr2_cnt_reg,          'h0b, "RO");
  default_map.add_reg(mem_size_reg,           'h0d, "RO");
  default_map.add_mem(yapp_pkt_mem,           'h10,  "RO");
  default_map.add_mem(yapp_mem,               'h100, "RW");
endfunction : build

//------------------------------------------------------------------------------
// yapp_router_regs_t.sv -- yapp_router_regs_t (register model)
// Split out of yapp_router_reg_pkg.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// yapp_router_regs_t : top-level block, places the register block at 0x1000
//------------------------------------------------------------------------------
class yapp_router_regs_t extends uvm_reg_block;

  rand yapp_regs_c router_yapp_regs;

  `uvm_object_utils(yapp_router_regs_t)

  extern function new(string name = "yapp_router_regs_t");
  extern virtual function void build();

endclass : yapp_router_regs_t

//------------------------------------------------------------------------------
// yapp_router_regs_t -- method implementations
//------------------------------------------------------------------------------

function yapp_router_regs_t::new(string name = "yapp_router_regs_t");
  super.new(name, UVM_NO_COVERAGE);
endfunction : new

//------------------------------------------------------------------------------
function void yapp_router_regs_t::build();
  default_map = create_map("default_map", 'h0, 1, UVM_LITTLE_ENDIAN, 0);
  router_yapp_regs = yapp_regs_c::type_id::create("router_yapp_regs");
  router_yapp_regs.configure(this, "");      // no extra HDL hierarchy level
  router_yapp_regs.build();
  default_map.add_submap(router_yapp_regs.default_map, 'h1000);
endfunction : build
```

(The same files, as written in Lab 11A, are in `labs/lab11a_rm_gen/`.)

The adapter (HBUS UVC)

```
//------------------------------------------------------------------------------
// hbus_reg_adapter.sv -- translates register-model operations into HBUS
// transactions and back (Lab 11B)
//
//   reg2bus : uvm_reg_bus_op  -> hbus_transaction  (front-door access)
//   bus2reg : hbus_transaction -> uvm_reg_bus_op   (prediction / read data)
//
// provides_responses = 0 : the driver writes read data straight into the
// request item, so the register map reads it back from the same object.
//------------------------------------------------------------------------------
class hbus_reg_adapter extends uvm_reg_adapter;

  `uvm_object_utils(hbus_reg_adapter)

  extern function new(string name = "hbus_reg_adapter");
  extern virtual function uvm_sequence_item reg2bus(const ref uvm_reg_bus_op rw);

  extern virtual function void bus2reg(uvm_sequence_item bus_item, ref uvm_reg_bus_op rw);

endclass : hbus_reg_adapter

//------------------------------------------------------------------------------
// hbus_reg_adapter -- method implementations
//------------------------------------------------------------------------------

function hbus_reg_adapter::new(string name = "hbus_reg_adapter");
  super.new(name);
  supports_byte_enable = 0;
  provides_responses   = 0;
endfunction : new

//------------------------------------------------------------------------------
function uvm_sequence_item hbus_reg_adapter::reg2bus(const ref uvm_reg_bus_op rw);
  hbus_transaction tr = hbus_transaction::type_id::create("tr");
  tr.m_haddr  = rw.addr[15:0];
  tr.m_hdata  = rw.data[7:0];
  tr.m_hwr_rd = (rw.kind == UVM_WRITE) ? HBUS_OP_WRITE : HBUS_OP_READ;
  return tr;
endfunction : reg2bus

//------------------------------------------------------------------------------
function void hbus_reg_adapter::bus2reg(uvm_sequence_item bus_item, ref uvm_reg_bus_op rw);
  hbus_transaction tr;
  if (!$cast(tr, bus_item)) begin
    `uvm_fatal("NOT_HBUS_TYPE", "Provided bus_item is not of type hbus_transaction")
    return;
  end
  rw.kind   = (tr.m_hwr_rd == HBUS_OP_WRITE) ? UVM_WRITE : UVM_READ;
  rw.addr   = tr.m_haddr;
  rw.data   = tr.m_hdata;
  rw.status = UVM_IS_OK;
endfunction : bus2reg
```

`provides_responses = 0` because the HBUS driver writes the read data straight into the request item: the map reads it back from the same object after `finish_item()`.

Integration (Lab 11B)

```
// router_tb::build_phase
m_yapp_rm = yapp_router_regs_t::type_id::create("yapp_rm");
m_yapp_rm.build();                              // registers, fields, maps
m_yapp_rm.lock_model();                         // freeze and compute addresses
m_yapp_rm.set_hdl_path_root("hw_top.dut.u_regs"); // backdoor root: the register block of the
router
m_yapp_rm.default_map.set_auto_predict(1);      // mirror follows every access
m_reg2hbus = hbus_reg_adapter::type_id::create("reg2hbus");

// router_tb::connect_phase
m_yapp_rm.default_map.set_sequencer(m_hbus.m_masters[0].m_sequencer, m_reg2hbus);
```

The access API (Lab 11C)

Call	Path	Bus traffic	Mirror
`write(status, value)`	front door	yes	updated (auto predict)
`read(status, value)`	front door	yes	updated; **checked** if `set_check_on_read(1)`
`poke(status, value)`	backdoor	no	updated
`peek(status, value)`	backdoor	no	updated
`predict(value)`	—	no	set by hand
`mirror(status, UVM_CHECK)`	front door	yes	compared

Backdoor access needs `-access +rwc` on the `xrun` command line.

Built-in sequences

Sequence	What it does	Expected on the YAPP router
`uvm_reg_hw_reset_seq`	reads every register and compares with the reset value	no errors right after reset
`uvm_mem_walk_seq`	walking-ones through every RW memory	`yapp_mem`: 511 writes, 255 reads; `-define INJECT_ERROR` makes it fail at `0x112a`

Both are started with `seq.model = tb.yapp_rm; seq.start(null);` — the map knows which sequencer to use.

Introspection

```
uvm_reg q[$];
m_tb.m_yapp_rm.get_registers(q);
rw_q = q.find(r) with (r.get_rights() == "RW");
```

`reg_introspection_test` builds RW and RO queues this way and runs the access checks over every register instead of two hand-picked ones.

Things that bite

- Register names in the model **must** equal the RTL variable names for the backdoor to resolve, and the root must be the module that holds them. Every register and memory of the router lives in the `yapp_hbus_regs` instance `u_regs`, so the root is `hw_top.dut.u_regs` and `ctrl_reg` resolves to `hw_top.dut.u_regs.ctrl_reg`. Move a register to another module and the model needs `add_hdl_path_slice` with the relative path, or a new root.
- Reserved / unused bits: `en_reg[3]` and `ctrl_reg[7:6]` are RW in the DUT here, so round trips are clean; on another DUT they might read as 0.
- `mem_size_reg` only receives 6 bits from the packet logic, but `poke` writes all 8 of the variable — expect `0x5a` back, not `0x1a`.
- Writing an RO register through the front door *does* create a bus write. The DUT ignores it, the mirror keeps its value, and the model may warn.

Random values — `rnd::`

Two kinds of randomness live in a UVM testbench:

- **Fields of an object** — a packet's `m_addr`, a sequence's `count`. They are `rand` members with constraints, drawn by `req.randomize() with { ... }`. That stays as it is: the constraints belong to the class, and a factory override can change them.
- **A loose value** — which parity bit to flip, how many bytes to fill, which element of a queue to pick. These come from the common library `common/rand_util_pkg.sv`, never from a bare `$urandom` / `$urandom_range`.

The library is one package with static functions, imported by every UVC package and by `tb_top`, so a component, a sequence or a test just calls it:

```
int unsigned bad_bit;
bad_bit = rnd::get_index(8, {get_full_name(), ".bad_parity_bit"});   // 0..7
m_parity[bad_bit] = ~m_parity[bad_bit];

m_payload = rnd::get_bytes(m_length, {get_full_name(), ".payload"});     // bit[7:0] []
```

The functions

Call	Returns
`rnd::get_bit(name)`	`bit` : 0 or 1
`rnd::get_int(min, max, name)`	`int` in `[min:max]`, both ends included (negative values allowed)
`rnd::get_uint(min, max, name)`	`int unsigned` in `[min:max]`
`rnd::get_byte(name)`	`bit[7:0]` : 0..255
`rnd::get_bits(width, name)`	`bit[63:0]` with `width` (1..64) random low bits, the rest 0
`rnd::get_index(size, name)`	`int unsigned` in `0 .. size-1`, an index into a collection

Call	Returns
`rnd::get_bytes(size, name)`	`rnd_bytes_t` (`bit[7:0] []`): `size` random bytes
`rnd::get_byte_queue(size, name)`	`rnd_byte_q_t` (`bit[7:0] [$]`): `size` random bytes
`rnd_array #(N)::get_bytes(name)`	`bit[7:0] [N]`: a fixed-size array

```
bit[7:0] key[8];
bit[7:0] hdr[$];
key = rnd_array #(8)::get_bytes("key_test.key");
hdr = rnd::get_byte_queue(4, "key_test.header");
```

The fixed-size array has its own parameterized class because a dynamic array cannot be assigned to a fixed-size one in every simulator (Verilator refuses it).

The name argument

The last argument is optional, but give it: many components draw a single bit or a small index, and when a draw fails the name tells which one. `{get_full_name(), ".field"}` gives the instance path for free. A failure stops the simulation with a `UVM_FATAL`:

```
UVM_FATAL rand_util_pkg.sv(98) @ 0: reporter [RND] uvm_test_top.env.delay: get_int(5, 3): min is greater than max
```

Without a name the message says `<unnamed>`.

How it works

Every function draws with `std::randomize()`, so the values follow the simulation seed (`+SVSEED` on Xcelium, `SEED=` with `make sim`) exactly like `$urandom` does. Arrays are sized first (`new[size]`) and randomized afterwards: a size constraint inside `std::randomize() with { q.size() == n; }` is not honoured by every simulator (Verilator 5.052 returns an empty queue).

On Verilator every `std::randomize()` call goes through the constraint solver (z3), which costs a few tenths of a millisecond. A call per packet is free; a loop of a million draws is not, and there `$urandom_range` inside the library would be the place to optimise.

Common mistakes

- **Drawing twice for one decision.** `parity[rnd::get_index(8)] = ~parity[rnd::get_index(8)];` picks two different bits, exactly like the old `$urandom_range` bug. Draw once into a variable.
- **Replacing an object's `randomize()`.** A packet's `m_length` must keep its constraint (`c_length`) and stay overridable (`short_yapp_packet`); it is not a loose value.
- **`min > max`.** `rnd::get_int(hi, lo)` is a fatal error, not an empty range.

Random tests

The directed tests of the test plan each check one rule with values chosen by hand. The random tests check all the rules at once, in combinations nobody chose: the **seed decides the shape of the test** (how many packets, how many bursts, which configuration, how each receiver behaves), not only the values inside it. Nothing in them is checked by hand: the reference model, the scoreboard and the error-pulse checker know the expected result of anything the seed produces.

Test	What it pushes
`random_test`	The balanced mix: 10..100 packets in 2..6 bursts, any configuration, any receiver profile
`random_config_test`	The configuration space: 4..8 short bursts, `maxpktsize` 0 / 1 / 62 / 63 most of the time, the router off now and then, mixed counter-enable masks, lengths at the boundaries, more packets to address 3
`random_backpressure_test`	Back-pressure: slow and stalling receivers on a random mix of channels, long packets (40..63 bytes) sent mostly back to back
`random_parity_test`	Bad parity everywhere: 30..60 % bad packets, on forwarded, dropped (address 3, oversized) and ignored (router off) packets, and storms of 1..3-byte bad packets back to back

The regression runs each of them with 8 seeds (`scripts/regress.yaml`), the directed tests with 4.

The scenario

`router_random_mcseq` (`yapp_project/tb/mcseqs/`) runs on the virtual sequencer, which drives the HBUS and the YAPP input:

```
draw: 10..100 packets, 2..6 bursts (every burst gets at least one packet)
for every burst:
    ctrl_reg = random (maxpktsize 0, 1, 62, 63 with extreme_pct, else 2..61; bits [7:6] random)
    en_reg   = random (router_en = 0 with router_off_pct; counter enables all / none / a random
mask)
    a few random HBUS reads: counters, mem_size_reg, yapp_pkt_mem bytes, unmapped addresses,
                             a yapp_mem write and its read back
    the burst: yapp_random_burst_seq (or a "storm" with storm_pct)
final reads: every register, then yapp_pkt_mem[0..mem_size]
```

The configuration only changes between bursts, when every packet already sent has been accepted by the router. The DUT decides at the header, the reference model when the monitor publishes the packet (after its parity byte); with no change in the middle of a packet they always agree. A change in the middle of a packet is undefined in the specification (DROP-05, excluded).

Each burst prints a `[TEST] STAGE:` line with its configuration and how many packets it sends, and another with what it sent:

```
[TEST] STAGE: random scenario: 22 packets in 6 bursts
[TEST] STAGE: burst 2/6: ctrl=0x12 (maxpktsize 18) en=0xfe (router OFF, counter enables 1111111),
3 packet(s)
[TEST] STAGE: burst 2/6 sent: addr0..3 0/0/3/0, bad parity 0, gap 0 1, length 40..63
[TEST] STAGE: done: 22 packets (7 forwarded, 15 dropped), 0 storm burst(s), 7 scoreboard matches,
78 HBUS reads checked, 2 error pulses for 2 bad packets
```

The knobs

`yapp_random_burst_seq` (YAPP UVC) draws, per packet:

Knob	Default	Meaning
`addr3_pct`	5	packets to the illegal address 3 (the rest to 0..2)
`bad_pct`	15	packets with bad parity (one bit flipped)
`short_pct`	0	1..3-byte packets
`long_pct`	0	40..63-byte packets, longer than a channel FIFO

Knob	Default	Meaning
`boundary_pct`	40	one of 1, maxpktsize-1, maxpktsize, maxpktsize+1, 63
(the rest)		any length 1..63
`gap0_pct` / `gap_short_pct`	25 / 35	gap before the packet: 0 (back to back), 1..3, else 4..20

Every choice comes from `rnd::` and is pinned with `==` in `randomize() with {...}` (the packet's own `c_addr_legal`, `c_parity_dist` and `c_delay` are switched off for that), so the payload stays random and nothing mixes `dist` with `==` (see the Verilator notes in Verilator).

`channel_rx_random_seq` (Channel UVC) gives every receiver a profile; a test sets one sequence object per channel as that channel's `default_sequence`:

Profile	Delay before reading a packet
`CHANNEL_RX_PROFILE_FAST`	0
`CHANNEL_RX_PROFILE_NORMAL`	0..10
`CHANNEL_RX_PROFILE_SLOW`	20..40
`CHANNEL_RX_PROFILE_STALLY`	usually 0..10, `m_stall_pct` (15 %) of the packets 50..150: the channel stops taking packets for a while

The wait is always on the header, before a packet: a receiver never suspends in the middle of one.

A new random test extends `random_test` and overrides two hooks: `configure_scenario()` sets the knobs of `router_random_mcseq` (the burst knobs above, `min_packets`, `max_bursts`, `extreme_pct`, `router_off_pct`, `storm_pct`, ...) and `pick_profile()` chooses the receivers.

The checking

Who	Checks	Where
Scoreboard	every forwarded packet comes out of its channel unchanged and in order; **nothing is left** in the queues at the end (`check_phase`)	`router_scoreboard`
Reference model	its model of `ctrl_reg`, `en_reg`, the six counters (each with its own enable bit, wrapping at 255), `mem_size_reg`, `yapp_pkt_mem` (the header and the payload of the last packet received while enabled, dropped ones too; a shorter packet leaves the old tail) and `yapp_mem`; **every HBUS read** is compared with it	`router_reference`, `m_check_reads = 1`
Error-pulse checker	one `error` pulse per bad-parity packet received while the router is on, 1..10 cycles after its parity byte, overlapping packets included	`error_pulse_checker`

The model follows the rules and the *Decisions* of the specification. The memories are not reset, so a byte the model has not seen written is not checked (the report counts those reads). The directed register tests poke the counters through the backdoor, which the model cannot see, so `check_reads` is off unless a test sets it:

```
uvm_config_db #(bit)::set(this, "tb.router_module.reference", "check_reads", 1'b1);
```

A wrong counter shows up as

```
UVM_ERROR ... [REF_READ] HBUS read 0x100b returned 0x03, the model expects 0x04
```

What the random tests found

The first `random_parity_test` storms found a real bug: a second bad packet that ended before the first `error` pulse re-armed the single timer of `yapp_error_timer`, and the first pulse was lost (one pulse for two packets, against "`error` pulses 1..10 cycles after a packet with bad parity"). The timer now keeps a schedule and gives every bad packet its own pulse (RTL walkthrough). Putting the old timer back makes `random_parity_test` fail with `no error pulse within 10 cycles of the bad packet`.

Coverage

The reference model publishes a `router_decision` per packet received while enabled; `router_coverage` (its own file, a subscriber of `decision_out`) samples `router_cfg_cg` and prints what the test reached. Nothing fails on it, it shows whether a seed hit the corners (`random_config_test`, one seed; only packets received with the router on are counted):

```
Coverage of the router configuration: 78.3%
  maxpktsize 0 / 1 / 2..61 / 62 / 63       : 0 / 12 / 6 / 0 / 7 packets
  length < max / = max / max+1 / > max+1   : 5 / 9 / 0 / 11
  forwarded good / bad                     : 11 / 0
  dropped address 3 good / bad             : 4 / 1
  dropped oversized good / bad             : 8 / 1
  counter enables none / mixed / all       : 13 / 12 / 0
```

The coverage of all the seeds together, with the bins no seed hit, is in Coverage results.

Running one seed again

```
make sim TEST=random_parity_test SEED=5 WAVES=1          # from yapp_project/tb
make regress ONLY=yapp_project WAVES=1 && make sim-report ONLY=yapp_project
```

The same seed gives the same scenario. The committed report (`yapp_project/regression/index.md`) links every run to its waves in Surfer.

Test plan

Open the Test plan view of the project map →

A verification plan links every **feature** of the DUT to the **stimulus** that exercises it, the **checker** that judges it and the **coverage** that proves it happened. This plan follows the pages of the course's DUT description (packet format, input and output protocols, registers, drop rules, counters, memories, host port) and the lab chapters that add requirements of their own (scoreboard, functional coverage, register tests). **12 feature groups, 64 items**: 63 COVERED, 1 EXCLUDED.

Every item says what is driven, what is checked and by which test; a *partial* item is exercised but not fully checked, a *gap* has no test yet (the note says what one would do), an *excluded* item is left out on purpose. The same data drives the **Test plan** view of the project map, where every test card lists the items it verifies.

Features

ROUTE — Routing

One input port, three output channels: the address in the header picks the channel and the packet comes out unchanged. *Project Overview p. 5 -- YAPP Router Description.* Read the specification → On the map →

Item	What is verified, and how	Tests
ROUTE-01 COVERED	**The address selects the output channel** *Stimulus:* Packets to addresses 0, 1 and 2 (yapp_012_seq, random sequences, yapp_boundary_seq). *Check:* Each channel monitor raises UVM_ERROR when a packet with another address appears on its channel; the scoreboard keeps one expected queue per channel. *Coverage:* `addr_cp.legal (yapp_pkt_cg)`	`yapp_012_tes` `simple_test` `router_simpl` `reg_function` `router_filte`
ROUTE-02 COVERED	**Payload and parity arrive unchanged** *Stimulus:* Every packet that is forwarded. *Check:* comp_equal() in the scoreboard: address, length, every payload byte and the parity byte of the channel packet equal the YAPP packet; one UVM_ERROR per difference. *Coverage:* `length_cp`	`router_simpl` `scoreboard_d` `coverage_tes` `backpressure` `router_filte`

Item	What is verified, and how	Tests
ROUTE-03 COVERED	**Order is kept per channel, channels are independent** *Stimulus:* Back-to-back packets to the same channel (0,1,2,0,1,2) and to different channels while the receivers are slow. *Check:* The scoreboard pops its per-channel queue in order: a swapped packet mismatches; the FIFO scoreboard (Lab 9D) blocks on the next packet of that channel.	`router_simpl` `coverage_tes` `router_filte` `backpressure` `random_backp`
ROUTE-04 COVERED	**Packets back to back with the minimum gap** *Stimulus:* yapp_gap_seq: gaps of 0 (the header in the cycle right after the parity byte -- the shortest the DUT accepts), 1, 2 and 3 cycles; m_packet_delay is the number of idle cycles. Random traffic keeps 1..20. *Check:* The YAPP monitor counts the idle cycles before every packet; the test compares them with the gaps the sequence asked for. Scoreboard: every packet delivered. *Coverage:* `gap_cp (yapp_gap_cg): back_to_back, one, two, longer`	`back_to_back` `coverage_tes` `scoreboard_d`

PKT — Packet format

Header {length[7:2], addr[1:0]}, 1..63 payload bytes, one even-parity byte over header and payload; address 3 is illegal. *Project Overview p. 6 -- Packet Data Specification.* Read the specification → On the map →

Item	What is verified, and how	Tests
PKT-01 COVERED	**Header: length in [7:2], address in [1:0]** *Stimulus:* Every packet; the driver packs {length, addr}, the channel monitor unpacks it again. *Check:* The channel monitor's length and address equal the YAPP packet's (scoreboard compare, channel address check); the DUT reads length +	`simple_test` `router_simple_mcseq` `coverage_test`

Item	What is verified, and how	Tests
	2 bytes exactly, so the next header is found where the monitor expects it.	
PKT-02 COVERED	**Every payload length 1..63, including both extremes** *Stimulus:* yapp_coverage_seq sends 1, 5, 20, 50 and 63 bytes; random sequences cover the rest; backpressure_test sends 40..63. *Check:* Scoreboard compare; a wrong length would shift the parity byte and the monitors would disagree. *Coverage:* `length_cp bins MIN, SMALL, MEDIUM, LARGE, MAX`	`coverage_test` `scoreboard_drop_tes` `backpressure_test` `random_test`
PKT-03 COVERED	**Address 3 is illegal** *Stimulus:* Packets with addr == 3 (c_addr_legal switched off): yapp_88_packets_seq, yapp_coverage_seq, yapp_boundary_seq. *Check:* No byte appears on any channel (scoreboard: nothing unexpected), the reference model drops it, addr3_cnt_reg counts it when addr3_cnt_en is set. *Coverage:* `addr_cp.illegal_addr`	`test_uvc_integratio` `coverage_test` `router_filter_test` `random_config_test`
PKT-04 COVERED	**Even bitwise parity over header and payload** *Stimulus:* Every good packet: parity = XOR of the header and all payload bytes (yapp_packet::calc_parity). *Check:* The channel monitor recomputes the parity and tags the packet CHANNEL_PARITY_TYPE_GOOD; the DUT counts nothing in parity_err_cnt_reg; no error pulse. *Coverage:* `parity_cp`	`reg_function_test` `coverage_test` `router_filter_test`

Item	What is verified, and how	Tests
PKT-05 COVERED	**A wrong parity byte is detected -- and the packet still delivered** *Stimulus:* m_parity_type == PARITY_TYPE_BAD flips one bit of the correct parity (yapp_pkt_seq, yapp_coverage_seq, 20 % in yapp_88_packets_seq). *Check:* parity_err_cnt_reg equals the monitor's num_bad_parity; error pulses once per bad packet (error_pulse_checker); the scoreboard still matches the packet (the router forwards it -- see Decisions in dut/spec.md). *Coverage:* `parity_cp, len_x_addr_x_parity (REQ3)`	`reg_function_test` `parity_error_test` `test_uvc_integratio` `random_parity_test`

IN — Input port protocol

Driven on the falling edge, sampled on the rising edge; in_data_vld frames header and payload; in_suspend stalls the sender; error pulses 1..10 cycles after a bad packet. *Project Overview p. 7 -- Input Port Protocol.*
Read the specification → On the map →

Item	What is verified, and how	Tests
IN-01 COVERED	**in_data_vld rises with the header; one byte per falling edge** *Stimulus:* yapp_if.send_to_dut in every packet test. *Check:* DUT, driver and monitor agree on one rule -- a byte is accepted at a rising edge with in_suspend low; a disagreement shows as a parity error or a scoreboard mismatch.	`yapp_012_test` `simple_test` `coverage_test`

Item	What is verified, and how	Tests
IN-02 COVERED	**in_data_vld is low with the parity byte** *Stimulus:* Every packet: the driver drops in_data_vld on the falling edge after the last payload byte. *Check:* The DUT's PARITY state takes the next accepted byte regardless of in_data_vld; the monitor samples the parity the same way; the packet count in the DUT (mem_size_reg, counters) matches.	`simple_test` `reg_function_test` `pkt_mem_test`
IN-03 COVERED	**in_suspend: the input holds while the target FIFO is full** *Stimulus:* Slow receivers (channel_rx_slow_seq, 20..40 cycles) and 40..63-byte packets, so the 16-byte FIFO fills before the receiver starts. *Check:* in_suspend rose at least once; scoreboard 6 matched, 0 mismatched, 0 unexpected; no parity error at the monitor (every byte accepted exactly once). *Note:* Writing this test showed that yapp_if.wait_accept() sampled in_suspend on the falling edge; it now samples the rising edge, like the DUT and the monitor (see components/driver.md).	`backpressure_test` `random_backpressure_test`
IN-04 COVERED	**error asserts 1 to 10 cycles after a bad-parity packet** *Stimulus:* 12 packets with bad parity (parity_error_test); in	`parity_error_test` `test_uvc_integration` `random_parity_test`

Item	What is verified, and how	Tests
	random_parity_test bad packets on forwarded and dropped packets and storms of 1..3-byte bad packets back to back. *Check:* error_pulse_checker: one pulse per bad packet received while enabled, 1..10 cycles after its parity byte; overlapping bad packets each get their own pulse (owed pulses are a queue); a pulse with nothing owed, an owed pulse older than 10 cycles, or one still owed at the end is an error. *Note:* The random storms found that a second bad packet ending before the first pulse re-armed the timer and the first pulse was lost; yapp_error_timer now schedules one pulse per packet (dut/rtl.md).	

OUT — Output port protocol

16-byte FIFO per channel; data_vld_x with the byte, the receiver reads on the falling edge and de-asserts suspend_x; a new byte every rising edge while suspend_x is low. *Project Overview p. 8 -- Output Port Protocol (Channel Ports).* Read the specification → On the map →

Item	What is verified, and how	Tests

OUT-01
COVERED
	data_vld_x, suspend_x and one byte per rising edge
	Stimulus: channel_if.receive_packet in every scoreboard test: the receiver reads length + 2 bytes after it drops suspend_x.
	Check: The channel monitor collects exactly length + 2 bytes (a wrong
		`simple_test`
		`router_simple_mcseq_test`
		`coverage_test`

Item	What is verified, and how	Tests
	handshake would duplicate or skip a byte and the scoreboard would mismatch); the address check never fires.	
OUT-02 COVERED	**The receiver may answer late: the header is held** *Stimulus:* channel_rx_resp_seq: response delay 0..10 cycles per packet; channel_rx_slow_seq: 20..40. *Check:* Scoreboard compare: the header read after the delay is still the header.	`simple_test` `router_simple_mcseq_test` `backpressure_test` `random_backpressure_test`
OUT-03 COVERED	**The FIFO is 16 bytes deep** *Stimulus:* Packets longer than 16 bytes with a receiver that waits 20..40 cycles: the FIFO fills, the router stalls the input. *Check:* in_suspend seen (backpressure_test); no byte lost (scoreboard); the FIFO never overflows (a lost byte would show as a parity error).	`backpressure_test` `scoreboard_drop_test`
OUT-04 COVERED	**suspend_x is re-asserted at the parity byte: the next packet waits** *Stimulus:* Back-to-back packets to the same channel (0,1,2,0,1,2 and the boundary sequence). *Check:* The second packet's header is held in the FIFO until the receiver answers again; the scoreboard matches both in order.	`router_simple_mcseq_test` `router_filter_test` `backpressure_test`

Item	What is verified, and how	Tests
OUT-05 COVERED	**All three channels busy at the same time** *Stimulus:* Random addresses with slow receivers: packets to channels 0, 1 and 2 are in flight together. *Check:* Scoreboard: every channel matched, nothing unexpected.	`backpressure_test` `coverage_test` `random_backpressure_test`

REG — Registers

ctrl_reg (maxpktsize) and en_reg are RW, the six counters and mem_size_reg are RO; reset values 0x3f, 0x01 and 0x00. *Project Overview p. 9, Lab 11 p. 49 -- Packet Router DUT Registers.* Read the specification → On the map →

Item	What is verified, and how	Tests
REG-01 COVERED	**Reset values** *Stimulus:* uvm_reg_hw_reset_seq reads every register right after reset. *Check:* The built-in sequence compares each read with the model's reset value: ctrl_reg 0x3f, en_reg 0x01, counters and mem_size_reg 0x00.	`uvm_reset_test`

Item	What is verified, and how	Tests
REG-02 COVERED	**RW registers: every bit can be written and read back** *Stimulus:* Walking one, walking zero, 0x00, 0xff, 0xaa, 0x55 through ctrl_reg and en_reg (20 patterns each). *Check:* After each write the front-door read and the backdoor peek return the pattern; the reserved bits ctrl_reg[7:6] and en_reg[3] hold their value (they are RW in the router).	`reg_bit_walk_test` `reg_access_test`
REG-03 COVERED	**RO registers ignore writes** *Stimulus:* Front-door writes to every counter and to mem_size_reg (register model) and a raw HBUS write to 0x1004. *Check:* peek after the write shows the poked value, the read shows 0x00 after the raw write.	`reg_access_test` `reg_introspection_test` `reg_bit_walk_test` `hbus_protocol_test`
REG-04 COVERED	**Front door and backdoor see the same flip-flops** *Stimulus:* write → peek and poke → read on a RW register; poke → read on a RO register (-access +rwc). *Check:* The values agree; the HDL paths of the model (hw_top.dut.u_regs.) resolve.	`reg_access_test` `reg_introspection_test`
REG-05 COVERED	**Unmapped addresses read 0x00 and ignore writes** *Stimulus:* Write 0xa5 then read 0x1002, 0x1003, 0x1007, 0x1008, 0x100c, 0x100e, 0x100f and 0x1050 with raw	`hbus_protocol_test` `random_test` `hbus_sweep_test`

Item	What is verified, and how	Tests
	HBUS sequences. *Check:* Every read returns 0x00.	
REG-06 COVERED	**mem_size_reg holds the payload length of the last packet** *Stimulus:* A 20-byte packet, then a 9-byte packet that is dropped as oversized. *Check:* mem_size_reg reads 20, then 9 (only bits [5:0] are used).	`pkt_mem_test` `random_test`
REG-07 COVERED	**Automatic checking on read (check_on_read + predict)** *Stimulus:* The reg_function_test stimulus with set_check_on_read(1); predict() loads the expected counter before each read. *Check:* The map compares every read with the mirror and raises UVM_ERROR on a difference.	`reg_function_check_test`

DROP — Drop rules

The whole packet is dropped when length > maxpktsize, when the router is disabled, or when the address is 3; changing the enables mid-packet is undefined. *Project Overview p. 10 -- maxpktsize, router_en, counters.*
Read the specification → On the map →

Item	What is verified, and how	Tests

DROP-01
COVERED
	length > maxpktsize drops the whole packet
	Stimulus: maxpktsize = 20 with full-size packets (scoreboard_drop_test, test_uvc_integration); maxpktsize = 10 with lengths 9, 10, 11 and 63 (router_filter_test).
		`router_filter_test`
		`scoreboard_drop_test`
		`test_uvc_integration`
		`random_config_test`

Item	What is verified, and how	Tests
	Check: Nothing of the packet appears on the channel: the reference model drops it and the scoreboard sees nothing unexpected; `ROUTER DROPS PACKET ... length > maxpktsize`; length == maxpktsize still passes.	
DROP-02 COVERED	**router_en = 0 drops every packet** *Stimulus:* en_reg = 0xfe, then yapp_012_seq; en_reg = 0xff, yapp_012_seq again. *Check:* Channel monitors collect nothing while disabled, the reference model reports 3 dropped (router disabled), no counter and not mem_size_reg moves; after re-enabling the packets flow and the address counters read 1.	`router_disable_test` `random_config_test`
DROP-03 COVERED	**Address 3 drops the packet** *Stimulus:* As PKT-03. *Check:* Reference model drop, no unexpected channel packet, `ROUTER DROPS PACKET ... illegal address`.	`router_filter_test` `coverage_test` `test_uvc_integratio` `random_config_test`
DROP-04 COVERED	**An oversized packet to address 3 counts in both counters** *Stimulus:* The 11- and 63-byte packets to address 3 of yapp_boundary_seq. *Check:* oversized_pkt_cnt_reg counts them and addr3_cnt_reg counts them (the DUT evaluates the conditions independently -- see Decisions in dut/spec.md).	`router_filter_test` `random_config_test`
DROP-05 EXCLUDED	**Changing router_en or a counter enable in the middle of a packet** *Note:* The specification declares the behaviour	—

Item	What is verified, and how	Tests
	undefined, so no test drives it; every test programs the registers between packets.	

CNT — Counters

Six 8-bit counters, each behind an enable bit of en_reg: parity errors, oversized packets, and one per address including the illegal 3. *Project Overview p. 10 -- The router counters.* Read the specification → On the map →

Item	What is verified, and how	Tests

CNT-01 COVERED	**A counter counts only while its enable bit is set**
	Stimulus: en_reg = 0x01 then 0xff around yapp_012_seq (address counters); 0xff then 0x01 around yapp_boundary_seq (oversized, addr3, addrN); 0x03 then 0x01 around bad packets (parity).
	Check: The counters read 0 (or stay frozen) while disabled and the exact packet count while enabled.
		`reg_function_test`
		`router_filter_test`
		`parity_error_test`
		`random_config_test`

CNT-02 COVERED	**addrN_cnt_reg counts every received packet with address N**
	Stimulus: yapp_boundary_seq: 4 packets per address, half of them dropped as oversized.
	Check: addr0/1/2_cnt_reg read 4 each: dropped packets are counted too (they were received -- see Decisions); yapp_012_seq twice gives 2 each.
		`router_filter_test`
		`reg_function_test`
		`random_config_test`

CNT-03 COVERED	**parity_err_cnt_reg counts the bad-parity packets**
	Stimulus: yapp_012_seq twice (random
		`reg_function_test`
		`parity_error_test`
		`random_parity_test`

Item	What is verified, and how	Tests
	parity) and 8 + 4 deliberately bad packets. *Check:* The register equals the YAPP monitor's num_bad_parity delta, then 8, then still 8 with the enable off.	
CNT-04 COVERED	**oversized_pkt_cnt_reg counts packets longer than maxpktsize** *Stimulus:* 8 oversized packets of yapp_boundary_seq (11 and 63 bytes against maxpktsize 10); none in reg_function_test (maxpktsize 63). *Check:* Reads 8, then 0 in reg_function_test.	`router_filter_test` `reg_function_test` `random_config_test`
CNT-05 COVERED	**addr3_cnt_reg counts packets to address 3** *Stimulus:* 4 packets to address 3 (yapp_boundary_seq); none in yapp_012_seq. *Check:* Reads 4, and 0 in reg_function_test.	`router_filter_test` `reg_function_test` `random_config_test`
CNT-06 COVERED	**Counter overflow after 255 packets** *Stimulus:* 257 one-byte packets to address 0 with every counter enabled. *Check:* addr0_cnt_reg reads 255 after 255 packets, 0 after the 256th (it wraps, it does not saturate), 1 after the 257th. *Note:* The specification is silent; wrapping is the decision recorded in dut/spec.md. Every counter is the same `+ 8'd1` in yapp_hbus_regs, so one counter stands for all six.	`counter_wrap_test` `hbus_sweep_test`

MEM — Memories

yapp_pkt_mem (64 bytes, RO) holds the last packet received; yapp_mem (256 bytes, RW) is scratch memory. *Project Overview p. 10, Lab 11 p. 50 -- Router Memories.* Read the specification → On the map →

Item	What is verified, and how	Tests
MEM-01 COVERED	**yapp_pkt_mem holds the last packet, byte by byte** *Stimulus:* A 20-byte packet to address 1 whose content the sequence keeps in req. *Check:* yapp_pkt_mem[0] == {length, addr}, yapp_pkt_mem[1..20] == payload[0..19], read through the register model.	`pkt_mem_test` `random_test`
MEM-02 COVERED	**yapp_pkt_mem is read-only** *Stimulus:* A raw HBUS write to 0x1010 with the inverted header. *Check:* yapp_pkt_mem[0] still reads the header.	`pkt_mem_test` `hbus_sweep_test`
MEM-03 COVERED	**A dropped packet is still stored** *Stimulus:* maxpktsize = 5 and a 9-byte packet. *Check:* mem_size_reg reads 9 and the bytes are in yapp_pkt_mem: the packet was received, just not forwarded (Decisions in dut/spec.md).	`pkt_mem_test` `random_config_test`
MEM-04 COVERED	**yapp_mem: every byte can be written and read back** *Stimulus:* uvm_mem_walk_seq: writes and reads the 256 bytes with a walking pattern; with +define+INJECT_ERROR the RTL flips bit 3 of address 0x2a. *Check:* The built-in sequence compares; 0 errors without the define, one UVM_ERROR with it (yapp_mem[42]). The regression runs both builds; the second one passes only if the error is reported.	`uvm_mem_walk_test`

HBUS — Host port (HBUS)

One-cycle writes and two-cycle reads driven on the falling edge; hdata is bidirectional and tri-stated by the DUT when hen is low. *Project Overview p. 11 -- Host Interface Port Protocol.* Read the specification → On the map →

Item	What is verified, and how	Tests
HBUS-01 COVERED	**Write: hen and hwr_rd for one cycle, data latched on the rising edge** *Stimulus:* Every register write (hbus_if.hbus_write): raw sequences and the register model's front door. *Check:* The value is read back (hbus_protocol_test, every reg test); the HBUS monitor decodes one WRITE per access.	`hbus_protocol_test` `router_simple_mcseq_test` `reg_access_test`
HBUS-02 COVERED	**Read: haddr sampled in cycle 1, hdata driven by the DUT in cycle 2** *Stimulus:* Every register read (hbus_if.hbus_read); hbus_read_max_pkt_seq after each maxpktsize write. *Check:* The value read equals what was written or poked (20 and 63 in router_simple_mcseq_test, 0x2a in hbus_protocol_test); the monitor decodes one READ.	`hbus_protocol_test` `router_simple_mcseq_test` `reg_access_test`
HBUS-03 COVERED	**hdata is tri-stated when hen drops** *Stimulus:* A read, then two idle cycles. *Check:* hbus_if.hdata_w reads 'z (the master's output enable is off, so only the DUT could still drive it).	`hbus_protocol_test`

Item	What is verified, and how	Tests
HBUS-04 COVERED	**Back-to-back accesses** *Stimulus:* uvm_mem_walk_seq (511 writes and 255 reads with no gap), the 20-pattern walks of reg_bit_walk_test. *Check:* Every read returns the expected value; the monitor count equals the number of accesses.	`uvm_mem_walk_test` `reg_bit_walk_test`
HBUS-05 COVERED	**The register model's front door is HBUS traffic** *Stimulus:* write()/read() through yapp_rm.default_map and the hbus_reg_adapter. *Check:* The HBUS monitor logs the cycles, and the reference model mirrors maxpktsize and router_en from them (the drop rules follow the model's writes in router_disable_test and router_filter_test).	`reg_function_test` `router_disable_test` `router_filter_test`

RND — Random scenarios

Random tests whose seed decides the shape: packet count, bursts, configuration, receiver profiles; every result checked by the reference model, the scoreboard and the error-pulse checker. *Project Overview pp. 5-11 -- every rule at once, in random combinations.* Read the specification → On the map →

Item	What is verified, and how
RND-01 COVERED	**Random configuration between bursts** *Stimulus:* router_random_mcseq: before each burst a random ctrl_reg (maxp random) and en_reg (router off now and then, counter-enable masks all / noı *Check:* router_reference models the counters, mem_size_reg, yapp_pkt_men HBUS read (check_reads = 1): random reads after each configuration, every the end.

Item	What is verified, and how
	Coverage: `router_coverage router_cfg_cg: maxpktsize, length vs maxpktsize, drop`
RND-02 COVERED	**One error pulse per bad packet when bad packets overlap** *Stimulus:* random_parity_test storms: 1..3-byte packets, 90% back to back, h *Check:* error_pulse_checker matches every cycle with error high to the oldes owed at the end.
RND-03 COVERED	**Bad parity on dropped and ignored packets** *Stimulus:* Bad parity on packets to address 3, oversized packets and packets *Check:* Dropped bad packets count in parity_err_cnt_reg (enable bit) and pul counts or pulses (reference model, error_pulse_checker).
RND-04 COVERED	**Receivers that stall, per channel** *Stimulus:* channel_rx_random_seq per channel: fast, normal, slow (20..40) or then), before a packet, never in the middle. *Check:* Scoreboard: every forwarded byte out once and in order; nothing left (check_phase).

COV — Functional coverage

The coverage models: yapp_pkt_coverage (Lab 10: length buckets, every address including the illegal one, every length to every legal address with bad parity) and the HBUS collectors of the router module (address map, register bits, yapp_mem data, counter values). *Lab 10 p. 47 -- REQ1, REQ2, REQ3.* The lab → On the map →

Item	What is verified, and how
COV-01 COVERED	**REQ1: all packet lengths, bucketed** *Stimulus:* yapp_coverage_seq: 1, 5, 20, 50, 63. *Check:* length_cp bins MIN, SMALL, MEDIUM, LARGE, MAX all hit; the pkt_cc *Coverage:* `length_cp`

Item	What is verified, and how
COV-02 COVERED	**REQ2: every address received a packet, including the illegal one** *Stimulus:* yapp_coverage_seq: addresses 0..3. *Check:* addr_cp bins legal[0..2] and illegal_addr hit. *Coverage:* `addr_cp`
COV-03 COVERED	**REQ3: every length bucket to every legal address with a parity error** *Stimulus:* yapp_coverage_seq: each length with PARITY_TYPE_BAD to addre *Check:* The cross len_x_addr_x_parity (good parity and address 3 ignored) re *Coverage:* `len_x_addr_x_parity`
COV-04 COVERED	**HBUS: every address of the router read and written** *Stimulus:* router_hbus_sweep_mcseq: every address 0x1000..0x100f, every b some middle addresses of 0x1050..0x10ff. *Check:* router_reference checks every read (check_reads = 1): read-only and is read back after each write. *Coverage:* `hbus_addr_cg: addr_cp (9 registers, 7 holes, first / middle / last of`
COV-05 COVERED	**HBUS: every bit of ctrl_reg and en_reg written as 0 and as 1** *Stimulus:* Patterns 0x00, 0xff, 0x55, 0xaa and walking one / walking zero on *Check:* Each write read back and compared with the model. *Coverage:* `hbus_bits_cg: reg_cp x bit_cp[0:7] x val_cp`
COV-06 COVERED	**HBUS: data patterns in yapp_mem** *Stimulus:* Every byte of yapp_mem written and read back with 0x00, 0xff, 0x *Check:* Every read equals what was written (reference model). *Coverage:* `hbus_data_cg: rw_cp x pattern_cp`
COV-07 COVERED	**HBUS: every counter read at 0, in between and at 255** *Stimulus:* maxpktsize = 1 and 2-byte bad-parity packets (oversized, dropped test. *Check:* Every counter read compared with the reference model (enables, wra *Coverage:* `hbus_counter_cg: counter_cp x value_cp (0, 1..254, 255)`

TB — Testbench mechanics

What the lab tests prove about the environment itself: topology, factory, configuration, sequences, virtual interfaces, the virtual sequencer and the scoreboard. *Labs 2-9 -- the course's own tests.* The lab → On the map →

Item	What is verified, and how	Tests
TB-01 COVERED	**The test builds the testbench and the run ends cleanly** *Stimulus:* base_test alone: no sequence, drain time 200 ns. *Check:* print_topology shows uvm_test_top with router_tb below; UVM_ERROR : 0; check_config_usage reports nothing unused.	`base_test`
TB-02 COVERED	**A factory type override replaces the packet type everywhere** *Stimulus:* set_type_override_by_type(yapp_packet, short_yapp_packet) before the environment is built. *Check:* Every printed packet is a short_yapp_packet with length < 15; the sequences were not touched.	`short_packet_test` `incr_payload_test` `exhaustive_seq_test` `simple_test` `router_simple_mcseq_test`
TB-03 COVERED	**Configuration through the database, checked for use** *Stimulus:* is_active = UVM_PASSIVE on the YAPP agent; configure_sequences() overridden to set nothing. *Check:* The topology has no driver and no sequencer under the agent; no `never used` configuration report.	`set_config_test`

Item	What is verified, and how	Tests
TB-04 COVERED	**The sequence library: sub-sequences and post_randomize shaping** *Stimulus:* yapp_exhaustive_seq starts every library sequence once; yapp_incr_payload_seq rewrites the payload to 0 1 2 ... *Check:* One `Executing` line per sequence, the expected packet counts, the incrementing payload.	`exhaustive_seq_test` `incr_payload_test`
TB-05 COVERED	**The virtual interface connects the driver to the DUT** *Stimulus:* yapp_012_seq through yapp_if into hw_top.dut. *Check:* Each packet appears on data_0, data_1, data_2 in the waveform; later the channel monitors confirm it.	`yapp_012_test` `simple_test`
TB-06 COVERED	**A multichannel sequence coordinates HBUS and YAPP** *Stimulus:* router_simple_mcseq on the virtual sequencer: program, read back, send, reprogram, send. *Check:* The log shows the sub-sequences in order; the read-backs return 20 and 63.	`router_simple_mcseq_test`
TB-07 COVERED	**The scoreboard notices a wrong packet** *Stimulus:* Full-size packets while maxpktsize is 20, without the reference model (Lab 9A). *Check:* Expected PKT_COMPARE errors	`scoreboard_drop_test`

Item	What is verified, and how	Tests
	and packets left in the queues: the negative test of the checker itself.	
TB-08 COVERED	**The register model describes itself** *Stimulus:* get_registers() and find() by get_rights(). *Check:* 2 RW and 7 RO registers listed, then the access checks run on all of them.	`reg_introspection_test`

Coverage matrix

Who verifies what, at a glance: one row per plan item, one column per test (in course order, the test-plan tests last); a dot where the test covers the item, in the colour of the item's status. The last column counts the tests of an item, the last row the items of a test. Hover a dot for the pair.

Item	base_test	set_config_test	short_packet_test	exhaustive_seq_test	incr_payload_test	yapp_012_test	simple_test	test_uvc_integration	router_simple_mcseq_test	scoreboard_drop_test	coverage_test	uvm_mem_walk_test	uvm_reset_test	reg_access_test	reg_function_check_test	reg_function_test	reg_introspection_test	back_to_back_test
ROUTE · Routing																		
ROUTE-01 The address selects the output ...						●	●		●							●		
ROUTE-02 Payload and parity arrive unch...									●	●	●							
ROUTE-03 Order is kept per channel, chan...									●		●							
ROUTE-04 Packets back to back with the ...										●	●							●
PKT · Packet format																		
PKT-01 Header: length in [7:2], address in ...							●		●		●							
PKT-02 Every payload length 1..63, includi...										●	●							
PKT-03 Address 3 is illegal								●			●							
PKT-04 Even bitwise parity over header a...											●					●		
PKT-05 A wrong parity byte is detected -- ...								●								●		
IN · Input port protocol																		

Item	base_test	set_config_test	short_packet_test	exhaustive_seq_test	incr_payload_test	yapp_012_test	simple_test	test_uvc_integration	router_simple_mcseq_test	scoreboard_drop_test	coverage_test	uvm_mem_walk_test	uvm_reset_test	reg_access_test	reg_function_check_test	reg_function_test	reg_introspection_test	back_to_back_test
IN-01 in_data_vld rises with the header; on...						●	●				●							
IN-02 in_data_vld is low with the parity byte							●									●		
IN-03 in_suspend: the input holds while th...																		
IN-04 error asserts 1 to 10 cycles after a b...								●										
OUT · Output port protocol																		
OUT-01 data_vld_x, suspend_x and one byt...							●		●		●							
OUT-02 The receiver may answer late: the...							●		●									
OUT-03 The FIFO is 16 bytes deep										●								
OUT-04 suspend_x is re-asserted at the p...									●									
OUT-05 All three channels busy at the sam...											●							
REG · Registers																		
REG-01 Reset values													●					
REG-02 RW registers: every bit can be writ...														●				
REG-03 RO registers ignore writes														●			●	
REG-04 Front door and backdoor see the s...														●			●	
REG-05 Unmapped addresses read 0x00 a...																		
REG-06 mem_size_reg holds the payload l...																		
REG-07 Automatic checking on read (chec...															●			
DROP · Drop rules																		
DROP-01 length > maxpktsize drops the w...								●		●								
DROP-02 router_en = 0 drops every packet																		
DROP-03 Address 3 drops the packet								●			●							
DROP-04 An oversized packet to address ...																		
DROP-05 Changing router_en or a counter ...																		
CNT · Counters																		
CNT-01 A counter counts only while its en...																●		
CNT-02 addrN_cnt_reg counts every recei...																●		
CNT-03 parity_err_cnt_reg counts the bad...																●		
CNT-04 oversized_pkt_cnt_reg counts pac...																●		
CNT-05 addr3_cnt_reg counts packets to ...																●		
CNT-06 Counter overflow after 255 packets																		
MEM · Memories																		
MEM-01 yapp_pkt_mem holds the last pac...																		

Item	base_test	set_config_test	short_packet_test	exhaustive_seq_test	incr_payload_test	yapp_012_test	simple_test	test_uvc_integration	router_simple_mcseq_test	scoreboard_drop_test	coverage_test	uvm_mem_walk_test	uvm_reset_test	reg_access_test	reg_function_check_test	reg_function_test	reg_introspection_test	back_to_back_test
MEM-02 yapp_pkt_mem is read-only																		
MEM-03 A dropped packet is still stored																		
MEM-04 yapp_mem: every byte can be wr...												●						
HBUS · Host port (HBUS)																		
HBUS-01 Write: hen and hwr_rd for one cy...									●					●				
HBUS-02 Read: haddr sampled in cycle 1, ...									●					●				
HBUS-03 hdata is tri-stated when hen drops																		
HBUS-04 Back-to-back accesses												●						
HBUS-05 The register model's front door i...																●		
RND · Random scenarios																		
RND-01 Random configuration between bu...																		
RND-02 One error pulse per bad packet w...																		
RND-03 Bad parity on dropped and ignore...																		
RND-04 Receivers that stall, per channel																		
COV · Functional coverage																		
COV-01 REQ1: all packet lengths, bucketed											●							
COV-02 REQ2: every address received a p...											●							
COV-03 REQ3: every length bucket to ever...											●							
COV-04 HBUS: every address of the route...																		
COV-05 HBUS: every bit of ctrl_reg and en...																		
COV-06 HBUS: data patterns in yapp_mem																		
COV-07 HBUS: every counter read at 0, in ...																		
TB · Testbench mechanics																		
TB-01 The test builds the testbench and th...	●																	
TB-02 A factory type override replaces the...			●	●	●		●		●									
TB-03 Configuration through the database...		●																
TB-04 The sequence library: sub-sequenc...				●	●													
TB-05 The virtual interface connects the d...						●	●											
TB-06 A multichannel sequence coordinat...									●									
TB-07 The scoreboard notices a wrong pa...										●								
TB-08 The register model describes itself																	●	
items per test	1	1	1	2	2	3	8	5	11	6	14	2	1	5	1	10	3	1

Tests

Every test extends `base_test` and only changes what runs. The lab is where the test was born; the test-plan tests exist only in `yapp_project/tb/tests`. Run one with `make run-project TEST=<name>` (or `make run LAB=<lab> TEST=<name>` for a lab).

Test	Lab	Purpose	Verifies
`base_test`	2	The skeleton every test extends: builds router_tb, keeps the run alive 200 ns after the last objection, reports unused settings.	TB-01
`set_config_test`	4	Lab 4: a passive agent through the configuration database -- and why the sequence setting lives in a virtual function.	TB-03
`short_packet_test`	4	Lab 4: the factory override in action -- the sequence asks for yapp_packet and gets short_yapp_packet.	TB-02
`exhaustive_seq_test`	5	Lab 5: one sequence that runs every sequence of the library once (sequences starting sub-sequences).	TB-02, TB-04

Test	Lab	Purpose	Verifies
`incr_payload_test`	5	Lab 5: a sequence that shapes the payload after randomisation.	TB-02, TB-04
`yapp_012_test`	6	Lab 6: the first packets into the real DUT -- one per channel, so the connection is easy to check in the waveform.	ROUTE-01, IN-01, TB-05
`simple_test`	7	Lab 7: every UVC talking to the DUT with the default sequences only.	ROUTE-01, PKT-01, IN-01, IN-02, OUT-01, OUT-02, TB-02, TB-05
`test_uvc_integration`	7	Lab 7: 88 packets of every length and address against a router programmed for small packets.	PKT-03, PKT-05, IN-04, DROP-01, DROP-03
`router_simple_mcseq_test`	8	Lab 8: one multichannel sequence on the virtual sequencer programs the router and sends the packets in order.	ROUTE-01, ROUTE-02, ROUTE-03, PKT-01, OUT-01, OUT-02, OUT-04, HBUS-01, HBUS-02,

Test	Lab	Purpose	Verifies
			TB-02, TB-06
`scoreboard_drop_test`	9A	Lab 9A: the negative check -- full-size packets while maxpktsize is 20, so the router drops some and the scoreboard must notice.	ROUTE-02, ROUTE-04, PKT-02, OUT-03, DROP-01, TB-07
`coverage_test`	10	Lab 10: stimulus written to close the functional coverage model of the YAPP packets (yapp_pkt_cg in yapp_pkt_coverage, fed by the monitor).	ROUTE-02, ROUTE-03, ROUTE-04, PKT-01, PKT-02, PKT-03, PKT-04, IN-01, OUT-01, OUT-05, DROP-03, COV-01, COV-02, COV-03
`uvm_mem_walk_test`	11B	Lab 11B: the built-in memory sequence walks yapp_mem; the RO yapp_pkt_mem is skipped.	MEM-04, HBUS-04
`uvm_reset_test`	11B	Lab 11B: the built-in register sequence that	REG-01

Test	Lab	Purpose	Verifies
		reads every register after reset.	
`reg_access_test`	11C	Lab 11C: hand-written front-door and backdoor checks on one RW and one RO register.	REG-02, REG-03, REG-04, HBUS-01, HBUS-02
`reg_function_check_test`	11C	Lab 11C (optional): the same test with the model checking every read itself.	REG-07
`reg_function_test`	11C	Lab 11C: do the enable bits and the counters behave? Packets in, counters read through the model.	ROUTE-01, PKT-04, PKT-05, IN-02, CNT-01, CNT-02, CNT-03, CNT-04, CNT-05, HBUS-05
`reg_introspection_test`	11C	Lab 11C (optional): the model tells the test which registers are RW and which are RO, then every one is checked.	REG-03, REG-04, TB-08
`back_to_back_test`	test plan	Test plan: the input port with the shortest gap -- a header in the cycle right after the parity	ROUTE-04

Test	Lab	Purpose	Verifies
		byte -- and gaps of 1, 2 and 3 cycles.	
`backpressure_test`	test plan	Test plan: the YAPP handshake under back-pressure -- slow receivers fill the 16-byte channel FIFOs, the router stalls the input with in_suspend and no byte is lost.	ROUTE-02, ROUTE-03, PKT-02, IN-03, OUT-02, OUT-03, OUT-04, OUT-05
`counter_wrap_test`	test plan	Test plan: an 8-bit counter wraps from 255 to 0 -- the specification is silent, the Decisions of dut/spec.md fix the choice.	CNT-06
`hbus_protocol_test`	test plan	Test plan: the host port on the wire -- one-cycle writes, two-cycle reads, tri-state after the access, unmapped addresses -- with raw HBUS sequences, no register model.	REG-03, REG-05, HBUS-01, HBUS-02, HBUS-03
`hbus_sweep_test`	test plan	Test plan: the HBUS side of the coverage -- every address of the router read and written, checked against the reference model, until	REG-05, CNT-06, MEM-02, COV-04, COV-05,

Test	Lab	Purpose	Verifies
		the four HBUS collectors reach 100 %.	COV-06, COV-07
`parity_error_test`	test plan	Test plan: what a wrong parity byte does -- the counter counts it (when enabled), error pulses 1..10 cycles later, and the packet is still delivered.	PKT-05, IN-04, CNT-01, CNT-03
`pkt_mem_test`	test plan	Test plan: the packet memory and mem_size_reg hold the last packet, byte by byte, even when the packet was dropped.	IN-02, REG-06, MEM-01, MEM-02, MEM-03
`random_backpressure_test`	test plan	Random tests: back-pressure on a random mix of channels -- slow and stalling receivers against long packets sent back to back.	ROUTE-03, IN-03, OUT-02, OUT-05, RND-04
`random_config_test`	test plan	Random tests: the configuration space -- every maxpktsize edge, the router off, mixed counter enables -- under many short bursts.	PKT-03, DROP-01, DROP-02, DROP-03, DROP-04, CNT-01, CNT-02, CNT-04, CNT-05,

Test	Lab	Purpose	Verifies
			MEM-03, RND-01, RND-03
`random_parity_test`	test plan	Random tests: bad parity where it can happen -- forwarded, dropped (address 3, oversized) and ignored (router off) -- and storms of overlapping bad packets.	PKT-05, IN-04, CNT-03, RND-02, RND-03
`random_test`	test plan	Random tests: the seed decides the shape of the test, and every result is checked automatically -- no expected value is written by hand.	PKT-02, REG-05, REG-06, MEM-01, RND-01, RND-04, COV-04
`reg_bit_walk_test`	test plan	Test plan: every bit of every register obeys its policy -- walking ones and zeros through the RW registers, writes ignored by the RO ones.	REG-02, REG-03, HBUS-04, COV-05
`router_disable_test`	test plan	Test plan: router_en = 0 makes the router deaf -- nothing reaches a channel, nothing is counted, nothing is stored; traffic resumes	DROP-02, HBUS-05

Test	Lab	Purpose	Verifies
		when it is enabled again.	
`router_filter_test`	test plan	Test plan: the drop rules around the maxpktsize limit and the counters behind them, with the counters enabled and then disabled.	ROUTE-01, ROUTE-02, ROUTE-03, PKT-03, PKT-04, OUT-04, DROP-01, DROP-03, DROP-04, CNT-01, CNT-02, CNT-04, CNT-05, HBUS-05

Writing a new test — checklist

1. Extend `base_test` (or `reg_function_test` when you need the register block, the YAPP sequencer and `check_counter()`); add `` `uvm_component_utils `` and the constructor.
2. Decide the packet type: factory override in `build_phase` *before* `super.build_phase(phase)`.
3. `configure_sequences()`: which sequencer runs what. Always `set_clock_and_channel_sequences()` in Labs 7+ (or your own channel sequence, as `backpressure_test` does).
4. For register tests: get handles in `connect_phase`, raise an objection in `run_phase`, use the model API.
5. Decide what *passes*: `UVM_ERROR : 0` plus the counts in the reports, and say so in the log (`uvm_info` at `UVM_NONE`).
6. Add the test to `annotations.yaml`: a line under `classes:`, its plan under `tests:`, and the items it verifies under `plan:`; `make map` regenerates this page and the map.

Glossary

The words of the course, in the sense they have in this project, each with the page that shows the thing itself. UVM names are the class or macro names; the YAPP names are the ones in the code.

UVM

Term	Meaning here	See
agent	One protocol's driver, sequencer and monitor in one component (`yapp_tx_agent`, `channel_rx_agent`, `hbus_master_agent`); `is_active` decides whether the driver and sequencer exist.	Agent and env
analysis port / imp / export	A one-to-many TLM connection: a monitor `write()`s to its `uvm_analysis_port`; every subscriber's `uvm_analysis_imp` receives it; an `uvm_analysis_export` passes it through an env boundary.	TLM connections, Lab 9A, Lab 9C
analysis FIFO	`uvm_tlm_analysis_fifo`: an analysis imp with a queue behind it, so a consumer can `get()` in its own time (the Lab 9D scoreboard).	Lab 9D
backdoor	Reading or writing a register through its HDL path (`hw_top.dut.u_regs.ctrl_reg`), without the bus; `peek()` / `poke()`.	Register model
config_db	`uvm_config_db#(T)::set/get` (and the course's `uvm_config_int`): a name-addressed database the test fills before `build_phase` and components read in theirs (`is_active`, `channel_id`, the virtual interfaces).	Factory and configuration

Term	Meaning here	See
coverage collector	A `uvm_subscriber` holding a covergroup and sampling it in `write()` (`yapp_pkt_coverage`, `router_coverage`, the HBUS collectors).	Functional coverage
drain time	How long the run phase stays alive after the last objection drops (`set_drain_time`), so the last packet reaches the output before the end.	Phases and objections
driver	Takes items from its sequencer (`get_next_item` / `item_done`) and drives the pins through the virtual interface.	Driver
env	The container that builds agents and connects them; `router_tb` is the testbench env, each UVC has its own (`yapp_env`, `hbus_env`).	Agent and env
factory	`type_id::create()` instead of `new()`, so a test can swap a class for a subclass with `set_type_override_by_type` without touching the code that creates it.	Factory and configuration, Lab 4
front door	A register access through the bus (the HBUS UVC and its adapter), as opposed to the backdoor.	Register model
monitor	Watches the interface, rebuilds the transaction and publishes it on an analysis port; never drives.	Monitor

Term	Meaning here	See
objection	`phase.raise_objection` / `drop_objection`: what keeps the run phase alive; a sequence raises one in `pre_body` and drops it in `post_body`.	Phases and objections
phase	The steps UVM runs every component through in order: `build`, `connect`, `end_of_elaboration`, `run` (the only one that takes time), `extract`, `check`, `report`.	Phases and objections
predictor	The part of the register model that keeps the mirror up to date (`auto_predict` here: the value is predicted from the access the model itself made).	Lab 11B
reference model	A transaction-level copy of the DUT's rules (`router_reference`): given what went in, it says what must come out.	Reference model
register model (RAL)	`uvm_reg_block` / `uvm_reg` / `uvm_reg_field` classes mirroring the DUT's registers (`yapp_router_regs_t`), with `write` / `read` / `mirror` and the built-in sequences.	Register model
scoreboard	Compares what the monitors saw on the outputs with what the reference model expected; counts matches and mismatches.	Scoreboard
sequence	A `uvm_sequence` whose `body()` creates items and sends them through `start_item` / `finish_item`; sequences nest (`seq.start(sequencer, this)`).	Sequencer and sequences

Term	Meaning here	See
sequence item	The transaction a sequence produces and a driver consumes (`yapp_packet`, `hbus_transaction`); randomized with constraints.	Packet
sequencer	Arbitrates between the sequences running on it and hands their items to the driver.	Sequencer and sequences
subscriber	`uvm_subscriber #(T)`: a component with a built-in analysis imp and a `write()` to override (the coverage collectors).	Functional coverage
test	A `uvm_test` subclass picked by `+UVM_TESTNAME`; builds the testbench and decides what runs (`base_test` and everything that extends it).	Lab 2, Test plan
UVC	UVM Verification Component: a reusable package with the agent(s), env, interface, sequences and transaction of one protocol (`yapp`, `hbus`, `channel`, `clock_and_reset`, and the router *module* UVC).	Project architecture
virtual interface	A class-side handle to an interface instance (`virtual yapp_if m_vif`), set by `tb_top` into the config_db and fetched by the driver and monitor.	Lab 6
virtual sequencer	A sequencer with no driver that holds handles to the real sequencers (`router_mcsequencer`), so one *multichannel* sequence can drive several UVCs.	Virtual sequencer

The project

Term	Meaning here	See
back-pressure	A channel receiver raising `suspend` so the router's output FIFO stops sending; the input side stalls when the FIFO is full.	DUT specification
channel	One of the three output ports (`ch0`, `ch1`, `ch2`); the packet's address selects it.	DUT specification
drop	A packet the router does not forward: address 3, bad parity, longer than `maxpktsize`, or `router_en` off; the reference model predicts every drop.	Test plan
gap	The idle cycles between two packets on the input (`packet_delay`); 0 means back to back.	Lab 10
HBUS	The host bus: the port through which the registers and memories are written and read (`haddr`, `hdata`, `hen`, `hwr_rd`).	Register map
`hdata_w`	The tri-state wire of `hbus_if` that is connected to the DUT, driven by the master only during writes (`hdata` and `hdata_oe` are the master's side).	Lab 7
`maxpktsize`	`ctrl_reg[5:0]`: the longest payload the router forwards.	Register map
mutant / bug hunt	A deliberate bug in the RTL behind a `` `define `` (`BUG_*`) that the regression must catch; one page lists every bug and the tests that find it.	Bug hunt
packet	Header (`addr[1:0]`, `length[5:0]`), `length` payload bytes, a parity byte that XORs everything	Packet

Term	Meaning here	See
	before it.	
parity	The XOR of the header and the payload; `PARITY_TYPE_BAD` packets carry a wrong one on purpose.	Packet
`rnd::`	The project's random-value library (`common/rand_util_pkg.sv`): every random number outside a `randomize()` goes through it.	Random values
`router_en`	`en_reg[0]`: the router forwards nothing while it is 0.	Register map
seed	The number that makes a random run reproducible (`SEED=`, `+SVSEED=`); the regression runs every test with several.	Regression results
`start/`	The directory a student works in: the lab with its method bodies emptied, `TASKS.md` next to it.	Getting started
test plan	The list of DUT features, each with its stimulus, checker, coverage and the tests that cover it.	Test plan

FAQ and troubleshooting

The questions that come up in the first weeks, with the answer and the place to read more. The pitfalls page lists the design mistakes the course warns about; this page is about what you see on the screen.

Running

- `make sim` says `verilator not found`
- The first `make sim` takes minutes
- My test runs `base_test` instead of the one I asked for
- The run never ends
- The last packet is missing from the output

Compiling

- `Unknown class or package 'yapp_packet'` / a class is not found
- `make lint` fails but the simulator is happy (or the other way round)
- `sv_style: ... R6 ...` — what is the style check complaining about?

Logs and results

- Where is the log?
- `UVM_WARNING ... configuration never used`
- `randomize() failed` / `UVM_ERROR ... Randomization failed`

- The scoreboard reports a mismatch — is it my code or the DUT?
- `make check` says `MISSING` but my test passes

Waves

- I cannot see `yapp_pkt_mem` / `yapp_mem` in the waves
- Which signals should I look at first?

The course material

- The course writes `` `uvm_do_with `` and `` `uvm_field_int ``; this code does not — why?
- Where is the Cadence lab PDF?

Results

Coverage results

The functional coverage of the project's regression, **merged over every test and seed**: which bins were hit, how often and by which tests, and which bins no run hit (the *holes*). Every run writes its covergroup bins to a coverage database; the report merges them. How the covergroups are built and where they live: Functional coverage. The bins of each coverage collector are also in its panel on the project map.

Which tests the coverage needs

222 bins were hit by at least one run. **Bins** is how many of them the test hits (over all its seeds); **only this test** counts the bins no other test reaches -- drop that test and they become holes. **Minimal set** is a greedy choice: the test that hits the most bins, then the one that adds the most to it, until every hit bin is covered -- the regression you would run when there is time for only a few tests. (Generated from `coverage.json` by `scripts/project_map/build.py`.)

Test	Bins	Only this test
hbus_sweep_test	197	**4**: `hbus_addr_cg.addr_x_rw.gap_last_x_write`, `hbus_addr_cg.addr_x_rw.gap_mid_x_write`, `hbus_addr_cg.addr_x_rw.pkt_mem_last_x_write` …
random_parity_test	184	0
random_config_test	182	0
random_test	182	0
random_backpressure_test	159	0
router_filter_test	99	0
reg_bit_walk_test	93	0

Test	Bins	Only this test
reg_function_check_test	90	0
reg_function_test	90	0
router_disable_test	80	0
reg_introspection_test	70	0
counter_wrap_test	64	0
parity_error_test	60	0
hbus_protocol_test	57	0
pkt_mem_test	52	0
uvm_reset_test	32	0
reg_access_test	30	0
uvm_mem_walk_test	28	0
back_to_back_test	19	0
backpressure_test	15	0

Minimal set, in order: `hbus_sweep_test` → `random_test` (2 of 20 tests hit every bin the regression hit).

Merged over **100 runs**: 222 of 222 bins hit. A percentage is covered bins / bins of the covergroup, as `get_inst_coverage()` gives it in the log of one run.

Annotated sources (genhtml): coverage/yapp_project_tb/index.html

Covergroup	Where	Bins hit	Coverage
`router_cfg_cg`	router_coverage.sv	23 / 23	**100.0%**
`hbus_addr_cg`	router_hbus_addr_coverage.sv	77 / 77	**100.0%**
`hbus_bits_cg`	router_hbus_bits_coverage.sv	44 / 44	**100.0%**
`hbus_counter_cg`	router_hbus_counter_coverage.sv	27 / 27	**100.0%**
`hbus_data_cg`	router_hbus_data_coverage.sv	17 / 17	**100.0%**
`yapp_gap_cg`	yapp_gap_coverage.sv	4 / 4	**100.0%**
`yapp_pkt_cg`	yapp_pkt_coverage.sv	30 / 30	**100.0%**

Holes

None: every bin was hit at least once.

`router_cfg_cg` : 100.0% (23 / 23 bins)

Source: yapp_project/uvc/router/router_coverage.sv

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it (mc first)
`max_cp` (5/5)	zero	148	10	random_test, random_config_test random_parity_test

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it (mc first)
	one	4167	12	hbus_sweep_test, random_parity_test, random_test, random_backpressu +1
	mid	1125	39	random_backpressu random_parity_test, random_test, random_config_test
	max62	209	16	random_config_test random_parity_test, random_test, random_backpressu
	max63	1422	45	counter_wrap_test, back_to_back_test, random_parity_test, parity_error_test +8
`len_cp` (4/4)	below	1983	71	counter_wrap_test, random_parity_test, random_test, random_config_test
	equal	152	33	random_parity_test, random_config_test router_filter_test, random_test +1

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it (mc first)
	plus_one	4253	33	hbus_sweep_test, random_config_test random_test, router_filter_test +2
	above	683	40	random_backpressu random_test, random_config_test random_parity_test
`reason_cp` (3/3)	forwarded	2061	71	counter_wrap_test, random_parity_test, random_test, random_config_test
	addr3	1171	33	hbus_sweep_test, random_parity_test, random_config_test router_filter_test +2
	oversized	3839	44	hbus_sweep_test, random_backpressu random_test, random_config_test
`parity_cp` (2/2)	good	2660	68	counter_wrap_test, random_backpressu random_parity_test, random_test +9
	bad	4411	44	hbus_sweep_test, random_parity_test,

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it (mc first)
				parity_error_test, random_config_test
`cnt_en_cp` (3/3)	none	348	44	back_to_back_test, router_filter_test, random_parity_test, random_config_test
	mixed	5135	38	hbus_sweep_test, random_parity_test, random_test, random_backpress +2
	all	1588	41	counter_wrap_test, random_config_test random_backpress random_parity_test
cross `reason_x_parity` (6/6)	addr3_x_bad	1052	15	hbus_sweep_test, random_parity_test, random_backpress random_config_test
	addr3_x_good	119	28	random_parity_test, random_config_test router_filter_test, random_test +1
	forwarded_x_bad	201	35	random_parity_test, parity_error_test, random_test,

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it (mc first)
				random_backpressu +4
	forwarded_x_good	1860	67	counter_wrap_test, random_parity_test, random_config_test random_test +9
	oversized_x_bad	3158	30	hbus_sweep_test, random_parity_test, random_config_test random_test +1
	oversized_x_good	681	40	random_backpressu random_test, random_config_test random_parity_test

`hbus_addr_cg` : 100.0% (77 / 77 bins)

Source: yapp_project/uvc/router/router_hbus_addr_coverage.sv

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it (mo first)
`addr_cp` (25/25)	ctrl	1128	64	hbus_sweep_test, reg_bit_walk_test, random_config_test random_parity_test
	en	1159	72	hbus_sweep_test, reg_bit_walk_test,

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it (mo first)
				random_config_test random_parity_test
	parity_cnt	711	76	hbus_sweep_test, reg_bit_walk_test, random_parity_test, random_backpressu +11
	oversized_cnt	703	68	hbus_sweep_test, random_config_test reg_bit_walk_test, random_parity_test
	addr3_cnt	720	68	hbus_sweep_test, random_config_test reg_bit_walk_test, random_test +9
	addr0_cnt	726	72	hbus_sweep_test, random_config_test reg_bit_walk_test, counter_wrap_test +
	addr1_cnt	708	68	hbus_sweep_test, reg_bit_walk_test, random_config_test random_backpressu +9
	addr2_cnt	714	68	hbus_sweep_test, random_config_test reg_bit_walk_test,

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it (mo first)
				random_backpressu +9
	mem_size	752	56	hbus_sweep_test, random_config_test random_test, random_backpressu +6
	reg_hole[0]	581	16	hbus_sweep_test, hbus_protocol_test, random_config_test random_parity_test
	reg_hole[1]	575	11	hbus_sweep_test, hbus_protocol_test, random_parity_test, random_config_test
	reg_hole[2]	579	14	hbus_sweep_test, hbus_protocol_test, random_backpressu random_config_test
	reg_hole[3]	581	16	hbus_sweep_test, hbus_protocol_test, random_config_test random_test +1
	reg_hole[4]	582	16	hbus_sweep_test, hbus_protocol_test, random_config_test

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it (mo first)
				random_backpressu +2
	reg_hole[5]	574	14	hbus_sweep_test, hbus_protocol_test, random_config_test random_parity_test
	reg_hole[6]	589	22	hbus_sweep_test, hbus_protocol_test, random_config_test random_parity_test
	pkt_mem_first	57	40	pkt_mem_test, random_parity_test, hbus_sweep_test, random_backpressu +2
	pkt_mem_mid	1840	40	hbus_sweep_test, random_backpressu random_config_test random_test +2
	pkt_mem_last	16	12	hbus_sweep_test, random_config_test random_backpressu random_parity_test
	gap_first	17	9	hbus_protocol_test, hbus_sweep_test, random_test

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it (mo first)
	gap_mid	112	33	hbus_sweep_test, random_config_test random_parity_test, random_backpressu +1
	gap_last	9	5	hbus_sweep_test, random_backpressu
	yapp_mem_first	64	13	hbus_sweep_test, uvm_mem_walk_tes random_test, random_parity_test
	yapp_mem_mid	14174	40	hbus_sweep_test, uvm_mem_walk_tes random_config_test random_backpressu +2
	yapp_mem_last	54	11	hbus_sweep_test, uvm_mem_walk_tes random_config_test random_parity_test
`rw_cp` (2/2)	read	18661	88	hbus_sweep_test, uvm_mem_walk_tes random_config_test random_backpressu +14

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it (mo first)
	write	9064	84	hbus_sweep_test, uvm_mem_walk_tes random_config_test reg_bit_walk_test +1
cross `addr_x_rw` (50/50)	addr0_cnt_x_read	694	72	hbus_sweep_test, random_config_test counter_wrap_test, random_backpressu +10
	addr0_cnt_x_write	32	16	hbus_sweep_test, reg_bit_walk_test, reg_access_test, reg_introspection_te
	addr1_cnt_x_read	680	68	hbus_sweep_test, random_config_test random_backpressu random_parity_test
	addr1_cnt_x_write	28	12	hbus_sweep_test, reg_bit_walk_test, reg_introspection_te
	addr2_cnt_x_read	686	68	hbus_sweep_test, random_config_test random_backpressu random_parity_test
	addr2_cnt_x_write	28	12	hbus_sweep_test, reg_bit_walk_test,

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it (mo first)
				reg_introspection_te
	addr3_cnt_x_read	692	68	hbus_sweep_test, random_config_test random_test, random_backpressu +9
	addr3_cnt_x_write	28	12	hbus_sweep_test, reg_bit_walk_test, reg_introspection_te
	ctrl_x_read	758	56	hbus_sweep_test, reg_bit_walk_test, random_parity_test, random_backpressu +6
	ctrl_x_write	370	60	hbus_sweep_test, reg_bit_walk_test, random_config_test random_parity_test
	en_x_read	761	56	hbus_sweep_test, reg_bit_walk_test, random_config_test random_parity_test
	en_x_write	398	68	hbus_sweep_test, reg_bit_walk_test, random_config_test random_parity_test

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it (mo first)
	gap_first_x_read	9	9	hbus_protocol_test, hbus_sweep_test, random_test
	gap_first_x_write	8	8	hbus_protocol_test, hbus_sweep_test
	gap_last_x_read	5	5	hbus_sweep_test, random_backpressu
	gap_last_x_write	4	4	hbus_sweep_test
	gap_mid_x_read	88	33	hbus_sweep_test, random_config_test random_parity_test, random_backpressu +1
	gap_mid_x_write	24	4	hbus_sweep_test
	mem_size_x_read	724	56	hbus_sweep_test, random_config_test random_test, random_backpressu +6
	mem_size_x_write	28	12	hbus_sweep_test, reg_bit_walk_test, reg_introspection_te
	oversized_cnt_x_read	675	68	hbus_sweep_test, random_config_test

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it (mo first)
				random_parity_test, random_backpressu +9
	oversized_cnt_x_write	28	12	hbus_sweep_test, reg_bit_walk_test, reg_introspection_te
	parity_cnt_x_read	679	76	hbus_sweep_test, random_parity_test, random_backpressu random_config_test
	parity_cnt_x_write	32	16	hbus_sweep_test, reg_bit_walk_test, hbus_protocol_test, reg_introspection_te
	pkt_mem_first_x_read	49	40	pkt_mem_test, random_parity_test, random_backpressu random_config_test
	pkt_mem_first_x_write	8	8	hbus_sweep_test, pkt_mem_test
	pkt_mem_last_x_read	12	12	hbus_sweep_test, random_config_test random_backpressu random_parity_test
	pkt_mem_last_x_write	4	4	hbus_sweep_test

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it (mo first)
	pkt_mem_mid_x_read	1592	40	random_backpressu random_config_test random_test, hbus_sweep_test +2
	pkt_mem_mid_x_write	248	4	hbus_sweep_test
	reg_hole[0]_x_read	561	16	hbus_sweep_test, random_config_test hbus_protocol_test, random_parity_test
	reg_hole[0]_x_write	20	8	hbus_sweep_test, hbus_protocol_test
	reg_hole[1]_x_read	555	11	hbus_sweep_test, hbus_protocol_test, random_parity_test, random_config_test
	reg_hole[1]_x_write	20	8	hbus_sweep_test, hbus_protocol_test
	reg_hole[2]_x_read	559	14	hbus_sweep_test, hbus_protocol_test, random_backpressu random_config_test
	reg_hole[2]_x_write	20	8	hbus_sweep_test, hbus_protocol_test

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it (mo first)
	reg_hole[3]_x_read	561	16	hbus_sweep_test, hbus_protocol_test, random_config_test random_test +1
	reg_hole[3]_x_write	20	8	hbus_sweep_test, hbus_protocol_test
	reg_hole[4]_x_read	562	16	hbus_sweep_test, hbus_protocol_test, random_config_test random_backpressu +2
	reg_hole[4]_x_write	20	8	hbus_sweep_test, hbus_protocol_test
	reg_hole[5]_x_read	554	14	hbus_sweep_test, hbus_protocol_test, random_config_test random_parity_test
	reg_hole[5]_x_write	20	8	hbus_sweep_test, hbus_protocol_test
	reg_hole[6]_x_read	569	22	hbus_sweep_test, random_config_test random_parity_test, random_backpressu +2

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it (mo first)
	reg_hole[6]_x_write	20	8	hbus_sweep_test, hbus_protocol_test
	yapp_mem_first_x_read	30	13	hbus_sweep_test, uvm_mem_walk_tes random_test, random_parity_test
	yapp_mem_first_x_write	34	13	hbus_sweep_test, uvm_mem_walk_tes random_test, random_parity_test
	yapp_mem_last_x_read	27	11	hbus_sweep_test, uvm_mem_walk_tes random_config_test random_parity_test
	yapp_mem_last_x_write	27	11	hbus_sweep_test, uvm_mem_walk_tes random_config_test random_parity_test
	yapp_mem_mid_x_read	6579	40	hbus_sweep_test, uvm_mem_walk_tes random_config_test random_backpressu +2
	yapp_mem_mid_x_write	7595	40	hbus_sweep_test, uvm_mem_walk_tes random_config_test

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it (mo first)
				random_backpressu +2

`hbus_bits_cg` : 100.0% (44 / 44 bins)

Source: yapp_project/uvc/router/router_hbus_bits_coverage.sv

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it first)
`reg_cp` (2/2)	ctrl_reg	2960	60	hbus_sweep_te reg_bit_walk_te random_config_ random_parity_
	en_reg	3184	68	hbus_sweep_te reg_bit_walk_te random_config_ random_parity_
`bit_cp` (8/8)	b[0]	768	80	hbus_sweep_te reg_bit_walk_te random_config_ random_parity_
	b[1]	768	80	hbus_sweep_te reg_bit_walk_te random_config_ random_parity_
	b[2]	768	80	hbus_sweep_te reg_bit_walk_te

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it first)
				random_config_ random_parity_
	b[3]	768	80	hbus_sweep_te reg_bit_walk_te random_config_ random_parity_
	b[4]	768	80	hbus_sweep_te reg_bit_walk_te random_config_ random_parity_
	b[5]	768	80	hbus_sweep_te reg_bit_walk_te random_config_ random_parity_
	b[6]	768	80	hbus_sweep_te reg_bit_walk_te random_config_ random_parity_
	b[7]	768	80	hbus_sweep_te reg_bit_walk_te random_config_ random_parity_

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it first)
`val_cp` (2/2)	zero	2906	76	hbus_sweep_te reg_bit_walk_te random_config_ random_backpr +11
	one	3238	80	hbus_sweep_te reg_bit_walk_te random_config_ random_parity_
cross `reg_x_bit_x_val` (32/32)	ctrl_reg_x_b[0]_x_one	191	58	hbus_sweep_te reg_bit_walk_te random_backpr random_config_
	ctrl_reg_x_b[0]_x_zero	179	45	hbus_sweep_te reg_bit_walk_te random_config_ random_test +4
	ctrl_reg_x_b[1]_x_one	202	50	hbus_sweep_te reg_bit_walk_te random_config_ random_parity_
	ctrl_reg_x_b[1]_x_zero	168	52	hbus_sweep_te reg_bit_walk_te random_parity_ random_backpr +5

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it first)
	ctrl_reg_x_b[2]_x_one	204	58	hbus_sweep_te reg_bit_walk_te random_config_ random_parity_
	ctrl_reg_x_b[2]_x_zero	166	45	hbus_sweep_te reg_bit_walk_te random_config_ random_backpr +4
	ctrl_reg_x_b[3]_x_one	201	51	hbus_sweep_te reg_bit_walk_te random_config_ random_parity_
	ctrl_reg_x_b[3]_x_zero	169	50	hbus_sweep_te reg_bit_walk_te random_backpr random_config_
	ctrl_reg_x_b[4]_x_one	189	51	hbus_sweep_te reg_bit_walk_te random_config_ random_parity_
	ctrl_reg_x_b[4]_x_zero	181	57	hbus_sweep_te reg_bit_walk_te random_config_ random_backpr +7

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it first)
	ctrl_reg_x_b[5]_x_one	201	57	hbus_sweep_te reg_bit_walk_te random_config_ random_parity_
	ctrl_reg_x_b[5]_x_zero	169	48	hbus_sweep_te reg_bit_walk_te random_config_ random_backpr +4
	ctrl_reg_x_b[6]_x_one	152	38	hbus_sweep_te reg_bit_walk_te random_parity_ random_backpr +2
	ctrl_reg_x_b[6]_x_zero	218	59	hbus_sweep_te reg_bit_walk_te random_config_ random_backpr +7
	ctrl_reg_x_b[7]_x_one	158	36	hbus_sweep_te reg_bit_walk_te random_parity_ random_config_
	ctrl_reg_x_b[7]_x_zero	212	57	hbus_sweep_te reg_bit_walk_te random_config_

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it first)
				random_backpr +7
	en_reg_x_b[0]_x_one	296	68	hbus_sweep_te reg_bit_walk_te random_config_ random_backpr +9
	en_reg_x_b[0]_x_zero	102	27	hbus_sweep_te reg_bit_walk_te random_config_ random_test +2
	en_reg_x_b[1]_x_one	214	63	hbus_sweep_te reg_bit_walk_te random_config_ random_backpr +8
	en_reg_x_b[1]_x_zero	184	57	hbus_sweep_te reg_bit_walk_te random_config_ random_parity_
	en_reg_x_b[2]_x_one	198	62	hbus_sweep_te reg_bit_walk_te random_config_ random_backpr +8
	en_reg_x_b[2]_x_zero	200	51	hbus_sweep_te reg_bit_walk_te

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it first)
				random_parity_ random_config_
	en_reg_x_b[3]_x_one	202	59	hbus_sweep_te reg_bit_walk_te random_config_ random_parity_
	en_reg_x_b[3]_x_zero	196	57	hbus_sweep_te reg_bit_walk_te random_backpr random_parity_
	en_reg_x_b[4]_x_one	207	60	hbus_sweep_te reg_bit_walk_te random_config_ random_parity_
	en_reg_x_b[4]_x_zero	191	54	hbus_sweep_te reg_bit_walk_te random_config_ random_parity_
	en_reg_x_b[5]_x_one	207	63	hbus_sweep_te reg_bit_walk_te random_config_ random_backpr +8

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it first)
	en_reg_x_b[5]_x_zero	191	50	hbus_sweep_te reg_bit_walk_te random_config_ random_parity_
	en_reg_x_b[6]_x_one	211	60	hbus_sweep_te reg_bit_walk_te random_config_ random_parity_
	en_reg_x_b[6]_x_zero	187	55	hbus_sweep_te reg_bit_walk_te random_config_ random_backpr +7
	en_reg_x_b[7]_x_one	205	60	hbus_sweep_te reg_bit_walk_te random_config_ random_parity_
	en_reg_x_b[7]_x_zero	193	56	hbus_sweep_te reg_bit_walk_te random_config_ random_parity_

`hbus_counter_cg`: 100.0% (27 / 27 bins)

Source: yapp_project/uvc/router/router_hbus_counter_coverage.sv

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it (mos first)
`counter_cp` (6/6)	parity_err	679	76	hbus_sweep_test, random_parity_test, random_backpressu random_config_test ·
	oversized	675	68	hbus_sweep_test, random_config_test, random_parity_test, random_backpressu +9
	addr0	694	72	hbus_sweep_test, random_config_test, counter_wrap_test, random_backpressu +10
	addr1	680	68	hbus_sweep_test, random_config_test, random_backpressu random_parity_test +
	addr2	686	68	hbus_sweep_test, random_config_test, random_backpressu random_parity_test +
	addr3	692	68	hbus_sweep_test, random_config_test, random_test, random_backpressu +9

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it (mos first)
`value_cp` (3/3)	zero	3575	68	hbus_sweep_test, router_disable_test, reg_function_check_ reg_function_test +9
	mid	359	68	random_config_test, random_parity_test, random_backpressu random_test +9
	max	172	12	hbus_sweep_test, reg_bit_walk_test, counter_wrap_test
cross `counter_x_value` (18/18)	addr0_x_max	40	12	hbus_sweep_test, counter_wrap_test, reg_bit_walk_test
	addr0_x_mid	75	62	random_config_test, random_backpressu random_parity_test, random_test +8
	addr0_x_zero	579	34	hbus_sweep_test, counter_wrap_test, reg_bit_walk_test, reg_function_check_
	addr1_x_max	28	8	hbus_sweep_test, reg_bit_walk_test

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it (mos first)
	addr1_x_mid	67	55	random_config_test, random_backpressu random_parity_test, router_filter_test +6
	addr1_x_zero	585	31	hbus_sweep_test, counter_wrap_test, reg_bit_walk_test, reg_function_check_
	addr2_x_max	20	8	hbus_sweep_test, reg_bit_walk_test
	addr2_x_mid	69	55	random_backpressu random_config_test, random_parity_test, random_test +6
	addr2_x_zero	597	36	hbus_sweep_test, random_config_test, counter_wrap_test, reg_bit_walk_test +7
	addr3_x_max	12	8	hbus_sweep_test, reg_bit_walk_test
	addr3_x_mid	42	29	random_config_test, random_parity_test, router_filter_test, hbus_sweep_test +3

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it (mos first)
	addr3_x_zero	638	50	hbus_sweep_test, random_backpressu random_test, reg_function_check_
	oversized_x_max	36	8	hbus_sweep_test, reg_bit_walk_test
	oversized_x_mid	54	40	random_config_test, random_backpressu router_filter_test, random_parity_test +
	oversized_x_zero	585	38	hbus_sweep_test, random_parity_test, counter_wrap_test, reg_bit_walk_test +7
	parity_err_x_max	36	8	hbus_sweep_test, reg_bit_walk_test
	parity_err_x_mid	52	43	random_parity_test, parity_error_test, random_config_test, random_test +5
	parity_err_x_zero	591	41	hbus_sweep_test, random_backpressu counter_wrap_test, hbus_protocol_test +

`hbus_data_cg` : 100.0% (17 / 17 bins)

Source: yapp_project/uvc/router/router_hbus_data_coverage.sv

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it (most hits first)
`rw_cp` (2/2)	read	6636	40	hbus_sweep_test, uvm_mem_walk_test, random_config_test, random_backpressure_test +2
	write	7656	40	hbus_sweep_test, uvm_mem_walk_test, random_config_test, random_backpressure_test +2
`pattern_cp` (5/5)	zeros	2064	9	hbus_sweep_test, uvm_mem_walk_test, random_parity_test
	ones	2060	9	hbus_sweep_test, uvm_mem_walk_test, random_parity_test
	x55	2066	10	hbus_sweep_test, uvm_mem_walk_test, random_config_test, random_test
	xaa	2068	9	hbus_sweep_test, uvm_mem_walk_test, random_config_test

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it (most hits first)
	other	6034	40	uvm_mem_walk_test, hbus_sweep_test, random_config_test, random_backpressure_test +2
cross `rw_x_pattern` (10/10)	read_x_ones	1030	9	hbus_sweep_test, uvm_mem_walk_test, random_parity_test
	read_x_other	2513	40	hbus_sweep_test, uvm_mem_walk_test, random_config_test, random_backpressure_test +2
	read_x_x55	1031	10	hbus_sweep_test, uvm_mem_walk_test, random_config_test, random_test
	read_x_xaa	1032	9	hbus_sweep_test, uvm_mem_walk_test, random_config_test
	read_x_zeros	1030	9	hbus_sweep_test, uvm_mem_walk_test, random_parity_test

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it (most hits first)
	write_x_ones	1030	9	hbus_sweep_test, uvm_mem_walk_test, random_parity_test
	write_x_other	3521	40	uvm_mem_walk_test, hbus_sweep_test, random_config_test, random_backpressure_test +2
	write_x_x55	1035	10	hbus_sweep_test, uvm_mem_walk_test, random_config_test, random_test
	write_x_xaa	1036	9	hbus_sweep_test, uvm_mem_walk_test, random_config_test
	write_x_zeros	1034	9	hbus_sweep_test, uvm_mem_walk_test, random_parity_test

`yapp_gap_cg`: 100.0% (4 / 4 bins)

Source: yapp_project/uvc/yapp/yapp_gap_coverage.sv

Coverpoint / cross	Bin	Hits	Runs	Tests that hit it (most hits first)
`gap_cp` (4/4)	back_to_back	774	36	random_parity_test, random_backpressure_test, random_test, random_config_test +1
	one	606	56	hbus_sweep_test, random_test, counter_wrap_test, random_config_test +8
	two	299	46	hbus_sweep_test, random_config_test, random_test, counter_wrap_test +8
	longer	5549	72	hbus_sweep_test, counter_wrap_test, random_config_test, random_test +10

`yapp_pkt_cg` : 100.0% (30 / 30 bins)

Source: yapp_project/uvc/yapp/yapp_pkt_coverage.sv

Coverpoint / cross	Bin	Hits	Runs	Tests t first)
`length_cp` (5/5)	MIN	1305	40	counte random random random

Coverpoint / cross	Bin	Hits	Runs	Tests t first)
	SMALL	4518	57	hbus_s random back_t router_
	MEDIUM	571	60	random random random random +7
	LARGE	565	52	random random random random
	MAX	341	35	random random random router_
`addr_cp` (4/4)	legal[0]	2547	67	counte hbus_s random random
	legal[1]	1581	68	hbus_s random random random

Coverpoint / cross	Bin	Hits	Runs	Tests t first)
	legal[2]	1976	68	hbus_s random random random +9
	illegal_addr	1196	33	hbus_s random random router_
`parity_cp` (2/2)	good	2849	68	counte random random random
	bad	4451	45	hbus_s random random random
`legal_addr_cp` (3/3)	legal[0]	2547	67	counte hbus_s random random
	legal[1]	1581	68	hbus_s random random random

Coverpoint / cross	Bin	Hits	Runs	Tests t first)
	legal[2]	1976	68	hbus_s random random random +9
`bad_parity_cp` (1/1)	bad	4451	45	hbus_s random random random
cross `len_x_addr_x_parity` (15/15)	LARGE_x_legal[0]_x_bad	22	15	parity_e random random random +1
	LARGE_x_legal[1]_x_bad	17	13	parity_e random random reg_fur
	LARGE_x_legal[2]_x_bad	38	23	random parity_e random random
	MAX_x_legal[0]_x_bad	12	8	random random random

Coverpoint / cross	Bin	Hits	Runs	Tests t first)
	MAX_x_legal[1]_x_bad	14	10	random random random random
	MAX_x_legal[2]_x_bad	25	17	random random random random
	MEDIUM_x_legal[0]_x_bad	21	15	random random parity_e reg_fur
	MEDIUM_x_legal[1]_x_bad	29	17	random parity_e random random
	MEDIUM_x_legal[2]_x_bad	43	28	parity_e random random random
	MIN_x_legal[0]_x_bad	12	9	random random random
	MIN_x_legal[1]_x_bad	23	10	random random

Coverpoint / cross	Bin	Hits	Runs	Tests t first)
				random parity_e
	MIN_x_legal[2]_x_bad	29	10	random random random
	SMALL_x_legal[0]_x_bad	1038	14	hbus_s random parity_e random
	SMALL_x_legal[1]_x_bad	1034	10	hbus_s random random
	SMALL_x_legal[2]_x_bad	1037	15	hbus_s random random parity_e

What each test adds

Bins hit over all the seeds of a test, and the bins no other test hits (a test with none adds nothing to the coverage; it may still check something no other test checks).

Test	Runs	Bins hit	Only this test
back_to_back_test	4	19 / 222	-

Test	Runs	Bins hit	Only this test
backpressure_test	4	15 / 222	-
base_test	4	0 / 222	-
counter_wrap_test	4	64 / 222	-
hbus_protocol_test	4	57 / 222	-
hbus_sweep_test	4	197 / 222	`hbus_addr_cg.addr_x_rw.gap_last` `hbus_addr_cg.addr_x_rw.gap_mid_` `hbus_addr_cg.addr_x_rw.pkt_mem_` `hbus_addr_cg.addr_x_rw.pkt_mem_`
parity_error_test	4	60 / 222	-
pkt_mem_test	4	52 / 222	-
random_backpressure_test	8	159 / 222	-
random_config_test	8	182 / 222	-
random_parity_test	8	184 / 222	-

Test	Runs	Bins hit	Only this test
random_test	8	182 / 222	-
reg_access_test	4	30 / 222	-
reg_bit_walk_test	4	93 / 222	-
reg_function_check_test	4	90 / 222	-
reg_function_test	4	90 / 222	-
reg_introspection_test	4	70 / 222	-
router_disable_test	4	80 / 222	-
router_filter_test	4	99 / 222	-
uvm_mem_walk_test	4	28 / 222	-
uvm_reset_test	4	32 / 222	-

Regression results

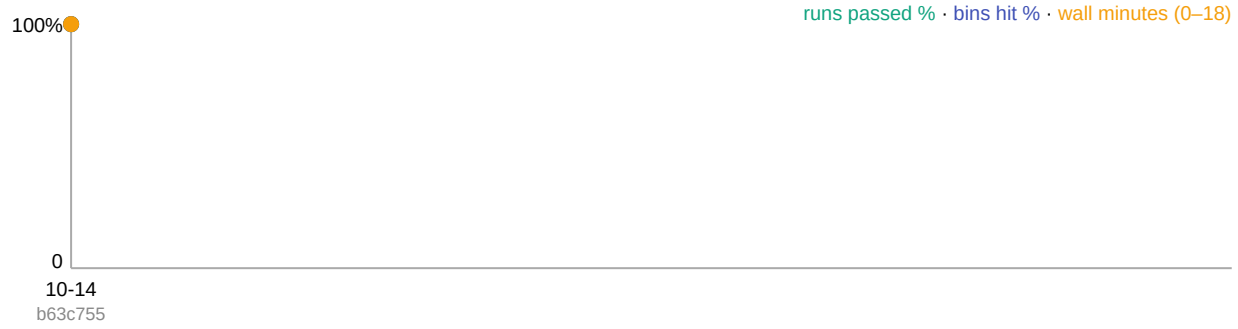
The report on GitHub →

100 of 100 runs passed · 2026-10-14 · commit `b63c755` · Verilator 5.052 · 17.8 min of simulation wall time · merged functional coverage: Coverage results.

Every test of the project runs with several seeds (`scripts/regress.yaml`); the page of a test lists its seeds with the log, the timeline and the waves of each one, the stages of the test and the wave pictures of its first seed. The whole report (logs, timelines, waves) is committed in `yapp_project/regression/`; how to read a log: Logs, timelines and waves.

History

Every committed regression report, newest last (`yapp_project/regression/history.csv`, appended by `make sim-report OUT=yapp_project/regression`): the share of runs that passed, the share of coverage bins hit, and the simulation wall time.



Date	Commit	Runs passed	Bins hit	Wall
2026-10-14	`b63c755`	100 / 100	222 / 222	17.8 min

Tests

Test	Lab	Seeds passed	Sim time	Wall	In / fwd / drop (seed 1)
`base_test`	2	4/4	0s	0.3 s	0 / 0 / 0
`uvm_mem_walk_test`	11B	4/4	18us	2.2 s	0 / 0 / 0
`uvm_reset_test`	11B	4/4	520ns	0.8 s	0 / 0 / 0
`reg_access_test`	11C	4/4	350ns	0.8 s	0 / 0 / 0
`reg_function_check_test`	11C	4/4	4us–5us	3.7 s	9 / 9 / 0
`reg_function_test`	11C	4/4	4us–5us	3.8 s	9 / 9 / 0
`reg_introspection_test`	11C	4/4	700ns	0.9 s	0 / 0 / 0
`back_to_back_test`	test plan	4/4	3us	6.1 s	24 / 24 / 0
`backpressure_test`	test plan	4/4	6us–8us	2.8 s	6 / 6 / 0
`counter_wrap_test`	test plan	4/4	40us–43us	59.3 s	257 / 257 / 0
`hbus_protocol_test`	test plan	4/4	790ns	2.1 s	0 / 0 / 0

Test	Lab	Seeds passed	Sim time	Wall	In / fwd / drop (seed 1)
`hbus_sweep_test`	test plan	4/4	314us–319us	461.8 s	1021 / 1 / 1020
`parity_error_test`	test plan	4/4	7us–8us	4.9 s	12 / 12 / 0
`pkt_mem_test`	test plan	4/4	2us	1.5 s	2 / 1 / 1
`random_backpressure_test`	test plan	8/8	13us–44us	106.6 s	25 / 8 / 17
`random_config_test`	test plan	8/8	16us–8us	136.3 s	22 / 6 / 16
`random_parity_test`	test plan	8/8	13us–37us	147.6 s	94 / 60 / 34
`random_test`	test plan	8/8	12us–6us	116.0 s	22 / 7 / 15
`reg_bit_walk_test`	test plan	4/4	3us	1.1 s	0 / 0 / 0
`router_disable_test`	test plan	4/4	7us–8us	2.4 s	6 / 3 / 3
`router_filter_test`	test plan	4/4	12us–13us	6.5 s	32 / 12 / 20

Regenerate: `make regress ONLY=yapp_project WAVES=1 && make sim-report ONLY=yapp_project OUT=yapp_project/regression && make map`.

Bug hunt

A testbench is only as good as the bugs it catches. The RTL of the router carries **10 deliberate bugs**, each behind a `` `define `` (`BUG_*` in `yapp_project/rtl`, listed in `scripts/mutants.yaml`): compile the design with one of them and the regression must fail. `make bug-hunt` (`scripts/bug_hunt.py`) compiles the design once per bug, runs every test of the project on it and records which tests catch it; the `bughunt` workflow does the same in CI and goes red when a bug survives. This page is that record.

> **For the mentor**
>
> Give a student a build with one bug (`make sim-project TEST=... DEFINES=BUG_...` in `yapp_project/tb`) and the log and waves of a failing test: finding *where* the design is wrong from *what* the testbench reports is the exercise. The sections below are the answers.

No results yet: run `make bug-hunt` and `make map`.

Labs

The labs

Eleven labs, one environment. Every lab page has the same layout so you always know where to look:

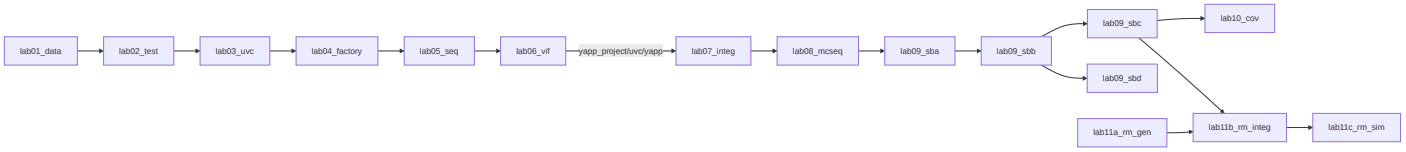
Objective → Concepts (with a diagram) → Solution, step by step → Run: the tests and the expected result → Checkpoint questions → Optional → What changed since the previous lab

Session plan

Session	Labs	Theme	Deliverable at the end
1	1, 2	data items, components, phases	a packet class and an empty test/testbench pair
2	3, 4	a UVC skeleton, factory, configuration	the YAPP agent printing packets; tests that reconfigure it
3	5	sequences, objections, randomization debug	the YAPP sequence library
4	6	interfaces, virtual interfaces, the DUT	packets flowing through the real router
5	7, 8	integrating UVCs, virtual sequencer	the full environment with a system-level sequence
6	9A, 9B (+ 9C, 9D)	TLM, scoreboard, reference model	self-checking tests
7	10	functional coverage	a coverage model and the stimulus that closes it

Session	Labs	Theme	Deliverable at the end
8	11A, 11B, 11C	register model	register tests through the RAL

Dependency chain



Labs 1–6 carry their own copy of the YAPP UVC in `labs/<lab>/sv`. From Lab 7 on, the finished UVC lives in `yapp_project/uvc/yapp` and the labs contain the testbench (`tb/`); their `run.f` compiles the UVCs from `yapp_project/uvc/` and the DUT from `yapp_project/rtl/yapp_router.f`. Labs 9A–9D carry the router module UVC as it grows (`labs/lab09_*/sv`); its finished form, with the full reference model and the coverage collectors, is `yapp_project/uvc/router`; the register model built in 11A is copied into the `tb/` of 11B and 11C, as the course does. The end of the chain, with every UVC, the virtual sequencer and the register model, is `yapp_project/tb/` (see Getting started).

Comparing two labs

The snapshots are made to be diffed:

```
diff -r labs/lab04_factory labs/lab05_seq
diff labs/lab09_sbb/sv/router_module_env.sv labs/lab09_sbc/sv/router_module_env.sv
```

Running a lab

```
cd labs/lab07_integ/tb
make run TEST=simple_test
make gui TEST=simple_test           # SimVision
make lint                           # slang elaboration check, no simulator
```

What the original manual does not give you

The course manual lists tasks, not code: "create a driver with a `run_phase` that uses `get_next_item` ...". These pages give the finished code *and* explain each decision, so a student can attempt the task, compare, and understand the difference.

Lab 1 — Creating a stimulus model

Open on the project map →

Directory: `labs/lab01_data` · **Files:** `sv/yapp_packet.sv`, `sv/yapp_pkg.sv`, `tb/top.sv`, `tb/run.f`

Objective

Model the YAPP packet as a UVM data item and explore what the base class gives you once the item describes its fields: printing, copying, cloning, comparing.

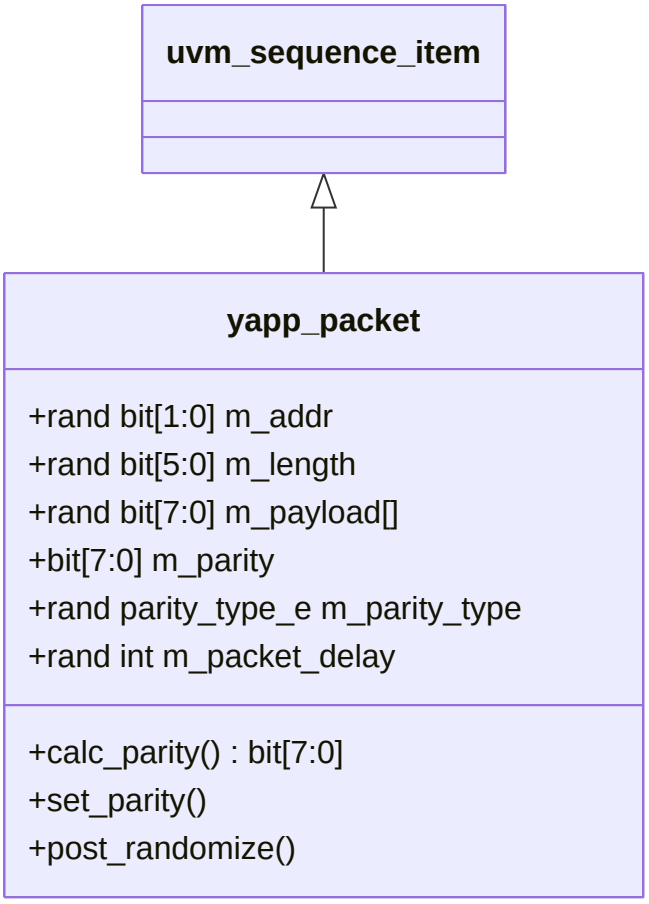
Concepts

`uvm_sequence_item` · `` `uvm_object_utils `` · `do_print` / `do_copy` / `do_compare` / `do_pack` / `do_unpack` / `do_record` · `rand` and constraints · `dist` · `post_randomize()` · control knobs · packages

No `uvm_field_*` macros here

The course material lists the fields between `` `uvm_object_utils_begin `` and `` `uvm_object_utils_end `` with `` `uvm_field_int(addr, UVM_ALL_ON) `` and so on, and the macros generate `print`, `copy`, `compare`, `pack` and `record` for you. This repository writes those methods out (`do_print` uses `printer.print_field`, `do_compare` uses `comparer.compare_field_int`, ...) so you see what each `pkt.print()` or `pkt.compare()` actually does — and what it costs to add a field.

The packet and its knobs



Field	On the wire?	Constraint
`m_addr`	yes (header `[1:0]`)	`!= 3`
`m_length`	yes (header `[7:2]`)	`1..63`, `m_payload.size() == m_length`
`payload[]`	yes	—
`m_parity`	yes	computed by `set_parity()`
`m_parity_type`	no — knob	`PARITY_TYPE_GOOD : PARITY_TYPE_BAD = 5 : 1`
`m_packet_delay`	no — knob	`1..20`

Solution

1. `sv/yapp_packet.sv`

```
//------------------------------------------------------------------------------
// yapp_packet.sv -- the YAPP packet as a UVM sequence item (Lab 1)
//
//          7   6   5   4   3   2   1   0
//        +-----------------------+-------+
// byte 0 |      length[5:0]      | addr  |  header
//        +-----------------------+-------+
// byte 1 |          payload[0]           |  \
//        |             ...               |   > N == length, 1..63 bytes
// byte N |         payload[N-1]          |  /
//        +-------------------------------+
// byte N+1 |          parity             |  even bitwise parity (XOR) of header and payload
//        +-------------------------------+
//   figure: docs/assets/packet_structure.svg (see README.md in this directory)
//
// Control knobs (not part of the packet on the wire):
//   parity_type  -- PARITY_TYPE_GOOD / PARITY_TYPE_BAD, decides what set_parity() writes
//   packet_delay -- idle clock cycles the driver inserts before the packet
//------------------------------------------------------------------------------

class yapp_packet extends uvm_sequence_item;

  // Packet fields
  rand bit[1:0] m_addr;
  rand bit[5:0] m_length;
  rand bit[7:0] m_payload[];
       bit[7:0] m_parity;          // computed, never randomized directly

  // Control knobs
  rand parity_type_e m_parity_type;
  rand int           m_packet_delay;

  `uvm_object_utils(yapp_packet)

  // Constraints (named so sequences can switch them off)
  extern constraint c_addr_legal;
  extern constraint c_length;
  extern constraint c_parity_defined;
  extern constraint c_parity_dist;
  extern constraint c_delay;

  // Printing, copying, comparing, packing and recording are written by hand
  // (do_print / do_copy / do_compare / do_pack / do_unpack / do_record below):
  // no uvm_field_* automation.
  extern virtual function void do_copy(uvm_object rhs);
  extern virtual function bit  do_compare(uvm_object rhs, uvm_comparer comparer);
  extern virtual function void do_print(uvm_printer printer);
  extern virtual function void do_pack(uvm_packer packer);
  extern virtual function void do_unpack(uvm_packer packer);
  extern virtual function void do_record(uvm_recorder recorder);

  extern function new(string name = "yapp_packet");

  // Correct parity: XOR of header and every payload byte
  extern function bit[7:0] calc_parity();

  // Write the parity byte according to the m_parity_type knob.
  // A bad parity flips one random bit of the correct value.
```

```
  extern function void set_parity();

  // Called automatically after every successful randomize(): sizes the
  // payload to `m_length` if the solver did not, then sets the parity
  extern function void post_randomize();

endclass : yapp_packet

//------------------------------------------------------------------------------
// yapp_packet -- constraints
//------------------------------------------------------------------------------

constraint yapp_packet::c_addr_legal {
  m_addr != 2'd3;
}

//------------------------------------------------------------------------------
constraint yapp_packet::c_length {
  m_length inside {[1:63]};
  m_payload.size() == m_length;
}

//------------------------------------------------------------------------------
constraint yapp_packet::c_parity_defined {
  m_parity_type != PARITY_TYPE_UNDEFINED;
}

//------------------------------------------------------------------------------
constraint yapp_packet::c_parity_dist {
  m_parity_type dist { PARITY_TYPE_GOOD := 5, PARITY_TYPE_BAD := 1 };
}

//------------------------------------------------------------------------------
constraint yapp_packet::c_delay {
  m_packet_delay inside {[1:20]};
}

//------------------------------------------------------------------------------
// yapp_packet -- method implementations
//------------------------------------------------------------------------------

function yapp_packet::new(string name = "yapp_packet");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
function bit[7:0] yapp_packet::calc_parity();
  calc_parity = {m_length, m_addr};
  foreach (m_payload[i]) begin
    calc_parity ^= m_payload[i];
  end
endfunction : calc_parity

//------------------------------------------------------------------------------
function void yapp_packet::set_parity();
  int unsigned bad_bit;
  m_parity = calc_parity();
  if (m_parity_type == PARITY_TYPE_BAD) begin
    // draw the bit once and flip it: two draws would pick two different bits
```

```
    // and leave the parity correct about half of the time
    bad_bit = rnd::get_index(8, {get_full_name(), ".bad_parity_bit"});
    m_parity[bad_bit] = ~m_parity[bad_bit];
  end
endfunction : set_parity

//------------------------------------------------------------------------------
function void yapp_packet::post_randomize();
  // Some solvers (Verilator 5.052) do not resize payload when randomize() is
  // called with an inline `with {...}`. Size it here: no sequence constrains
  // the payload bytes, so nothing is lost.
  if (m_payload.size() != m_length) begin
    m_payload = rnd::get_bytes(m_length, {get_full_name(), ".payload"});
  end
  set_parity();
endfunction : post_randomize

//------------------------------------------------------------------------------
function void yapp_packet::do_copy(uvm_object rhs);
  yapp_packet rhs_;
  if (!$cast(rhs_, rhs)) begin
    `uvm_fatal(get_type_name(), "do_copy: rhs is not a yapp_packet")
  end
  super.do_copy(rhs);
  m_addr         = rhs_.m_addr;
  m_length       = rhs_.m_length;
  m_payload      = rhs_.m_payload;
  m_parity       = rhs_.m_parity;
  m_parity_type  = rhs_.m_parity_type;
  m_packet_delay = rhs_.m_packet_delay;
endfunction : do_copy

//------------------------------------------------------------------------------
function bit yapp_packet::do_compare(uvm_object rhs, uvm_comparer comparer);
  yapp_packet rhs_;
  if (!$cast(rhs_, rhs)) begin
    return 0;
  end
  do_compare = super.do_compare(rhs, comparer);
  do_compare &= comparer.compare_field_int("addr", m_addr, rhs_.m_addr, $bits(m_addr), UVM_HEX);
  do_compare &= comparer.compare_field_int("length", m_length, rhs_.m_length, $bits(m_length),
UVM_DEC);
  do_compare &= comparer.compare_field_int("payload.size()", m_payload.size(),
rhs_.m_payload.size(), 32, UVM_DEC);
  foreach (m_payload[i]) begin
    if (i < rhs_.m_payload.size()) begin
      do_compare &= comparer.compare_field_int($sformatf("payload[%0d]", i), m_payload[i],
rhs_.m_payload[i], $bits(m_payload[i]), UVM_HEX);
    end
  end
  do_compare &= comparer.compare_field_int("parity", m_parity, rhs_.m_parity, $bits(m_parity),
UVM_HEX);
  do_compare &= comparer.compare_field_int("parity_type", m_parity_type, rhs_.m_parity_type,
$bits(m_parity_type));
endfunction : do_compare

//------------------------------------------------------------------------------
function void yapp_packet::do_print(uvm_printer printer);
  super.do_print(printer);
```

```
  printer.print_field("addr", m_addr, $bits(m_addr), UVM_HEX);
  printer.print_field("length", m_length, $bits(m_length), UVM_DEC);
  printer.print_array_header("payload", m_payload.size());
  foreach (m_payload[i]) begin
    printer.print_field($sformatf("[%0d]", i), m_payload[i], $bits(m_payload[i]), UVM_HEX, "[");
  end
  printer.print_array_footer(m_payload.size());
  printer.print_field("parity", m_parity, $bits(m_parity), UVM_HEX);
  printer.print_generic("parity_type", "parity_type_e", $bits(m_parity_type),
m_parity_type.name());
  printer.print_field("packet_delay", m_packet_delay, $bits(m_packet_delay), UVM_DEC);
endfunction : do_print

//------------------------------------------------------------------------------
function void yapp_packet::do_pack(uvm_packer packer);
  super.do_pack(packer);
  packer.pack_field_int(m_addr, $bits(m_addr));
  packer.pack_field_int(m_length, $bits(m_length));
  packer.pack_field_int(m_payload.size(), 32);
  foreach (m_payload[i]) begin
    packer.pack_field_int(m_payload[i], $bits(m_payload[i]));
  end
  packer.pack_field_int(m_parity, $bits(m_parity));
  packer.pack_field_int(m_parity_type, $bits(m_parity_type));
  packer.pack_field_int(m_packet_delay, $bits(m_packet_delay));
endfunction : do_pack

//------------------------------------------------------------------------------
function void yapp_packet::do_unpack(uvm_packer packer);
  super.do_unpack(packer);
  m_addr = packer.unpack_field_int($bits(m_addr));
  m_length = packer.unpack_field_int($bits(m_length));
  m_payload = new[packer.unpack_field_int(32)];
  foreach (m_payload[i]) begin
    m_payload[i] = packer.unpack_field_int($bits(m_payload[i]));
  end
  m_parity = packer.unpack_field_int($bits(m_parity));
  m_parity_type = parity_type_e'(packer.unpack_field_int($bits(m_parity_type)));
  m_packet_delay = packer.unpack_field_int($bits(m_packet_delay));
endfunction : do_unpack

//------------------------------------------------------------------------------
function void yapp_packet::do_record(uvm_recorder recorder);
  super.do_record(recorder);
  recorder.record_field("addr", m_addr, $bits(m_addr), UVM_HEX);
  recorder.record_field("length", m_length, $bits(m_length), UVM_DEC);
  foreach (m_payload[i]) begin
    recorder.record_field($sformatf("payload[%0d]", i), m_payload[i], $bits(m_payload[i]),
UVM_HEX);
  end
  recorder.record_field("parity", m_parity, $bits(m_parity), UVM_HEX);
  recorder.record_string("parity_type", m_parity_type.name());
  recorder.record_field("packet_delay", m_packet_delay, $bits(m_packet_delay), UVM_DEC);
endfunction : do_record
```

Points worth a second look:

- **`parity_type_e` is declared outside the class** so that tests and sequences can write `PARITY_TYPE_GOOD` without a scope prefix.
- **`m_parity` is not `rand`.** It is derived: `post_randomize()` calls `set_parity()`, which uses `calc_parity()` for a good packet and flips one bit for a bad one. The bit is drawn **once**, from the common random library (`rnd::get_index(8, ...)`, see Random values), and then flipped. Writing `m_parity[$urandom_range(7,0)] = ~m_parity[$urandom_range(7,0)]` draws twice, picks two different bits and leaves the parity correct about half of the time (a real bug, found by the first simulation of this code).
- **Named constraints** can be disabled later (`c_addr_legal.constraint_mode(0)`).
- `do_compare` does not look at `m_packet_delay` (the course's `UVM_NOCOMPARE`): two identical packets sent with different gaps still compare equal.
- The class body only *declares* the methods (`extern`); their bodies follow `endclass` as `function yapp_packet::set_parity();`. Read the class to learn what a packet can do, read below it to learn how.

2. `sv/yapp_pkg.sv`

```
//------------------------------------------------------------------------------
// yapp_pkg.sv -- YAPP UVC package (Lab 1: only the packet)
//------------------------------------------------------------------------------
package yapp_pkg;

  import uvm_pkg::*;
  import rand_util_pkg::*;   // rnd:: random values (common/rand_util_pkg.sv)
  `include "uvm_macros.svh"
  `include "yapp_defines.sv"   // typedefs and enums of the package
  `include "yapp_packet.sv"

endpackage : yapp_pkg
```

3. `tb/top.sv`

```
//------------------------------------------------------------------------------
// top.sv -- Lab 1: randomize and print five YAPP packets
//------------------------------------------------------------------------------
`timescale 1ns/1ns

module top;

  import uvm_pkg::*;
  import rand_util_pkg::*;   // rnd:: random values (common/rand_util_pkg.sv)
  `include "uvm_macros.svh"
  import yapp_pkg::*;

  yapp_packet m_pkt;

  initial begin
    m_pkt = new("pkt");
    repeat (5) begin
      if (!m_pkt.randomize()) begin
        `uvm_error("TOP", "Packet randomization failed")
      end
      m_pkt.print();
    end

    //---------------------------------------------------------------------
    // Optional: explore copy / compare / print (do_copy, do_compare, do_print in yapp_packet)
    //---------------------------------------------------------------------
    begin
      yapp_packet copy_pkt, clone_pkt;

      // copy(): field-by-field copy into an EXISTING object
      copy_pkt = new("copy_pkt");
      copy_pkt.copy(m_pkt);

      // clone(): allocate a NEW object and copy into it (returns uvm_object)
      $cast(clone_pkt, m_pkt.clone());
      clone_pkt.set_name("clone_pkt");

      // compare(): returns 1 when every field flagged for compare matches
      `uvm_info("TOP", $sformatf("copy  compare -> %0d", m_pkt.compare(copy_pkt)),  UVM_NONE)
      `uvm_info("TOP", $sformatf("clone compare -> %0d", m_pkt.compare(clone_pkt)), UVM_NONE)
      clone_pkt.m_payload[0] = ~clone_pkt.m_payload[0];
      `uvm_info("TOP", $sformatf("modified clone compare -> %0d (miscompare is reported above)",
                                 m_pkt.compare(clone_pkt)), UVM_NONE)

      // The same packet with the table and the tree printer
      m_pkt.print(uvm_default_table_printer);
      m_pkt.print(uvm_default_tree_printer);
    end
  end

endmodule : top
```

4. `tb/run.f`

```
// Lab 1 -- xrun command file
// The course recommends the Cadence UVM 1.1d library; use CDNS-1.2 if you prefer.
-uvmhome CDNS-1.1d
-timescale 1ns/1ns

-incdir ../sv             // include directory for sv files
-incdir ../../../common    // rand_util_defines.sv, uvm_version_compat.svh
../../../common/rand_util_pkg.sv  // rnd:: random values (before the packages that use it)
../sv/yapp_pkg.sv         // compile YAPP package
top.sv                    // compile top level module
```

Run

```
cd labs/lab01_data/tb && make run
```

Expected: five packet tables. Every `m_addr` is 0, 1 or 2; `m_payload` has exactly `m_length` entries; about one packet in six shows `PARITY_TYPE_BAD` with a parity byte that differs from the XOR of the other bytes. Then the optional section prints `copy compare -> 1`, `clone compare -> 1`, a miscompare message for the modified clone, and the same packet in table and tree layout.

See it run

The lines that matter from the real log of every test of this lab (Verilator, seed 1, from the last `make regress`; the paths and the packet tables are left out). Your own log must show the same lines -- `make check LAB=lab01_data START=1` looks for them.

default

```
  addr          integral       2     'h1
  length        integral       6     'd20
  parity_type   parity_type_e  32    GOOD_PARITY
  addr          integral       2     'h1
  length        integral       6     'd30
  parity_type   parity_type_e  32    GOOD_PARITY
  length        integral       6     'd7
@0 reporter [TOP] copy  compare -> 1
@0 reporter [TOP] clone compare -> 1
@0 reporter [MISCMP] Miscompare for pkt.payload[0]: lhs = 'hb6 : rhs = 'h49
@0 reporter [MISCMP] 1 Miscompare(s) for object pkt@62 vs. clone_pkt@188
@0 reporter [TOP] modified clone compare -> 0 (miscompare is reported above)
... (11 more lines of the kinds above)
```

Try this

Small changes to make in `start/` once the lab works, each with a question; the answer unfolds.

Make bad parity rarer: change the `dist` in `c_parity_type` so that only one packet in twenty has `PARITY_TYPE_BAD`. Run again -- how many of the five packets are bad now, and why can the answer still be two?

Add a constraint that forbids `m_length == 1`. Which packets change, and what does `print()` show for `m_payload` now?

Checkpoint questions

Why does `m_parity` not have the `rand` qualifier?

What does `m_payload.size() == m_length` do during randomization?

What is the difference between `copy()` and `clone()`?

Where does `print()` get the field names and formats from?

Optional

`copy()`, `clone()`, `compare()`, table vs tree printer — all in `top.sv` above.

What changed since the previous lab

This is the first lab. Lab 2 removes the randomize-and-print loop from `top.sv` and replaces it with `run_test()`.

Lab 2 — Creating test and testbench components

Open on the project map →

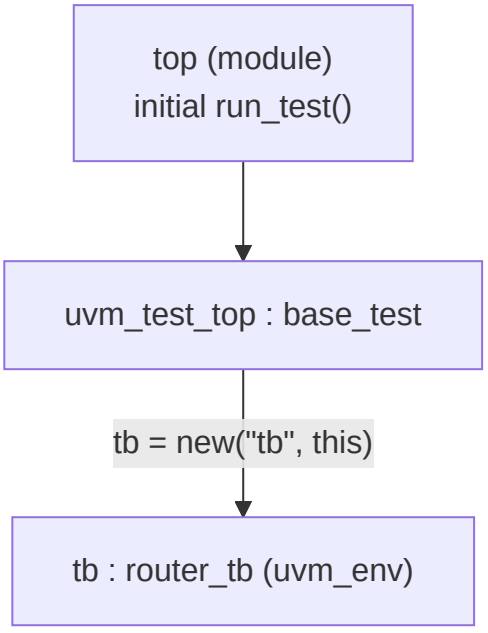
Directory: `labs/lab02_test` · **New files:** `tb/router_tb.sv`, `tb/router_test_lib.sv`

Objective

Start the UVM component hierarchy: a test that owns a testbench, started by `run_test()`, with the phases doing the work.

Concepts

`uvm_env` · `uvm_test` · `` `uvm_component_utils `` · component constructor `(name, parent)` · `build_phase` · `end_of_elaboration_phase` · `uvm_top.print_topology()` · `run_test()` · `+UVM_TESTNAME` · verbosity



Solution

1. `tb/router_tb.sv`

```
//------------------------------------------------------------------------------
// router_tb.sv -- the testbench environment (Lab 2)
//
// For now it is empty: the point of this lab is the component hierarchy
// (test -> testbench) and the phases that build it.
//------------------------------------------------------------------------------
class router_tb extends uvm_env;

  `uvm_component_utils(router_tb)

  extern function new(string name, uvm_component parent);
  extern function void build_phase(uvm_phase phase);

endclass : router_tb

//------------------------------------------------------------------------------
// router_tb -- method implementations
//------------------------------------------------------------------------------

function router_tb::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//------------------------------------------------------------------------------
function void router_tb::build_phase(uvm_phase phase);
  super.build_phase(phase);
  `uvm_info(get_type_name(), "Executing the build phase of the testbench", UVM_HIGH)
endfunction : build_phase
```

2. `tb/router_test_lib.sv`

```
//------------------------------------------------------------------------------
// router_test_lib.sv -- test library (Lab 2)
//
// One class per file: the classes live in tests/<class>.sv and are included below.
//------------------------------------------------------------------------------

`include "tests/base_test.sv"
`include "tests/test2.sv"

//------------------------------------------------------------------------------
// base_test.sv -- base_test (test library (Lab 2))
// Split out of router_test_lib.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// base_test: builds the testbench and prints the topology
//------------------------------------------------------------------------------
class base_test extends uvm_test;

  router_tb m_tb;

  `uvm_component_utils(base_test)

  extern function new(string name, uvm_component parent);
  extern function void build_phase(uvm_phase phase);

  extern function void end_of_elaboration_phase(uvm_phase phase);

endclass : base_test

//------------------------------------------------------------------------------
// base_test -- method implementations
//------------------------------------------------------------------------------

function base_test::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//------------------------------------------------------------------------------
function void base_test::build_phase(uvm_phase phase);
  super.build_phase(phase);
  `uvm_info(get_type_name(), "Executing the build phase of the test", UVM_HIGH)
  m_tb = new("tb", this);          // the test OWNS the testbench
endfunction : build_phase

//------------------------------------------------------------------------------
function void base_test::end_of_elaboration_phase(uvm_phase phase);
  uvm_top.print_topology();      // the whole component tree, once it is built
endfunction : end_of_elaboration_phase

//------------------------------------------------------------------------------
// test2.sv -- test2 (test library (Lab 2))
// Split out of router_test_lib.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// test2 (optional): the smallest possible test -- everything is inherited.
// Select it with +UVM_TESTNAME=test2, no recompilation needed.
```

```
//------------------------------------------------------------------------------
class test2 extends base_test;

  `uvm_component_utils(test2)

  extern function new(string name, uvm_component parent);

endclass : test2

//------------------------------------------------------------------------------
// test2 -- method implementations
//------------------------------------------------------------------------------

function test2::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new
```

The testbench is constructed **after** `super.build_phase(phase)`: the base implementation applies configuration to the test's own fields first.

3. `tb/top.sv`

```
//------------------------------------------------------------------------------
// top.sv -- Lab 2: start UVM phasing with run_test()
//
// Include order matters: the test library refers to router_tb, so the
// testbench must be included first.
//------------------------------------------------------------------------------
`timescale 1ns/1ns

module top;

  import uvm_pkg::*;
  import rand_util_pkg::*;   // rnd:: random values (common/rand_util_pkg.sv)
  `include "uvm_macros.svh"
  import yapp_pkg::*;

  `include "router_tb.sv"
  `include "router_test_lib.sv"

  initial begin
    run_test();      // the test is chosen with +UVM_TESTNAME=<test>
  end

endmodule : top
```

The order of the two `` `include ``s matters: `base_test` has a `router_tb` handle, so `router_tb` must be declared first.

4. `tb/run.f`

```
// Lab 2 -- xrun command file
-uvmhome CDNS-1.1d
-timescale 1ns/1ns

-incdir ../sv
-incdir ../../../common    // rand_util_defines.sv, uvm_version_compat.svh
../../../common/rand_util_pkg.sv  // rnd:: random values (before the packages that use it)
../sv/yapp_pkg.sv
top.sv

+UVM_TESTNAME=base_test
+UVM_VERBOSITY=UVM_HIGH     // try UVM_LOW: no recompile needed
```

Run

```
make run                                    # +UVM_TESTNAME=base_test +UVM_VERBOSITY=UVM_HIGH from
run.f
make run TEST=test2
make run XRUN_OPTS=+UVM_VERBOSITY=UVM_LOW   # overrides the run.f setting, no recompile
```

Expected (UVM_HIGH):

```
UVM_INFO ... [RNTST] Running test base_test...
UVM_INFO ... [base_test] Executing the build phase of the test
UVM_INFO ... [router_tb] Executing the build phase of the testbench
UVM_INFO ... [UVMTOP] UVM testbench topology:
--------------------------------
Name           Type        Size  Value
--------------------------------
uvm_test_top   base_test   -     @...
  tb           router_tb   -     @...
--------------------------------
```

With `UVM_LOW` the two "Executing the build phase" lines disappear; the topology stays (it is printed at `UVM_NONE`).

See it run

The lines that matter from the real log of every test of this lab (Verilator, seed 1, from the last `make regress`; the paths and the packet tables are left out). Your own log must show the same lines -- `make check LAB=lab02_test START=1` looks for them.

base_test

```
@0 reporter [RNTST] Running test base_test...
@0 uvm_test_top [base_test] Executing the build phase of the test
@0 uvm_test_top.tb [router_tb] Executing the build phase of the testbench
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
```

Try this

Small changes to make in `start/` once the lab works, each with a question; the answer unfolds.

Run the test with `+UVM_VERBOSITY=UVM_HIGH` and then with `UVM_LOW`. Which lines disappear, and where does the build-phase message of `router_tb` come from?

Checkpoint questions

Does the printed topology match the hierarchy you expected?

Which test class is being executed?

Do you see build-phase reports from both test and testbench?

What is the *minimum* code for `test2`?

Optional

`test2` is in the solution. Switch between tests with `+UVM_TESTNAME` only.

What changed since the previous lab

```
diff -r labs/lab01_data labs/lab02_test
```

- `tb/top.sv`: the randomize/print loop is gone; `run_test()` and the two includes are in.
- new `tb/router_tb.sv`, `tb/router_test_lib.sv`.
- `tb/run.f`: `+UVM_TESTNAME`, `+UVM_VERBOSITY`.

Lab 3 — Creating a simple UVC

Open on the project map →

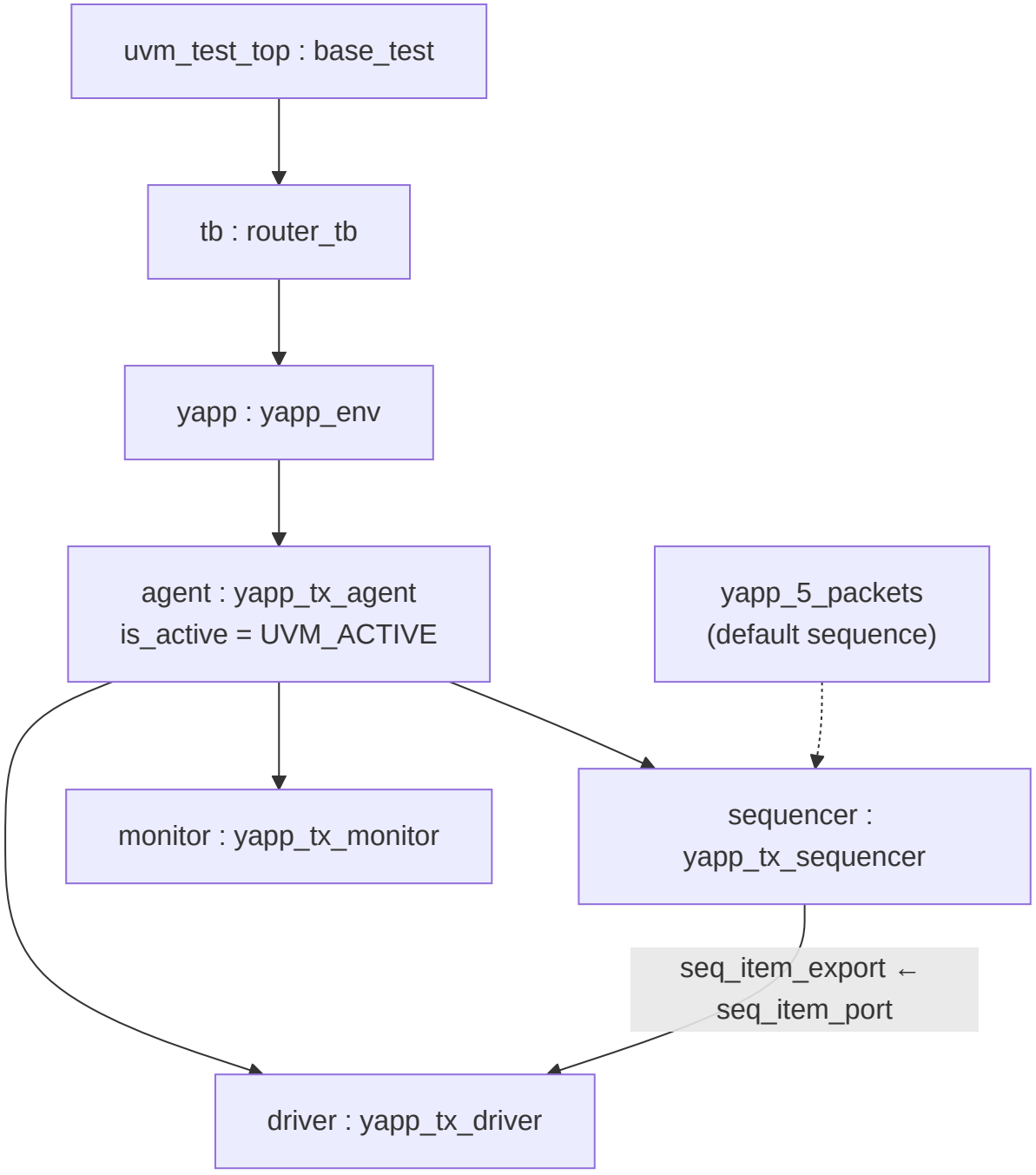
Directory: `labs/lab03_uvc` · **New files:** `sv/yapp_tx_driver.sv`, `yapp_tx_sequencer.sv`, `yapp_tx_monitor.sv`, `yapp_tx_agent.sv`, `yapp_env.sv`, `yapp_tx_seqs.sv`

Objective

Build the front end of the YAPP input UVC — driver, sequencer, monitor, agent, env — and watch the component phases run.

Concepts

`uvm_driver #(T)` · `uvm_sequencer #(T)` · `uvm_monitor` · `uvm_agent` · `is_active` · `seq_item_port` / `seq_item_export` · `get_next_item()` / `item_done()` · `sprint()` · default sequence through `uvm_config_wrapper` · phase order



Solution

1. Driver — `sv/yapp_tx_driver.sv`

For now the driver only prints the packet; pins arrive in Lab 6.

```
//------------------------------------------------------------------------------
// yapp_tx_driver.sv -- Lab 3: pulls packets from the sequencer and (for now)
// only prints them. The real pin wiggling arrives in Lab 6.
//------------------------------------------------------------------------------
class yapp_tx_driver extends uvm_driver #(yapp_packet);

  `uvm_component_utils(yapp_tx_driver)

  extern function new(string name, uvm_component parent);
  extern task run_phase(uvm_phase phase);

  extern task send_to_dut(yapp_packet pkt);

  // Optional: which component reports first / last and why?
  extern function void start_of_simulation_phase(uvm_phase phase);

endclass : yapp_tx_driver

//------------------------------------------------------------------------------
// yapp_tx_driver -- method implementations
//------------------------------------------------------------------------------

function yapp_tx_driver::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//------------------------------------------------------------------------------
task yapp_tx_driver::run_phase(uvm_phase phase);
  forever begin
    seq_item_port.get_next_item(req);     // blocks until a sequence sends one
    send_to_dut(req);
    seq_item_port.item_done();            // unblocks the sequence
  end
endtask : run_phase

//------------------------------------------------------------------------------
task yapp_tx_driver::send_to_dut(yapp_packet pkt);
  `uvm_info(get_type_name(), $sformatf("Packet is \n%s", pkt.sprint()), UVM_LOW)
  #10ns;                                  // makes the log easier to follow
endtask : send_to_dut

//------------------------------------------------------------------------------
function void yapp_tx_driver::start_of_simulation_phase(uvm_phase phase);
  `uvm_info(get_type_name(), {"start of simulation for ", get_full_name()}, UVM_HIGH)
endfunction : start_of_simulation_phase
```

2. Sequencer — `sv/yapp_tx_sequencer.sv`

```
//------------------------------------------------------------------------------
// yapp_tx_sequencer.sv -- arbitrates yapp_packet items towards the driver (Lab 3)
//
// Nothing to add: uvm_sequencer #(T) already contains the whole machinery.
//------------------------------------------------------------------------------
class yapp_tx_sequencer extends uvm_sequencer #(yapp_packet);

  `uvm_component_utils(yapp_tx_sequencer)

  extern function new(string name, uvm_component parent);

  // Optional: which component reports first / last and why?
  extern function void start_of_simulation_phase(uvm_phase phase);

endclass : yapp_tx_sequencer

//------------------------------------------------------------------------------
// yapp_tx_sequencer -- method implementations
//------------------------------------------------------------------------------

function yapp_tx_sequencer::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//------------------------------------------------------------------------------
function void yapp_tx_sequencer::start_of_simulation_phase(uvm_phase phase);
  `uvm_info(get_type_name(), {"start of simulation for ", get_full_name()}, UVM_HIGH)
endfunction : start_of_simulation_phase
```

3. Monitor — `sv/yapp_tx_monitor.sv`

```
//------------------------------------------------------------------------------
// yapp_tx_monitor.sv -- Lab 3: the monitor shell (no interface yet)
//
// Monitors do NOT take a type parameter: uvm_monitor is a plain component.
//------------------------------------------------------------------------------
class yapp_tx_monitor extends uvm_monitor;

  `uvm_component_utils(yapp_tx_monitor)

  extern function new(string name, uvm_component parent);
  extern task run_phase(uvm_phase phase);

  // Optional: which component reports first / last and why?
  extern function void start_of_simulation_phase(uvm_phase phase);

endclass : yapp_tx_monitor

//------------------------------------------------------------------------------
// yapp_tx_monitor -- method implementations
//------------------------------------------------------------------------------

function yapp_tx_monitor::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//------------------------------------------------------------------------------
task yapp_tx_monitor::run_phase(uvm_phase phase);
  `uvm_info(get_type_name(), "Inside the monitor run_phase", UVM_LOW)
endtask : run_phase

//------------------------------------------------------------------------------
function void yapp_tx_monitor::start_of_simulation_phase(uvm_phase phase);
  `uvm_info(get_type_name(), {"start of simulation for ", get_full_name()}, UVM_HIGH)
endfunction : start_of_simulation_phase
```

4. Agent — `sv/yapp_tx_agent.sv`

```
//------------------------------------------------------------------------------
// yapp_tx_agent.sv -- the YAPP transmit agent (Lab 3)
//
//   is_active == UVM_ACTIVE  : monitor + driver + sequencer (generates traffic)
//   is_active == UVM_PASSIVE : monitor only (just watches the bus)
//------------------------------------------------------------------------------
class yapp_tx_agent extends uvm_agent;

  yapp_tx_monitor   m_monitor;
  yapp_tx_driver    m_driver;
  yapp_tx_sequencer m_sequencer;

  // is_active is inherited from uvm_agent (default UVM_ACTIVE); build_phase
  // picks up an override set with uvm_config_int::set(..., "is_active", ...)
  `uvm_component_utils(yapp_tx_agent)

  // The fields are printed by do_print() below and read from uvm_config_db in build_phase():
  // no uvm_field_* automation.
  extern virtual function void do_print(uvm_printer printer);
  extern function new(string name, uvm_component parent);

  extern function void build_phase(uvm_phase phase);
  extern function void connect_phase(uvm_phase phase);

  // Optional: which component reports first / last and why?
  extern function void start_of_simulation_phase(uvm_phase phase);

endclass : yapp_tx_agent

//------------------------------------------------------------------------------
// yapp_tx_agent -- method implementations
//------------------------------------------------------------------------------

function yapp_tx_agent::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//------------------------------------------------------------------------------
function void yapp_tx_agent::build_phase(uvm_phase phase);
  uvm_bitstream_t cfg_is_active;
  super.build_phase(phase);
  // overrides set with uvm_config_int::set(...)
  if (uvm_config_int::get(this, "", "is_active", cfg_is_active)) begin
    is_active = uvm_active_passive_enum'(cfg_is_active);
  end
  m_monitor = new("monitor", this);
  if (is_active == UVM_ACTIVE) begin
    m_driver    = new("driver", this);
    m_sequencer = new("sequencer", this);
  end
endfunction : build_phase

//------------------------------------------------------------------------------
function void yapp_tx_agent::connect_phase(uvm_phase phase);
  if (is_active == UVM_ACTIVE) begin
    m_driver.seq_item_port.connect(m_sequencer.seq_item_export);
  end
endfunction : connect_phase
```

```
//------------------------------------------------------------------------------
function void yapp_tx_agent::start_of_simulation_phase(uvm_phase phase);
  `uvm_info(get_type_name(), {"start of simulation for ", get_full_name()}, UVM_HIGH)
endfunction : start_of_simulation_phase

//------------------------------------------------------------------------------
function void yapp_tx_agent::do_print(uvm_printer printer);
  super.do_print(printer);
  printer.print_generic("is_active", "uvm_active_passive_enum", $bits(is_active),
is_active.name());
endfunction : do_print
```

5. Env — `sv/yapp_env.sv`

```
//------------------------------------------------------------------------------
// yapp_env.sv -- YAPP UVC top level (Lab 3)
//------------------------------------------------------------------------------
class yapp_env extends uvm_env;

  yapp_tx_agent m_agent;

  `uvm_component_utils(yapp_env)

  extern function new(string name, uvm_component parent);
  extern function void build_phase(uvm_phase phase);

  // Optional: which component reports first / last and why?
  extern function void start_of_simulation_phase(uvm_phase phase);

endclass : yapp_env

//------------------------------------------------------------------------------
// yapp_env -- method implementations
//------------------------------------------------------------------------------

function yapp_env::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//------------------------------------------------------------------------------
function void yapp_env::build_phase(uvm_phase phase);
  super.build_phase(phase);
  m_agent = new("agent", this);
endfunction : build_phase

//------------------------------------------------------------------------------
function void yapp_env::start_of_simulation_phase(uvm_phase phase);
  `uvm_info(get_type_name(), {"start of simulation for ", get_full_name()}, UVM_HIGH)
endfunction : start_of_simulation_phase
```

6. The provided sequences — `sv/yapp_tx_seqs.sv`

```
//------------------------------------------------------------------------------
// yapp_tx_seqs.sv -- YAPP sequence library (Lab 3: provided base + 5 packets)
//
// One class per file: the classes live in seqs/<class>.sv and are included below.
//------------------------------------------------------------------------------

`include "seqs/yapp_base_seq.sv"
`include "seqs/yapp_5_packets.sv"
```

```
//------------------------------------------------------------------------------
// yapp_base_seq.sv -- YAPP sequence library (Lab 3: provided base + 5 packets)
// Split out of yapp_tx_seqs.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// yapp_base_seq: raises an objection while the sequence runs, so run_phase
// does not end before the last packet was delivered.
//------------------------------------------------------------------------------
class yapp_base_seq extends uvm_sequence #(yapp_packet);

  `uvm_object_utils(yapp_base_seq)

  extern function new(string name = "yapp_base_seq");
  extern task pre_body();

  extern task post_body();

endclass : yapp_base_seq

//------------------------------------------------------------------------------
// yapp_base_seq -- method implementations
//------------------------------------------------------------------------------

function yapp_base_seq::new(string name = "yapp_base_seq");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task yapp_base_seq::pre_body();
  uvm_phase phase = `YAPP_STARTING_PHASE;
  if (phase != null) begin
    phase.raise_objection(this, get_type_name());
  end
endtask : pre_body

//------------------------------------------------------------------------------
task yapp_base_seq::post_body();
  uvm_phase phase = `YAPP_STARTING_PHASE;
  if (phase != null) begin
    phase.drop_objection(this, get_type_name());
  end
endtask : post_body
```

```
//------------------------------------------------------------------------------
// yapp_5_packets.sv -- YAPP sequence library (Lab 3: provided base + 5 packets)
// Split out of yapp_tx_seqs.sv: one class per file.
//------------------------------------------------------------------------------
```

```
//------------------------------------------------------------------------------
// yapp_5_packets: five random packets (provided in Lab 3)
//   Test class configuration template:
//     uvm_config_wrapper::set(this, "<path>.run_phase",
//                             "default_sequence", yapp_5_packets::get_type());
//------------------------------------------------------------------------------
class yapp_5_packets extends yapp_base_seq;

  `uvm_object_utils(yapp_5_packets)

  extern function new(string name = "yapp_5_packets");
  extern task body();

endclass : yapp_5_packets

//------------------------------------------------------------------------------
// yapp_5_packets -- method implementations
//------------------------------------------------------------------------------

function yapp_5_packets::new(string name = "yapp_5_packets");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task yapp_5_packets::body();
  `uvm_info(get_type_name(), "Executing yapp_5_packets sequence", UVM_LOW)
  repeat (5) begin
    req = yapp_packet::type_id::create("req");
    start_item(req);
    if (!req.randomize()) begin
      `uvm_error(get_type_name(), "req.randomize() failed")
    end
    finish_item(req);
  end
endtask : body
```

7. Package include order — `sv/yapp_pkg.sv`

```
//-----------------------------------------------------------------------------
// yapp_pkg.sv -- YAPP UVC package (Lab 3)
//
// Include order matters: a class must be included before any file that
// uses it (packet -> monitor -> sequencer -> sequences -> driver -> agent -> env).
//-----------------------------------------------------------------------------
package yapp_pkg;

  import uvm_pkg::*;
  import rand_util_pkg::*;   // rnd:: random values (common/rand_util_pkg.sv)
  `include "uvm_macros.svh"
  `include "uvm_version_compat.svh"
  `include "yapp_defines.sv"   // typedefs and enums of the package
  `include "yapp_packet.sv"
  `include "yapp_tx_monitor.sv"
  `include "yapp_tx_sequencer.sv"
  `include "yapp_tx_seqs.sv"
  `include "yapp_tx_driver.sv"
  `include "yapp_tx_agent.sv"
  `include "yapp_env.sv"

endpackage : yapp_pkg
```

8. Testbench and test

`router_tb` gets a `yapp_env yapp` handle built in `build_phase`. `base_test` sets the **default sequence** of the sequencer *before* building the testbench:

```
uvm_config_wrapper::set(this, "tb.yapp.agent.sequencer.run_phase",
                        "default_sequence", yapp_5_packets::get_type());
m_tb = new("tb", this);
```

The path `tb.yapp.agent.sequencer` is the one you read off the topology report when you ran `base_test` before adding the sequence.

Run

```
make run                                    # base_test, 5 packets
make run XRUN_OPTS="+SVSEED=random"         # different packets
make run XRUM_OPTS="+UVM_VERBOSITY=UVM_HIGH"    # see the start_of_simulation messages
```

Expected: the topology now shows `tb.yapp.agent.{driver,monitor,sequencer}` with `is_active UVM_ACTIVE`; then `Executing yapp_5_packets sequence`, the monitor's "Inside the monitor run_phase", and five `TX pkt #n: addr=… len=…` lines from the driver, 10 ns apart (at `+UVM_VERBOSITY=UVM_HIGH` each one is followed by the full `Packet is` table).

See it run

The lines that matter from the real log of every test of this lab (Verilator, seed 1, from the last `make regress`; the paths and the packet tables are left out). Your own log must show the same lines -- `make check LAB=lab03_uvc START=1` looks for them.

base_test

```
@0 reporter [RNTST] Running test base_test...
@0 uvm_test_top.tb.yapp.agent.sequencer@@yapp_5_packets [yapp_5_packets] Executing yapp_5_packets
sequence
@0 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                          integral       2     'h0
  length                        integral       6     40
  parity_type                   parity_type_e  32    GOOD_PARITY
@10 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                          integral       2     'h0
  length                        integral       6     60
  parity_type                   parity_type_e  32    GOOD_PARITY
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (12 more lines of the kinds above)
```

Try this

Small changes to make in `start/` once the lab works, each with a question; the answer unfolds.

Create the agent passive from the test (`uvm_config_int::set(this, "tb.yapp.agent", "is_active", UVM_PASSIVE)` before `super.build_phase`). Which components vanish from the topology, and what does the monitor print?

Checkpoint questions

Full hierarchical path from the test to the sequencer?

What value does `is_active` have in the topology print?

Which `start_of_simulation_phase` runs first, which last, why?

Why `monitor` unconditionally but `driver` / `sequencer` only when active?

Optional

`start_of_simulation_phase()` is present in every component of this lab (removed again from Lab 4 on, to keep the code lean).

What changed since the previous lab

```
diff -r labs/lab02_test labs/lab03_uvc
```

- six new files in `sv/`, included by `yapp_pkg.sv` in dependency order
- `router_tb` builds `yapp_env`; `base_test` sets the default sequence
- `run.f`: `-incdir ../../../common` (the `starting_phase` shim) and `+SVSEED=random`

Lab 4 — Using factories

Open on the project map →

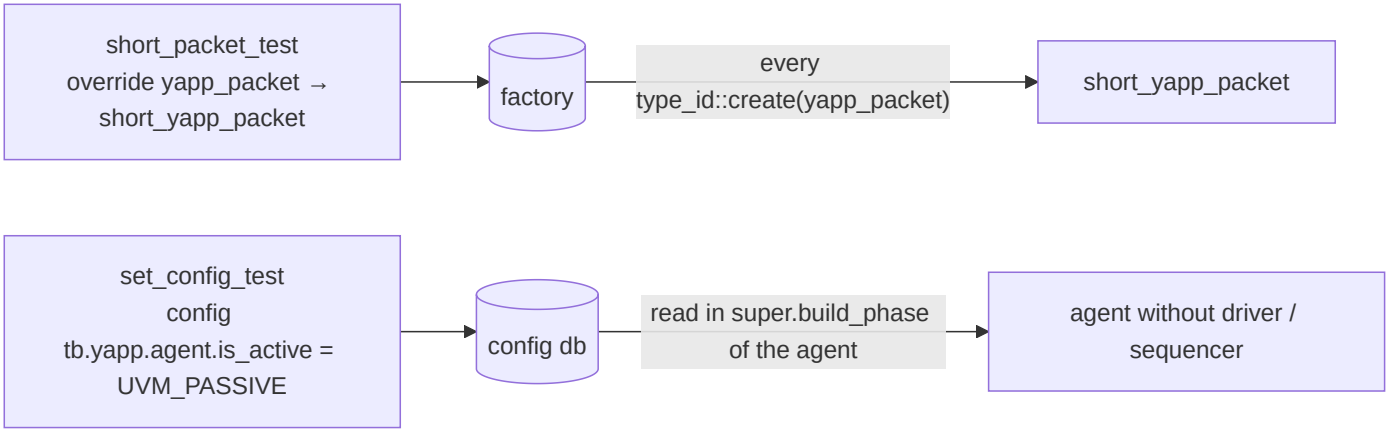
Directory: `labs/lab04_factory` · **Changed:** every `new()` of a component → `type_id::create()`; `sv/yapp_packet.sv` (+ `short_yapp_packet`); `tb/router_test_lib.sv` (+ 2 tests)

Objective

Create components and data through the factory so that a test can replace types and change configuration without touching the UVC.

Concepts

`type_id::create()` · `set_type_override_by_type()` · `uvm_config_int::set()` · `check_config_usage()` · `recording_detail`



Solution

1. `new()` → `type_id::create()`

In `router_test_lib.sv`, `router_tb.sv`, `yapp_env.sv`, `yapp_tx_agent.sv`:

```
m_tb        = router_tb::type_id::create("tb", this);
m_yapp      = yapp_env::type_id::create("yapp", this);
m_agent     = yapp_tx_agent::type_id::create("agent", this);
m_monitor   = yapp_tx_monitor::type_id::create("monitor", this);
```

2. `short_yapp_packet` — `sv/short_yapp_packet.sv`

`c_no_addr2` is removed again in Lab 5.

```
//------------------------------------------------------------------------------
// short_yapp_packet.sv -- short_yapp_packet (Lab 4)
// Split out of yapp_packet.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// short_yapp_packet (Lab 4): same packet, payload shorter than 15 bytes and
// never addressed to channel 2. Because it is a subclass, the factory can
// substitute it for every yapp_packet without touching the sequences.
//------------------------------------------------------------------------------
class short_yapp_packet extends yapp_packet;

  `uvm_object_utils(short_yapp_packet)

  extern constraint c_short_length;
  extern constraint c_no_addr2;

  extern function new(string name = "short_yapp_packet");

endclass : short_yapp_packet

//------------------------------------------------------------------------------
// short_yapp_packet -- constraints
//------------------------------------------------------------------------------

constraint short_yapp_packet::c_short_length {
  m_length < 15;
}

//------------------------------------------------------------------------------
constraint short_yapp_packet::c_no_addr2 {
  m_addr != 2'd2;
}

//------------------------------------------------------------------------------
// short_yapp_packet -- method implementations
//------------------------------------------------------------------------------

function short_yapp_packet::new(string name = "short_yapp_packet");
  super.new(name);
endfunction : new
```

3. The tests — `tb/router_test_lib.sv`

```
//------------------------------------------------------------------------------
// router_test_lib.sv -- test library (Lab 4: factory, overrides, configuration)
//
// One class per file: the classes live in tests/<class>.sv and are included below.
//------------------------------------------------------------------------------

`include "tests/base_test.sv"
`include "tests/short_packet_test.sv"
`include "tests/set_config_test.sv"

//------------------------------------------------------------------------------
// base_test.sv -- base_test (test library (Lab 4: factory, overrides, configuration))
// Split out of router_test_lib.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// base_test
//   * builds the testbench through the factory
//   * turns transaction recording on for every component
//   * the default sequence is set in configure_sequences(), a virtual method,
//     so a derived test can replace or skip it (see set_config_test)
//   * check_config_usage() reports settings nobody consumed
//------------------------------------------------------------------------------
class base_test extends uvm_test;

  router_tb m_tb;

  `uvm_component_utils(base_test)

  extern function new(string name, uvm_component parent);
  extern function void build_phase(uvm_phase phase);

  // Default sequences for the sequencers in the testbench
  extern virtual function void configure_sequences();

  extern function void end_of_elaboration_phase(uvm_phase phase);
  extern function void check_phase(uvm_phase phase);

endclass : base_test

//------------------------------------------------------------------------------
// base_test -- method implementations
//------------------------------------------------------------------------------

function base_test::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//------------------------------------------------------------------------------
function void base_test::build_phase(uvm_phase phase);
  super.build_phase(phase);
  `uvm_info(get_type_name(), "Executing the build phase of the test", UVM_HIGH)
  uvm_config_int::set(this, "*", "recording_detail", 1);   // transaction recording
  configure_sequences();                                   // before the tb is built!
  m_tb = router_tb::type_id::create("tb", this);
endfunction : build_phase

//------------------------------------------------------------------------------
```

```
function void base_test::configure_sequences();
  uvm_config_wrapper::set(this, "tb.yapp.agent.sequencer.run_phase",
                          "default_sequence", yapp_5_packets::get_type());
endfunction : configure_sequences

//--------------------------------------------------------------------------------
function void base_test::end_of_elaboration_phase(uvm_phase phase);
  uvm_top.print_topology();
endfunction : end_of_elaboration_phase

//--------------------------------------------------------------------------------
function void base_test::check_phase(uvm_phase phase);
  check_config_usage();      // lists configuration settings that were never read
endfunction : check_phase

//--------------------------------------------------------------------------------
// short_packet_test.sv -- short_packet_test (test library (Lab 4: factory, overrides,
configuration))
// Split out of router_test_lib.sv: one class per file.
//--------------------------------------------------------------------------------

//--------------------------------------------------------------------------------
// short_packet_test: every yapp_packet the factory creates becomes a
// short_yapp_packet. Nothing in the UVC changes.
//--------------------------------------------------------------------------------
class short_packet_test extends base_test;

  `uvm_component_utils(short_packet_test)

  extern function new(string name, uvm_component parent);
  extern function void build_phase(uvm_phase phase);

endclass : short_packet_test

//--------------------------------------------------------------------------------
// short_packet_test -- method implementations
//--------------------------------------------------------------------------------

function short_packet_test::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//--------------------------------------------------------------------------------
function void short_packet_test::build_phase(uvm_phase phase);
  set_type_override_by_type(yapp_packet::get_type(), short_yapp_packet::get_type());
  super.build_phase(phase);
endfunction : build_phase

//--------------------------------------------------------------------------------
// set_config_test.sv -- set_config_test (test library (Lab 4: factory, overrides,
configuration))
// Split out of router_test_lib.sv: one class per file.
//--------------------------------------------------------------------------------

//--------------------------------------------------------------------------------
// set_config_test: the YAPP agent becomes passive -> no driver, no sequencer.
// A passive agent has no sequencer, so no default sequence is configured;
// otherwise check_config_usage() would (rightly) complain about an unused
// "default_sequence" setting.
```

```
//------------------------------------------------------------------------------
class set_config_test extends base_test;

  `uvm_component_utils(set_config_test)

  extern function new(string name, uvm_component parent);
  extern function void build_phase(uvm_phase phase);

  extern function void configure_sequences();

endclass : set_config_test

//------------------------------------------------------------------------------
// set_config_test -- method implementations
//------------------------------------------------------------------------------

function set_config_test::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//------------------------------------------------------------------------------
function void set_config_test::build_phase(uvm_phase phase);
  // Must be set BEFORE the agent is built (i.e. before super.build_phase)
  uvm_config_int::set(this, "tb.yapp.agent", "is_active", UVM_PASSIVE);
  super.build_phase(phase);
endfunction : build_phase

//------------------------------------------------------------------------------
function void set_config_test::configure_sequences();
  // intentionally empty: there is no sequencer to configure
endfunction : configure_sequences
```

4. The testbench — `tb/router_tb.sv`

```
//------------------------------------------------------------------------------
// router_tb.sv -- the testbench environment (Lab 4: built through the factory)
//------------------------------------------------------------------------------
class router_tb extends uvm_env;

  yapp_env m_yapp;

  `uvm_component_utils(router_tb)

  extern function new(string name, uvm_component parent);
  extern function void build_phase(uvm_phase phase);

endclass : router_tb

//------------------------------------------------------------------------------
// router_tb -- method implementations
//------------------------------------------------------------------------------

function router_tb::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//------------------------------------------------------------------------------
function void router_tb::build_phase(uvm_phase phase);
  super.build_phase(phase);
  `uvm_info(get_type_name(), "Executing the build phase of the testbench", UVM_HIGH)
  // type_id::create() instead of new(): a test can now override the type
  m_yapp = yapp_env::type_id::create("yapp", this);
endfunction : build_phase
```

Run

```
make run                          # base_test: 5 packets, same as Lab 3
make run TEST=short_packet_test   # 5 packets of type short_yapp_packet
make run TEST=set_config_test     # passive agent, nothing runs
```

Expected:

- `short_packet_test`: every `TX pkt` line has `len` < 15 and never `addr=2`; at `UVM_HIGH` each `Packet is` table starts with `short_yapp_packet`.
- `set_config_test`: topology shows `tb.yapp.agent` with `is_active` `UVM_PASSIVE` and only a `monitor` below it. **No** configuration-usage report at the end.

The configuration-usage question

If `base_test` set the default sequence unconditionally (as Lab 3 did), `set_config_test` would print:

```
UVM_INFO ... [CFGNRD] ::: The following resources have at least one write and no reads :::
default_sequence [/^uvm_test_top\.tb\.yapp\.agent\.sequencer\.run_phase$/] : (class
uvm_pkg::uvm_object_wrapper) ...
```

The sequencer does not exist in a passive agent, so nobody reads the setting. The fix here: `base_test` puts its sequence configuration into a **virtual function** `configure_sequences()`; `set_config_test` overrides it with an empty body. Every test from now on uses the same hook to choose its sequences.

See it run

The lines that matter from the real log of every test of this lab (Verilator, seed 1, from the last `make regress`; the paths and the packet tables are left out). Your own log must show the same lines -- `make check LAB=lab04_factory START=1` looks for them.

base_test

```
@0 reporter [RNTST] Running test base_test...
@0 uvm_test_top.tb.yapp.agent.sequencer@@yapp_5_packets [yapp_5_packets] Executing yapp_5_packets
sequence
@0 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h0
  length                      integral       6     40
  parity_type                 parity_type_e  32    GOOD_PARITY
@10 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h0
  length                      integral       6     60
  parity_type                 parity_type_e  32    GOOD_PARITY
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (12 more lines of the kinds above)
```

short_packet_test

```
@0 reporter [RNTST] Running test short_packet_test...
@0 uvm_test_top.tb.yapp.agent.sequencer@@yapp_5_packets [yapp_5_packets] Executing yapp_5_packets
sequence
@0 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral           2     'h1
  length                      integral           6     'd13
  parity_type                 parity_type_e      32    GOOD_PARITY
@10 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral           2     'h0
  length                      integral           6     'd9
  parity_type                 parity_type_e      32    GOOD_PARITY
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (12 more lines of the kinds above)
```

set_config_test

```
@0 reporter [RNTST] Running test set_config_test...
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
```

Try this

Small changes to make in `start/` once the lab works, each with a question; the answer unfolds.

Write `long_yapp_packet` (a `yapp_packet` whose `m_length` is at least 32) and override the packet type with it in a new test. Which of the existing tests does the override change, and which one does it not?

Checkpoint questions

Why did `check_config_usage()` report an unused setting in `set_config_test`?

Why must the override / configuration be set *before* `super.build_phase(phase)`?

What is `recording_detail` for?

What changed since the previous lab

```
diff -r labs/lab03_uvc labs/lab04_factory
```

- `type_id::create()` everywhere, `start_of_simulation_phase` experiments removed
- `short_yapp_packet`
- `base_test`: `recording_detail`, `configure_sequences()`, `check_phase`
- `short_packet_test`, `set_config_test`

Lab 5 — Generating UVM sequences

Open on the project map →

Directory: `labs/lab05_seq` · **Changed:** `sv/yapp_tx_seqs.sv` (the library), `sv/yapp_packet.sv` (constraint fix), `tb/router_test_lib.sv` (+ 2 tests)

Objective

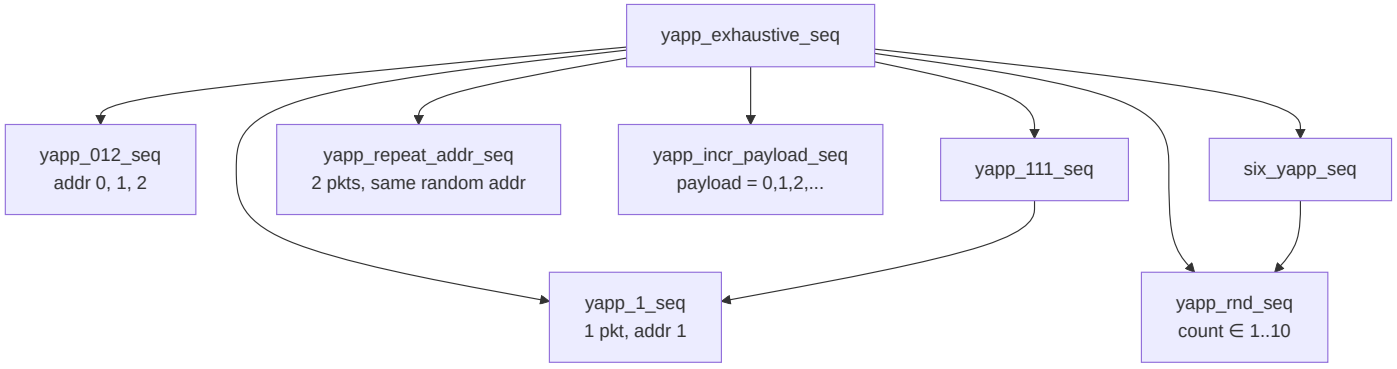
Write a library of sequences, control which one runs from the test, and understand objections and randomization failures.

Concepts

`uvm_sequence #(T)` · `body()` · `type_id::create` → `start_item` → `randomize() with` → `finish_item` · `seq.start(m_sequencer, this)` for a nested sequence · random sequence properties · objections (`yapp_base_seq`) · randomization debugging

No `uvm_do` macros here

The course material writes `` `uvm_do_with(req, { req.addr == 2'd1; }) ``. This repository writes the four steps the macro hides, so you see them: create the item through the factory, `start_item(req)` (wait for the sequencer's grant), `req.randomize() with {…}` (check the result), `finish_item(req)` (hand it to the driver, wait for `item_done`). A nested sequence is created the same way and started with `seq.start(m_sequencer, this)` — that is what `` `uvm_do(seq_1) `` would do.



Solution

1. The library — `sv/yapp_tx_seqs.sv`

```
//------------------------------------------------------------------------------
// yapp_tx_seqs.sv -- YAPP sequence library (Lab 3 base, Lab 5 library)
//
// Every sequence:
//   * extends yapp_base_seq (objection handling),
//   * has an object utils macro and a constructor,
//   * prints its name with `uvm_info at UVM_LOW at the start of body().
//
// One class per file: the classes live in seqs/<class>.sv and are included below.
//------------------------------------------------------------------------------

`include "seqs/yapp_base_seq.sv"
`include "seqs/yapp_5_packets.sv"

//------------------------------------------------------------------------------
// Lab 5 sequences
//------------------------------------------------------------------------------

`include "seqs/yapp_1_seq.sv"
`include "seqs/yapp_012_seq.sv"
`include "seqs/yapp_111_seq.sv"
`include "seqs/yapp_repeat_addr_seq.sv"
`include "seqs/yapp_incr_payload_seq.sv"
`include "seqs/yapp_rnd_seq.sv"
`include "seqs/six_yapp_seq.sv"
`include "seqs/yapp_exhaustive_seq.sv"

//------------------------------------------------------------------------------
// yapp_base_seq.sv -- YAPP sequence library (Lab 3 base, Lab 5 library)
// Split out of yapp_tx_seqs.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// yapp_base_seq: raises an objection while the sequence runs, so run_phase
// does not end before the last packet was delivered.
//------------------------------------------------------------------------------
class yapp_base_seq extends uvm_sequence #(yapp_packet);

  `uvm_object_utils(yapp_base_seq)

  extern function new(string name = "yapp_base_seq");
  extern task pre_body();

  extern task post_body();

endclass : yapp_base_seq

//------------------------------------------------------------------------------
// yapp_base_seq -- method implementations
//------------------------------------------------------------------------------

function yapp_base_seq::new(string name = "yapp_base_seq");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task yapp_base_seq::pre_body();
  uvm_phase phase = `YAPP_STARTING_PHASE;
```

```
  if (phase != null) begin
    phase.raise_objection(this, get_type_name());
  end
endtask : pre_body

//------------------------------------------------------------------------------
task yapp_base_seq::post_body();
  uvm_phase phase = `YAPP_STARTING_PHASE;
  if (phase != null) begin
    phase.drop_objection(this, get_type_name());
  end
endtask : post_body

//------------------------------------------------------------------------------
// yapp_5_packets.sv -- YAPP sequence library (Lab 3 base, Lab 5 library)
// Split out of yapp_tx_seqs.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// yapp_5_packets: five random packets (provided in Lab 3)
//   Test class configuration template:
//     uvm_config_wrapper::set(this, "tb.yapp.agent.sequencer.run_phase",
//                             "default_sequence", yapp_5_packets::get_type());
//------------------------------------------------------------------------------
class yapp_5_packets extends yapp_base_seq;

  `uvm_object_utils(yapp_5_packets)

  extern function new(string name = "yapp_5_packets");
  extern task body();

endclass : yapp_5_packets

//------------------------------------------------------------------------------
// yapp_5_packets -- method implementations
//------------------------------------------------------------------------------

function yapp_5_packets::new(string name = "yapp_5_packets");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task yapp_5_packets::body();
  `uvm_info(get_type_name(), "Executing yapp_5_packets sequence", UVM_LOW)
  repeat (5) begin
    req = yapp_packet::type_id::create("req");
    start_item(req);
    if (!req.randomize()) begin
      `uvm_error(get_type_name(), "req.randomize() failed")
    end
    finish_item(req);
  end
endtask : body

//------------------------------------------------------------------------------
// yapp_1_seq.sv -- YAPP sequence library (Lab 3 base, Lab 5 library)
// Split out of yapp_tx_seqs.sv: one class per file.
//------------------------------------------------------------------------------
```

```
// yapp_1_seq: one packet to address 1
class yapp_1_seq extends yapp_base_seq;

  `uvm_object_utils(yapp_1_seq)

  extern function new(string name = "yapp_1_seq");
  extern task body();

endclass : yapp_1_seq

//------------------------------------------------------------------------------
// yapp_1_seq -- method implementations
//------------------------------------------------------------------------------

function yapp_1_seq::new(string name = "yapp_1_seq");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task yapp_1_seq::body();
  `uvm_info(get_type_name(), "Executing yapp_1_seq sequence", UVM_LOW)
  req = yapp_packet::type_id::create("req");
  start_item(req);
  if (!req.randomize() with { req.m_addr == 2'd1; }) begin
    `uvm_error(get_type_name(), "req.randomize() failed")
  end
  finish_item(req);
endtask : body

//------------------------------------------------------------------------------
// yapp_012_seq.sv -- YAPP sequence library (Lab 3 base, Lab 5 library)
// Split out of yapp_tx_seqs.sv: one class per file.
//------------------------------------------------------------------------------

// yapp_012_seq: three packets, to address 0, then 1, then 2
class yapp_012_seq extends yapp_base_seq;

  `uvm_object_utils(yapp_012_seq)

  extern function new(string name = "yapp_012_seq");
  extern task body();

endclass : yapp_012_seq

//------------------------------------------------------------------------------
// yapp_012_seq -- method implementations
//------------------------------------------------------------------------------

function yapp_012_seq::new(string name = "yapp_012_seq");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task yapp_012_seq::body();
  `uvm_info(get_type_name(), "Executing yapp_012_seq sequence", UVM_LOW)
  req = yapp_packet::type_id::create("req");
  start_item(req);
  if (!req.randomize() with { req.m_addr == 2'd0; }) begin
    `uvm_error(get_type_name(), "req.randomize() failed")
```

```
  end
  finish_item(req);

  req = yapp_packet::type_id::create("req");
  start_item(req);
  if (!req.randomize() with { req.m_addr == 2'd1; }) begin
    `uvm_error(get_type_name(), "req.randomize() failed")
  end
  finish_item(req);

  req = yapp_packet::type_id::create("req");
  start_item(req);
  if (!req.randomize() with { req.m_addr == 2'd2; }) begin
    `uvm_error(get_type_name(), "req.randomize() failed")
  end
  finish_item(req);
endtask : body

//------------------------------------------------------------------------------
// yapp_111_seq.sv -- YAPP sequence library (Lab 3 base, Lab 5 library)
// Split out of yapp_tx_seqs.sv: one class per file.
//------------------------------------------------------------------------------

// yapp_111_seq: nested sequence -- runs yapp_1_seq three times
class yapp_111_seq extends yapp_base_seq;

  yapp_1_seq m_seq_1;

  `uvm_object_utils(yapp_111_seq)

  extern function new(string name = "yapp_111_seq");
  extern task body();

endclass : yapp_111_seq

//------------------------------------------------------------------------------
// yapp_111_seq -- method implementations
//------------------------------------------------------------------------------

function yapp_111_seq::new(string name = "yapp_111_seq");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task yapp_111_seq::body();
  `uvm_info(get_type_name(), "Executing yapp_111_seq sequence", UVM_LOW)
  repeat (3) begin
    m_seq_1 = yapp_1_seq::type_id::create("seq_1");
    if (!m_seq_1.randomize()) begin
      `uvm_error(get_type_name(), "seq_1.randomize() failed")
    end
    m_seq_1.start(m_sequencer, this);
  end
endtask : body

//------------------------------------------------------------------------------
// yapp_repeat_addr_seq.sv -- YAPP sequence library (Lab 3 base, Lab 5 library)
// Split out of yapp_tx_seqs.sv: one class per file.
//------------------------------------------------------------------------------
```

```
// yapp_repeat_addr_seq: two packets to the same random (legal) address
class yapp_repeat_addr_seq extends yapp_base_seq;

  // A random SEQUENCE property, randomized when the sequence is randomized
  rand bit[1:0] m_seq_addr;

  `uvm_object_utils(yapp_repeat_addr_seq)

  extern constraint c_seq_addr;

  extern function new(string name = "yapp_repeat_addr_seq");
  extern task body();

endclass : yapp_repeat_addr_seq

//------------------------------------------------------------------------------
// yapp_repeat_addr_seq -- constraints
//------------------------------------------------------------------------------

constraint yapp_repeat_addr_seq::c_seq_addr {
  m_seq_addr != 2'd3;
}

//------------------------------------------------------------------------------
// yapp_repeat_addr_seq -- method implementations
//------------------------------------------------------------------------------

function yapp_repeat_addr_seq::new(string name = "yapp_repeat_addr_seq");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task yapp_repeat_addr_seq::body();
  `uvm_info(get_type_name(),
            $sformatf("Executing yapp_repeat_addr_seq sequence (addr %0d)", m_seq_addr), UVM_LOW)
  req = yapp_packet::type_id::create("req");
  start_item(req);
  if (!req.randomize() with { req.m_addr == m_seq_addr; }) begin
    `uvm_error(get_type_name(), "req.randomize() failed")
  end
  finish_item(req);

  req = yapp_packet::type_id::create("req");
  start_item(req);
  if (!req.randomize() with { req.m_addr == m_seq_addr; }) begin
    `uvm_error(get_type_name(), "req.randomize() failed")
  end
  finish_item(req);
endtask : body

//------------------------------------------------------------------------------
// yapp_incr_payload_seq.sv -- YAPP sequence library (Lab 3 base, Lab 5 library)
// Split out of yapp_tx_seqs.sv: one class per file.
//------------------------------------------------------------------------------

// yapp_incr_payload_seq: one packet whose payload is 0, 1, 2, ... length-1
class yapp_incr_payload_seq extends yapp_base_seq;
```

```
  `uvm_object_utils(yapp_incr_payload_seq)

  extern function new(string name = "yapp_incr_payload_seq");
  extern task body();

endclass : yapp_incr_payload_seq

//------------------------------------------------------------------------------
// yapp_incr_payload_seq -- method implementations
//------------------------------------------------------------------------------

function yapp_incr_payload_seq::new(string name = "yapp_incr_payload_seq");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task yapp_incr_payload_seq::body();
  `uvm_info(get_type_name(), "Executing yapp_incr_payload_seq sequence", UVM_LOW)
  req = yapp_packet::type_id::create("req");   // build through the factory
  if (!req.randomize()) begin
    `uvm_error(get_type_name(), "Randomization failed")
  end
  foreach (req.m_payload[i]) begin
    req.m_payload[i] = i;
  end
  req.set_parity();                        // payload changed -> recompute parity
  start_item(req);   // hand it to the driver
  finish_item(req);
endtask : body

//------------------------------------------------------------------------------
// yapp_rnd_seq.sv -- YAPP sequence library (Lab 3 base, Lab 5 library)
// Split out of yapp_tx_seqs.sv: one class per file.
//------------------------------------------------------------------------------

// yapp_rnd_seq (optional): a random number (1..10) of random packets
class yapp_rnd_seq extends yapp_base_seq;

  rand int m_count;

  `uvm_object_utils(yapp_rnd_seq)

  extern constraint c_count;

  extern function new(string name = "yapp_rnd_seq");
  extern task body();

endclass : yapp_rnd_seq

//------------------------------------------------------------------------------
// yapp_rnd_seq -- constraints
//------------------------------------------------------------------------------

constraint yapp_rnd_seq::c_count {
  m_count inside {[1:10]};
}

//------------------------------------------------------------------------------
// yapp_rnd_seq -- method implementations
```

```
//------------------------------------------------------------------------------

function yapp_rnd_seq::new(string name = "yapp_rnd_seq");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task yapp_rnd_seq::body();
  `uvm_info(get_type_name(),
            $sformatf("Executing yapp_rnd_seq sequence (%0d packets)", m_count), UVM_LOW)
  repeat (m_count) begin
    req = yapp_packet::type_id::create("req");
    start_item(req);
    if (!req.randomize()) begin
      `uvm_error(get_type_name(), "req.randomize() failed")
    end
    finish_item(req);
  end
endtask : body

//------------------------------------------------------------------------------
// six_yapp_seq.sv -- YAPP sequence library (Lab 3 base, Lab 5 library)
// Split out of yapp_tx_seqs.sv: one class per file.
//------------------------------------------------------------------------------

// six_yapp_seq (optional): yapp_rnd_seq constrained to exactly six packets
class six_yapp_seq extends yapp_base_seq;

  yapp_rnd_seq m_rnd_seq;

  `uvm_object_utils(six_yapp_seq)

  extern function new(string name = "six_yapp_seq");
  extern task body();

endclass : six_yapp_seq

//------------------------------------------------------------------------------
// six_yapp_seq -- method implementations
//------------------------------------------------------------------------------

function six_yapp_seq::new(string name = "six_yapp_seq");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task six_yapp_seq::body();
  `uvm_info(get_type_name(), "Executing six_yapp_seq sequence", UVM_LOW)
  m_rnd_seq = yapp_rnd_seq::type_id::create("rnd_seq");
  if (!m_rnd_seq.randomize() with { m_rnd_seq.m_count == 6; }) begin
    `uvm_error(get_type_name(), "rnd_seq.randomize() failed")
  end
  m_rnd_seq.start(m_sequencer, this);
endtask : body

//------------------------------------------------------------------------------
// yapp_exhaustive_seq.sv -- YAPP sequence library (Lab 3 base, Lab 5 library)
// Split out of yapp_tx_seqs.sv: one class per file.
//------------------------------------------------------------------------------
```

```
// yapp_exhaustive_seq: runs every sequence above once
class yapp_exhaustive_seq extends yapp_base_seq;

  yapp_1_seq            m_seq_1;
  yapp_012_seq          m_seq_012;
  yapp_111_seq          m_seq_111;
  yapp_repeat_addr_seq  m_seq_repeat_addr;
  yapp_incr_payload_seq m_seq_incr_payload;
  yapp_rnd_seq          m_seq_rnd;
  six_yapp_seq          m_seq_six;

  `uvm_object_utils(yapp_exhaustive_seq)

  extern function new(string name = "yapp_exhaustive_seq");
  extern task body();

endclass : yapp_exhaustive_seq

//------------------------------------------------------------------------------
// yapp_exhaustive_seq -- method implementations
//------------------------------------------------------------------------------

function yapp_exhaustive_seq::new(string name = "yapp_exhaustive_seq");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task yapp_exhaustive_seq::body();
  `uvm_info(get_type_name(), "Executing yapp_exhaustive_seq sequence", UVM_LOW)
  m_seq_1 = yapp_1_seq::type_id::create("seq_1");
  if (!m_seq_1.randomize()) begin
    `uvm_error(get_type_name(), "seq_1.randomize() failed")
  end
  m_seq_1.start(m_sequencer, this);

  m_seq_012 = yapp_012_seq::type_id::create("seq_012");
  if (!m_seq_012.randomize()) begin
    `uvm_error(get_type_name(), "seq_012.randomize() failed")
  end
  m_seq_012.start(m_sequencer, this);

  m_seq_111 = yapp_111_seq::type_id::create("seq_111");
  if (!m_seq_111.randomize()) begin
    `uvm_error(get_type_name(), "seq_111.randomize() failed")
  end
  m_seq_111.start(m_sequencer, this);

  m_seq_repeat_addr = yapp_repeat_addr_seq::type_id::create("seq_repeat_addr");
  if (!m_seq_repeat_addr.randomize()) begin
    `uvm_error(get_type_name(), "seq_repeat_addr.randomize() failed")
  end
  m_seq_repeat_addr.start(m_sequencer, this);

  m_seq_incr_payload = yapp_incr_payload_seq::type_id::create("seq_incr_payload");
  if (!m_seq_incr_payload.randomize()) begin
    `uvm_error(get_type_name(), "seq_incr_payload.randomize() failed")
  end
  m_seq_incr_payload.start(m_sequencer, this);
```

```
  m_seq_rnd = yapp_rnd_seq::type_id::create("seq_rnd");
  if (!m_seq_rnd.randomize()) begin
    `uvm_error(get_type_name(), "seq_rnd.randomize() failed")
  end
  m_seq_rnd.start(m_sequencer, this);

  m_seq_six = six_yapp_seq::type_id::create("seq_six");
  if (!m_seq_six.randomize()) begin
    `uvm_error(get_type_name(), "seq_six.randomize() failed")
  end
  m_seq_six.start(m_sequencer, this);
endtask : body
```

Sequence	Technique
`yapp_1_seq`	one item: `start_item(req)` → `req.randomize() with { req.m_addr == 2'd1; }` → `finish_item(req)`
`yapp_012_seq`	the same three times with different inline constraints
`yapp_111_seq`	nested: `seq_1 = yapp_1_seq::type_id::create("seq_1")` → `m_seq_1.start(m_sequencer, this)`, three times
`yapp_repeat_addr_seq`	`rand bit[1:0] m_seq_addr` with `!= 3`; both items use it
`yapp_incr_payload_seq`	`create` → `randomize()` → edit payload → `set_parity()` → `start_item` / `finish_item`
`yapp_rnd_seq`	`rand int count` in 1..10, printed in the info message
`six_yapp_seq`	`m_rnd_seq.randomize() with { m_rnd_seq.m_count == 6; }` before `m_rnd_seq.start(m_sequencer, this)`
`yapp_exhaustive_seq`	runs all of the above, with named handles

2. The tests

```
class incr_payload_test extends base_test;
  ...
  function void build_phase(uvm_phase phase);
    set_type_override_by_type(yapp_packet::get_type(), short_yapp_packet::get_type());
    super.build_phase(phase);
  endfunction
  function void configure_sequences();
    uvm_config_wrapper::set(this, "tb.yapp.agent.sequencer.run_phase",
                            "default_sequence", yapp_incr_payload_seq::get_type());
  endfunction
endclass
```

`exhaustive_seq_test` is identical with `yapp_exhaustive_seq`.

Run

```
make run TEST=incr_payload_test XRUN_OPTS=+UVM_VERBOSITY=UVM_FULL
make run TEST=exhaustive_seq_test
make gui TEST=exhaustive_seq_test        # SimVision: -gui -access rwc
```

Expected: `incr_payload_test` prints one short packet whose payload reads `0 1 2 3 …`; at `UVM_FULL` you can see `Starting default sequence` `yapp_incr_payload_seq`. `exhaustive_seq_test` prints one `Executing …` line per sequence and 1 + 3 + 3 + 2 + 1 + *count* + 6 packets, no warnings.

The planned failure

With the Lab 4 version of `short_yapp_packet` (`m_addr != 2`) the run shows:

```
UVM_ERROR ... [yapp_012_seq] req.randomize() failed
```

for every `req.randomize() with { req.m_addr == 2'd2; }`: the inline constraint wants address 2 and the class constraint forbids it, so `randomize()` returns 0 and the `if (!req.randomize() …)` guard reports it (the simulator prints its own constraint-solver warning next to it; with the course's `` `uvm_do_with `` macro you would see `UVM_WARNING [RNDFLD] Randomization failed in uvm_do_with action` instead). The simulation **does not stop** in batch mode; the packet keeps its previous values and `finish_item` still sends it.

In the GUI (`-gui -access rwc`) the simulation stops at the failure and the *Constraints Manager* lists the two conflicting constraints — `short_yapp_packet::c_no_addr2` and the inline `req.m_addr == 2'b10`.

Fix: short packets must go to every address, so the `m_addr != 2` constraint is removed from `short_yapp_packet` (that is the version in this lab's `sv/yapp_packet.sv`). Do this before Lab 6.

See it run

The lines that matter from the real log of every test of this lab (Verilator, seed 1, from the last `make regress`; the paths and the packet tables are left out). Your own log must show the same lines -- `make check LAB=lab05_seq START=1` looks for them.

base_test

```
@0 reporter [RNTST] Running test base_test...
@0 uvm_test_top.tb.yapp.agent.sequencer@@yapp_5_packets [yapp_5_packets] Executing yapp_5_packets
sequence
@0 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                          integral       2     'h0
  length                        integral       6     40
  parity_type                   parity_type_e  32    GOOD_PARITY
@10 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                          integral       2     'h0
  length                        integral       6     60
  parity_type                   parity_type_e  32    GOOD_PARITY
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (12 more lines of the kinds above)
```

short_packet_test

```
@0 reporter [RNTST] Running test short_packet_test...
@0 uvm_test_top.tb.yapp.agent.sequencer@@yapp_5_packets [yapp_5_packets] Executing yapp_5_packets
sequence
@0 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                          integral           2     'h1
  length                        integral           6     'd13
  parity_type                   parity_type_e      32    GOOD_PARITY
@10 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                          integral           2     'h0
  length                        integral           6     'd9
  parity_type                   parity_type_e      32    GOOD_PARITY
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (12 more lines of the kinds above)
```

set_config_test

```
@0 reporter [RNTST] Running test set_config_test...
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
```

incr_payload_test

```
@0 reporter [RNTST] Running test incr_payload_test...
@0 uvm_test_top.tb.yapp.agent.sequencer@@yapp_incr_payload_seq [yapp_incr_payload_seq] Executing
yapp_incr_payload_seq sequence
@0 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                          integral           2     'h2
  length                        integral           6     'd4
  parity_type                   parity_type_e      32    GOOD_PARITY
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
```

exhaustive_seq_test

```
@0 reporter [RNTST] Running test exhaustive_seq_test...
@0 uvm_test_top.tb.yapp.agent.sequencer@@yapp_exhaustive_seq [yapp_exhaustive_seq] Executing
yapp_exhaustive_seq sequence
@0 uvm_test_top.tb.yapp.agent.sequencer@@yapp_exhaustive_seq.seq_1 [yapp_1_seq] Executing
yapp_1_seq sequence
@0 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral          2     'h1
  length                      integral          6     'd14
  parity_type                 parity_type_e     32    GOOD_PARITY
@10 uvm_test_top.tb.yapp.agent.sequencer@@yapp_exhaustive_seq.seq_012 [yapp_012_seq] Executing
yapp_012_seq sequence
@10 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral          2     'h0
  length                      integral          6     'd4
  parity_type                 parity_type_e     32    GOOD_PARITY
@70 uvm_test_top.tb.yapp.agent.sequencer@@yapp_exhaustive_seq.seq_repeat_addr
[yapp_repeat_addr_seq] Executing yapp_repeat_addr_seq sequence (addr 2)
@90 uvm_test_top.tb.yapp.agent.sequencer@@yapp_exhaustive_seq.seq_incr_payload
[yapp_incr_payload_seq] Executing yapp_incr_payload_seq sequence
@100 uvm_test_top.tb.yapp.agent.sequencer@@yapp_exhaustive_seq.seq_rnd [yapp_rnd_seq] Executing
yapp_rnd_seq sequence (6 packets)
  parity_type                 parity_type_e     32    BAD_PARITY
  parity_type                 parity_type_e     32    BAD_PARITY
@160 uvm_test_top.tb.yapp.agent.sequencer@@yapp_exhaustive_seq.seq_six [six_yapp_seq] Executing
six_yapp_seq sequence
@160 uvm_test_top.tb.yapp.agent.sequencer@@yapp_exhaustive_seq.seq_six.rnd_seq [yapp_rnd_seq]
Executing yapp_rnd_seq sequence (6 packets)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (82 more lines of the kinds above)
```

Try this

Small changes to make in `start/` once the lab works, each with a question; the answer unfolds.

Make `yapp_012_seq` send the addresses in the order 2, 1, 0. Does the driver print them in that order? Why?

Start `yapp_1_seq` twice from a new sequence, the second time with `req.m_addr == 2'd2` in the `randomize() with`. What happens before the Lab 5 fix of `short_yapp_packet`?

Checkpoint questions

Why do you get randomization violations?

What happens to the packet when a constraint violation is found?

How do objections keep the simulation alive here?

Why does `yapp_incr_payload_seq` randomize *before* `start_item`?

Optional

`yapp_rnd_seq` and `six_yapp_seq` are in the library and part of `yapp_exhaustive_seq`.

What changed since the previous lab

```
diff -r labs/lab04_factory labs/lab05_seq
```

- `yapp_tx_seqs.sv`: the whole library
- `yapp_packet.sv`: `c_no_addr2` removed from `short_yapp_packet`
- `incr_payload_test`, `exhaustive_seq_test`

Lab 6 — Connecting to the DUT using virtual interfaces

Open on the project map →

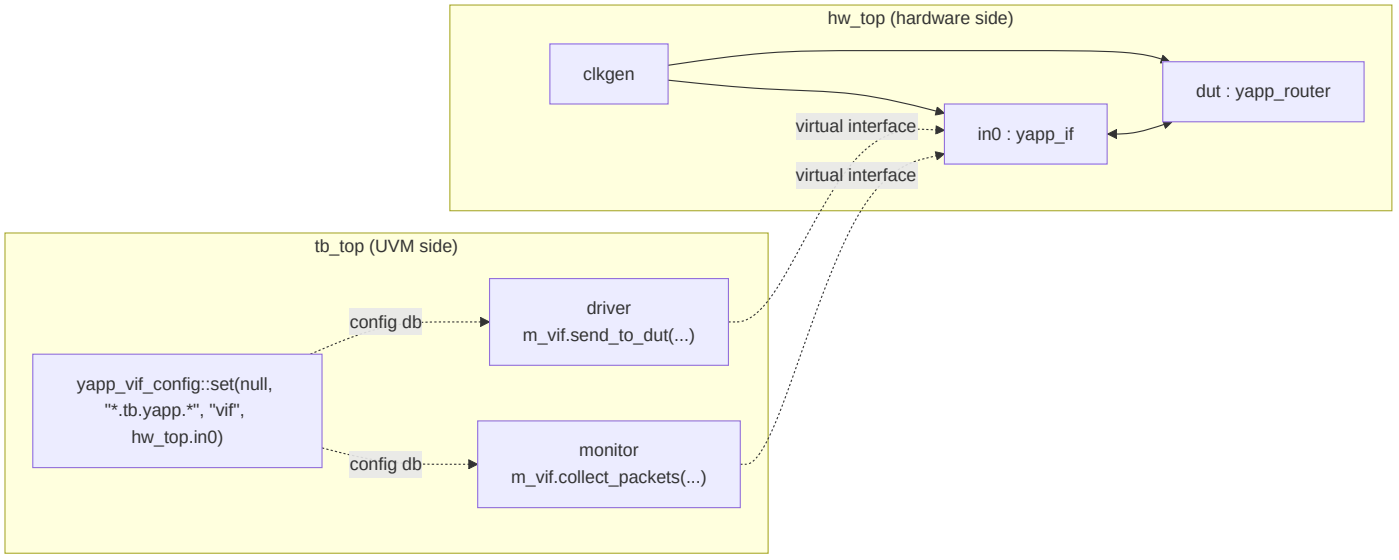
Directory: `labs/lab06_vif` · **New:** `sv/yapp_if.sv`, `tb/hw_top.sv`, `tb/clkgen.sv`, `tb/tb_top.sv` (was `top.sv`), `tb/yapp_router_instance.txt` · **Changed:** monitor, driver, `yapp_pkg.sv`, tests, `run.f`

Objective

Connect the YAPP UVC to the router's input port: an interface in the hardware world, a virtual interface handle in the class world, the configuration database between them.

Concepts

SystemVerilog `interface` with tasks · `virtual interface` · `uvm_config_db #(virtual yapp_if)` · `connect_phase` `get()` · drain time · `hw_top` / `tb_top` split · transaction recording



Solution

1. The interface — `sv/yapp_if.sv`

```
//------------------------------------------------------------------------------
// yapp_if.sv -- YAPP input-port interface
//
// Holds the three DUT input-port signals and the protocol knowledge:
//   * send_to_dut()    -- used by the driver to transmit one packet
//   * collect_packets()-- used by the monitor to observe one packet
//   * yapp_reset()     -- used by the driver while reset is active
//
// Protocol recap (see yapp_router.sv for the DUT view):
//   * The driver changes in_data / in_data_vld on the FALLING edge.
//   * The DUT accepts the byte on the next RISING edge unless it is asserting
//     in_suspend (target FIFO full). While in_suspend is high the byte must be
//     held unchanged.
//   * in_data_vld is high for the header and payload bytes and low for the
//     parity byte, which is driven on the falling edge right after the last
//     payload byte.
//------------------------------------------------------------------------------
`timescale 1ns/1ns

interface yapp_if (input logic clock, input logic reset);

  logic [7:0] in_data;
  logic       in_data_vld;
  logic       in_suspend;

  //--------------------------------------------------------------------------
  // Driver side
  //--------------------------------------------------------------------------
  task automatic yapp_reset();
    @(negedge clock);
    in_data     <= 8'h00;
    in_data_vld <= 1'b0;
  endtask : yapp_reset

  // Hold the current byte until the DUT has accepted it (in_suspend low on a
  // falling edge means the byte was taken at the preceding rising edge).
  task automatic wait_accept();
    @(negedge clock);
    while (in_suspend) begin
      @(negedge clock);
    end
  endtask : wait_accept

  task automatic send_to_dut(input bit [1:0] addr,
                             input bit [5:0] length,
                             input bit [7:0] payload[],
                             input bit [7:0] parity,
                             input int       packet_delay);
    // Inter-packet gap
    repeat (packet_delay) begin
      @(negedge clock);
    end

    // Header: {length, addr} together with in_data_vld
    @(negedge clock);
    in_data_vld <= 1'b1;
    in_data     <= {length, addr};
```

```
    // Payload, one byte per accepted cycle
    foreach (payload[i]) begin
      wait_accept();
      in_data <= payload[i];
    end

    // Parity with in_data_vld low
    wait_accept();
    in_data_vld <= 1'b0;
    in_data     <= parity;

    // Make sure the parity byte was accepted before returning to idle
    wait_accept();
    in_data <= 8'h00;
  endtask : send_to_dut

  //------------------------------------------------------------------------
  // Monitor side: sample on the RISING edge. A byte is accepted by the DUT
  // when it is not suspending the input at that edge.
  //------------------------------------------------------------------------
  task automatic collect_packets(output bit [1:0] addr,
                                 output bit [5:0] length,
                                 output bit [7:0] payload[],
                                 output bit [7:0] parity);
    // Wait for an accepted header
    do @(posedge clock); while (!(in_data_vld && !in_suspend));
    length = in_data[7:2];
    addr   = in_data[1:0];
    payload = new[length];

    // Payload bytes
    for (int i = 0; i < length; i++) begin
      do @(posedge clock); while (!(in_data_vld && !in_suspend));
      payload[i] = in_data;
    end

    // Parity byte: first accepted cycle after the payload
    do @(posedge clock); while (in_suspend);
    parity = in_data;
  endtask : collect_packets

endinterface : yapp_if
```

2. The typedef — `sv/yapp_pkg.sv`

```
typedef uvm_config_db #(virtual yapp_if) yapp_vif_config;
```

placed before the includes so the driver, the monitor and `tb_top` can use it.

3. Monitor — `sv/yapp_tx_monitor.sv`

```
//------------------------------------------------------------------------------
// yapp_tx_monitor.sv -- observes packets on the YAPP input port (Lab 6)
//
// The monitor reaches the DUT pins through a VIRTUAL INTERFACE handle that
// tb_top stores in the configuration database and connect_phase() fetches.
//------------------------------------------------------------------------------
class yapp_tx_monitor extends uvm_monitor;

  // Virtual interface, set by tb_top through yapp_vif_config
  virtual interface yapp_if m_vif;

  // Statistics
  int m_num_pkt_col;

  `uvm_component_utils(yapp_tx_monitor)

  // The fields are printed by do_print() below:
  // no uvm_field_* automation.
  extern virtual function void do_print(uvm_printer printer);
  extern function new(string name, uvm_component parent);

  extern function void connect_phase(uvm_phase phase);
  extern task run_phase(uvm_phase phase);

  extern task collect_packets();
  extern function void report_phase(uvm_phase phase);

endclass : yapp_tx_monitor

//------------------------------------------------------------------------------
// yapp_tx_monitor -- method implementations
//------------------------------------------------------------------------------

function yapp_tx_monitor::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//------------------------------------------------------------------------------
function void yapp_tx_monitor::connect_phase(uvm_phase phase);
  // get() returns 1 on success: always check it, a missing vif is fatal later
  if (!yapp_vif_config::get(this, "", "vif", m_vif)) begin
    `uvm_error("NOVIF", {"virtual interface must be set for: ", get_full_name(), ".vif"})
  end
endfunction : connect_phase

//------------------------------------------------------------------------------
task yapp_tx_monitor::run_phase(uvm_phase phase);
  `uvm_info(get_type_name(), "YAPP monitor running", UVM_LOW)
  collect_packets();
endtask : run_phase

//------------------------------------------------------------------------------
task yapp_tx_monitor::collect_packets();
  yapp_packet pkt;
  // Nothing to observe while reset is active
  @(posedge m_vif.clock);
  wait (m_vif.reset === 1'b0);
  forever begin
```

```
    pkt = yapp_packet::type_id::create("pkt", this);
    m_vif.collect_packets(pkt.m_addr, pkt.m_length, pkt.m_payload, pkt.m_parity);
    void'(begin_tr(pkt, "Monitor_YAPP_Packet"));
    pkt.m_parity_type = (pkt.m_parity == pkt.calc_parity()) ? PARITY_TYPE_GOOD : PARITY_TYPE_BAD;
    m_num_pkt_col++;
    `uvm_info(get_type_name(), $sformatf("Packet collected:\n%s", pkt.sprint()), UVM_LOW)
    end_tr(pkt);
  end
endtask : collect_packets

//------------------------------------------------------------------------------
function void yapp_tx_monitor::report_phase(uvm_phase phase);
  `uvm_info(get_type_name(),
            $sformatf("YAPP monitor report: %0d packets collected", m_num_pkt_col), UVM_LOW)
endfunction : report_phase

//------------------------------------------------------------------------------
function void yapp_tx_monitor::do_print(uvm_printer printer);
  super.do_print(printer);
  printer.print_field("num_pkt_col", m_num_pkt_col, $bits(m_num_pkt_col), UVM_DEC);
endfunction : do_print
```

4. Driver — `sv/yapp_tx_driver.sv`

```
//------------------------------------------------------------------------------
// yapp_tx_driver.sv -- pulls packets from the sequencer and drives the
// YAPP input port through the virtual interface (Lab 3 skeleton, Lab 6 protocol)
//------------------------------------------------------------------------------
class yapp_tx_driver extends uvm_driver #(yapp_packet);

  // Virtual interface, set by tb_top through yapp_vif_config
  virtual interface yapp_if m_vif;

  // Statistics
  int m_num_sent;

  `uvm_component_utils(yapp_tx_driver)

  // The fields are printed by do_print() below:
  // no uvm_field_* automation.
  extern virtual function void do_print(uvm_printer printer);
  extern function new(string name, uvm_component parent);

  extern function void connect_phase(uvm_phase phase);
  extern task run_phase(uvm_phase phase);

  // Drive idle values and stay quiet while reset is active
  extern task reset_signals();

  // The classic driver loop
  extern task get_and_drive();
  extern task send_to_dut(yapp_packet pkt);

  extern function void report_phase(uvm_phase phase);

endclass : yapp_tx_driver

//------------------------------------------------------------------------------
// yapp_tx_driver -- method implementations
//------------------------------------------------------------------------------

function yapp_tx_driver::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//------------------------------------------------------------------------------
function void yapp_tx_driver::connect_phase(uvm_phase phase);
  if (!yapp_vif_config::get(this, "", "vif", m_vif)) begin
    `uvm_error("NOVIF", {"virtual interface must be set for: ", get_full_name(), ".vif"})
  end
endfunction : connect_phase

//------------------------------------------------------------------------------
task yapp_tx_driver::run_phase(uvm_phase phase);
  reset_signals();
  get_and_drive();
endtask : run_phase

//------------------------------------------------------------------------------
task yapp_tx_driver::reset_signals();
  m_vif.yapp_reset();
  wait (m_vif.reset === 1'b0);
```

```
endtask : reset_signals

//------------------------------------------------------------------------------
task yapp_tx_driver::get_and_drive();
  forever begin
    seq_item_port.get_next_item(req);     // blocks until a sequence sends one
    send_to_dut(req);
    seq_item_port.item_done();            // unblocks the sequence
  end
endtask : get_and_drive

//------------------------------------------------------------------------------
task yapp_tx_driver::send_to_dut(yapp_packet pkt);
  void'(begin_tr(pkt, "Driver_YAPP_Packet"));   // transaction recording
  `uvm_info(get_type_name(), $sformatf("Packet is \n%s", pkt.sprint()), UVM_LOW)
  m_vif.send_to_dut(pkt.m_addr, pkt.m_length, pkt.m_payload, pkt.m_parity, pkt.m_packet_delay);
  m_num_sent++;
  end_tr(pkt);
endtask : send_to_dut

//------------------------------------------------------------------------------
function void yapp_tx_driver::report_phase(uvm_phase phase);
  `uvm_info(get_type_name(), $sformatf("YAPP driver report: %0d packets sent", m_num_sent),
UVM_LOW)
endfunction : report_phase

//------------------------------------------------------------------------------
function void yapp_tx_driver::do_print(uvm_printer printer);
  super.do_print(printer);
  printer.print_field("num_sent", m_num_sent, $bits(m_num_sent), UVM_DEC);
endfunction : do_print
```

(The pieces to copy are also in `sv/driver_example.sv` and `sv/monitor_example.sv`, as the course supplies them.)

5. Tests — drain time and `yapp_012_test`

```
task run_phase(uvm_phase phase);
  uvm_objection obj = phase.get_objection();
  obj.set_drain_time(this, 200ns);        // let the last packet leave the router
endtask
```

`yapp_012_test` sets `yapp_012_seq` as the default sequence: one packet per channel makes the DUT connection easy to check.

6. Hardware top — `tb/hw_top.sv`

```
//-------------------------------------------------------------------------------
// hw_top.sv -- hardware side of the testbench (Lab 6)
//
//   * clkgen produces the clock
//   * a simple initial block produces the reset (the Clock & Reset UVC takes
//     over in Lab 7)
//   * in0 is the YAPP interface instance; the UVM side reaches it through
//     the virtual interface set in tb_top
//   * the router DUT (step 11 of the lab). Before the DUT was added, the
//     initial block also drove `in0.in_suspend <= 0`; the DUT drives it now.
//-------------------------------------------------------------------------------
`timescale 1ns/1ns

module hw_top;

  logic clock;
  logic reset;

  // Clock
  clkgen clkgen (.run_clock(1'b1), .clock_period(10), .clock(clock));

  // Reset: active for the first 5 clock cycles
  initial begin
    reset = 1'b1;
    repeat (5) begin
      @(negedge clock);
    end
    reset = 1'b0;
  end

  // YAPP interface instance
  yapp_if in0 (clock, reset);

  // The DUT -- port list from yapp_router_instance.txt
  yapp_router dut (
    .clock       (clock),
    .reset       (reset),
    // YAPP input port
    .in_data     (in0.in_data),
    .in_data_vld (in0.in_data_vld),
    .in_suspend  (in0.in_suspend),
    // Output channels (not monitored yet)
    .data_0      (),
    .data_vld_0  (),
    .suspend_0   (1'b0),
    .data_1      (),
    .data_vld_1  (),
    .suspend_1   (1'b0),
    .data_2      (),
    .data_vld_2  (),
    .suspend_2   (1'b0),
    // HBUS host port (not used yet)
    .hdata       (),
    .haddr       (16'h0000),
    .hen         (1'b0),
    .hwr_rd      (1'b0),
    // Status
    .error       ()
```

```
  );

endmodule : hw_top
```

The lab does this in two steps. **Without the DUT** the module only has the clock, the reset and the interface, plus `in0.in_suspend <= 0` in the initial block (nobody else drives it). **With the DUT** that line is removed — the DUT drives `in_suspend` — the instance from `yapp_router_instance.txt` is pasted in, and `suspend_0/1/2` are tied to `1'b0` so packets flow out.

7. UVM top — `tb/tb_top.sv`

```
//------------------------------------------------------------------------------
// tb_top.sv -- UVM side of the testbench (Lab 6, formerly top.sv)
//
// Two jobs: publish the virtual interface(s) in the configuration database
// and start UVM phasing.
//------------------------------------------------------------------------------
`timescale 1ns/1ns

module tb_top;

  import uvm_pkg::*;
  import rand_util_pkg::*;   // rnd:: random values (common/rand_util_pkg.sv)
  `include "uvm_macros.svh"
  import yapp_pkg::*;

  `include "router_tb.sv"
  `include "router_test_lib.sv"

  initial begin
    // context = null (top), wildcard instance path so that BOTH the driver and
    // the monitor get the same handle, absolute path to the interface instance
    yapp_vif_config::set(null, "*.tb.yapp.*", "vif", hw_top.in0);
    run_test();
  end

endmodule : tb_top
```

8. `tb/run.f`

```
// Lab 6 -- xrun command file
-uvmhome CDNS-1.1d
-timescale 1ns/1ns            // avoids timescale errors between files

-incdir ../../../common
../../../common/rand_util_pkg.sv   // rnd:: random values, used by every UVC and test
-incdir ../sv
../sv/yapp_pkg.sv
../sv/yapp_if.sv              // interfaces are COMPILED, never `included
-F ../../../yapp_project/rtl/yapp_router.f   // the DUT: one module per file
clkgen.sv
hw_top.sv
tb_top.sv

// Tests: base_test | short_packet_test | set_config_test |
//        incr_payload_test | exhaustive_seq_test | yapp_012_test
+UVM_TESTNAME=yapp_012_test
+UVM_VERBOSITY=UVM_MEDIUM    // one line per packet; UVM_HIGH adds the full packet tables
+SVSEED=random
```

Interfaces are **compiled**, not `` `include ``d.

Run

```
make run TEST=yapp_012_test
make gui TEST=yapp_012_test
```

Expected: three `TX pkt` lines from the driver and three `RX pkt … on the input` lines from the monitor with the same address, length and parity (at `UVM_HIGH` the full `Packet is` / `Packet collected` tables follow). In the waveform viewer, `hw_top.in0.in_data` shows `H D D … P` for each packet and the same bytes appear a few cycles later on `hw_top.dut.data_0`, then `data_1`, then `data_2`. The transaction streams `Driver_YAPP_Packet` and `Monitor_YAPP_Packet` show one bar per packet.

See it run

The lines that matter from the real log of every test of this lab (Verilator, seed 1, from the last `make regress`; the paths and the packet tables are left out). Your own log must show the same lines -- `make check LAB=lab06_vif START=1` looks for them.

base_test

```
@0 reporter [RNTST] Running test base_test...
@0 uvm_test_top.tb.yapp.agent.sequencer@@yapp_5_packets [yapp_5_packets] Executing yapp_5_packets
sequence
@50 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                          integral       2     'h0
  length                        integral       6     40
  parity_type                   parity_type_e  32    GOOD_PARITY
@675 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral       2     'h0
  length        integral       6     40
  parity_type   parity_type_e  32    GOOD_PARITY
@680 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                          integral       2     'h0
  length                        integral       6     60
  parity_type                   parity_type_e  32    GOOD_PARITY
@1325 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral       2     'h0
  length        integral       6     60
  parity_type   parity_type_e  32    GOOD_PARITY
@3480 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] YAPP driver report: 5 packets sent
@3480 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP monitor report: 5 packets
collected
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (24 more lines of the kinds above)
```

short_packet_test

```
@0 reporter [RNTST] Running test short_packet_test...
@0 uvm_test_top.tb.yapp.agent.sequencer@@yapp_5_packets [yapp_5_packets] Executing yapp_5_packets
sequence
@50 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral          2     'h1
  length                      integral          6     'd13
  parity_type                 parity_type_e     32    GOOD_PARITY
@265 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral          2     'h1
  length        integral          6     'd13
  parity_type   parity_type_e     32    GOOD_PARITY
@270 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral          2     'h0
  length                      integral          6     'd9
  parity_type                 parity_type_e     32    GOOD_PARITY
@425 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral          2     'h0
  length        integral          6     'd9
  parity_type   parity_type_e     32    GOOD_PARITY
  length        integral          6     'd13
  length        integral          6     'd1
@1400 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] YAPP driver report: 5 packets sent
@1400 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP monitor report: 5 packets
collected
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (22 more lines of the kinds above)
```

set_config_test

```
@0 reporter [RNTST] Running test set_config_test...
@0 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP monitor report: 0 packets collected
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
```

incr_payload_test

```
@0 reporter [RNTST] Running test incr_payload_test...
@0 uvm_test_top.tb.yapp.agent.sequencer@@yapp_incr_payload_seq [yapp_incr_payload_seq] Executing
yapp_incr_payload_seq sequence
@50 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral          2     'h0
  length                      integral          6     'd5
  parity_type                 parity_type_e     32    GOOD_PARITY
@295 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral          2     'h0
  length        integral          6     'd5
  parity_type   parity_type_e     32    GOOD_PARITY
@500 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] YAPP driver report: 1 packets sent
@500 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP monitor report: 1 packets
collected
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
```

exhaustive_seq_test

```
@0 reporter [RNTST] Running test exhaustive_seq_test...
@0 uvm_test_top.tb.yapp.agent.sequencer@@yapp_exhaustive_seq [yapp_exhaustive_seq] Executing
yapp_exhaustive_seq sequence
@0 uvm_test_top.tb.yapp.agent.sequencer@@yapp_exhaustive_seq.seq_1 [yapp_1_seq] Executing
yapp_1_seq sequence
@50 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral           2     'h1
  length                      integral           6     'd14
  parity_type                 parity_type_e      32    GOOD_PARITY
@335 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral           2     'h1
  length        integral           6     'd14
  parity_type   parity_type_e      32    GOOD_PARITY
@340 uvm_test_top.tb.yapp.agent.sequencer@@yapp_exhaustive_seq.seq_012 [yapp_012_seq] Executing
yapp_012_seq sequence
@340 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral           2     'h0
  length                      integral           6     'd4
  parity_type                 parity_type_e      32    GOOD_PARITY
@605 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral           2     'h0
  length        integral           6     'd4
  parity_type   parity_type_e      32    GOOD_PARITY
  length        integral           6     'd12
  length        integral           6     'd9
@1840 uvm_test_top.tb.yapp.agent.sequencer@@yapp_exhaustive_seq.seq_repeat_addr
[yapp_repeat_addr_seq] Executing yapp_repeat_addr_seq sequence (addr …
  parity_type                 parity_type_e      32    BAD_PARITY
... (16 lines left out)
@6670 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP monitor report: 25 packets
collected
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (179 more lines of the kinds above)
```

yapp_012_test

```
@0 reporter [RNTST] Running test yapp_012_test...
@0 uvm_test_top.tb.yapp.agent.sequencer@@yapp_012_seq [yapp_012_seq] Executing yapp_012_seq
sequence
@50 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h0
  length                      integral       6     'd3
  parity_type                 parity_type_e  32    BAD_PARITY
@235 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral       2     'h0
  length        integral       6     'd3
  parity_type   parity_type_e  32    BAD_PARITY
@240 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h1
  length                      integral       6     'd4
  parity_type                 parity_type_e  32    GOOD_PARITY
@405 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral       2     'h1
  length        integral       6     'd4
  parity_type   parity_type_e  32    GOOD_PARITY
  parity_type                 parity_type_e  32    GOOD_PARITY
  addr          integral       2     'h2
  length        integral       6     'd12
  parity_type   parity_type_e  32    GOOD_PARITY
@960 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] YAPP driver report: 3 packets sent
@960 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP monitor report: 3 packets
collected
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (4 more lines of the kinds above)
```

Try this

Small changes to make in `start/` once the lab works, each with a question; the answer unfolds.

Set the drain time to 0 in `base_test` (`set_drain_time(this, 0)`). Which packet is lost, and in which report do you see it?

Open the waves (`make sim TEST=yapp_012_test WAVES=1`) and find the first packet on `in0`. How many clock cycles does a packet of length 4 take on `in_data`, and what marks its end?

Checkpoint questions

Why `set(null, "*.tb.yapp.*", …)` from `tb_top`?

Why test the return value of `get()`?

What goes wrong without the drain time?

Why does the monitor sample on the rising edge when the driver drives on the falling edge?

What changed since the previous lab

```
diff -r labs/lab05_seq labs/lab06_vif
```

- `yapp_if.sv`, `yapp_vif_config`, `vif` in driver and monitor
- `top.sv` → `tb_top.sv` with the `set()`, new `hw_top.sv`, `clkgen.sv`
- `base_test::run_phase` drain time, `yapp_012_test`
- `run.f`: interface, RTL, `clkgen.sv`, `hw_top.sv`, `tb_top.sv`

Lab 7 — Integrating multiple UVCs

Open on the project map →

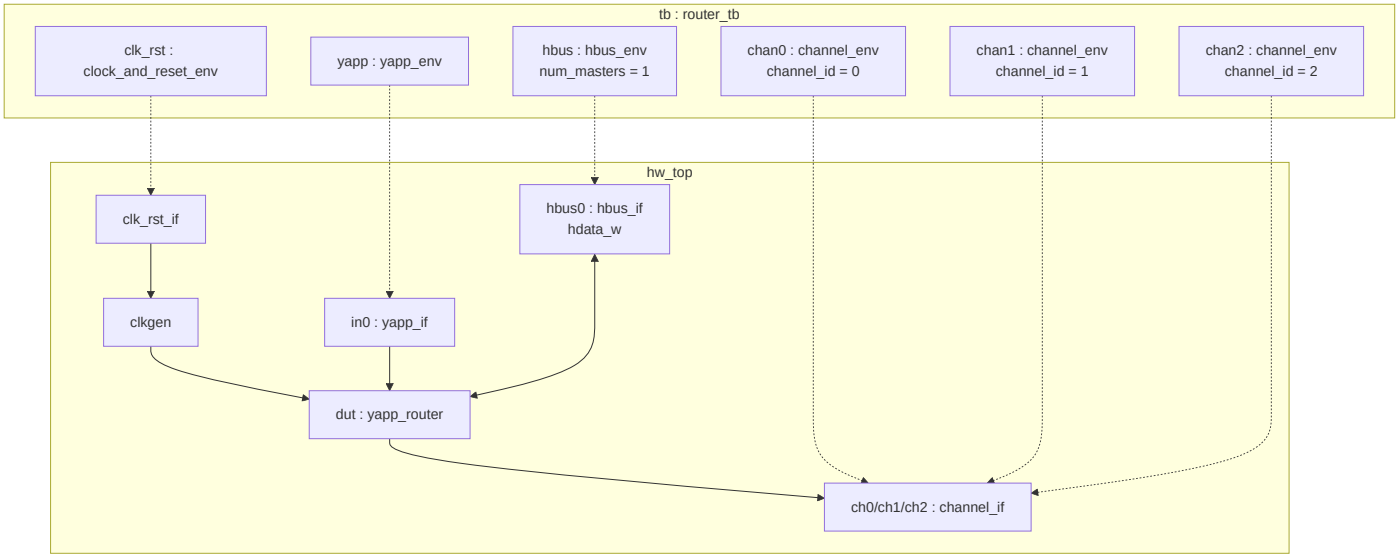
Directory: `labs/lab07_integ` (only `tb/`; the YAPP UVC moved to `yapp_project/uvc/yapp`) · **UVCs added:** `yapp_project/uvc/hbus`, `yapp_project/uvc/channel`, `yapp_project/uvc/clock_and_reset`

Objective

Add the three "provided" UVCs — HBUS, Channel (×3) and Clock & Reset — configure them, wire their interfaces in `hw_top` and run a test on the full system.

Concepts

reusing UVCs · `uvm_config_int::set` before build · several interfaces in `hw_top` · one `set()` per virtual interface in `tb_top` · default sequences per UVC · the `hdata_w` wire



The provided UVCs at a glance

UVC	Env / agent	Interface
`hbus_pkg`	`hbus_env` with `masters[0]` (driver + sequencer) and a shared `monitor`	`hbus_if` — `haddr, hen, hwr_rd, hdata, hdata_oe`

UVC	Env / agent	Interface
`channel_pkg`	`channel_env` → `rx_agent` (driver plays the receiver, monitor, sequencer)	`channel_if` — `data, data_vld, suspe`
`clock_and_reset_pkg`	`clock_and_reset_env` → `agent`	`clock_and_reset_if` — `clock, reset, run_clock, clock_peri`

Solution

1. `tb/router_tb.sv`

```
//------------------------------------------------------------------------------
// router_tb.sv -- the testbench environment (Lab 7: all four UVCs)
//
//   yapp     : YAPP input UVC (built in Labs 1-6, now in yapp_project/uvc/yapp)
//   chan0..2 : one Channel UVC per router output, configured with channel_id
//   hbus     : HBUS UVC, one master agent, no slave (the DUT is the slave)
//   clk_rst  : Clock & Reset UVC, needs no configuration
//------------------------------------------------------------------------------
class router_tb extends uvm_env;

  yapp_env            m_yapp;
  channel_env         m_chan0;
  channel_env         m_chan1;
  channel_env         m_chan2;
  hbus_env            m_hbus;
  clock_and_reset_env m_clk_rst;

  `uvm_component_utils(router_tb)

  extern function new(string name, uvm_component parent);
  extern function void build_phase(uvm_phase phase);

endclass : router_tb

//------------------------------------------------------------------------------
// router_tb -- method implementations
//------------------------------------------------------------------------------

function router_tb::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//------------------------------------------------------------------------------
function void router_tb::build_phase(uvm_phase phase);
  super.build_phase(phase);
  `uvm_info(get_type_name(), "Executing the build phase of the testbench", UVM_HIGH)

  // Configuration BEFORE construction
  uvm_config_int::set(this, "chan0", "channel_id", 0);
  uvm_config_int::set(this, "chan1", "channel_id", 1);
  uvm_config_int::set(this, "chan2", "channel_id", 2);
  uvm_config_int::set(this, "hbus",  "num_masters", 1);

  m_yapp    = yapp_env::type_id::create("yapp", this);
  m_chan0   = channel_env::type_id::create("chan0", this);
  m_chan1   = channel_env::type_id::create("chan1", this);
  m_chan2   = channel_env::type_id::create("chan2", this);
  m_hbus    = hbus_env::type_id::create("hbus", this);
  m_clk_rst = clock_and_reset_env::type_id::create("clk_rst", this);
endfunction : build_phase
```

2. `tb/hw_top.sv`

```
//-------------------------------------------------------------------------------
// hw_top.sv -- hardware side of the testbench (Lab 7: every interface wired)
//
// The Clock & Reset UVC now owns reset, the clock period and run_clock, so the
// hand-written reset block from Lab 6 is gone.
//
// WARNING: the HBUS data bus is bidirectional. Connect the DUT to the WIRE
// `hdata_w` of the interface, never to the logic variable `hdata`.
//-------------------------------------------------------------------------------
`timescale 1ns/1ns

module hw_top;

  logic clock;
  logic reset;
  logic run_clock;
  int   clock_period;

  // Clock & Reset UVC interface drives reset, run_clock and clock_period
  clock_and_reset_if clk_rst_if (
    .clock        (clock),
    .reset        (reset),
    .run_clock    (run_clock),
    .clock_period (clock_period)
  );

  // Clock generator controlled by the Clock & Reset UVC
  clkgen clkgen (.run_clock(run_clock), .clock_period(clock_period), .clock(clock));

  // Interface instances, all on the same clock and reset
  yapp_if    in0   (clock, reset);
  hbus_if    hbus0 (clock, reset);
  channel_if ch0   (clock, reset);
  channel_if ch1   (clock, reset);
  channel_if ch2   (clock, reset);

  // Status output of the router
  logic error;

  yapp_router dut (
    .clock       (clock),
    .reset       (reset),
    // YAPP input port
    .in_data     (in0.in_data),
    .in_data_vld (in0.in_data_vld),
    .in_suspend  (in0.in_suspend),
    // Output channels
    .data_0      (ch0.data),
    .data_vld_0  (ch0.data_vld),
    .suspend_0   (ch0.suspend),
    .data_1      (ch1.data),
    .data_vld_1  (ch1.data_vld),
    .suspend_1   (ch1.suspend),
    .data_2      (ch2.data),
    .data_vld_2  (ch2.data_vld),
    .suspend_2   (ch2.suspend),
    // HBUS host port
    .hdata       (hbus0.hdata_w),        // the WIRE, not hbus0.hdata
```

```
    .haddr       (hbus0.haddr),
    .hen         (hbus0.hen),
    .hwr_rd      (hbus0.hwr_rd),
    // Status
    .error       (error)
  );

endmodule : hw_top
```

> ⚠ **`hdata_w`, not `hdata`**
>
> `hbus_if.hdata` is the logic variable the master drives; `hbus_if.hdata_w` is the tri-state wire. Connecting the DUT's `inout hdata` to the variable gives a multiple-driver conflict; connect the **wire**.

3. `tb/tb_top.sv`

```
//------------------------------------------------------------------------------
// tb_top.sv -- UVM side of the testbench (Lab 7)
//
// Every UVC package has a `<uvc>_vif_config` typedef. One set() per interface,
// each with a wildcard so every component of that UVC gets the handle.
//------------------------------------------------------------------------------
`timescale 1ns/1ns

module tb_top;

  import uvm_pkg::*;
  import rand_util_pkg::*;   // rnd:: random values (common/rand_util_pkg.sv)
  `include "uvm_macros.svh"
  import yapp_pkg::*;
  import channel_pkg::*;
  import hbus_pkg::*;
  import clock_and_reset_pkg::*;

  `include "router_tb.sv"
  `include "router_test_lib.sv"

  initial begin
    yapp_vif_config::set(null,            "*.tb.yapp.*",    "vif", hw_top.in0);
    hbus_vif_config::set(null,            "*.tb.hbus.*",    "vif", hw_top.hbus0);
    channel_vif_config::set(null,         "*.tb.chan0.*",   "vif", hw_top.ch0);
    channel_vif_config::set(null,         "*.tb.chan1.*",   "vif", hw_top.ch1);
    channel_vif_config::set(null,         "*.tb.chan2.*",   "vif", hw_top.ch2);
    clock_and_reset_vif_config::set(null, "*.tb.clk_rst.*", "vif", hw_top.clk_rst_if);
    run_test();
  end

endmodule : tb_top
```

4. `tb/run.f` — one incdir, one package, one interface per UVC

```
// Lab 7 -- xrun command file: every UVC adds an incdir, a package and an interface
-uvmhome CDNS-1.1d
-timescale 1ns/1ns

-incdir ../../../common
../../../common/rand_util_pkg.sv   // rnd:: random values, used by every UVC and test

// YAPP UVC (your files from lab06_vif, now standalone in yapp_project/uvc/yapp)
-incdir ../../../yapp_project/uvc/yapp
../../../yapp_project/uvc/yapp/yapp_pkg.sv
../../../yapp_project/uvc/yapp/yapp_if.sv

// Channel UVC
-incdir ../../../yapp_project/uvc/channel
../../../yapp_project/uvc/channel/channel_pkg.sv
../../../yapp_project/uvc/channel/channel_if.sv

// HBUS UVC
-incdir ../../../yapp_project/uvc/hbus
../../../yapp_project/uvc/hbus/hbus_pkg.sv
../../../yapp_project/uvc/hbus/hbus_if.sv

// Clock & Reset UVC
-incdir ../../../yapp_project/uvc/clock_and_reset
../../../yapp_project/uvc/clock_and_reset/clock_and_reset_pkg.sv
../../../yapp_project/uvc/clock_and_reset/clock_and_reset_if.sv
../../../yapp_project/uvc/clock_and_reset/clkgen.sv

// DUT and top levels
-F ../../../yapp_project/rtl/yapp_router.f   // the DUT: one module per file
hw_top.sv
tb_top.sv

// Tests: base_test | simple_test | test_uvc_integration
+UVM_TESTNAME=simple_test
+UVM_VERBOSITY=UVM_MEDIUM    // one line per packet; UVM_HIGH adds the full packet tables
+SVSEED=random
```

5. Tests

```
//------------------------------------------------------------------------------
// router_test_lib.sv -- test library (Lab 7: multi-UVC tests)
//
// Sequencer paths (from the base_test topology report):
//   tb.yapp.agent.sequencer
//   tb.chan0.rx_agent.sequencer   (chan1, chan2 likewise -> "tb.chan*.rx_agent.sequencer")
//   tb.hbus.masters[0].sequencer
//   tb.clk_rst.agent.sequencer
//
// One class per file: the classes live in tests/<class>.sv and are included below.
//------------------------------------------------------------------------------

`include "tests/base_test.sv"
`include "tests/simple_test.sv"
`include "tests/test_uvc_integration.sv"

//------------------------------------------------------------------------------
// base_test.sv -- base_test (test library (Lab 7: multi-UVC tests))
// Split out of router_test_lib.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// base_test: builds the testbench, prints the topology, no default sequences.
// Without a clock sequence nothing runs, which is exactly what you want when
// checking the topology report.
//------------------------------------------------------------------------------
class base_test extends uvm_test;

  router_tb m_tb;

  `uvm_component_utils(base_test)

  extern function new(string name, uvm_component parent);
  extern function void build_phase(uvm_phase phase);

  extern virtual function void configure_sequences();

  // Helpers shared by the tests below
  extern function void set_clock_and_channel_sequences();
  extern function void end_of_elaboration_phase(uvm_phase phase);

  extern task run_phase(uvm_phase phase);
  extern function void check_phase(uvm_phase phase);

endclass : base_test

//------------------------------------------------------------------------------
// base_test -- method implementations
//------------------------------------------------------------------------------

function base_test::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//------------------------------------------------------------------------------
function void base_test::build_phase(uvm_phase phase);
  super.build_phase(phase);
  `uvm_info(get_type_name(), "Executing the build phase of the test", UVM_HIGH)
```

```
  uvm_config_int::set(this, "*", "recording_detail", 1);
  configure_sequences();
  m_tb = router_tb::type_id::create("tb", this);
endfunction : build_phase

//------------------------------------------------------------------------------
function void base_test::configure_sequences();
  // nothing: derived tests decide what runs
endfunction : configure_sequences

//------------------------------------------------------------------------------
function void base_test::set_clock_and_channel_sequences();
  uvm_config_wrapper::set(this, "tb.clk_rst.agent.sequencer.run_phase",
                          "default_sequence", clk10_rst5_seq::get_type());
  // one wildcard statement for all three channels
  uvm_config_wrapper::set(this, "tb.chan*.rx_agent.sequencer.run_phase",
                          "default_sequence", channel_rx_resp_seq::get_type());
endfunction : set_clock_and_channel_sequences

//------------------------------------------------------------------------------
function void base_test::end_of_elaboration_phase(uvm_phase phase);
  uvm_top.print_topology();
endfunction : end_of_elaboration_phase

//------------------------------------------------------------------------------
task base_test::run_phase(uvm_phase phase);
  uvm_objection obj = phase.get_objection();
  obj.set_drain_time(this, 200ns);
endtask : run_phase

//------------------------------------------------------------------------------
function void base_test::check_phase(uvm_phase phase);
  check_config_usage();
endfunction : check_phase

//------------------------------------------------------------------------------
// simple_test.sv -- simple_test (test library (Lab 7: multi-UVC tests))
// Split out of router_test_lib.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// simple_test: short packets to channels 0, 1, 2; channels respond; clock and
// reset from the UVC; the HBUS stays idle (router uses its reset values).
//------------------------------------------------------------------------------
class simple_test extends base_test;

  `uvm_component_utils(simple_test)

  extern function new(string name, uvm_component parent);
  extern function void build_phase(uvm_phase phase);

  extern function void configure_sequences();

endclass : simple_test

//------------------------------------------------------------------------------
// simple_test -- method implementations
//------------------------------------------------------------------------------
```

```
function simple_test::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//------------------------------------------------------------------------------
function void simple_test::build_phase(uvm_phase phase);
  set_type_override_by_type(yapp_packet::get_type(), short_yapp_packet::get_type());
  super.build_phase(phase);
endfunction : build_phase

//------------------------------------------------------------------------------
function void simple_test::configure_sequences();
  uvm_config_wrapper::set(this, "tb.yapp.agent.sequencer.run_phase",
                          "default_sequence", yapp_012_seq::get_type());
  set_clock_and_channel_sequences();
  // no default sequence for the HBUS UVC
endfunction : configure_sequences

//------------------------------------------------------------------------------
// test_uvc_integration.sv -- test_uvc_integration (test library (Lab 7: multi-UVC tests))
// Split out of router_test_lib.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// test_uvc_integration (optional): 88 packets to all four addresses with
// lengths 1..22 and 20% bad parity, while the HBUS sets maxpktsize = 20 and
// enables the router. Check: routing, `error` on bad parity, drops for
// length > 20 and for address 3 ("ROUTER DROPS PACKET" in the log).
//------------------------------------------------------------------------------
class test_uvc_integration extends base_test;

  `uvm_component_utils(test_uvc_integration)

  extern function new(string name, uvm_component parent);
  extern function void configure_sequences();

endclass : test_uvc_integration

//------------------------------------------------------------------------------
// test_uvc_integration -- method implementations
//------------------------------------------------------------------------------

function test_uvc_integration::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//------------------------------------------------------------------------------
function void test_uvc_integration::configure_sequences();
  uvm_config_wrapper::set(this, "tb.yapp.agent.sequencer.run_phase",
                          "default_sequence", yapp_88_packets_seq::get_type());
  uvm_config_wrapper::set(this, "tb.hbus.masters[0].sequencer.run_phase",
                          "default_sequence", hbus_small_packet_seq::get_type());
  set_clock_and_channel_sequences();
endfunction : configure_sequences
```

Run

```
make run TEST=base_test            # topology only -- save it, later labs need the paths
make run TEST=simple_test
make gui TEST=simple_test          # add YAPP + channel monitor transactions to the waveform
make run TEST=test_uvc_integration # optional
```

Expected for `simple_test`: `clk10_rst5_seq` starts the clock and holds reset 5 cycles; `yapp_012_seq` sends three short packets; each channel monitor prints exactly one `Channel N collected packet` whose fields equal the YAPP packet with `m_addr == N`; the channel reports say `1 packets collected` each.

Expected for `test_uvc_integration`: HBUS `WRITE addr=0x1000 data=0x14` and `0x1001 data=0x01`; 88 packets; `ROUTER DROPS PACKET … length > maxpktsize` for lengths 21 and 22, `… illegal address` for every address-3 packet; `error` pulses in the waveform after bad-parity packets; the channel monitors together collect 3 × 20 = 60 packets.

See it run

The lines that matter from the real log of every test of this lab (Verilator, seed 1, from the last `make regress`; the paths and the packet tables are left out). Your own log must show the same lines -- `make check LAB=lab07_integ START=1` looks for them.

base_test

```
@0 reporter [RNTST] Running test base_test...
@0 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 report: 0 packets
collected
@0 uvm_test_top.tb.chan1.rx_agent.monitor [channel_rx_monitor] Channel 1 report: 0 packets
collected
@0 uvm_test_top.tb.hbus.monitor [hbus_monitor] HBUS report: 0 writes, 0 reads
@0 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] YAPP driver report: 0 packets sent
@0 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP monitor report: 0 packets collected,
coverage 0.0%
@0 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 0 of 1 cycle,
0 of 2, 0 longer (gap coverage 0.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (1 more lines of the kinds above)
```

simple_test

```
@0 reporter [RNTST] Running test simple_test...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@0 uvm_test_top.tb.yapp.agent.sequencer@@yapp_012_seq [yapp_012_seq] Executing yapp_012_seq
sequence
@1140 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 report: 1 packets
collected
@1140 uvm_test_top.tb.chan1.rx_agent.monitor [channel_rx_monitor] Channel 1 report: 1 packets
collected
@1140 uvm_test_top.tb.hbus.monitor [hbus_monitor] HBUS report: 0 writes, 0 reads
@1140 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] YAPP driver report: 3 packets sent
@1140 uvm_test_top.tb.yapp.agent.gap_cov [yapp_gap_coverage] YAPP gap coverage: 2 gaps sampled,
coverage 25.0%
@1140 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP monitor report: 3 packets
collected, 1 with bad parity
@1140 uvm_test_top.tb.yapp.agent.pkt_cov [yapp_pkt_coverage] YAPP packet coverage: 3 packets
sampled, coverage 40.0%
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (2 more lines of the kinds above)
```

test_uvc_integration

```
@0 reporter [RNTST] Running test test_uvc_integration...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@0 uvm_test_top.tb.hbus.masters[0].sequencer@@hbus_small_packet_seq [hbus_small_packet_seq]
Executing hbus_small_packet_seq (maxpktsize=20, router_en…
@0 uvm_test_top.tb.yapp.agent.sequencer@@yapp_88_packets_seq [yapp_88_packets_seq] Executing
yapp_88_packets_seq sequence
@50 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h0
  length                      integral       6     'd1
  parity_type                 parity_type_e  32    GOOD_PARITY
@75 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1000 data=0x14
@95 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1001 data=0x01
@285 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral       2     'h0
  length        integral       6     'd1
  parity_type   parity_type_e  32    GOOD_PARITY
@290 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h0
  length                      integral       6     'd2
  parity_type                 parity_type_e  32    BAD_PARITY
@415 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 collected packet:
  addr         integral               2     'h0
  length       integral               6     'd1
  parity_type  channel_parity_type_e  32    CHANNEL_GOOD_PARITY
... (30 lines left out)
@24160 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 5 of 1
cycle, 3 of 2, 79 longer (gap coverage 75.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (910 more lines of the kinds above)
```

Try this

Small changes to make in `start/` once the lab works, each with a question; the answer unfolds.

Build the HBUS env with `num_masters = 2`. What does the topology show, and why does the second master never do anything?

Checkpoint questions

Why `uvm_config_int::set(this, "chan0", "channel_id", 0)` and not a constructor argument?

Why no default sequence for the HBUS in `simple_test`?

How can one statement configure three channel sequencers?

Where does the reset come from now?

What changed since the previous lab

```
diff -r labs/lab06_vif/tb labs/lab07_integ/tb
diff -r labs/lab06_vif/sv yapp_project/uvc/yapp          # the UVC became standalone (+ later
additions)
```

- `router_tb`: three channel envs, HBUS env, Clock & Reset env, their configuration
- `hw_top`: all interfaces, `clkgen` driven by the clock interface, no reset block, full DUT connection
- `tb_top`: imports and `set()` for every interface
- tests: `simple_test`, `test_uvc_integration`; older tests removed

Lab 8 — Multichannel sequences and system-level tests

Open on the project map →

Directory: `labs/lab08_mcseq` · **New:** `tb/router_mcsequencer.sv`, `tb/router_mcseqs_lib.sv` · **Changed:** `router_tb.sv`, `router_test_lib.sv`, `tb_top.sv`

Objective

Coordinate the HBUS and YAPP traffic from one place: a multichannel (virtual) sequencer and a sequence that programs the router, sends packets, reprograms it and sends more.

Concepts

virtual sequencer with sub-sequencer handles · `` `uvm_declare_p_sequencer `` · `seq.start(p_sequencer.m_hbus_seqr, this)` (the course's `` `uvm_do_on `` / `` `uvm_do_on_with ``) · objection on the starting phase · hierarchical references in `connect_phase`



Solution

1. `tb/router_mcsequencer.sv`

```
//------------------------------------------------------------------------------
// router_mcsequencer.sv -- multichannel (virtual) sequencer (Lab 8)
//
// It drives no interface itself: it only holds handles to the sequencers it
// coordinates. A multichannel sequence runs on it and starts sub-sequences on
// those handles. The Channel UVCs run a fixed response sequence and the
// Clock & Reset UVC is left out to keep things simple.
//------------------------------------------------------------------------------
class router_mcsequencer extends uvm_sequencer;

  hbus_master_sequencer m_hbus_seqr;   // assigned by router_tb in connect_phase
  yapp_tx_sequencer     m_yapp_seqr;

  `uvm_component_utils(router_mcsequencer)

  extern function new(string name, uvm_component parent);

endclass : router_mcsequencer

//------------------------------------------------------------------------------
// router_mcsequencer -- method implementations
//------------------------------------------------------------------------------

function router_mcsequencer::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new
```

2. `tb/router_mcseqs_lib.sv`

```
//------------------------------------------------------------------------------
// router_mcseqs_lib.sv -- multichannel sequence library (Lab 8)
//
// `uvm_declare_p_sequencer gives every sequence a typed `p_sequencer` handle,
// so `p_sequencer.hbus_seqr` / `p_sequencer.yapp_seqr` are available.
// seq.start(p_sequencer.<seqr>, this) runs a sub-sequence on another sequencer.
//
// One class per file: the classes live in mcseqs/<class>.sv and are included below.
//------------------------------------------------------------------------------

`include "mcseqs/router_mcseq_base.sv"
`include "mcseqs/router_simple_mcseq.sv"

//------------------------------------------------------------------------------
// router_mcseq_base.sv -- router_mcseq_base (multichannel sequence library (Lab 8))
// Split out of router_mcseqs_lib.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// router_mcseq_base: objection on the starting phase + p_sequencer
//------------------------------------------------------------------------------
class router_mcseq_base extends uvm_sequence;

  `uvm_object_utils(router_mcseq_base)
  `uvm_declare_p_sequencer(router_mcsequencer)

  extern function new(string name = "router_mcseq_base");
  extern task pre_body();

  extern task post_body();

endclass : router_mcseq_base

//------------------------------------------------------------------------------
// router_mcseq_base -- method implementations
//------------------------------------------------------------------------------

function router_mcseq_base::new(string name = "router_mcseq_base");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task router_mcseq_base::pre_body();
  uvm_phase phase = `YAPP_STARTING_PHASE;
  if (phase != null) begin
    phase.raise_objection(this, get_type_name());
  end
endtask : pre_body

//------------------------------------------------------------------------------
task router_mcseq_base::post_body();
  uvm_phase phase = `YAPP_STARTING_PHASE;
  if (phase != null) begin
    phase.drop_objection(this, get_type_name());
  end
endtask : post_body

//------------------------------------------------------------------------------
```

```
// router_simple_mcseq.sv -- router_simple_mcseq (multichannel sequence library (Lab 8))
// Split out of router_mcseqs_lib.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// router_simple_mcseq: program the router over HBUS, then send YAPP traffic
//   1. maxpktsize = 20, router enabled        (hbus_small_packet_seq)
//   2. read MAXPKTSIZE back                    (hbus_read_max_pkt_seq)
//   3. 6 packets to addresses 0,1,2            (yapp_012_seq twice)
//   4. maxpktsize = 63                         (hbus_large_packet_seq)
//   5. read MAXPKTSIZE back                    (hbus_read_max_pkt_seq)
//   6. 6 random packets                        (yapp_rnd_seq, count == 6)
//------------------------------------------------------------------------------
class router_simple_mcseq extends router_mcseq_base;

  hbus_small_packet_seq m_hbus_small_seq;
  hbus_large_packet_seq m_hbus_large_seq;
  hbus_read_max_pkt_seq m_hbus_read_seq;
  yapp_012_seq          m_yapp_012;
  yapp_rnd_seq          m_yapp_rnd;

  `uvm_object_utils(router_simple_mcseq)

  extern function new(string name = "router_simple_mcseq");
  extern task body();

endclass : router_simple_mcseq

//------------------------------------------------------------------------------
// router_simple_mcseq -- method implementations
//------------------------------------------------------------------------------

function router_simple_mcseq::new(string name = "router_simple_mcseq");
  super.new(name);
endfunction : new

//------------------------------------------------------------------------------
task router_simple_mcseq::body();
  `uvm_info(get_type_name(), "Executing router_simple_mcseq sequence", UVM_LOW)

  // Small packets allowed, router enabled -- and check the register
  m_hbus_small_seq = hbus_small_packet_seq::type_id::create("hbus_small_seq");
  if (!m_hbus_small_seq.randomize()) begin
    `uvm_error(get_type_name(), "hbus_small_seq.randomize() failed")
  end
  m_hbus_small_seq.start(p_sequencer.m_hbus_seqr, this);

  m_hbus_read_seq = hbus_read_max_pkt_seq::type_id::create("hbus_read_seq");
  if (!m_hbus_read_seq.randomize()) begin
    `uvm_error(get_type_name(), "hbus_read_seq.randomize() failed")
  end
  m_hbus_read_seq.start(p_sequencer.m_hbus_seqr, this);

  // Six packets to channels 0, 1, 2
  repeat (2) begin
    m_yapp_012 = yapp_012_seq::type_id::create("yapp_012");
    if (!m_yapp_012.randomize()) begin
      `uvm_error(get_type_name(), "yapp_012.randomize() failed")
    end
```

```
    m_yapp_012.start(p_sequencer.m_yapp_seqr, this);
  end

  // Large packets allowed -- and check the register
  m_hbus_large_seq = hbus_large_packet_seq::type_id::create("hbus_large_seq");
  if (!m_hbus_large_seq.randomize()) begin
    `uvm_error(get_type_name(), "hbus_large_seq.randomize() failed")
  end
  m_hbus_large_seq.start(p_sequencer.m_hbus_seqr, this);

  m_hbus_read_seq = hbus_read_max_pkt_seq::type_id::create("hbus_read_seq");
  if (!m_hbus_read_seq.randomize()) begin
    `uvm_error(get_type_name(), "hbus_read_seq.randomize() failed")
  end
  m_hbus_read_seq.start(p_sequencer.m_hbus_seqr, this);

  // Six random packets
  m_yapp_rnd = yapp_rnd_seq::type_id::create("yapp_rnd");
  if (!m_yapp_rnd.randomize() with { m_yapp_rnd.m_count == 6; }) begin
    `uvm_error(get_type_name(), "yapp_rnd.randomize() failed")
  end
  m_yapp_rnd.start(p_sequencer.m_yapp_seqr, this);
endtask : body
```

3. `router_tb` : build and connect

```
m_mcseqr = router_mcsequencer::type_id::create("mcseqr", this);
...
function void connect_phase(uvm_phase phase);
  // Hierarchical references, NOT configuration strings: no wildcards here
  m_mcseqr.m_hbus_seqr = m_hbus.m_masters[0].m_sequencer;
  m_mcseqr.m_yapp_seqr = m_yapp.m_agent.m_sequencer;
endfunction
```

4. The test

```
class router_simple_mcseq_test extends base_test;
  ...
  function void build_phase(uvm_phase phase);
    set_type_override_by_type(yapp_packet::get_type(), short_yapp_packet::get_type());
    super.build_phase(phase);
  endfunction
  function void configure_sequences();
    set_clock_and_channel_sequences();
    uvm_config_wrapper::set(this, "tb.mcseqr.run_phase",
                            "default_sequence", router_simple_mcseq::get_type());
    // no default sequence for YAPP or HBUS: the mcseq controls them
  endfunction
endclass
```

5. Include order in `tb_top.sv`

```
`include "router_mcsequencer.sv"     // router_tb has a handle of this type
`include "router_mcseqs_lib.sv"
`include "router_tb.sv"
`include "router_test_lib.sv"
```

Run

```
make run TEST=router_simple_mcseq_test
```

Expected: in this order in the log —

```
[hbus_small_packet_seq] Executing ...
[hbus_monitor] Collected WRITE addr=0x1000 data=0x14
[hbus_monitor] Collected WRITE addr=0x1001 data=0x01
[hbus_read_max_pkt_seq] MAXPKTSIZE register reads 20
[yapp_012_seq] Executing ...  (×2, six packets, channel monitors echo them)
[hbus_monitor] Collected WRITE addr=0x1000 data=0x3f
[hbus_read_max_pkt_seq] MAXPKTSIZE register reads 63
[yapp_rnd_seq] Executing yapp_rnd_seq sequence (6 packets)
```

12 YAPP packets in total, 12 channel packets, `UVM_ERROR : 0`.

See it run

The lines that matter from the real log of every test of this lab (Verilator, seed 1, from the last `make regress`; the paths and the packet tables are left out). Your own log must show the same lines -- `make check LAB=lab08_mcseq START=1` looks for them.

base_test

```
@0 reporter [RNTST] Running test base_test...
@0 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 report: 0 packets
collected
@0 uvm_test_top.tb.chan1.rx_agent.monitor [channel_rx_monitor] Channel 1 report: 0 packets
collected
@0 uvm_test_top.tb.hbus.monitor [hbus_monitor] HBUS report: 0 writes, 0 reads
@0 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] YAPP driver report: 0 packets sent
@0 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP monitor report: 0 packets collected,
coverage 0.0%
@0 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 0 of 1 cycle,
0 of 2, 0 longer (gap coverage 0.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (1 more lines of the kinds above)
```

simple_test

```
@0 reporter [RNTST] Running test simple_test...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@0 uvm_test_top.tb.yapp.agent.sequencer@@yapp_012_seq [yapp_012_seq] Executing yapp_012_seq
sequence
@50 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral           2     'h0
  length                      integral           6     'd9
  parity_type                 parity_type_e      32    BAD_PARITY
@225 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral           2     'h0
  length        integral           6     'd9
  parity_type   parity_type_e      32    BAD_PARITY
@230 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral           2     'h1
  length                      integral           6     'd14
  parity_type                 parity_type_e      32    GOOD_PARITY
@355 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 collected packet:
  addr         integral              2     'h0
  length       integral              6     'd9
  parity_type  channel_parity_type_e  32    CHANNEL_BAD_PARITY
@575 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral           2     'h1
  length        integral           6     'd14
... (19 lines left out)
@1140 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 0 of 1
cycle, 0 of 2, 2 longer (gap coverage 25.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (10 more lines of the kinds above)
```

test_uvc_integration

```
@0 reporter [RNTST] Running test test_uvc_integration...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@0 uvm_test_top.tb.hbus.masters[0].sequencer@@hbus_small_packet_seq [hbus_small_packet_seq]
Executing hbus_small_packet_seq (maxpktsize=20, router_en…
@0 uvm_test_top.tb.yapp.agent.sequencer@@yapp_88_packets_seq [yapp_88_packets_seq] Executing
yapp_88_packets_seq sequence
@50 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h0
  length                      integral       6     'd1
  parity_type                 parity_type_e  32    GOOD_PARITY
@75 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1000 data=0x14
@95 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1001 data=0x01
@285 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral       2     'h0
  length        integral       6     'd1
  parity_type   parity_type_e  32    GOOD_PARITY
@290 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h0
  length                      integral       6     'd2
  parity_type                 parity_type_e  32    BAD_PARITY
@415 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 collected packet:
  addr         integral               2     'h0
  length       integral               6     'd1
  parity_type  channel_parity_type_e  32    CHANNEL_GOOD_PARITY
... (30 lines left out)
@24160 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 5 of 1
cycle, 3 of 2, 79 longer (gap coverage 75.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (910 more lines of the kinds above)
```

router_simple_mcseq_test

```
@0 reporter [RNTST] Running test router_simple_mcseq_test...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@0 uvm_test_top.tb.mcseqr@@router_simple_mcseq [router_simple_mcseq] Executing
router_simple_mcseq sequence
@0 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_small_seq
[hbus_small_packet_seq] Executing hbus_small_packet_seq (maxpktsize=…
@75 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1000 data=0x14
@90 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_read_seq
[hbus_read_max_pkt_seq] Executing hbus_read_max_pkt_seq
@95 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1001 data=0x01
@120 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_read_seq
[hbus_read_max_pkt_seq] MAXPKTSIZE register reads 20
@120 uvm_test_top.tb.yapp.agent.sequencer@@router_simple_mcseq.yapp_012 [yapp_012_seq] Executing
yapp_012_seq sequence
@120 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral          2     'h0
  length                      integral          6     'd1
  parity_type                 parity_type_e     32    GOOD_PARITY
@125 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected READ  addr=0x1000 data=0x14
@335 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral          2     'h0
  length        integral          6     'd1
  parity_type   parity_type_e     32    GOOD_PARITY
@340 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral          2     'h1
  length                      integral          6     'd14
  parity_type                 parity_type_e     32    GOOD_PARITY
... (39 lines left out)
@3140 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 2 of 1
cycle, 0 of 2, 9 longer (gap coverage 50.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (109 more lines of the kinds above)
```

Try this

Small changes to make in `start/` once the lab works, each with a question; the answer unfolds.

In `router_simple_mcseq`, start `hbus_read_max_pkt_seq` before `hbus_small_packet_seq`. What does the read return?

Checkpoint questions

Why can the mcsequencer's handles not be set with `uvm_config_*`?

What does `` `uvm_declare_p_sequencer `` add?

Who raises the objection?

Why six packets with `yapp_012_seq` twice rather than one sequence of six?

What changed since the previous lab

```
diff -r labs/lab07_integ/tb labs/lab08_mcseq/tb
```

Lab 9A — Creating a scoreboard using TLM

Open on the project map →

Directory: `labs/lab09_sba` · **New:** `sv/router_scoreboard.sv`, `sv/packet_compare.sv` · **Changed:** `yapp_project/uvc/yapp/yapp_tx_monitor.sv` (analysis port), `tb/router_tb.sv`, `tb/tb_top.sv`, tests

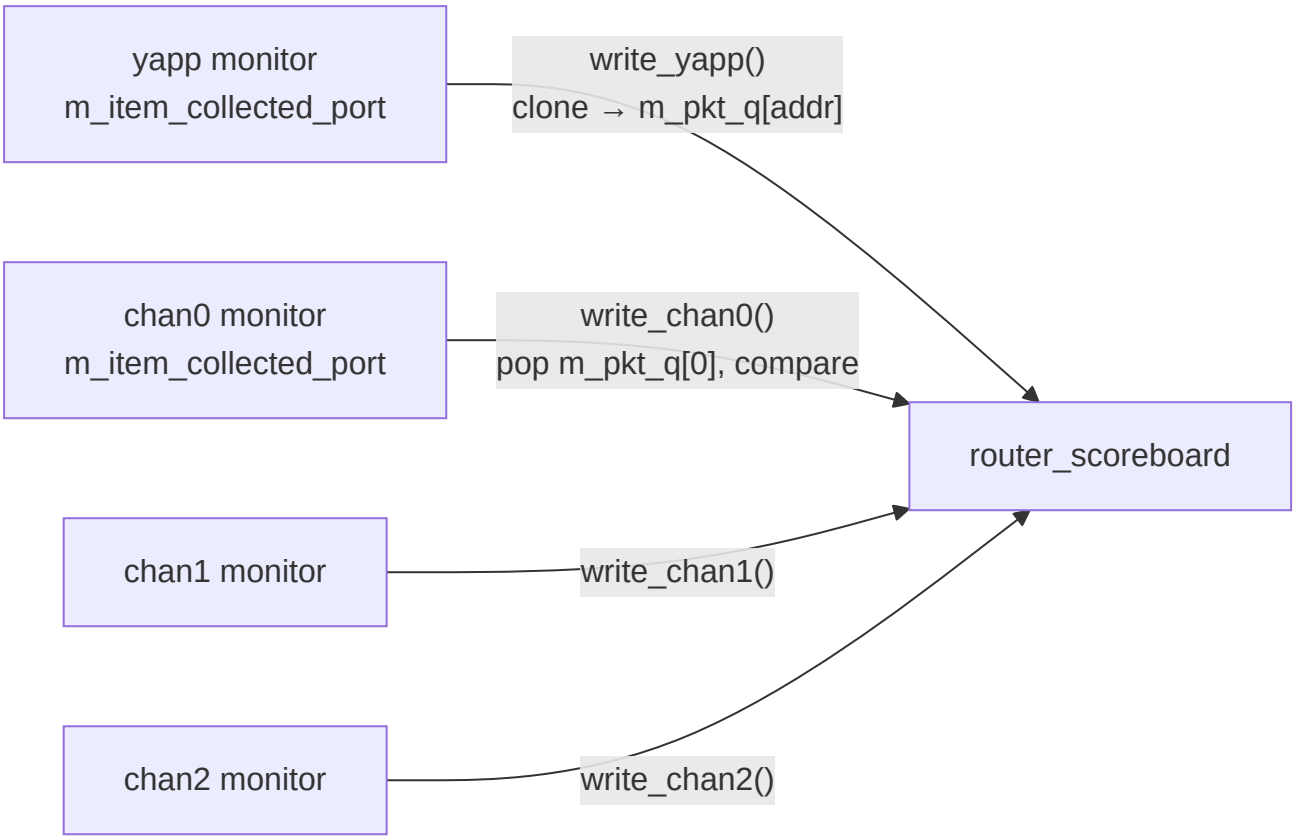
Objective

Make the tests self-checking: a scoreboard that receives every packet sent into the router and every packet that came out, and compares.

For this first step all traffic is legal (short packets, legal addresses): the router drops nothing.

Concepts

`uvm_analysis_port #(T)` · `write()` · `` `uvm_analysis_imp_decl(_suffix) `` · `uvm_analysis_imp_suffix #(T, IMP)` · `clone()` · per-address queues · `report_phase`



Solution

1. Analysis port in the YAPP monitor (`yapp_project/uvc/yapp/yapp_tx_monitor.sv`)

```
uvm_analysis_port #(yapp_packet) m_item_collected_port;
...
function new(string name, uvm_component parent);
  super.new(name, parent);
  m_item_collected_port = new("item_collected_port", this);
endfunction
...
m_item_collected_port.write(pkt);      // after each collected packet
```

The channel monitor already had one (`yapp_project/uvc/channel/channel_rx_monitor.sv`).

2. The scoreboard — `sv/router_scoreboard.sv`

```
//------------------------------------------------------------------------------
// router_scoreboard.sv -- compares what went into the router with what came
// out of each channel (Lab 9A)
//
//   YAPP monitor ──port──► yapp_in  (imp)  : clone and queue per address
//   ChanN monitor──port──► chanN_in (imp)  : pop queue N and compare
//
// An analysis imp needs a write_<suffix>() method per suffix; the suffixes are
// created by the `uvm_analysis_imp_decl macros below (outside the class).
//------------------------------------------------------------------------------
`uvm_analysis_imp_decl(_yapp)
`uvm_analysis_imp_decl(_chan0)
`uvm_analysis_imp_decl(_chan1)
`uvm_analysis_imp_decl(_chan2)

class router_scoreboard extends uvm_scoreboard;

  // TLM connection points
  uvm_analysis_imp_yapp  #(yapp_packet,    router_scoreboard) m_yapp_in;
  uvm_analysis_imp_chan0 #(channel_packet, router_scoreboard) m_chan0_in;
  uvm_analysis_imp_chan1 #(channel_packet, router_scoreboard) m_chan1_in;
  uvm_analysis_imp_chan2 #(channel_packet, router_scoreboard) m_chan2_in;

  // Expected packets, one queue per output channel
  yapp_packet m_pkt_q[3][$];

  // Statistics
  int m_packets_in;         // received from the YAPP monitor
  int m_packets_matched;    // compared OK
  int m_packets_mismatched; // compare failed
  int m_packets_unexpected; // channel packet with an empty queue
  int m_packets_dropped_addr3;

  `uvm_component_utils(router_scoreboard)

  extern function new(string name, uvm_component parent);

  // The comparison function supplied with the lab
  `include "packet_compare.sv"

  //--------------------------------------------------------------------------
  // YAPP side: the monitor passes a HANDLE, so clone before storing it
  //--------------------------------------------------------------------------
  extern function void write_yapp(yapp_packet packet);

  //--------------------------------------------------------------------------
  // Channel side
  //--------------------------------------------------------------------------
  extern function void write_chan0(channel_packet packet);
  extern function void write_chan1(channel_packet packet);

  extern function void write_chan2(channel_packet packet);
  extern function void check_channel(int ch, channel_packet cp);

  //--------------------------------------------------------------------------
  // End of test: the numbers must add up
  //   packets_in == matched + mismatched + left in queues (+ addr 3)
  //--------------------------------------------------------------------------
```

```
  extern function void report_phase(uvm_phase phase);

endclass : router_scoreboard

//------------------------------------------------------------------------------
// router_scoreboard -- method implementations
//------------------------------------------------------------------------------

function router_scoreboard::new(string name, uvm_component parent);
  super.new(name, parent);
  m_yapp_in  = new("yapp_in",  this);
  m_chan0_in = new("chan0_in", this);
  m_chan1_in = new("chan1_in", this);
  m_chan2_in = new("chan2_in", this);
endfunction : new

//------------------------------------------------------------------------------
function void router_scoreboard::write_yapp(yapp_packet packet);
  yapp_packet pkt;
  $cast(pkt, packet.clone());
  m_packets_in++;
  if (pkt.m_addr == 2'd3) begin
    // Lab 9A assumes legal traffic only; keep the count honest anyway
    m_packets_dropped_addr3++;
    `uvm_warning(get_type_name(), "Packet to illegal address 3 is not expected on any channel")
    return;
  end
  m_pkt_q[pkt.m_addr].push_back(pkt);
  `uvm_info(get_type_name(),
            $sformatf("Queued packet for channel %0d (%0d waiting)", pkt.m_addr,
m_pkt_q[pkt.m_addr].size()),
            UVM_MEDIUM)
endfunction : write_yapp

//------------------------------------------------------------------------------
function void router_scoreboard::write_chan0(channel_packet packet);
  check_channel(0, packet);
endfunction : write_chan0

//------------------------------------------------------------------------------
function void router_scoreboard::write_chan1(channel_packet packet);
  check_channel(1, packet);
endfunction : write_chan1

//------------------------------------------------------------------------------
function void router_scoreboard::write_chan2(channel_packet packet);
  check_channel(2, packet);
endfunction : write_chan2

//------------------------------------------------------------------------------
function void router_scoreboard::check_channel(int ch, channel_packet cp);
  yapp_packet expected;
  if (m_pkt_q[ch].size() == 0) begin
    m_packets_unexpected++;
    `uvm_error(get_type_name(),
               $sformatf("Channel %0d packet received but nothing was expected:\n%s", ch,
cp.sprint()))
    return;
  end
```

```
  expected = m_pkt_q[ch].pop_front();
  if (comp_equal(expected, cp)) begin
    m_packets_matched++;
    `uvm_info(get_type_name(), $sformatf("Channel %0d packet matched", ch), UVM_MEDIUM)
  end else begin
    m_packets_mismatched++;
    `uvm_error(get_type_name(),
               $sformatf("Channel %0d packet MISMATCH\nexpected:\n%s\nreceived:\n%s",
                         ch, expected.sprint(), cp.sprint()))
  end
endfunction : check_channel

//------------------------------------------------------------------------------
function void router_scoreboard::report_phase(uvm_phase phase);
  `uvm_info(get_type_name(), $sformatf({"\n--- Scoreboard report ---\n",
    "  packets received  : %0d\n",
    "  packets matched   : %0d\n",
    "  packets mismatched: %0d\n",
    "  packets unexpected: %0d\n",
    "  packets to addr 3 : %0d\n",
    "  left in queue 0/1/2: %0d / %0d / %0d"},
    m_packets_in, m_packets_matched, m_packets_mismatched, m_packets_unexpected,
m_packets_dropped_addr3,
    m_pkt_q[0].size(), m_pkt_q[1].size(), m_pkt_q[2].size()), UVM_LOW)
endfunction : report_phase
```

3. The comparison — `sv/packet_compare.sv` (included in the class)

```
//------------------------------------------------------------------------------
// packet_compare.sv -- supplied for Lab 9A: compare a YAPP packet (what went
// into the router) with a Channel packet (what came out).
//
// The two are different classes, so the built-in compare() cannot be used
// directly. This file is `included INSIDE the scoreboard class body.
//------------------------------------------------------------------------------

  // Plain SystemVerilog comparison, one `uvm_error per difference
  function bit comp_equal(input yapp_packet yp, input channel_packet cp);
    bit ok = 1;
    if (yp.m_addr != cp.m_addr) begin
      `uvm_error("PKT_COMPARE", $sformatf("Address mismatch: YAPP %0d, Channel %0d", yp.m_addr,
cp.m_addr))
      ok = 0;
    end
    if (yp.m_length != cp.m_length) begin
      `uvm_error("PKT_COMPARE", $sformatf("Length mismatch: YAPP %0d, Channel %0d", yp.m_length,
cp.m_length))
      ok = 0;
    end
    foreach (yp.m_payload[i]) begin
      if (i >= cp.m_payload.size()) begin
        break;
      end
      if (yp.m_payload[i] != cp.m_payload[i]) begin
        `uvm_error("PKT_COMPARE", $sformatf("Payload[%0d] mismatch: YAPP 0x%02h, Channel 0x%02h",
                                            i, yp.m_payload[i], cp.m_payload[i]))
        ok = 0;
      end
    end
    if (yp.m_parity != cp.m_parity) begin
      `uvm_error("PKT_COMPARE", $sformatf("Parity mismatch: YAPP 0x%02h, Channel 0x%02h",
yp.m_parity, cp.m_parity))
      ok = 0;
    end
    return ok;
  endfunction : comp_equal

  // Optional: the same check with the uvm_comparer policy object
  function bit comp_equal_uvm(input yapp_packet yp, input channel_packet cp);
    uvm_comparer comparer = new();
    bit ok = 1;
    ok &= comparer.compare_field_int("addr",   yp.m_addr,   cp.m_addr,   2);
    ok &= comparer.compare_field_int("length", yp.m_length, cp.m_length, 6);
    ok &= comparer.compare_field_int("parity", yp.m_parity, cp.m_parity, 8);
    if (yp.m_payload.size() == cp.m_payload.size()) begin
      foreach (yp.m_payload[i]) begin
        ok &= comparer.compare_field_int($sformatf("payload[%0d]", i), yp.m_payload[i],
cp.m_payload[i], 8);
      end
    end else begin
      ok = 0;
    end
    if (!ok) begin
      `uvm_error("PKT_COMPARE", "uvm_comparer found differences between the YAPP and Channel
packets")
    end
```

```
    return ok;
  endfunction : comp_equal_uvm
```

4. `router_tb` : build and connect

```
m_scoreboard = router_scoreboard::type_id::create("scoreboard", this);
...
m_yapp.m_agent.m_monitor.m_item_collected_port.connect(m_scoreboard.m_yapp_in);
m_chan0.m_rx_agent.m_monitor.m_item_collected_port.connect(m_scoreboard.m_chan0_in);
m_chan1.m_rx_agent.m_monitor.m_item_collected_port.connect(m_scoreboard.m_chan1_in);
m_chan2.m_rx_agent.m_monitor.m_item_collected_port.connect(m_scoreboard.m_chan2_in);
```

`tb_top.sv` includes `router_scoreboard.sv` (before `router_tb.sv`), `run.f` gets `-incdir ../sv`.

Run

```
make run TEST=router_simple_mcseq_test    # legal traffic
make run TEST=scoreboard_drop_test        # negative check (step 7)
```

Expected, `router_simple_mcseq_test`:

```
--- Scoreboard report ---
  packets received  : 12
  packets matched   : 12
  packets mismatched: 0
  packets unexpected: 0
  left in queue 0/1/2: 0 / 0 / 0
```

Expected, `scoreboard_drop_test` (no short-packet override, `maxpktsize` = 20 during the first six packets): `yapp_packet` tables with random lengths; `ROUTER DROPS PACKET … length > maxpktsize` for every packet longer than 20; `UVM_ERROR … [PKT_COMPARE] Length mismatch` / `… MISMATCH` when the next packet on that channel is compared against the dropped one; at the end `packets received == matched + mismatched + left in queues`.

See it run

The lines that matter from the real log of every test of this lab (Verilator, seed 1, from the last `make regress`; the paths and the packet tables are left out). Your own log must show the same lines -- `make check LAB=lab09_sba START=1` looks for them.

base_test

```
@0 reporter [RNTST] Running test base_test...
@0 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 report: 0 packets
collected
@0 uvm_test_top.tb.chan1.rx_agent.monitor [channel_rx_monitor] Channel 1 report: 0 packets
collected
@0 uvm_test_top.tb.hbus.monitor [hbus_monitor] HBUS report: 0 writes, 0 reads
--- Scoreboard report ---
  packets matched   : 0
  packets mismatched: 0
@0 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] YAPP driver report: 0 packets sent
@0 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP monitor report: 0 packets collected,
coverage 0.0%
@0 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 0 of 1 cycle,
0 of 2, 0 longer (gap coverage 0.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (1 more lines of the kinds above)
```

simple_test

```
@0 reporter [RNTST] Running test simple_test...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@0 uvm_test_top.tb.yapp.agent.sequencer@@yapp_012_seq [yapp_012_seq] Executing yapp_012_seq
sequence
@50 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral           2     'h0
  length                      integral           6     'd9
  parity_type                 parity_type_e      32    BAD_PARITY
@225 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral           2     'h0
  length        integral           6     'd9
  parity_type   parity_type_e      32    BAD_PARITY
@230 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral           2     'h1
  length                      integral           6     'd14
  parity_type                 parity_type_e      32    GOOD_PARITY
@355 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 collected packet:
  addr         integral              2     'h0
  length       integral              6     'd9
  parity_type  channel_parity_type_e  32    CHANNEL_BAD_PARITY
@575 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral           2     'h1
  length        integral           6     'd14
... (22 lines left out)
@1140 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 0 of 1
cycle, 0 of 2, 2 longer (gap coverage 25.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (10 more lines of the kinds above)
```

test_uvc_integration

```
@0 reporter [RNTST] Running test test_uvc_integration...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@0 uvm_test_top.tb.hbus.masters[0].sequencer@@hbus_small_packet_seq [hbus_small_packet_seq]
Executing hbus_small_packet_seq (maxpktsize=20, router_en…
@0 uvm_test_top.tb.yapp.agent.sequencer@@yapp_88_packets_seq [yapp_88_packets_seq] Executing
yapp_88_packets_seq sequence
@50 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h0
  length                      integral       6     'd1
  parity_type                 parity_type_e  32    GOOD_PARITY
@75 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1000 data=0x14
@95 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1001 data=0x01
@285 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral       2     'h0
  length        integral       6     'd1
  parity_type   parity_type_e  32    GOOD_PARITY
@290 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h0
  length                      integral       6     'd2
  parity_type                 parity_type_e  32    BAD_PARITY
@415 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 collected packet:
  addr         integral               2     'h0
  length       integral               6     'd1
  parity_type  channel_parity_type_e  32    CHANNEL_GOOD_PARITY
... (33 lines left out)
@24160 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 5 of 1
cycle, 3 of 2, 79 longer (gap coverage 75.0%)
UVM_WARNING :   22
UVM_ERROR :    0
UVM_FATAL :    0
... (910 more lines of the kinds above)
```

router_simple_mcseq_test

```
@0 reporter [RNTST] Running test router_simple_mcseq_test...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@0 uvm_test_top.tb.mcseqr@@router_simple_mcseq [router_simple_mcseq] Executing
router_simple_mcseq sequence
@0 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_small_seq
[hbus_small_packet_seq] Executing hbus_small_packet_seq (maxpktsize=…
@75 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1000 data=0x14
@90 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_read_seq
[hbus_read_max_pkt_seq] Executing hbus_read_max_pkt_seq
@95 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1001 data=0x01
@120 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_read_seq
[hbus_read_max_pkt_seq] MAXPKTSIZE register reads 20
@120 uvm_test_top.tb.yapp.agent.sequencer@@router_simple_mcseq.yapp_012 [yapp_012_seq] Executing
yapp_012_seq sequence
@120 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral          2     'h0
  length                      integral          6     'd1
  parity_type                 parity_type_e     32    GOOD_PARITY
@125 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected READ  addr=0x1000 data=0x14
@335 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral          2     'h0
  length        integral          6     'd1
  parity_type   parity_type_e     32    GOOD_PARITY
@340 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral          2     'h1
  length                      integral          6     'd14
  parity_type                 parity_type_e     32    GOOD_PARITY
... (42 lines left out)
@3140 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 2 of 1
cycle, 0 of 2, 9 longer (gap coverage 50.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (109 more lines of the kinds above)
```

scoreboard_drop_test

```
@0 reporter [RNTST] Running test scoreboard_drop_test...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@0 uvm_test_top.tb.mcseqr@@router_simple_mcseq [router_simple_mcseq] Executing
router_simple_mcseq sequence
@0 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_small_seq
[hbus_small_packet_seq] Executing hbus_small_packet_seq (maxpktsize=…
@75 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1000 data=0x14
@90 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_read_seq
[hbus_read_max_pkt_seq] Executing hbus_read_max_pkt_seq
@95 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1001 data=0x01
@120 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_read_seq
[hbus_read_max_pkt_seq] MAXPKTSIZE register reads 20
@120 uvm_test_top.tb.yapp.agent.sequencer@@router_simple_mcseq.yapp_012 [yapp_012_seq] Executing
yapp_012_seq sequence
@120 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h0
  length                      integral       6     43
  parity_type                 parity_type_e  32    GOOD_PARITY
@125 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected READ  addr=0x1000 data=0x14
@635 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral       2     'h0
  length        integral       6     43
  parity_type   parity_type_e  32    GOOD_PARITY
@640 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h1
  length                      integral       6     'd17
  parity_type                 parity_type_e  32    GOOD_PARITY
... (55 lines left out)
@5730 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 0 of 1
cycle, 0 of 2, 11 longer (gap coverage 25.0%)
UVM_WARNING :    0
UVM_ERROR :   49
UVM_FATAL :    0
... (145 more lines of the kinds above)
```

Try this

Small changes to make in `start/` once the lab works, each with a question; the answer unfolds.

Send one packet with `PARITY_TYPE_BAD` to address 0 (constrain it in a sequence). What does the scoreboard say, and why is it wrong?

Checkpoint questions

Why clone in `write_yapp()`?

Why four imps instead of one?

Why a queue per address and not one queue?

Do the numbers add up in the negative test?

Optional

`comp_equal_uvm()` in `packet_compare.sv` uses `uvm_comparer` instead of `!=`. Swap the call in `check_channel()` to try it.

What changed since the previous lab

```
diff -r labs/lab08_mcseq/tb labs/lab09_sba/tb
```

Lab 9B — Router module UVC

Open on the project map →

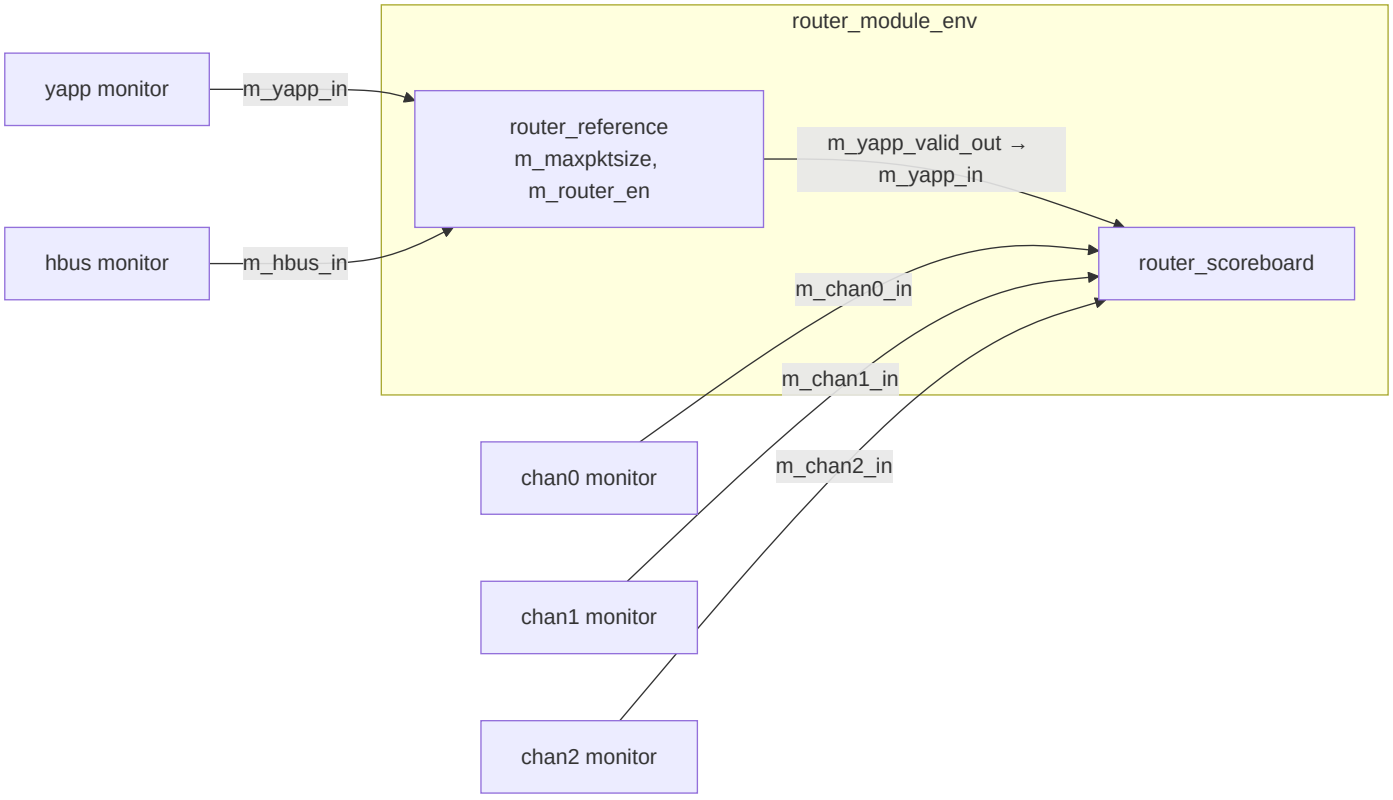
Directory: `labs/lab09_sbb` · **New:** `sv/router_reference.sv`, `sv/router_module_env.sv`, `sv/router_module_pkg.sv` · **Changed:** `tb/router_tb.sv`, `tb/tb_top.sv`, `run.f` · **Result copied to:** `yapp_project/uvc/router` (where the reference model has since grown into a full model of the registers, counters and memories — see Reference model)

Objective

Turn the scoreboard into a reusable **module UVC**: an env that also contains a reference model which knows — from the HBUS traffic — which packets the router will drop.

Concepts

module UVC vs interface UVC · reference model · mirroring register settings from bus traffic · layered TLM connections · UVC package



Solution

1. The reference model — `sv/router_reference.sv`

```
//------------------------------------------------------------------------------
// router_reference.sv -- reference model of the router's packet filtering
// (Lab 9B)
//
//   HBUS monitor ──port──► hbus_in (imp) : mirror maxpktsize and router_en
//   YAPP monitor ──port──► yapp_in (imp) : forward the packet on
//                                          yapp_valid_out only if the router
//                                          will route it
//
// `_yapp` was declared by router_scoreboard.sv (same package); only `_hbus`
// is new here.
//------------------------------------------------------------------------------
`uvm_analysis_imp_decl(_hbus)

class router_reference extends uvm_component;

  // Inputs
  uvm_analysis_imp_yapp #(yapp_packet,      router_reference) m_yapp_in;
  uvm_analysis_imp_hbus #(hbus_transaction, router_reference) m_hbus_in;
  // Output: packets the router is expected to deliver
  uvm_analysis_port #(yapp_packet) m_yapp_valid_out;

  // Mirrored register fields (reset values of the DUT)
  bit[5:0] m_maxpktsize = 6'h3f;
  bit      m_router_en  = 1'b1;

  // Statistics
  int m_packets_forwarded;
  int m_dropped_disabled;
  int m_dropped_size;
  int m_dropped_addr;

  `uvm_component_utils(router_reference)

  extern function new(string name, uvm_component parent);

  // Keep the register mirror up to date from the bus traffic
  extern function void write_hbus(hbus_transaction tr);

  // Same decision the RTL makes when the header arrives
  extern function void write_yapp(yapp_packet pkt);
  extern function void report_phase(uvm_phase phase);

endclass : router_reference

//------------------------------------------------------------------------------
// router_reference -- method implementations
//------------------------------------------------------------------------------

function router_reference::new(string name, uvm_component parent);
  super.new(name, parent);
  m_yapp_in        = new("yapp_in", this);
  m_hbus_in        = new("hbus_in", this);
  m_yapp_valid_out = new("yapp_valid_out", this);
endfunction : new

//------------------------------------------------------------------------------
function void router_reference::write_hbus(hbus_transaction tr);
```

```
  if (tr.m_hwr_rd != HBUS_OP_WRITE) begin
    return;
  end
  case (tr.m_haddr)
    16'h1000: begin
      m_maxpktsize = tr.m_hdata[5:0];
      `uvm_info(get_type_name(), $sformatf("maxpktsize is now %0d", m_maxpktsize), UVM_LOW)
    end
    16'h1001: begin
      m_router_en = tr.m_hdata[0];
      `uvm_info(get_type_name(), $sformatf("router_en is now %0d", m_router_en), UVM_LOW)
    end
    default: ;
  endcase
endfunction : write_hbus

//------------------------------------------------------------------------------
function void router_reference::write_yapp(yapp_packet pkt);
  if (!m_router_en) begin
    m_dropped_disabled++;
    `uvm_info(get_type_name(), "Packet dropped: router disabled", UVM_LOW)
  end else if (pkt.m_addr == 2'd3) begin
    m_dropped_addr++;
    `uvm_info(get_type_name(), "Packet dropped: illegal address 3", UVM_LOW)
  end else if (pkt.m_length > m_maxpktsize) begin
    m_dropped_size++;
    `uvm_info(get_type_name(),
              $sformatf("Packet dropped: length %0d > maxpktsize %0d", pkt.m_length,
m_maxpktsize), UVM_LOW)
  end else begin
    m_packets_forwarded++;
    m_yapp_valid_out.write(pkt);
  end
endfunction : write_yapp

//------------------------------------------------------------------------------
function void router_reference::report_phase(uvm_phase phase);
  `uvm_info(get_type_name(), $sformatf({"\n--- Reference model report ---\n",
    "  forwarded to scoreboard   : %0d\n",
    "  dropped (router disabled) : %0d\n",
    "  dropped (oversized)       : %0d\n",
    "  dropped (address 3)       : %0d"},
    m_packets_forwarded, m_dropped_disabled, m_dropped_size, m_dropped_addr), UVM_LOW)
endfunction : report_phase
```

2. The env — `sv/router_module_env.sv`

```
//------------------------------------------------------------------------------
// router_module_env.sv -- the router MODULE UVC (Lab 9B)
//
// A module UVC has no interface: it is a bundle of analysis components
// (reference model, scoreboard, later coverage) that work on the transactions
// the interface UVCs collect.
//
//   yapp ──►(yapp_in) reference ──yapp_valid_out──►(yapp_in) scoreboard ◄──(chanN_in) chanN
//   hbus ──►(hbus_in)
//------------------------------------------------------------------------------
class router_module_env extends uvm_env;

  router_reference  m_reference;
  router_scoreboard m_scoreboard;

  `uvm_component_utils(router_module_env)

  extern function new(string name, uvm_component parent);
  extern function void build_phase(uvm_phase phase);

  extern function void connect_phase(uvm_phase phase);

endclass : router_module_env

//------------------------------------------------------------------------------
// router_module_env -- method implementations
//------------------------------------------------------------------------------

function router_module_env::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//------------------------------------------------------------------------------
function void router_module_env::build_phase(uvm_phase phase);
  super.build_phase(phase);
  m_reference  = router_reference::type_id::create("reference", this);
  m_scoreboard = router_scoreboard::type_id::create("scoreboard", this);
endfunction : build_phase

//------------------------------------------------------------------------------
function void router_module_env::connect_phase(uvm_phase phase);
  // Only packets the router will route reach the scoreboard
  m_reference.m_yapp_valid_out.connect(m_scoreboard.m_yapp_in);
endfunction : connect_phase
```

3. The package — `sv/router_module_pkg.sv`

```
//------------------------------------------------------------------------------
// router_module_pkg.sv -- router module UVC package (Lab 9B)
//
// Needs the packet types of the interface UVCs, hence the imports.
// Include order: scoreboard first (declares the _yapp/_chanN imps), then the
// reference model (declares _hbus), then the env that holds both.
//------------------------------------------------------------------------------
package router_module_pkg;

  import uvm_pkg::*;
  import rand_util_pkg::*;   // rnd:: random values (common/rand_util_pkg.sv)
  `include "uvm_macros.svh"
  import yapp_pkg::*;
  import channel_pkg::*;
  import hbus_pkg::*;

  `include "router_scoreboard.sv"
  `include "router_reference.sv"
  `include "router_module_env.sv"

endpackage : router_module_pkg
```

4. `router_tb`: connect through the layer

```
m_router_module = router_module_env::type_id::create("router_module", this);
...
m_yapp.m_agent.m_monitor.m_item_collected_port.connect(m_router_module.m_reference.m_yapp_in);
m_hbus.m_monitor.m_item_collected_port.connect(m_router_module.m_reference.m_hbus_in);
m_chan0.m_rx_agent.m_monitor.m_item_collected_port.connect(m_router_module.m_scoreboard.m_chan0_i
n);
m_chan1.m_rx_agent.m_monitor.m_item_collected_port.connect(m_router_module.m_scoreboard.m_chan1_i
n);
m_chan2.m_rx_agent.m_monitor.m_item_collected_port.connect(m_router_module.m_scoreboard.m_chan2_i
n);
```

`tb_top.sv` imports `router_module_pkg` instead of including the scoreboard; `run.f` compiles `../sv/router_module_pkg.sv` after the three UVC packages it imports.

Run

```
make run TEST=router_simple_mcseq_test
make run TEST=scoreboard_drop_test
```

Expected: `router_simple_mcseq_test` as in 9A (12/12). The interesting one is **`scoreboard_drop_test`**: the same oversized packets are dropped by the router, but now the reference model drops them too —

```
[router_reference] Packet dropped: length 47 > maxpktsize 20
...
--- Reference model report ---
  forwarded to scoreboard   : N
  dropped (oversized)       : 12 - N
--- Scoreboard report ---
  packets received  : N
  packets matched   : N
  packets mismatched: 0
  left in queue 0/1/2: 0 / 0 / 0
```

`UVM_ERROR : 0`. The test is self-checking even with illegal traffic.

See it run

The lines that matter from the real log of every test of this lab (Verilator, seed 1, from the last `make regress`; the paths and the packet tables are left out). Your own log must show the same lines -- `make check LAB=lab09_sbb START=1` looks for them.

base_test

```
@0 reporter [RNTST] Running test base_test...
@0 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 report: 0 packets
collected
@0 uvm_test_top.tb.chan1.rx_agent.monitor [channel_rx_monitor] Channel 1 report: 0 packets
collected
@0 uvm_test_top.tb.hbus.monitor [hbus_monitor] HBUS report: 0 writes, 0 reads
--- Reference model report ---
--- Scoreboard report ---
  packets matched   : 0
  packets mismatched: 0
@0 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] YAPP driver report: 0 packets sent
@0 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP monitor report: 0 packets collected,
coverage 0.0%
@0 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 0 of 1 cycle,
0 of 2, 0 longer (gap coverage 0.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (1 more lines of the kinds above)
```

simple_test

```
@0 reporter [RNTST] Running test simple_test...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@0 uvm_test_top.tb.yapp.agent.sequencer@@yapp_012_seq [yapp_012_seq] Executing yapp_012_seq
sequence
@50 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral          2     'h0
  length                      integral          6     'd9
  parity_type                 parity_type_e     32    BAD_PARITY
@225 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral          2     'h0
  length        integral          6     'd9
  parity_type   parity_type_e     32    BAD_PARITY
@230 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral          2     'h1
  length                      integral          6     'd14
  parity_type                 parity_type_e     32    GOOD_PARITY
@355 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 collected packet:
  addr         integral              2     'h0
  length       integral              6     'd9
  parity_type  channel_parity_type_e  32    CHANNEL_BAD_PARITY
@575 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral          2     'h1
  length        integral          6     'd14
... (23 lines left out)
@1140 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 0 of 1
cycle, 0 of 2, 2 longer (gap coverage 25.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (10 more lines of the kinds above)
```

test_uvc_integration

```
@0 reporter [RNTST] Running test test_uvc_integration...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@0 uvm_test_top.tb.hbus.masters[0].sequencer@@hbus_small_packet_seq [hbus_small_packet_seq]
Executing hbus_small_packet_seq (maxpktsize=20, router_en…
@0 uvm_test_top.tb.yapp.agent.sequencer@@yapp_88_packets_seq [yapp_88_packets_seq] Executing
yapp_88_packets_seq sequence
@50 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h0
  length                      integral       6     'd1
  parity_type                 parity_type_e  32    GOOD_PARITY
@75 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1000 data=0x14
@75 uvm_test_top.tb.router_module.reference [router_reference] maxpktsize is now 20
@95 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1001 data=0x01
@95 uvm_test_top.tb.router_module.reference [router_reference] router_en is now 1
@285 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral       2     'h0
  length        integral       6     'd1
  parity_type   parity_type_e  32    GOOD_PARITY
@290 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h0
  length                      integral       6     'd2
  parity_type                 parity_type_e  32    BAD_PARITY
@415 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 collected packet:
  addr         integral               2     'h0
... (36 lines left out)
@24160 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 5 of 1
cycle, 3 of 2, 79 longer (gap coverage 75.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (910 more lines of the kinds above)
```

router_simple_mcseq_test

```
@0 reporter [RNTST] Running test router_simple_mcseq_test...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@0 uvm_test_top.tb.mcseqr@@router_simple_mcseq [router_simple_mcseq] Executing
router_simple_mcseq sequence
@0 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_small_seq
[hbus_small_packet_seq] Executing hbus_small_packet_seq (maxpktsize=…
@75 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1000 data=0x14
@75 uvm_test_top.tb.router_module.reference [router_reference] maxpktsize is now 20
@90 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_read_seq
[hbus_read_max_pkt_seq] Executing hbus_read_max_pkt_seq
@95 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1001 data=0x01
@95 uvm_test_top.tb.router_module.reference [router_reference] router_en is now 1
@120 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_read_seq
[hbus_read_max_pkt_seq] MAXPKTSIZE register reads 20
@120 uvm_test_top.tb.yapp.agent.sequencer@@router_simple_mcseq.yapp_012 [yapp_012_seq] Executing
yapp_012_seq sequence
@120 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral           2     'h0
  length                      integral           6     'd1
  parity_type                 parity_type_e      32    GOOD_PARITY
@125 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected READ  addr=0x1000 data=0x14
@335 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral           2     'h0
  length        integral           6     'd1
  parity_type   parity_type_e      32    GOOD_PARITY
@340 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral           2     'h1
... (47 lines left out)
@3140 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 2 of 1
cycle, 0 of 2, 9 longer (gap coverage 50.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (109 more lines of the kinds above)
```

scoreboard_drop_test

```
@0 reporter [RNTST] Running test scoreboard_drop_test...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@0 uvm_test_top.tb.mcseqr@@router_simple_mcseq [router_simple_mcseq] Executing
router_simple_mcseq sequence
@0 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_small_seq
[hbus_small_packet_seq] Executing hbus_small_packet_seq (maxpktsize=…
@75 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1000 data=0x14
@75 uvm_test_top.tb.router_module.reference [router_reference] maxpktsize is now 20
@90 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_read_seq
[hbus_read_max_pkt_seq] Executing hbus_read_max_pkt_seq
@95 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1001 data=0x01
@95 uvm_test_top.tb.router_module.reference [router_reference] router_en is now 1
@120 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_read_seq
[hbus_read_max_pkt_seq] MAXPKTSIZE register reads 20
@120 uvm_test_top.tb.yapp.agent.sequencer@@router_simple_mcseq.yapp_012 [yapp_012_seq] Executing
yapp_012_seq sequence
@120 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h0
  length                      integral       6     43
  parity_type                 parity_type_e  32    GOOD_PARITY
@125 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected READ  addr=0x1000 data=0x14
@635 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral       2     'h0
  length        integral       6     43
  parity_type   parity_type_e  32    GOOD_PARITY
@640 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h1
... (52 lines left out)
@5730 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 0 of 1
cycle, 0 of 2, 11 longer (gap coverage 25.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (92 more lines of the kinds above)
```

Try this

Small changes to make in `start/` once the lab works, each with a question; the answer unfolds.

Write `0x00` to `en_reg` through `hbus_write_seq` before the packets. What does the reference model predict, and what does the scoreboard report?

Checkpoint questions

What is the difference between an interface UVC and a module UVC?

Why does the reference model mirror the registers from the monitor rather than ask the sequence?

Why can `router_reference` reuse `uvm_analysis_imp_yapp`?

Where should the decision rules live — reference model or scoreboard?

What changed since the previous lab

```
diff -r labs/lab09_sba labs/lab09_sbb
```

Lab 9C — Using TLM export connectors *(optional)*

Open on the project map →

Directory: `labs/lab09_sbc` · **Changed:** `sv/router_module_env.sv`, `tb/router_tb.sv`

Objective

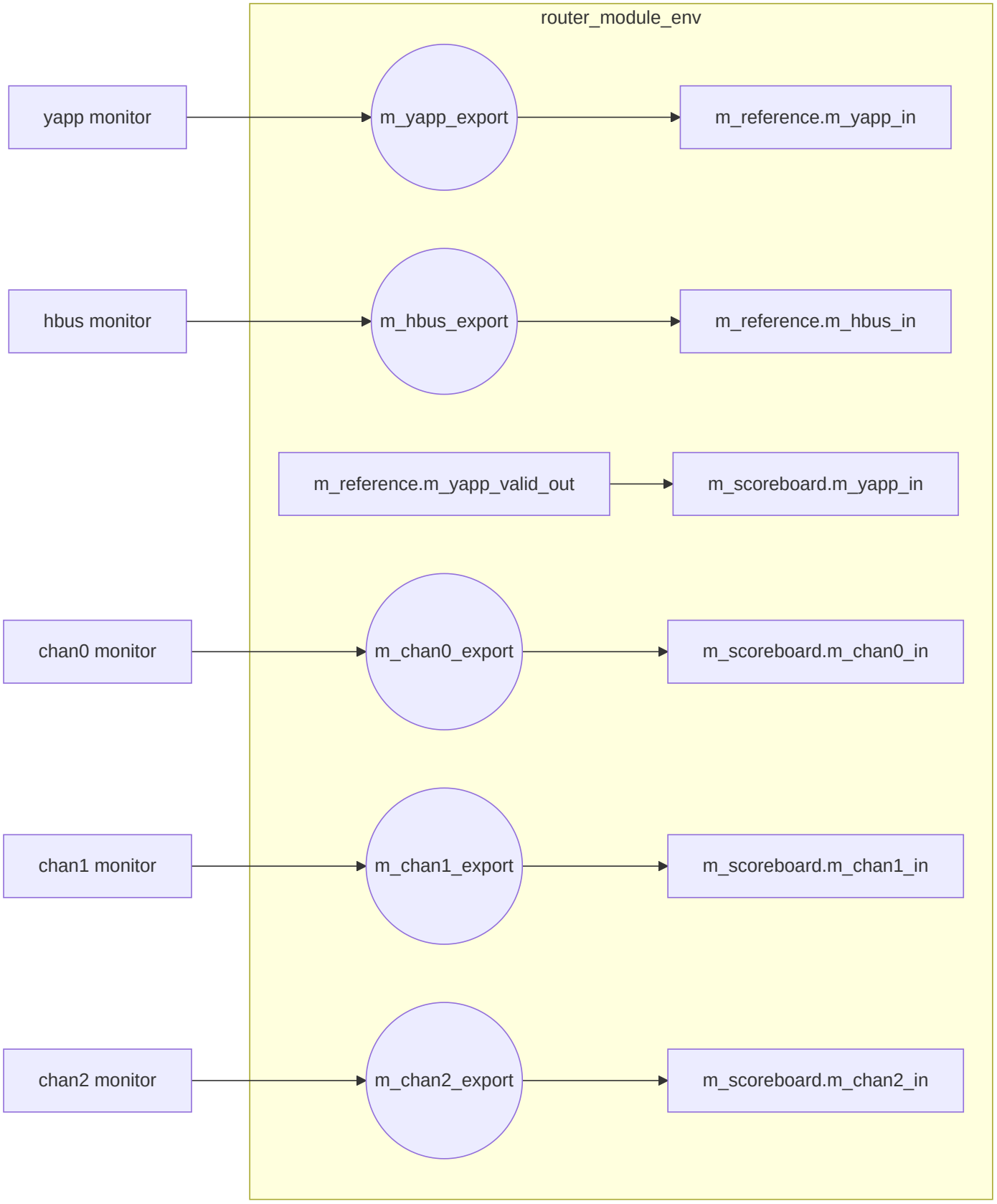
Expose all connection points of the module UVC at the env level with **exports**, so the testbench never reaches into the env.

Concepts

- An `uvm_analysis_export` is a **pass-through**: it has no `write()` of its own; in `connect_phase` the env connects it to the imp inside (`yapp_export.connect(reference.yapp_in)`).
- The testbench then connects monitors to the env's exports and never names a child of the env: the env can be reorganised without touching `router_tb`.

Why

An interface UVC always has its analysis ports in the monitor — you know where to look. In a module UVC the imps can be anywhere inside. Exports put them at the top.



Solution

1. `sv/router_module_env.sv`

```
//------------------------------------------------------------------------------
// router_module_env.sv -- the router MODULE UVC with exports (Lab 9C)
//
// Every external connection point is now an analysis EXPORT on the env, so
// the testbench never has to know where the imps live inside the module UVC:
//
//   yapp_export ──► reference.yapp_in       chan0_export ──► scoreboard.chan0_in
//   hbus_export ──► reference.hbus_in       chan1_export ──► scoreboard.chan1_in
//                                           chan2_export ──► scoreboard.chan2_in
//   reference.yapp_valid_out ──► scoreboard.yapp_in   (internal, unchanged)
//------------------------------------------------------------------------------
class router_module_env extends uvm_env;

  router_reference  m_reference;
  router_scoreboard m_scoreboard;

  // Top-level connection points
  uvm_analysis_export #(yapp_packet)      m_yapp_export;
  uvm_analysis_export #(hbus_transaction) m_hbus_export;
  uvm_analysis_export #(channel_packet)   m_chan0_export;
  uvm_analysis_export #(channel_packet)   m_chan1_export;
  uvm_analysis_export #(channel_packet)   m_chan2_export;

  `uvm_component_utils(router_module_env)

  extern function new(string name, uvm_component parent);
  extern function void build_phase(uvm_phase phase);

  extern function void connect_phase(uvm_phase phase);

endclass : router_module_env

//------------------------------------------------------------------------------
// router_module_env -- method implementations
//------------------------------------------------------------------------------

function router_module_env::new(string name, uvm_component parent);
  super.new(name, parent);
  m_yapp_export  = new("yapp_export",  this);
  m_hbus_export  = new("hbus_export",  this);
  m_chan0_export = new("chan0_export", this);
  m_chan1_export = new("chan1_export", this);
  m_chan2_export = new("chan2_export", this);
endfunction : new

//------------------------------------------------------------------------------
function void router_module_env::build_phase(uvm_phase phase);
  super.build_phase(phase);
  m_reference  = router_reference::type_id::create("reference", this);
  m_scoreboard = router_scoreboard::type_id::create("scoreboard", this);
endfunction : build_phase

//------------------------------------------------------------------------------
function void router_module_env::connect_phase(uvm_phase phase);
  // exports -> internal imps
  m_yapp_export.connect(m_reference.m_yapp_in);
  m_hbus_export.connect(m_reference.m_hbus_in);
  m_chan0_export.connect(m_scoreboard.m_chan0_in);
```

```
  m_chan1_export.connect(m_scoreboard.m_chan1_in);
  m_chan2_export.connect(m_scoreboard.m_chan2_in);
  // internal link stays as in Lab 9B
  m_reference.m_yapp_valid_out.connect(m_scoreboard.m_yapp_in);
endfunction : connect_phase
```

2. `router_tb.connect_phase`

```
m_yapp.m_agent.m_monitor.m_item_collected_port.connect(m_router_module.m_yapp_export);
m_hbus.m_monitor.m_item_collected_port.connect(m_router_module.m_hbus_export);
m_chan0.m_rx_agent.m_monitor.m_item_collected_port.connect(m_router_module.m_chan0_export);
m_chan1.m_rx_agent.m_monitor.m_item_collected_port.connect(m_router_module.m_chan1_export);
m_chan2.m_rx_agent.m_monitor.m_item_collected_port.connect(m_router_module.m_chan2_export);
```

Run

```
make run TEST=router_simple_mcseq_test
make run TEST=scoreboard_drop_test
```

Expected: identical results to Lab 9B — this lab changes the wiring, not the behaviour.

See it run

The lines that matter from the real log of every test of this lab (Verilator, seed 1, from the last `make regress`; the paths and the packet tables are left out). Your own log must show the same lines -- `make check LAB=lab09_sbc START=1` looks for them.

base_test

```
@0 reporter [RNTST] Running test base_test...
@0 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 report: 0 packets
collected
@0 uvm_test_top.tb.chan1.rx_agent.monitor [channel_rx_monitor] Channel 1 report: 0 packets
collected
@0 uvm_test_top.tb.hbus.monitor [hbus_monitor] HBUS report: 0 writes, 0 reads
--- Reference model report ---
--- Scoreboard report ---
  packets matched   : 0
  packets mismatched: 0
@0 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] YAPP driver report: 0 packets sent
@0 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP monitor report: 0 packets collected,
coverage 0.0%
@0 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 0 of 1 cycle,
0 of 2, 0 longer (gap coverage 0.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (1 more lines of the kinds above)
```

simple_test

```
@0 reporter [RNTST] Running test simple_test...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@0 uvm_test_top.tb.yapp.agent.sequencer@@yapp_012_seq [yapp_012_seq] Executing yapp_012_seq
sequence
@50 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral          2     'h0
  length                      integral          6     'd9
  parity_type                 parity_type_e     32    BAD_PARITY
@225 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral          2     'h0
  length        integral          6     'd9
  parity_type   parity_type_e     32    BAD_PARITY
@230 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral          2     'h1
  length                      integral          6     'd14
  parity_type                 parity_type_e     32    GOOD_PARITY
@355 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 collected packet:
  addr         integral              2     'h0
  length       integral              6     'd9
  parity_type  channel_parity_type_e  32    CHANNEL_BAD_PARITY
@575 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral          2     'h1
  length        integral          6     'd14
... (23 lines left out)
@1140 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 0 of 1
cycle, 0 of 2, 2 longer (gap coverage 25.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (10 more lines of the kinds above)
```

test_uvc_integration

```
@0 reporter [RNTST] Running test test_uvc_integration...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@0 uvm_test_top.tb.hbus.masters[0].sequencer@@hbus_small_packet_seq [hbus_small_packet_seq]
Executing hbus_small_packet_seq (maxpktsize=20, router_en…
@0 uvm_test_top.tb.yapp.agent.sequencer@@yapp_88_packets_seq [yapp_88_packets_seq] Executing
yapp_88_packets_seq sequence
@50 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h0
  length                      integral       6     'd1
  parity_type                 parity_type_e  32    GOOD_PARITY
@75 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1000 data=0x14
@75 uvm_test_top.tb.router_module.reference [router_reference] maxpktsize is now 20
@95 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1001 data=0x01
@95 uvm_test_top.tb.router_module.reference [router_reference] router_en is now 1
@285 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral       2     'h0
  length        integral       6     'd1
  parity_type   parity_type_e  32    GOOD_PARITY
@290 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h0
  length                      integral       6     'd2
  parity_type                 parity_type_e  32    BAD_PARITY
@415 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 collected packet:
  addr         integral               2     'h0
... (36 lines left out)
@24160 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 5 of 1
cycle, 3 of 2, 79 longer (gap coverage 75.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (910 more lines of the kinds above)
```

router_simple_mcseq_test

```
@0 reporter [RNTST] Running test router_simple_mcseq_test...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@0 uvm_test_top.tb.mcseqr@@router_simple_mcseq [router_simple_mcseq] Executing
router_simple_mcseq sequence
@0 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_small_seq
[hbus_small_packet_seq] Executing hbus_small_packet_seq (maxpktsize=…
@75 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1000 data=0x14
@75 uvm_test_top.tb.router_module.reference [router_reference] maxpktsize is now 20
@90 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_read_seq
[hbus_read_max_pkt_seq] Executing hbus_read_max_pkt_seq
@95 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1001 data=0x01
@95 uvm_test_top.tb.router_module.reference [router_reference] router_en is now 1
@120 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_read_seq
[hbus_read_max_pkt_seq] MAXPKTSIZE register reads 20
@120 uvm_test_top.tb.yapp.agent.sequencer@@router_simple_mcseq.yapp_012 [yapp_012_seq] Executing
yapp_012_seq sequence
@120 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral           2     'h0
  length                      integral           6     'd1
  parity_type                 parity_type_e      32    GOOD_PARITY
@125 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected READ  addr=0x1000 data=0x14
@335 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral           2     'h0
  length        integral           6     'd1
  parity_type   parity_type_e      32    GOOD_PARITY
@340 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral           2     'h1
... (47 lines left out)
@3140 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 2 of 1
cycle, 0 of 2, 9 longer (gap coverage 50.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (109 more lines of the kinds above)
```

scoreboard_drop_test

```
@0 reporter [RNTST] Running test scoreboard_drop_test...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@0 uvm_test_top.tb.mcseqr@@router_simple_mcseq [router_simple_mcseq] Executing
router_simple_mcseq sequence
@0 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_small_seq
[hbus_small_packet_seq] Executing hbus_small_packet_seq (maxpktsize=…
@75 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1000 data=0x14
@75 uvm_test_top.tb.router_module.reference [router_reference] maxpktsize is now 20
@90 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_read_seq
[hbus_read_max_pkt_seq] Executing hbus_read_max_pkt_seq
@95 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1001 data=0x01
@95 uvm_test_top.tb.router_module.reference [router_reference] router_en is now 1
@120 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_read_seq
[hbus_read_max_pkt_seq] MAXPKTSIZE register reads 20
@120 uvm_test_top.tb.yapp.agent.sequencer@@router_simple_mcseq.yapp_012 [yapp_012_seq] Executing
yapp_012_seq sequence
@120 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h0
  length                      integral       6     43
  parity_type                 parity_type_e  32    GOOD_PARITY
@125 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected READ  addr=0x1000 data=0x14
@635 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral       2     'h0
  length        integral       6     43
  parity_type   parity_type_e  32    GOOD_PARITY
@640 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h1
... (52 lines left out)
@5730 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 0 of 1
cycle, 0 of 2, 11 longer (gap coverage 25.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (92 more lines of the kinds above)
```

Try this

Small changes to make in `start/` once the lab works, each with a question; the answer unfolds.

Connect only `chan0`'s analysis port to the module UVC's export and leave `chan1` / `chan2` unconnected. Does it compile, and what do the counts look like?

Checkpoint questions

What does an export add that a port or an imp does not?

Why does the internal `yapp_valid_out → scoreboard.yapp_in` link stay inside?

Exports are constructed in `new()` — why not in `build_phase`?

What changed since the previous lab

```
diff labs/lab09_sbb/sv/router_module_env.sv labs/lab09_sbc/sv/router_module_env.sv
diff labs/lab09_sbb/tb/router_tb.sv labs/lab09_sbc/tb/router_tb.sv
```

Lab 9D — Using TLM analysis FIFOs *(optional)*

Open on the project map →

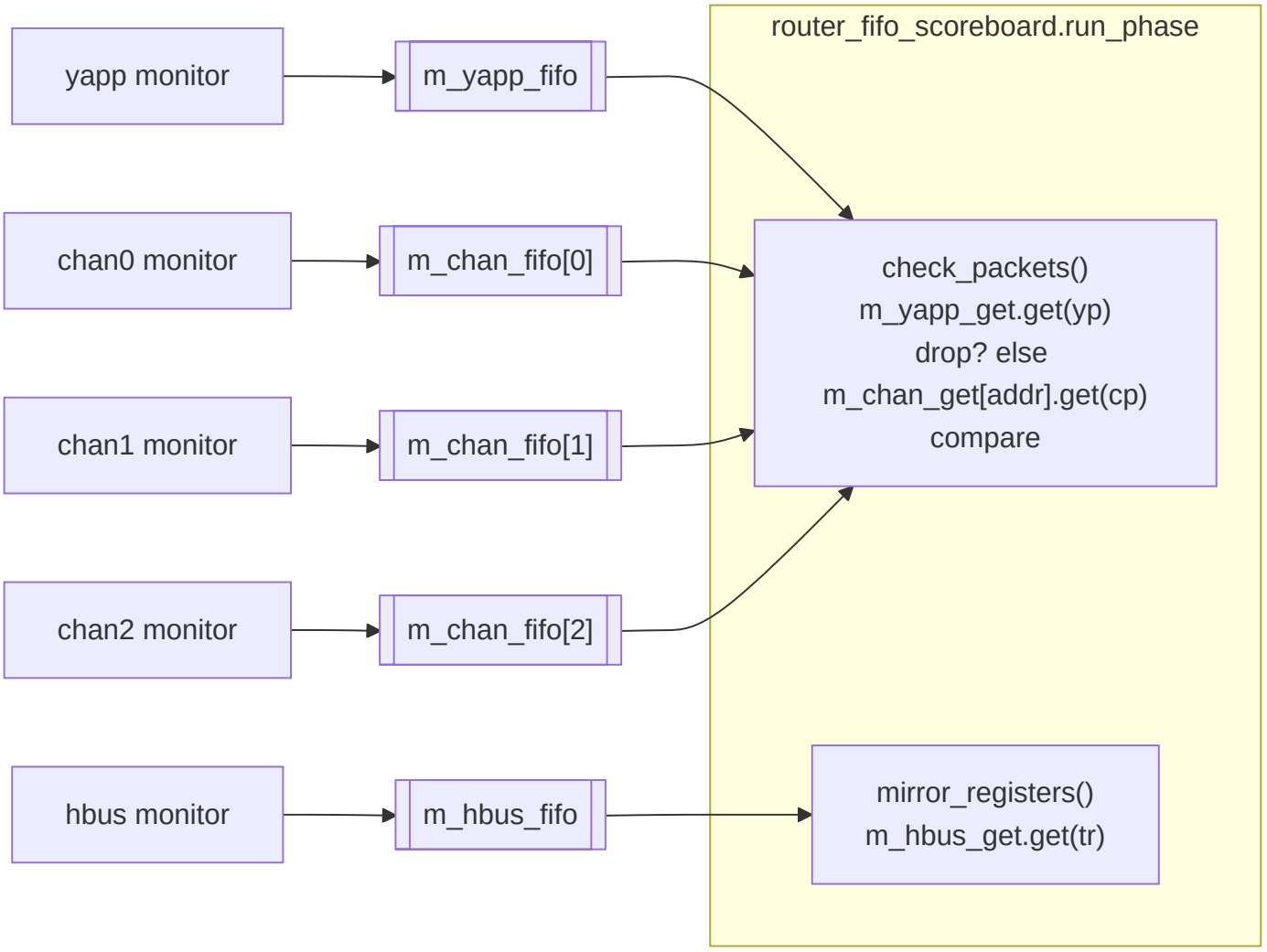
Directory: `labs/lab09_sbd` · **New:** `sv/router_fifo_scoreboard.sv` · **Changed:** `tb/router_tb.sv`

Objective

Build the same checker as a **process** on top of `uvm_tlm_analysis_fifo`s and blocking `get()` calls instead of `write()` callbacks.

Concepts

`uvm_tlm_analysis_fifo #(T)` · `analysis_export` / `get_peek_export` · `uvm_get_port #(T)` · blocking `get()` · `fork` · `check_phase`



Solution

1. `sv/router_fifo_scoreboard.sv`

```
//------------------------------------------------------------------------------
// router_fifo_scoreboard.sv -- scoreboard + reference model built on TLM
// analysis FIFOs and blocking get() calls (Lab 9D)
//
// Instead of reacting inside write() callbacks, the checking runs as a
// PROCESS in run_phase:
//
//   yapp_export ──► yapp_fifo ──get──► [drop?] ──► chan_fifo[addr] ──get──► compare
//   hbus_export ──► hbus_fifo ──get──► mirror maxpktsize / router_en   (parallel thread)
//
// Analysis FIFOs do NOT clone: every monitor must create a new object per
// transaction (ours do), otherwise the FIFO contents get overwritten.
//------------------------------------------------------------------------------
class router_fifo_scoreboard extends uvm_scoreboard;

  // External connection points
  uvm_analysis_export #(yapp_packet)      m_yapp_export;
  uvm_analysis_export #(hbus_transaction) m_hbus_export;
  uvm_analysis_export #(channel_packet)   m_chan_export[3];

  // Storage
  uvm_tlm_analysis_fifo #(yapp_packet)      m_yapp_fifo;
  uvm_tlm_analysis_fifo #(hbus_transaction) m_hbus_fifo;
  uvm_tlm_analysis_fifo #(channel_packet)   m_chan_fifo[3];

  // Blocking readers
  uvm_get_port #(yapp_packet)      m_yapp_get;
  uvm_get_port #(hbus_transaction) m_hbus_get;
  uvm_get_port #(channel_packet)   m_chan_get[3];

  // Register mirror (DUT reset values)
  bit[5:0] m_maxpktsize = 6'h3f;
  bit      m_router_en  = 1'b1;

  // Statistics
  int m_packets_in, m_packets_matched, m_packets_mismatched;
  int m_dropped_disabled, m_dropped_size, m_dropped_addr;

  `uvm_component_utils(router_fifo_scoreboard)

  extern function new(string name, uvm_component parent);

  `include "packet_compare.sv"

  extern function void connect_phase(uvm_phase phase);
  extern task run_phase(uvm_phase phase);

  // Parallel thread: keep the register mirror current
  extern task mirror_registers();

  // Main thread: one YAPP packet at a time, in order
  extern task check_packets();

  // Every FIFO must be empty when the test ends
  extern function void check_phase(uvm_phase phase);
  extern function void report_phase(uvm_phase phase);

endclass : router_fifo_scoreboard
```

```
//------------------------------------------------------------------------------
// router_fifo_scoreboard -- method implementations
//------------------------------------------------------------------------------

function router_fifo_scoreboard::new(string name, uvm_component parent);
  super.new(name, parent);
  m_yapp_export = new("yapp_export", this);
  m_hbus_export = new("hbus_export", this);
  m_yapp_fifo   = new("yapp_fifo", this);
  m_hbus_fifo   = new("hbus_fifo", this);
  m_yapp_get    = new("yapp_get", this);
  m_hbus_get    = new("hbus_get", this);
  foreach (m_chan_fifo[i]) begin
    m_chan_export[i] = new($sformatf("chan%0d_export", i), this);
    m_chan_fifo[i]   = new($sformatf("chan%0d_fifo", i), this);
    m_chan_get[i]    = new($sformatf("chan%0d_get", i), this);
  end
endfunction : new

//------------------------------------------------------------------------------
function void router_fifo_scoreboard::connect_phase(uvm_phase phase);
  // exports -> FIFO analysis exports (monitors write into the FIFOs)
  m_yapp_export.connect(m_yapp_fifo.analysis_export);
  m_hbus_export.connect(m_hbus_fifo.analysis_export);
  // get ports -> FIFO get/peek exports (we read out of the FIFOs)
  m_yapp_get.connect(m_yapp_fifo.get_peek_export);
  m_hbus_get.connect(m_hbus_fifo.get_peek_export);
  foreach (m_chan_fifo[i]) begin
    m_chan_export[i].connect(m_chan_fifo[i].analysis_export);
    m_chan_get[i].connect(m_chan_fifo[i].get_peek_export);
  end
endfunction : connect_phase

//------------------------------------------------------------------------------
task router_fifo_scoreboard::run_phase(uvm_phase phase);
  fork
    mirror_registers();
    check_packets();
  join
endtask : run_phase

//------------------------------------------------------------------------------
task router_fifo_scoreboard::mirror_registers();
  hbus_transaction tr;
  forever begin
    m_hbus_get.get(tr);
    if (tr.m_hwr_rd == HBUS_OP_WRITE) begin
      case (tr.m_haddr)
        16'h1000: m_maxpktsize = tr.m_hdata[5:0];
        16'h1001: m_router_en  = tr.m_hdata[0];
        default: ;
      endcase
    end
  end
endtask : mirror_registers

//------------------------------------------------------------------------------
task router_fifo_scoreboard::check_packets();
```

```
  yapp_packet    yp;
  channel_packet cp;
  forever begin
    m_yapp_get.get(yp);                    // blocks until the monitor delivers one
    m_packets_in++;
    if (!m_router_en) begin
      m_dropped_disabled++;
      `uvm_info(get_type_name(), "Packet dropped: router disabled", UVM_LOW)
    end else if (yp.m_addr == 2'd3) begin
      m_dropped_addr++;
      `uvm_info(get_type_name(), "Packet dropped: illegal address 3", UVM_LOW)
    end else if (yp.m_length > m_maxpktsize) begin
      m_dropped_size++;
      `uvm_info(get_type_name(), $sformatf("Packet dropped: length %0d > maxpktsize %0d",
                                           yp.m_length, m_maxpktsize), UVM_LOW)
    end else begin
      m_chan_get[yp.m_addr].get(cp);         // blocks until the channel delivers it
      if (comp_equal(yp, cp)) begin
        m_packets_matched++;
        `uvm_info(get_type_name(), $sformatf("Channel %0d packet matched", yp.m_addr),
UVM_MEDIUM)
      end else begin
        m_packets_mismatched++;
        `uvm_error(get_type_name(), $sformatf("Channel %0d packet
MISMATCH\nexpected:\n%s\nreceived:\n%s",
                                              yp.m_addr, yp.sprint(), cp.sprint()))
      end
    end
  end
endtask : check_packets

//------------------------------------------------------------------------------
function void router_fifo_scoreboard::check_phase(uvm_phase phase);
  if (m_yapp_fifo.used() != 0) begin
    `uvm_error(get_type_name(), $sformatf("%0d YAPP packets still unchecked",
m_yapp_fifo.used()))
  end
  foreach (m_chan_fifo[i]) begin
    if (m_chan_fifo[i].used() != 0) begin
      `uvm_error(get_type_name(), $sformatf("%0d packets left in channel %0d FIFO",
m_chan_fifo[i].used(), i))
    end
  end
endfunction : check_phase

//------------------------------------------------------------------------------
function void router_fifo_scoreboard::report_phase(uvm_phase phase);
  `uvm_info(get_type_name(), $sformatf({"\n--- FIFO scoreboard report ---\n",
    "  packets received  : %0d\n",
    "  packets matched   : %0d\n",
    "  packets mismatched: %0d\n",
    "  dropped (disabled/oversized/addr3): %0d / %0d / %0d"},
    m_packets_in, m_packets_matched, m_packets_mismatched,
    m_dropped_disabled, m_dropped_size, m_dropped_addr), UVM_LOW)
endfunction : report_phase
```

2. `router_tb` : FIFO scoreboard instead of the module env

```
m_fifo_sb = router_fifo_scoreboard::type_id::create("fifo_sb", this);
...
m_yapp.m_agent.m_monitor.m_item_collected_port.connect(m_fifo_sb.m_yapp_export);
m_hbus.m_monitor.m_item_collected_port.connect(m_fifo_sb.m_hbus_export);
m_chan0.m_rx_agent.m_monitor.m_item_collected_port.connect(m_fifo_sb.m_chan_export[0]);
m_chan1.m_rx_agent.m_monitor.m_item_collected_port.connect(m_fifo_sb.m_chan_export[1]);
m_chan2.m_rx_agent.m_monitor.m_item_collected_port.connect(m_fifo_sb.m_chan_export[2]);
```

Run

```
make run TEST=router_simple_mcseq_test
make run TEST=scoreboard_drop_test
```

Expected: `--- FIFO scoreboard report ---` with 12 received / 12 matched (first test) or N matched + the dropped counts (second), every FIFO empty in `check_phase`, `UVM_ERROR : 0`.

See it run

The lines that matter from the real log of every test of this lab (Verilator, seed 1, from the last `make regress`; the paths and the packet tables are left out). Your own log must show the same lines -- `make check LAB=lab09_sbd START=1` looks for them.

base_test

```
@0 reporter [RNTST] Running test base_test...
@0 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 report: 0 packets
collected
@0 uvm_test_top.tb.chan1.rx_agent.monitor [channel_rx_monitor] Channel 1 report: 0 packets
collected
--- FIFO scoreboard report ---
  packets matched   : 0
  packets mismatched: 0
@0 uvm_test_top.tb.hbus.monitor [hbus_monitor] HBUS report: 0 writes, 0 reads
@0 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] YAPP driver report: 0 packets sent
@0 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP monitor report: 0 packets collected,
coverage 0.0%
@0 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 0 of 1 cycle,
0 of 2, 0 longer (gap coverage 0.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (1 more lines of the kinds above)
```

simple_test

```
@0 reporter [RNTST] Running test simple_test...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@0 uvm_test_top.tb.yapp.agent.sequencer@@yapp_012_seq [yapp_012_seq] Executing yapp_012_seq
sequence
@50 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral          2     'h0
  length                      integral          6     'd9
  parity_type                 parity_type_e     32    BAD_PARITY
@225 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral          2     'h0
  length        integral          6     'd9
  parity_type   parity_type_e     32    BAD_PARITY
@230 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral          2     'h1
  length                      integral          6     'd14
  parity_type                 parity_type_e     32    GOOD_PARITY
@355 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 collected packet:
  addr         integral              2     'h0
  length       integral              6     'd9
  parity_type  channel_parity_type_e  32    CHANNEL_BAD_PARITY
@575 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral          2     'h1
  length        integral          6     'd14
... (22 lines left out)
@1140 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 0 of 1
cycle, 0 of 2, 2 longer (gap coverage 25.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (10 more lines of the kinds above)
```

test_uvc_integration

```
@0 reporter [RNTST] Running test test_uvc_integration...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@0 uvm_test_top.tb.hbus.masters[0].sequencer@@hbus_small_packet_seq [hbus_small_packet_seq]
Executing hbus_small_packet_seq (maxpktsize=20, router_en…
@0 uvm_test_top.tb.yapp.agent.sequencer@@yapp_88_packets_seq [yapp_88_packets_seq] Executing
yapp_88_packets_seq sequence
@50 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h0
  length                      integral       6     'd1
  parity_type                 parity_type_e  32    GOOD_PARITY
@75 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1000 data=0x14
@95 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1001 data=0x01
@285 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral       2     'h0
  length        integral       6     'd1
  parity_type   parity_type_e  32    GOOD_PARITY
@290 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h0
  length                      integral       6     'd2
  parity_type                 parity_type_e  32    BAD_PARITY
@415 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 collected packet:
  addr         integral               2     'h0
  length       integral               6     'd1
  parity_type  channel_parity_type_e  32    CHANNEL_GOOD_PARITY
... (33 lines left out)
@24160 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 5 of 1
cycle, 3 of 2, 79 longer (gap coverage 75.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (910 more lines of the kinds above)
```

router_simple_mcseq_test

```
@0 reporter [RNTST] Running test router_simple_mcseq_test...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@0 uvm_test_top.tb.mcseqr@@router_simple_mcseq [router_simple_mcseq] Executing
router_simple_mcseq sequence
@0 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_small_seq
[hbus_small_packet_seq] Executing hbus_small_packet_seq (maxpktsize=…
@75 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1000 data=0x14
@90 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_read_seq
[hbus_read_max_pkt_seq] Executing hbus_read_max_pkt_seq
@95 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1001 data=0x01
@120 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_read_seq
[hbus_read_max_pkt_seq] MAXPKTSIZE register reads 20
@120 uvm_test_top.tb.yapp.agent.sequencer@@router_simple_mcseq.yapp_012 [yapp_012_seq] Executing
yapp_012_seq sequence
@120 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral          2     'h0
  length                      integral          6     'd1
  parity_type                 parity_type_e     32    GOOD_PARITY
@125 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected READ  addr=0x1000 data=0x14
@335 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral          2     'h0
  length        integral          6     'd1
  parity_type   parity_type_e     32    GOOD_PARITY
@340 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral          2     'h1
  length                      integral          6     'd14
  parity_type                 parity_type_e     32    GOOD_PARITY
... (42 lines left out)
@3140 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 2 of 1
cycle, 0 of 2, 9 longer (gap coverage 50.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (109 more lines of the kinds above)
```

scoreboard_drop_test

```
@0 reporter [RNTST] Running test scoreboard_drop_test...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@0 uvm_test_top.tb.mcseqr@@router_simple_mcseq [router_simple_mcseq] Executing
router_simple_mcseq sequence
@0 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_small_seq
[hbus_small_packet_seq] Executing hbus_small_packet_seq (maxpktsize=…
@75 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1000 data=0x14
@90 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_read_seq
[hbus_read_max_pkt_seq] Executing hbus_read_max_pkt_seq
@95 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1001 data=0x01
@120 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_read_seq
[hbus_read_max_pkt_seq] MAXPKTSIZE register reads 20
@120 uvm_test_top.tb.yapp.agent.sequencer@@router_simple_mcseq.yapp_012 [yapp_012_seq] Executing
yapp_012_seq sequence
@120 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h0
  length                      integral       6     43
  parity_type                 parity_type_e  32    GOOD_PARITY
@125 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected READ  addr=0x1000 data=0x14
@635 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral       2     'h0
  length        integral       6     43
  parity_type   parity_type_e  32    GOOD_PARITY
@640 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h1
  length                      integral       6     'd17
  parity_type                 parity_type_e  32    GOOD_PARITY
... (47 lines left out)
@5730 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 0 of 1
cycle, 0 of 2, 11 longer (gap coverage 25.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (92 more lines of the kinds above)
```

Try this

Small changes to make in `start/` once the lab works, each with a question; the answer unfolds.

Send one packet to address 3. Which analysis FIFO stays empty, and does the test end on its own?

Checkpoint questions

Why must every monitor create a new object per transaction here?

What can the process style do that the callback style cannot?

Why is the HBUS handled in a parallel thread?

What does `check_phase` add that `report_phase` does not?

What changed since the previous lab

```
diff -r labs/lab09_sbc labs/lab09_sbd
```

Lab 10 — A simple functional coverage model *(optional)*

Open on the project map →

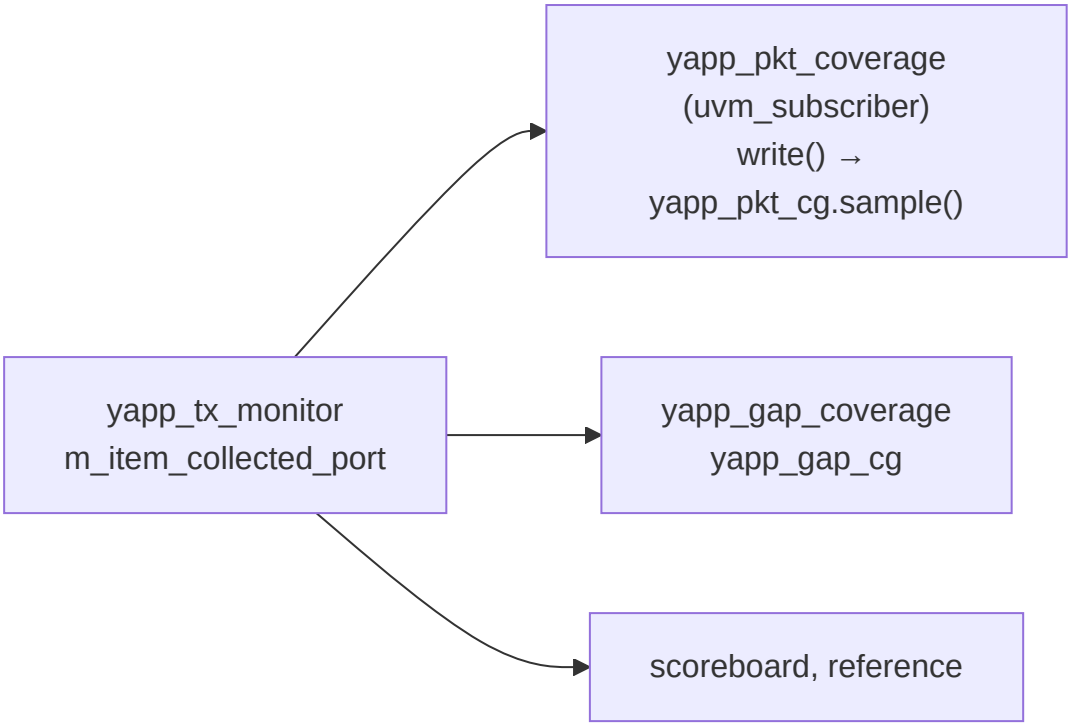
Directory: `labs/lab10_cov` · **Changed:** `yapp_project/uvc/yapp/yapp_pkt_coverage.sv` (covergroup, a subscriber of the monitor), `yapp_project/uvc/yapp/yapp_tx_agent.sv` (builds and connects it), `yapp_project/uvc/yapp/yapp_tx_seqs.sv` (`yapp_coverage_seq`), `tb/router_test_lib.sv` , `run.f` (`-coverage U`)

Objective

Learn where coverage lives in a UVM environment (the course puts it in the monitor; this repository next to it, in a subscriber of the monitor), write a covergroup for three requirements, find the holes and close them with better stimulus.

Concepts

- A **covergroup** is a set of **coverpoints**; every coverpoint sorts the sampled value into **bins**. A **cross** pairs the bins of two coverpoints. `sample()` is called by the testbench, here once per collected packet.
- The monitor is the right source: it reports what the DUT really received, whatever the sequence intended. A `uvm_subscriber` with an `analysis_export` hangs on the monitor's `m_item_collected_port` and samples in `write()` .
- **Holes** are bins no run hit; **closing** them means writing stimulus that reaches them (`yapp_coverage_seq`) — the regression merges every run.
- The general picture (what coverage answers, the vocabulary, the coverage-driven loop) is on Functional coverage.



Requirements

Req	Statement
REQ1	all packet lengths were sent: bins for min, small, medium, large, max
REQ2	every address received a packet — **including the illegal address 3**
REQ3	every length bin was sent to every legal address **with a parity error**

Solution

1. The covergroup in `yapp_pkt_coverage`

The Cadence lab writes the covergroup **inside the monitor**. This repository puts it in its own file: `yapp_pkt_coverage`, a `uvm_subscriber

(yapp_packet)` that the agent connects to the monitor's

`m_item_collected_port`. The bins are the same, so the percentage is the same; the monitor only observes and publishes, and the coverage model can be replaced or left out without touching it (see Functional coverage).

```
covergroup yapp_pkt_cg with function sample(bit[5:0] length,
                                            bit[1:0] addr,
                                            parity_type_e parity_type);
  option.per_instance = 1;
  length_cp : coverpoint length {
    bins MIN    = {1};
    bins SMALL  = {[2:10]};
    bins MEDIUM = {[11:40]};
    bins LARGE  = {[41:62]};
    bins MAX    = {63};
  }
  addr_cp : coverpoint addr {
    bins legal[]      = {[0:2]};
    bins illegal_addr = {3};
  }
  parity_cp : coverpoint parity_type {
    bins good = {PARITY_TYPE_GOOD};
    bins bad  = {PARITY_TYPE_BAD};
  }
  len_x_addr_x_parity : cross length_cp, addr_cp, parity_cp {
    ignore_bins good_parity  = binsof(parity_cp) intersect {PARITY_TYPE_GOOD};
    ignore_bins illegal_addr = binsof(addr_cp.illegal_addr);
  }
endgroup

function yapp_pkt_coverage::new(string name, uvm_component parent);
  super.new(name, parent);
  yapp_pkt_cg = new();              // a class covergroup is created with new()
endfunction

function void yapp_pkt_coverage::write(yapp_packet t);   // every packet the monitor publishes
  yapp_pkt_cg.sample(t.m_length, t.m_addr, t.m_parity_type);
endfunction

// yapp_tx_agent::connect_phase
m_monitor.m_item_collected_port.connect(m_pkt_cov.analysis_export);
```

2. Stimulus that closes the model — `yapp_coverage_seq`

Every length bucket × every address (0..3) × good/bad parity = 40 packets. The packet's `c_addr_legal` constraint is switched off to reach address 3.

```
task yapp_coverage_seq::body();
  int lengths[5] = '{1, 5, 20, 50, 63};   // one value inside each bin
  for (int a = 0; a < 4; a++)
    foreach (lengths[i])
      for (int bad = 0; bad < 2; bad++) begin
        req = yapp_packet::type_id::create("req");
        req.c_addr_legal.constraint_mode(0);        // before randomize(): allow addr 3
        if (!req.randomize() with { req.m_addr == a; req.m_length == lengths[i];
                                    req.m_parity_type == (bad ? PARITY_TYPE_BAD :
PARITY_TYPE_GOOD); })
          `uvm_error(get_type_name(), "Randomization failed")
        start_item(req);
        finish_item(req);
      end
endtask : body
```

The constraint is switched off on the object *before* `randomize()`, which is why the sequence randomizes first and only then does `start_item` / `finish_item`.

3. `coverage_test`

Full-size packets (no override), `yapp_coverage_seq` on the YAPP sequencer, channels and clock as usual. `maxpktsize` keeps its reset value 63, so only the address-3 packets are dropped (the reference model expects that).

4. `run.f`

```
-coverage U        // collect functional coverage
-covoverwrite
```

Run

```
make run TEST=router_simple_mcseq_test   # see the holes
make run TEST=coverage_test              # close them
make gui TEST=coverage_test              # Windows -> Tools -> Coverage
imc -load cov_work/scope/coverage_test   # or IMC standalone
```

With Verilator every run writes its bins to a coverage database; `make regress` then `make sim-report` merges them into `coverage.md` and an HTML page (see Functional coverage).

In IMC: *Verification Hierarchy* → *uvm_pkg* → … *yapp_pkt_coverage* → right-click → *Cover Group Analysis* → *Items* of `yapp_pkt_cg`.

Expected: with `router_simple_mcseq_test` (short packets) `LARGE`, `MAX` and `illegal_addr` are empty and most of the cross is uncovered; the collector's report line (`YAPP packet coverage: ... coverage N%`) shows the percentage. With `coverage_test` all 5 length bins, all 4 address bins, both parity values and all 15 cross bins are hit: 100%.

See it run

The lines that matter from the real log of every test of this lab (Verilator, seed 1, from the last `make regress`; the paths and the packet tables are left out). Your own log must show the same lines -- `make check LAB=lab10_cov START=1` looks for them.

base_test

```
@0 reporter [RNTST] Running test base_test...
@0 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 report: 0 packets
collected
@0 uvm_test_top.tb.chan1.rx_agent.monitor [channel_rx_monitor] Channel 1 report: 0 packets
collected
@0 uvm_test_top.tb.hbus.monitor [hbus_monitor] HBUS report: 0 writes, 0 reads
--- Reference model report ---
--- Scoreboard report ---
  packets matched   : 0
  packets mismatched: 0
@0 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] YAPP driver report: 0 packets sent
@0 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP monitor report: 0 packets collected,
coverage 0.0%
@0 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 0 of 1 cycle,
0 of 2, 0 longer (gap coverage 0.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (1 more lines of the kinds above)
```

router_simple_mcseq_test

```
@0 reporter [RNTST] Running test router_simple_mcseq_test...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@0 uvm_test_top.tb.mcseqr@@router_simple_mcseq [router_simple_mcseq] Executing
router_simple_mcseq sequence
@0 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_small_seq
[hbus_small_packet_seq] Executing hbus_small_packet_seq (maxpktsize=…
@75 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1000 data=0x14
@75 uvm_test_top.tb.router_module.reference [router_reference] maxpktsize is now 20
@90 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_read_seq
[hbus_read_max_pkt_seq] Executing hbus_read_max_pkt_seq
@95 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1001 data=0x01
@95 uvm_test_top.tb.router_module.reference [router_reference] router_en is now 1
@120 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_read_seq
[hbus_read_max_pkt_seq] MAXPKTSIZE register reads 20
@120 uvm_test_top.tb.yapp.agent.sequencer@@router_simple_mcseq.yapp_012 [yapp_012_seq] Executing
yapp_012_seq sequence
@120 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral           2     'h0
  length                      integral           6     'd1
  parity_type                 parity_type_e      32    GOOD_PARITY
@125 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected READ  addr=0x1000 data=0x14
@335 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral           2     'h0
  length        integral           6     'd1
  parity_type   parity_type_e      32    GOOD_PARITY
@340 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral           2     'h1
... (47 lines left out)
@3140 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 2 of 1
cycle, 0 of 2, 9 longer (gap coverage 50.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (109 more lines of the kinds above)
```

test_uvc_integration

```
@0 reporter [RNTST] Running test test_uvc_integration...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@0 uvm_test_top.tb.hbus.masters[0].sequencer@@hbus_small_packet_seq [hbus_small_packet_seq]
Executing hbus_small_packet_seq (maxpktsize=20, router_en…
@0 uvm_test_top.tb.yapp.agent.sequencer@@yapp_88_packets_seq [yapp_88_packets_seq] Executing
yapp_88_packets_seq sequence
@50 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h0
  length                      integral       6     'd1
  parity_type                 parity_type_e  32    GOOD_PARITY
@75 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1000 data=0x14
@75 uvm_test_top.tb.router_module.reference [router_reference] maxpktsize is now 20
@95 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1001 data=0x01
@95 uvm_test_top.tb.router_module.reference [router_reference] router_en is now 1
@285 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral       2     'h0
  length        integral       6     'd1
  parity_type   parity_type_e  32    GOOD_PARITY
@290 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h0
  length                      integral       6     'd2
  parity_type                 parity_type_e  32    BAD_PARITY
@415 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 collected packet:
  addr         integral               2     'h0
... (36 lines left out)
@24160 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 5 of 1
cycle, 3 of 2, 79 longer (gap coverage 75.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (910 more lines of the kinds above)
```

coverage_test

```
@0 reporter [RNTST] Running test coverage_test...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@0 uvm_test_top.tb.yapp.agent.sequencer@@yapp_coverage_seq [yapp_coverage_seq] Executing
yapp_coverage_seq sequence
@50 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h0
  length                      integral       6     'd1
  parity_type                 parity_type_e  32    GOOD_PARITY
@275 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral       2     'h0
  length        integral       6     'd1
  parity_type   parity_type_e  32    GOOD_PARITY
@280 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h0
  length                      integral       6     'd1
  parity_type                 parity_type_e  32    BAD_PARITY
@405 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 collected packet:
  addr         integral               2     'h0
  length       integral               6     'd1
  parity_type  channel_parity_type_e  32    CHANNEL_GOOD_PARITY
@455 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral       2     'h0
  length        integral       6     'd1
... (39 lines left out)
@17710 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 3 of 1
cycle, 0 of 2, 36 longer (gap coverage 50.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (398 more lines of the kinds above)
```

Try this

Small changes to make in `start/` once the lab works, each with a question; the answer unfolds.

Add a bin for the maximum length (`bins max = {63};`) to the length coverpoint. What does `coverage_test` report now, and which sequence would close the hole?

Checkpoint questions

Why is the covergroup fed by the monitor and not by the sequence or the driver?

Why `bins illegal_addr = {3}` and not `illegal_bins`?

What do the `ignore_bins` in the cross achieve?

Why `option.per_instance = 1`?

What changed since the previous lab

```
diff labs/lab06_vif/sv/yapp_tx_monitor.sv yapp_project/uvc/yapp/yapp_tx_monitor.sv   # analysis
port, gap in packet_delay
cat yapp_project/uvc/yapp/yapp_pkt_coverage.sv                                       # the
covergroup
diff -r labs/lab09_sbc/tb labs/lab10_cov/tb
```

Lab 11A — Register model: generation

Open on the project map →

Directory: `labs/lab11a_rm_gen` · **Files:** `yapp_router_regs.xml`, `yapp_router_reg_pkg.sv`, `quicktest.sv`, `run.f`, `Makefile`

Objective

Get a UVM register model of the router and inspect it: hierarchy, reset values, access policies, address map.

Concepts

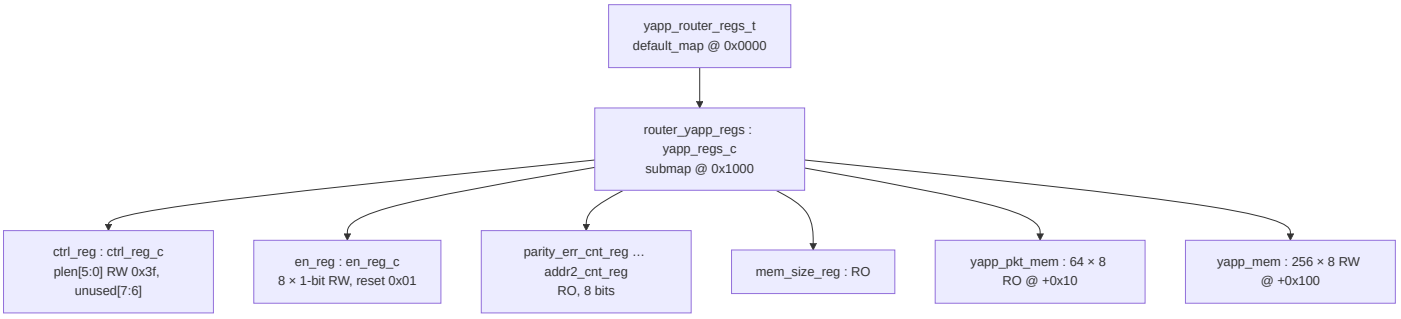
- A **register model** mirrors the DUT's registers as objects: `uvm_reg_block` → `uvm_reg` → `uvm_reg_field`, and `uvm_mem` for memories, placed in a `uvm_reg_map` with their offsets and access policies (RW, RO).
- The model is **generated** from a description of the register map (IP-XACT here), so the testbench, the documentation and the RTL share one source.

About the generation step

The course generates the model from an IP-XACT description with Cadence's `reg_verifier`:

```
reg_verifier -domain uvmreg -top yapp_router_regs.xml -dut yapp_router_regs \
             -out_file yapp_router_regs -quicktest -cov -pkg yapp_router_reg_pkg
```

That tool is not available here, so `yapp_router_reg_pkg.sv` was **written by hand** with the same structure and class names the manual refers to. The IP-XACT file is kept as the single human-readable source of the register map.



Solution

1. The IP-XACT file — `yapp_router_regs.xml`

yapp_router_regs.xml

2. The model — `yapp_router_reg_pkg.sv`

```
//------------------------------------------------------------------------------
// yapp_router_reg_pkg.sv -- UVM register model (RAL) of the YAPP router
//
// In the Cadence flow this file is GENERATED from yapp_router_regs.xml by
// reg_verifier (as yapp_router_regs_rdb.sv). It is written by hand here so the
// course does not depend on that tool; the class names match what the lab
// manual expects:
//
//   yapp_router_regs_t              top block, owns default_map (HBUS, base 0x1000)
//   └── router_yapp_regs            yapp_regs_c
//       ├── ctrl_reg                ctrl_reg_c      0x1000  field plen[5:0] (a.k.a. maxpktsize)
//       ├── en_reg                  en_reg_c        0x1001  8 enable bits
//       ├── parity_err_cnt_reg      ..._c           0x1004  RO
//       ├── oversized_pkt_cnt_reg   ..._c           0x1005  RO
//       ├── addr3_cnt_reg           ..._c           0x1006  RO
//       ├── addr0_cnt_reg           ..._c           0x1009  RO
//       ├── addr1_cnt_reg           ..._c           0x100a  RO
//       ├── addr2_cnt_reg           ..._c           0x100b  RO
//       ├── mem_size_reg            ..._c           0x100d  RO (DUT uses bits [5:0])
//       ├── yapp_pkt_mem            yapp_pkt_mem_c  0x1010  64 x 8  RO
//       └── yapp_mem                yapp_mem_c      0x1100  256 x 8 RW
//
// Backdoor: every register / memory is configured with the HDL name of the
// matching RTL variable, so after
//     yapp_rm.set_hdl_path_root("hw_top.dut.u_regs");
// peek()/poke() resolve to hw_top.dut.u_regs.ctrl_reg, hw_top.dut.u_regs.yapp_mem[i], ...
//------------------------------------------------------------------------------
package yapp_router_reg_pkg;

  import uvm_pkg::*;
  `include "uvm_macros.svh"
  `include "reg/ctrl_reg_c.sv"
  `include "reg/en_reg_c.sv"
  `include "reg/ro_byte_reg_c.sv"
  `include "reg/parity_err_cnt_reg_c.sv"
  `include "reg/oversized_pkt_cnt_reg_c.sv"
  `include "reg/addr3_cnt_reg_c.sv"
  `include "reg/addr0_cnt_reg_c.sv"
  `include "reg/addr1_cnt_reg_c.sv"
  `include "reg/addr2_cnt_reg_c.sv"
  `include "reg/mem_size_reg_c.sv"
  `include "reg/yapp_pkt_mem_c.sv"
  `include "reg/yapp_mem_c.sv"
  `include "reg/yapp_regs_c.sv"
  `include "reg/yapp_router_regs_t.sv"

endpackage : yapp_router_reg_pkg

//------------------------------------------------------------------------------
// ctrl_reg_c.sv -- ctrl_reg_c (register model)
// Split out of yapp_router_reg_pkg.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// ctrl_reg : maximum packet length
//------------------------------------------------------------------------------
class ctrl_reg_c extends uvm_reg;
```

```
  rand uvm_reg_field plen;     // [5:0]  maximum payload length ("maxpktsize")
  rand uvm_reg_field unused;   // [7:6]

  `uvm_object_utils(ctrl_reg_c)

  extern function new(string name = "ctrl_reg_c");
  extern virtual function void build();

endclass : ctrl_reg_c

//------------------------------------------------------------------------------
// ctrl_reg_c -- method implementations
//------------------------------------------------------------------------------

function ctrl_reg_c::new(string name = "ctrl_reg_c");
  super.new(name, 8, UVM_NO_COVERAGE);
endfunction : new

//------------------------------------------------------------------------------
function void ctrl_reg_c::build();
  plen   = uvm_reg_field::type_id::create("plen");
  unused = uvm_reg_field::type_id::create("unused");
  //               parent size lsb access volatile reset  has_reset is_rand
individually_accessible
  plen.configure  (this,  6,   0,  "RW",  0,       6'h3f, 1,        1,      0);
  unused.configure(this,  2,   6,  "RW",  0,       2'h0,  1,        1,      0);
endfunction : build

//------------------------------------------------------------------------------
// en_reg_c.sv -- en_reg_c (register model)
// Split out of yapp_router_reg_pkg.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------
// en_reg : router enable and counter enables
//------------------------------------------------------------------------
class en_reg_c extends uvm_reg;

  rand uvm_reg_field router_en;             // [0]
  rand uvm_reg_field parity_err_cnt_en;     // [1]
  rand uvm_reg_field oversized_pkt_cnt_en;  // [2]
  rand uvm_reg_field reserved;              // [3]
  rand uvm_reg_field addr0_cnt_en;          // [4]
  rand uvm_reg_field addr1_cnt_en;          // [5]
  rand uvm_reg_field addr2_cnt_en;          // [6]
  rand uvm_reg_field addr3_cnt_en;          // [7]

  `uvm_object_utils(en_reg_c)

  extern function new(string name = "en_reg_c");
  extern virtual function void build();

endclass : en_reg_c

//------------------------------------------------------------------------------
// en_reg_c -- method implementations
//------------------------------------------------------------------------------

function en_reg_c::new(string name = "en_reg_c");
```

```
  super.new(name, 8, UVM_NO_COVERAGE);
endfunction : new

//------------------------------------------------------------------------------
function void en_reg_c::build();
  router_en            = uvm_reg_field::type_id::create("router_en");
  parity_err_cnt_en    = uvm_reg_field::type_id::create("parity_err_cnt_en");
  oversized_pkt_cnt_en = uvm_reg_field::type_id::create("oversized_pkt_cnt_en");
  reserved             = uvm_reg_field::type_id::create("reserved");
  addr0_cnt_en         = uvm_reg_field::type_id::create("addr0_cnt_en");
  addr1_cnt_en         = uvm_reg_field::type_id::create("addr1_cnt_en");
  addr2_cnt_en         = uvm_reg_field::type_id::create("addr2_cnt_en");
  addr3_cnt_en         = uvm_reg_field::type_id::create("addr3_cnt_en");
  router_en.configure           (this, 1, 0, "RW", 0, 1'b1, 1, 1, 0);
  parity_err_cnt_en.configure   (this, 1, 1, "RW", 0, 1'b0, 1, 1, 0);
  oversized_pkt_cnt_en.configure(this, 1, 2, "RW", 0, 1'b0, 1, 1, 0);
  reserved.configure            (this, 1, 3, "RW", 0, 1'b0, 1, 1, 0);
  addr0_cnt_en.configure        (this, 1, 4, "RW", 0, 1'b0, 1, 1, 0);
  addr1_cnt_en.configure        (this, 1, 5, "RW", 0, 1'b0, 1, 1, 0);
  addr2_cnt_en.configure        (this, 1, 6, "RW", 0, 1'b0, 1, 1, 0);
  addr3_cnt_en.configure        (this, 1, 7, "RW", 0, 1'b0, 1, 1, 0);
endfunction : build

//------------------------------------------------------------------------------
// ro_byte_reg_c.sv -- ro_byte_reg_c (register model)
// Split out of yapp_router_reg_pkg.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------
// Read-only 8-bit counter registers: one small class each, sharing a base
//------------------------------------------------------------------------
virtual class ro_byte_reg_c extends uvm_reg;

  rand uvm_reg_field value;   // [7:0]

  extern function new(string name);
  extern virtual function void build();

endclass : ro_byte_reg_c

//------------------------------------------------------------------------------
// ro_byte_reg_c -- method implementations
//------------------------------------------------------------------------------

function ro_byte_reg_c::new(string name);
  super.new(name, 8, UVM_NO_COVERAGE);
endfunction : new

//------------------------------------------------------------------------------
function void ro_byte_reg_c::build();
  value = uvm_reg_field::type_id::create("value");
  value.configure(this, 8, 0, "RO", 0, 8'h00, 1, 0, 0);
endfunction : build

//------------------------------------------------------------------------------
// parity_err_cnt_reg_c.sv -- parity_err_cnt_reg_c (register model)
// Split out of yapp_router_reg_pkg.sv: one class per file.
//------------------------------------------------------------------------------
```

```
class parity_err_cnt_reg_c extends ro_byte_reg_c;
  `uvm_object_utils(parity_err_cnt_reg_c)
  extern function new(string name = "parity_err_cnt_reg_c");
endclass : parity_err_cnt_reg_c

//------------------------------------------------------------------------------
// parity_err_cnt_reg_c -- method implementations
//------------------------------------------------------------------------------

function parity_err_cnt_reg_c::new(string name = "parity_err_cnt_reg_c"); super.new(name);
endfunction : new

//------------------------------------------------------------------------------
// oversized_pkt_cnt_reg_c.sv -- oversized_pkt_cnt_reg_c (register model)
// Split out of yapp_router_reg_pkg.sv: one class per file.
//------------------------------------------------------------------------------

class oversized_pkt_cnt_reg_c extends ro_byte_reg_c;
  `uvm_object_utils(oversized_pkt_cnt_reg_c)
  extern function new(string name = "oversized_pkt_cnt_reg_c");
endclass : oversized_pkt_cnt_reg_c

//------------------------------------------------------------------------------
// oversized_pkt_cnt_reg_c -- method implementations
//------------------------------------------------------------------------------

function oversized_pkt_cnt_reg_c::new(string name = "oversized_pkt_cnt_reg_c"); super.new(name);
endfunction : new

//------------------------------------------------------------------------------
// addr3_cnt_reg_c.sv -- addr3_cnt_reg_c (register model)
// Split out of yapp_router_reg_pkg.sv: one class per file.
//------------------------------------------------------------------------------

class addr3_cnt_reg_c extends ro_byte_reg_c;
  `uvm_object_utils(addr3_cnt_reg_c)
  extern function new(string name = "addr3_cnt_reg_c");
endclass : addr3_cnt_reg_c

//------------------------------------------------------------------------------
// addr3_cnt_reg_c -- method implementations
//------------------------------------------------------------------------------

function addr3_cnt_reg_c::new(string name = "addr3_cnt_reg_c"); super.new(name);
endfunction : new

//------------------------------------------------------------------------------
// addr0_cnt_reg_c.sv -- addr0_cnt_reg_c (register model)
// Split out of yapp_router_reg_pkg.sv: one class per file.
//------------------------------------------------------------------------------

class addr0_cnt_reg_c extends ro_byte_reg_c;
  `uvm_object_utils(addr0_cnt_reg_c)
  extern function new(string name = "addr0_cnt_reg_c");
endclass : addr0_cnt_reg_c

//------------------------------------------------------------------------------
// addr0_cnt_reg_c -- method implementations
//------------------------------------------------------------------------------
```

```
function addr0_cnt_reg_c::new(string name = "addr0_cnt_reg_c"); super.new(name);
endfunction : new

//------------------------------------------------------------------------------
// addr1_cnt_reg_c.sv -- addr1_cnt_reg_c (register model)
// Split out of yapp_router_reg_pkg.sv: one class per file.
//------------------------------------------------------------------------------

class addr1_cnt_reg_c extends ro_byte_reg_c;
  `uvm_object_utils(addr1_cnt_reg_c)
  extern function new(string name = "addr1_cnt_reg_c");
endclass : addr1_cnt_reg_c

//------------------------------------------------------------------------------
// addr1_cnt_reg_c -- method implementations
//------------------------------------------------------------------------------

function addr1_cnt_reg_c::new(string name = "addr1_cnt_reg_c"); super.new(name);
endfunction : new

//------------------------------------------------------------------------------
// addr2_cnt_reg_c.sv -- addr2_cnt_reg_c (register model)
// Split out of yapp_router_reg_pkg.sv: one class per file.
//------------------------------------------------------------------------------

class addr2_cnt_reg_c extends ro_byte_reg_c;
  `uvm_object_utils(addr2_cnt_reg_c)
  extern function new(string name = "addr2_cnt_reg_c");
endclass : addr2_cnt_reg_c

//------------------------------------------------------------------------------
// addr2_cnt_reg_c -- method implementations
//------------------------------------------------------------------------------

function addr2_cnt_reg_c::new(string name = "addr2_cnt_reg_c"); super.new(name);
endfunction : new

//------------------------------------------------------------------------------
// mem_size_reg_c.sv -- mem_size_reg_c (register model)
// Split out of yapp_router_reg_pkg.sv: one class per file.
//------------------------------------------------------------------------------

// mem_size_reg: length of the last packet. The DUT only implements [5:0];
// the model keeps the full byte like the register map does.
class mem_size_reg_c extends ro_byte_reg_c;
  `uvm_object_utils(mem_size_reg_c)
  extern function new(string name = "mem_size_reg_c");
endclass : mem_size_reg_c

//------------------------------------------------------------------------------
// mem_size_reg_c -- method implementations
//------------------------------------------------------------------------------

function mem_size_reg_c::new(string name = "mem_size_reg_c"); super.new(name);
endfunction : new

//------------------------------------------------------------------------------
// yapp_pkt_mem_c.sv -- yapp_pkt_mem_c (register model)
```

```
// Split out of yapp_router_reg_pkg.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------
// Memories
//------------------------------------------------------------------------
class yapp_pkt_mem_c extends uvm_mem;
  `uvm_object_utils(yapp_pkt_mem_c)
  extern function new(string name = "yapp_pkt_mem_c");
endclass : yapp_pkt_mem_c

//------------------------------------------------------------------------------
// yapp_pkt_mem_c -- method implementations
//------------------------------------------------------------------------------

function yapp_pkt_mem_c::new(string name = "yapp_pkt_mem_c");
  super.new(name, 64, 8, "RO", UVM_NO_COVERAGE);   // bytes of the last packet
endfunction : new

//------------------------------------------------------------------------------
// yapp_mem_c.sv -- yapp_mem_c (register model)
// Split out of yapp_router_reg_pkg.sv: one class per file.
//------------------------------------------------------------------------------

class yapp_mem_c extends uvm_mem;
  `uvm_object_utils(yapp_mem_c)
  extern function new(string name = "yapp_mem_c");
endclass : yapp_mem_c

//------------------------------------------------------------------------------
// yapp_mem_c -- method implementations
//------------------------------------------------------------------------------

function yapp_mem_c::new(string name = "yapp_mem_c");
  super.new(name, 256, 8, "RW", UVM_NO_COVERAGE);  // scratch memory
endfunction : new

//------------------------------------------------------------------------------
// yapp_regs_c.sv -- yapp_regs_c (register model)
// Split out of yapp_router_reg_pkg.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------
// yapp_regs_c : the register block (offsets relative to its base)
//------------------------------------------------------------------------
class yapp_regs_c extends uvm_reg_block;

  rand ctrl_reg_c              ctrl_reg;
  rand en_reg_c                en_reg;
  rand parity_err_cnt_reg_c    parity_err_cnt_reg;
  rand oversized_pkt_cnt_reg_c oversized_pkt_cnt_reg;
  rand addr3_cnt_reg_c         addr3_cnt_reg;
  rand addr0_cnt_reg_c         addr0_cnt_reg;
  rand addr1_cnt_reg_c         addr1_cnt_reg;
  rand addr2_cnt_reg_c         addr2_cnt_reg;
  rand mem_size_reg_c          mem_size_reg;
       yapp_pkt_mem_c          yapp_pkt_mem;
       yapp_mem_c              yapp_mem;
```

```
  `uvm_object_utils(yapp_regs_c)

  extern function new(string name = "yapp_regs_c");
  extern virtual function void build();

endclass : yapp_regs_c

//------------------------------------------------------------------------------
// yapp_regs_c -- method implementations
//------------------------------------------------------------------------------

function yapp_regs_c::new(string name = "yapp_regs_c");
  super.new(name, UVM_NO_COVERAGE);
endfunction : new

//------------------------------------------------------------------------------
function void yapp_regs_c::build();
  // one byte per address, little endian
  default_map = create_map("default_map", 'h0, 1, UVM_LITTLE_ENDIAN, 0);

  ctrl_reg              = ctrl_reg_c::type_id::create("ctrl_reg");
  en_reg                = en_reg_c::type_id::create("en_reg");
  parity_err_cnt_reg    = parity_err_cnt_reg_c::type_id::create("parity_err_cnt_reg");
  oversized_pkt_cnt_reg = oversized_pkt_cnt_reg_c::type_id::create("oversized_pkt_cnt_reg");
  addr3_cnt_reg         = addr3_cnt_reg_c::type_id::create("addr3_cnt_reg");
  addr0_cnt_reg         = addr0_cnt_reg_c::type_id::create("addr0_cnt_reg");
  addr1_cnt_reg         = addr1_cnt_reg_c::type_id::create("addr1_cnt_reg");
  addr2_cnt_reg         = addr2_cnt_reg_c::type_id::create("addr2_cnt_reg");
  mem_size_reg          = mem_size_reg_c::type_id::create("mem_size_reg");
  yapp_pkt_mem          = yapp_pkt_mem_c::type_id::create("yapp_pkt_mem");
  yapp_mem              = yapp_mem_c::type_id::create("yapp_mem");

  // configure(parent block, register file, HDL path of the RTL variable)
  ctrl_reg.configure             (this, null, "ctrl_reg");
  en_reg.configure               (this, null, "en_reg");
  parity_err_cnt_reg.configure   (this, null, "parity_err_cnt_reg");
  oversized_pkt_cnt_reg.configure(this, null, "oversized_pkt_cnt_reg");
  addr3_cnt_reg.configure        (this, null, "addr3_cnt_reg");
  addr0_cnt_reg.configure        (this, null, "addr0_cnt_reg");
  addr1_cnt_reg.configure        (this, null, "addr1_cnt_reg");
  addr2_cnt_reg.configure        (this, null, "addr2_cnt_reg");
  mem_size_reg.configure         (this, null, "mem_size_reg");
  yapp_pkt_mem.configure         (this, "yapp_pkt_mem");
  yapp_mem.configure             (this, "yapp_mem");

  ctrl_reg.build();
  en_reg.build();
  parity_err_cnt_reg.build();
  oversized_pkt_cnt_reg.build();
  addr3_cnt_reg.build();
  addr0_cnt_reg.build();
  addr1_cnt_reg.build();
  addr2_cnt_reg.build();
  mem_size_reg.build();

  default_map.add_reg(ctrl_reg,              'h00, "RW");
  default_map.add_reg(en_reg,                'h01, "RW");
  default_map.add_reg(parity_err_cnt_reg,    'h04, "RO");
  default_map.add_reg(oversized_pkt_cnt_reg, 'h05, "RO");
```

```
  default_map.add_reg(addr3_cnt_reg,          'h06, "RO");
  default_map.add_reg(addr0_cnt_reg,          'h09, "RO");
  default_map.add_reg(addr1_cnt_reg,          'h0a, "RO");
  default_map.add_reg(addr2_cnt_reg,          'h0b, "RO");
  default_map.add_reg(mem_size_reg,           'h0d, "RO");
  default_map.add_mem(yapp_pkt_mem,           'h10,  "RO");
  default_map.add_mem(yapp_mem,               'h100, "RW");
endfunction : build

//------------------------------------------------------------------------------
// yapp_router_regs_t.sv -- yapp_router_regs_t (register model)
// Split out of yapp_router_reg_pkg.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// yapp_router_regs_t : top-level block, places the register block at 0x1000
//------------------------------------------------------------------------------
class yapp_router_regs_t extends uvm_reg_block;

  rand yapp_regs_c router_yapp_regs;

  `uvm_object_utils(yapp_router_regs_t)

  extern function new(string name = "yapp_router_regs_t");
  extern virtual function void build();

endclass : yapp_router_regs_t

//------------------------------------------------------------------------------
// yapp_router_regs_t -- method implementations
//------------------------------------------------------------------------------

function yapp_router_regs_t::new(string name = "yapp_router_regs_t");
  super.new(name, UVM_NO_COVERAGE);
endfunction : new

//------------------------------------------------------------------------------
function void yapp_router_regs_t::build();
  default_map = create_map("default_map", 'h0, 1, UVM_LITTLE_ENDIAN, 0);
  router_yapp_regs = yapp_regs_c::type_id::create("router_yapp_regs");
  router_yapp_regs.configure(this, "");      // no extra HDL hierarchy level
  router_yapp_regs.build();
  default_map.add_submap(router_yapp_regs.default_map, 'h1000);
endfunction : build
```

3. The quick test — `quicktest.sv`

```
//------------------------------------------------------------------------------
// quicktest.sv -- Lab 11A: build the register model standalone, reset it and
// print the model and its address map. No DUT, no bus: pure introspection.
//
//   % xrun -f run.f
//
// Use the printout to answer the lab questions: model type, reset value of
// ctrl_reg.plen, size of yapp_pkt_mem, access policy of addr3_cnt_reg,
// address of mem_size_reg, start address of the packet memory.
//------------------------------------------------------------------------------
`timescale 1ns/1ns

module quicktest;

  import uvm_pkg::*;
  `include "uvm_macros.svh"
  import yapp_router_reg_pkg::*;

  class qt_test extends uvm_test;

    yapp_router_regs_t model;

    `uvm_component_utils(qt_test)

    extern function new(string name, uvm_component parent);
    extern function void build_phase(uvm_phase phase);

    extern task run_phase(uvm_phase phase);

  endclass : qt_test

//------------------------------------------------------------------------------
// qt_test -- method implementations
//------------------------------------------------------------------------------

function qt_test::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//------------------------------------------------------------------------------
function void qt_test::build_phase(uvm_phase phase);
  super.build_phase(phase);
  model = yapp_router_regs_t::type_id::create("model");
  model.build();          // creates registers, fields, memories, maps
  model.lock_model();     // no more changes; computes the address map
endfunction : build_phase

//------------------------------------------------------------------------------
task qt_test::run_phase(uvm_phase phase);
  phase.raise_objection(this);
  model.reset();          // mirrored values <- reset values
  model.print();          // hierarchy: block -> registers -> fields
  model.default_map.print();   // addresses as seen from the HBUS
  phase.drop_objection(this);
endtask : run_phase

  initial run_test("qt_test");
```

```
endmodule : quicktest
```

`model.print()` is called **after** `model.reset()` so the printed mirrored values are the reset values; `model.default_map.print()` shows the addresses.

Run

```
cd labs/lab11a_rm_gen && make run_test
```

Expected: a tree `model (yapp_router_regs_t) → router_yapp_regs (yapp_regs_c) → ctrl_reg … yapp_mem`, each register with its fields, access and reset value; then the map with `ctrl_reg @0x1000`, `en_reg @0x1001`, …, `mem_size_reg @0x100d`, `yapp_pkt_mem @0x1010`, `yapp_mem @0x1100`.

Try this

Small changes to make in `start/` once the lab works, each with a question; the answer unfolds.

Add a one-bit field `dbg_en` at bit 3 of `en_reg` in the XML and regenerate. What changes in the package, and what does the DUT do with a write to that bit?

Checkpoint questions

Access policy of `ctrl_reg`? Of `addr0_cnt_reg`?

What is the model's top type?

Reset value of `ctrl_reg.plen`? Size of `yapp_pkt_mem`? Policy of `addr3_cnt_reg`?

Address of `mem_size_reg`? Start of the packet memory?

Why `plen` and not `maxpktsize`?

What changed since the previous lab

This lab is standalone: no DUT, no bus, no testbench.

Lab 11B — Register model: integration

Open on the project map →

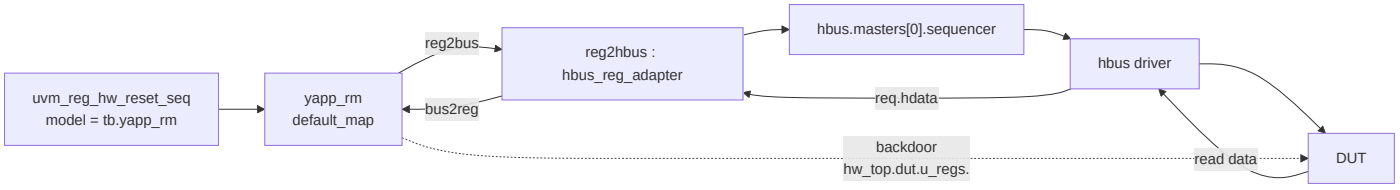
Directory: `labs/lab11b_rm_integ` · **New in `tb/`:** `yapp_router_reg_pkg.sv` (copied from 11A) · **Changed:** `router_tb.sv`, `tb_top.sv`, `router_test_lib.sv`, `run.f` (`-access +rwc`)

Objective

Put the register model into the testbench, connect its front door to the HBUS UVC through an adapter, and run the built-in reset and memory-walk sequences.

Concepts

`uvm_reg_block::build()` / `lock_model()` · `set_hdl_path_root()` · `set_auto_predict()` · `uvm_reg_adapter` · `uvm_reg_map::set_sequencer()` · `uvm_reg_hw_reset_seq` · `uvm_mem_walk_seq`



Solution

1. `router_tb`

```
//------------------------------------------------------------------------------
// router_tb.sv -- the testbench environment (Lab 11B: + register model)
//
//   yapp_rm  : the register model (yapp_router_regs_t)
//   reg2hbus : adapter translating register operations to HBUS transactions
//
// Front-door accesses go  model -> default_map -> reg2hbus -> HBUS sequencer.
// Backdoor accesses use the HDL paths rooted at "hw_top.dut.u_regs" (the register block of the
router).
//------------------------------------------------------------------------------
class router_tb extends uvm_env;

  yapp_env            m_yapp;
  channel_env         m_chan0;
  channel_env         m_chan1;
  channel_env         m_chan2;
  hbus_env            m_hbus;
  clock_and_reset_env m_clk_rst;
  router_mcsequencer  m_mcseqr;
  router_module_env   m_router_module;

  yapp_router_regs_t  m_yapp_rm;
  hbus_reg_adapter    m_reg2hbus;

  `uvm_component_utils(router_tb)

  // The fields are printed by do_print() below:
  // no uvm_field_* automation.
  extern virtual function void do_print(uvm_printer printer);
  extern function new(string name, uvm_component parent);

  extern function void build_phase(uvm_phase phase);
  extern function void connect_phase(uvm_phase phase);

endclass : router_tb

//------------------------------------------------------------------------------
// router_tb -- method implementations
//------------------------------------------------------------------------------

function router_tb::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//------------------------------------------------------------------------------
function void router_tb::build_phase(uvm_phase phase);
  super.build_phase(phase);
  `uvm_info(get_type_name(), "Executing the build phase of the testbench", UVM_HIGH)

  uvm_config_int::set(this, "chan0", "channel_id", 0);
  uvm_config_int::set(this, "chan1", "channel_id", 1);
  uvm_config_int::set(this, "chan2", "channel_id", 2);
  uvm_config_int::set(this, "hbus",  "num_masters", 1);

  m_yapp          = yapp_env::type_id::create("yapp", this);
  m_chan0         = channel_env::type_id::create("chan0", this);
  m_chan1         = channel_env::type_id::create("chan1", this);
  m_chan2         = channel_env::type_id::create("chan2", this);
```

```
  m_hbus          = hbus_env::type_id::create("hbus", this);
  m_clk_rst       = clock_and_reset_env::type_id::create("clk_rst", this);
  m_mcseqr        = router_mcsequencer::type_id::create("mcseqr", this);
  m_router_module = router_module_env::type_id::create("router_module", this);

  // Register model: build, lock, backdoor root, implicit prediction
  m_yapp_rm = yapp_router_regs_t::type_id::create("yapp_rm");
  m_yapp_rm.build();
  m_yapp_rm.lock_model();
  m_yapp_rm.set_hdl_path_root("hw_top.dut.u_regs");
  m_yapp_rm.default_map.set_auto_predict(1);

  m_reg2hbus = hbus_reg_adapter::type_id::create("reg2hbus");
endfunction : build_phase

//------------------------------------------------------------------------------
function void router_tb::connect_phase(uvm_phase phase);
  m_mcseqr.m_hbus_seqr = m_hbus.m_masters[0].m_sequencer;
  m_mcseqr.m_yapp_seqr = m_yapp.m_agent.m_sequencer;

  m_yapp.m_agent.m_monitor.m_item_collected_port.connect(m_router_module.m_yapp_export);
  m_hbus.m_monitor.m_item_collected_port.connect(m_router_module.m_hbus_export);
  m_chan0.m_rx_agent.m_monitor.m_item_collected_port.connect(m_router_module.m_chan0_export);
  m_chan1.m_rx_agent.m_monitor.m_item_collected_port.connect(m_router_module.m_chan1_export);
  m_chan2.m_rx_agent.m_monitor.m_item_collected_port.connect(m_router_module.m_chan2_export);

  // Front door: the model's map drives the HBUS master sequencer via the adapter
  m_yapp_rm.default_map.set_sequencer(m_hbus.m_masters[0].m_sequencer, m_reg2hbus);
endfunction : connect_phase

//------------------------------------------------------------------------------
function void router_tb::do_print(uvm_printer printer);
  super.do_print(printer);
  printer.print_object("yapp_rm", m_yapp_rm);
endfunction : do_print
```

Step by step in `build_phase`:

Call	Effect
`yapp_rm = yapp_router_regs_t::type_id::create("yapp_rm")`	the model is an object, nc parent
`yapp_rm.build()`	creates sub-block, regist fields, memories, maps
`yapp_rm.lock_model()`	no more structural chang addresses are computed

Call	Effect
`yapp_rm.set_hdl_path_root("hw_top.dut.u_regs")`	backdoor paths become `hw_top.dut.u_regs.ctrl_` … — `u_regs` is the `yapp_hbus_regs` instance holds every register and memory of the router
`yapp_rm.default_map.set_auto_predict(1)`	the mirror follows every front-door access
`reg2hbus = hbus_reg_adapter::type_id::create("reg2hbus")`	the translator

and in `connect_phase` : `yapp_rm.default_map.set_sequencer(hbus.masters[0].sequencer, reg2hbus)` .

2. The adapter (already in the HBUS UVC)

```
//------------------------------------------------------------------------------
// hbus_reg_adapter.sv -- translates register-model operations into HBUS
// transactions and back (Lab 11B)
//
//   reg2bus : uvm_reg_bus_op  -> hbus_transaction  (front-door access)
//   bus2reg : hbus_transaction -> uvm_reg_bus_op   (prediction / read data)
//
// provides_responses = 0 : the driver writes read data straight into the
// request item, so the register map reads it back from the same object.
//------------------------------------------------------------------------------
class hbus_reg_adapter extends uvm_reg_adapter;

  `uvm_object_utils(hbus_reg_adapter)

  extern function new(string name = "hbus_reg_adapter");
  extern virtual function uvm_sequence_item reg2bus(const ref uvm_reg_bus_op rw);

  extern virtual function void bus2reg(uvm_sequence_item bus_item, ref uvm_reg_bus_op rw);

endclass : hbus_reg_adapter

//------------------------------------------------------------------------------
// hbus_reg_adapter -- method implementations
//------------------------------------------------------------------------------

function hbus_reg_adapter::new(string name = "hbus_reg_adapter");
  super.new(name);
  supports_byte_enable = 0;
  provides_responses   = 0;
endfunction : new

//------------------------------------------------------------------------------
function uvm_sequence_item hbus_reg_adapter::reg2bus(const ref uvm_reg_bus_op rw);
  hbus_transaction tr = hbus_transaction::type_id::create("tr");
  tr.m_haddr  = rw.addr[15:0];
  tr.m_hdata  = rw.data[7:0];
  tr.m_hwr_rd = (rw.kind == UVM_WRITE) ? HBUS_OP_WRITE : HBUS_OP_READ;
  return tr;
endfunction : reg2bus

//------------------------------------------------------------------------------
function void hbus_reg_adapter::bus2reg(uvm_sequence_item bus_item, ref uvm_reg_bus_op rw);
  hbus_transaction tr;
  if (!$cast(tr, bus_item)) begin
    `uvm_fatal("NOT_HBUS_TYPE", "Provided bus_item is not of type hbus_transaction")
    return;
  end
  rw.kind   = (tr.m_hwr_rd == HBUS_OP_WRITE) ? UVM_WRITE : UVM_READ;
  rw.addr   = tr.m_haddr;
  rw.data   = tr.m_hdata;
  rw.status = UVM_IS_OK;
endfunction : bus2reg
```

3. `tb_top` : import the package before `router_tb`

```
import router_module_pkg::*;
import yapp_router_reg_pkg::*;     // before router_tb, which uses yapp_router_regs_t
```

4. The tests

```
class uvm_reset_test extends base_test;
  ...
  task run_phase(uvm_phase phase);
    uvm_reg_hw_reset_seq reset_seq;
    super.run_phase(phase);
    phase.raise_objection(this);
    reset_seq = uvm_reg_hw_reset_seq::type_id::create("reset_seq");
    reset_seq.model = m_tb.m_yapp_rm;
    reset_seq.start(null);          // the map knows the sequencer
    phase.drop_objection(this);
  endtask
endclass
```

`uvm_mem_walk_test` is the same with `uvm_mem_walk_seq`.

5. `run.f`

`yapp_router_reg_pkg.sv` is compiled from `tb/`; `-access +rwc` is required for backdoor access.

Run

```
make run TEST=uvm_reset_test
make run TEST=uvm_mem_walk_test
make run TEST=uvm_mem_walk_test XRUN_OPTS="-define INJECT_ERROR"
```

With Verilator: `make sim TEST=uvm_mem_walk_test DEFINES=INJECT_ERROR` (a separate build in `build/sim/<dir>__INJECT_ERROR/`). `make regress` runs that build too and counts it as passed only when the walk reports the error.

Expected:

- `uvm_reset_test`: the topology shows `yapp_rm` under `tb` (`router_tb::do_print` calls `printer.print_object("yapp_rm", yapp_rm)`); the HBUS monitor logs one `READ` per register (`0x1000`, `0x1001`, `0x1004` … `0x100d`); `UVM_ERROR : 0`.
- `uvm_mem_walk_test`: HBUS traffic over `0x1100..0x11ff` only (`yapp_pkt_mem` is RO and skipped); the HBUS report says **511 writes, 255 reads**.
- with `INJECT_ERROR`: `UVM_ERROR … [uvm_mem_walk_seq] Memory … read back as … instead of …` at address `0x112a`.

See it run

The lines that matter from the real log of every test of this lab (Verilator, seed 1, from the last `make regress`; the paths and the packet tables are left out). Your own log must show the same lines -- `make check LAB=lab11b_rm_integ START=1` looks for them.

base_test

```
@0 reporter [RNTST] Running test base_test...
@0 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 report: 0 packets
collected
@0 uvm_test_top.tb.chan1.rx_agent.monitor [channel_rx_monitor] Channel 1 report: 0 packets
collected
@0 uvm_test_top.tb.hbus.monitor [hbus_monitor] HBUS report: 0 writes, 0 reads
--- Reference model report ---
--- Scoreboard report ---
  packets matched   : 0
  packets mismatched: 0
@0 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] YAPP driver report: 0 packets sent
@0 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP monitor report: 0 packets collected,
coverage 0.0%
@0 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 0 of 1 cycle,
0 of 2, 0 longer (gap coverage 0.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (1 more lines of the kinds above)
```

router_simple_mcseq_test

```
@0 reporter [RNTST] Running test router_simple_mcseq_test...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@0 uvm_test_top.tb.mcseqr@@router_simple_mcseq [router_simple_mcseq] Executing
router_simple_mcseq sequence
@0 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_small_seq
[hbus_small_packet_seq] Executing hbus_small_packet_seq (maxpktsize=…
@75 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1000 data=0x14
@75 uvm_test_top.tb.router_module.reference [router_reference] maxpktsize is now 20
@90 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_read_seq
[hbus_read_max_pkt_seq] Executing hbus_read_max_pkt_seq
@95 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1001 data=0x01
@95 uvm_test_top.tb.router_module.reference [router_reference] router_en is now 1
@120 uvm_test_top.tb.hbus.masters[0].sequencer@@router_simple_mcseq.hbus_read_seq
[hbus_read_max_pkt_seq] MAXPKTSIZE register reads 20
@120 uvm_test_top.tb.yapp.agent.sequencer@@router_simple_mcseq.yapp_012 [yapp_012_seq] Executing
yapp_012_seq sequence
@120 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral           2     'h0
  length                      integral           6     'd1
  parity_type                 parity_type_e      32    GOOD_PARITY
@125 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected READ  addr=0x1000 data=0x14
@335 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral           2     'h0
  length        integral           6     'd1
  parity_type   parity_type_e      32    GOOD_PARITY
@340 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral           2     'h1
... (47 lines left out)
@3140 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 2 of 1
cycle, 0 of 2, 9 longer (gap coverage 50.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (109 more lines of the kinds above)
```

uvm_reset_test

```
@0 reporter [RNTST] Running test uvm_reset_test...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@85 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected READ  addr=0x1000 data=0x3f
@115 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected READ  addr=0x1001 data=0x01
@520 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 report: 0 packets
collected
@520 uvm_test_top.tb.chan1.rx_agent.monitor [channel_rx_monitor] Channel 1 report: 0 packets
collected
@520 uvm_test_top.tb.hbus.monitor [hbus_monitor] HBUS report: 0 writes, 9 reads
--- Reference model report ---
--- Scoreboard report ---
  packets matched   : 0
  packets mismatched: 0
@520 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] YAPP driver report: 0 packets sent
@520 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP monitor report: 0 packets
collected, coverage 0.0%
@520 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 0 of 1
cycle, 0 of 2, 0 longer (gap coverage 0.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (9 more lines of the kinds above)
```

uvm_mem_walk_test

```
@0 reporter [RNTST] Running test uvm_mem_walk_test...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@75 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1100 data=0xff
@95 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1101 data=0xfe
@125 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected READ  addr=0x1100 data=0xff
@195 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected READ  addr=0x1101 data=0xfe
@18150 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 report: 0 packets
collected
@18150 uvm_test_top.tb.chan1.rx_agent.monitor [channel_rx_monitor] Channel 1 report: 0 packets
collected
@18150 uvm_test_top.tb.hbus.monitor [hbus_monitor] HBUS report: 511 writes, 256 reads
--- Reference model report ---
--- Scoreboard report ---
  packets matched   : 0
  packets mismatched: 0
@18150 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] YAPP driver report: 0 packets sent
@18150 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP monitor report: 0 packets
collected, coverage 0.0%
@18150 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 0 of 1
cycle, 0 of 2, 0 longer (gap coverage 0.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (765 more lines of the kinds above)
```

Try this

Small changes to make in `start/` once the lab works, each with a question; the answer unfolds.

Write `ctrl_reg` through the backdoor (`poke`) and then read it through the front door. What is the value in the register model's mirror after each step?

Checkpoint questions

Why does the register sequence start with `start(null)`?

Why `provides_responses = 0` in the adapter?

Why 511 writes and 255 reads for a 256-location memory?

What does `set_auto_predict(1)` change?

What changed since the previous lab

```
diff -r labs/lab09_sbc/tb labs/lab11b_rm_integ/tb
```

Lab 11C — Register model: simulation

Open on the project map →

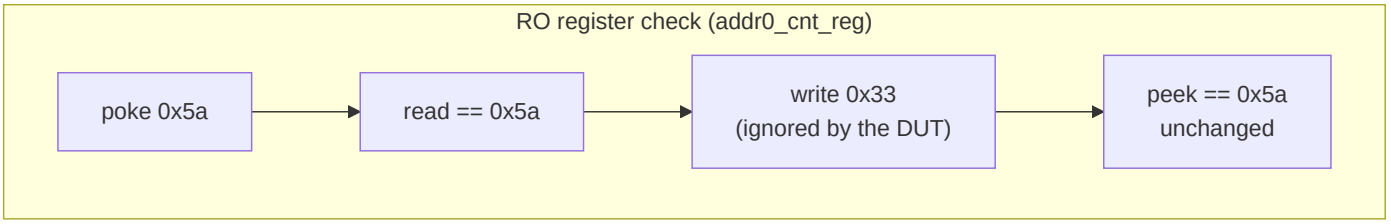
Directory: `labs/lab11c_rm_sim` · **Changed:** `tb/router_test_lib.sv` (+ 4 tests)

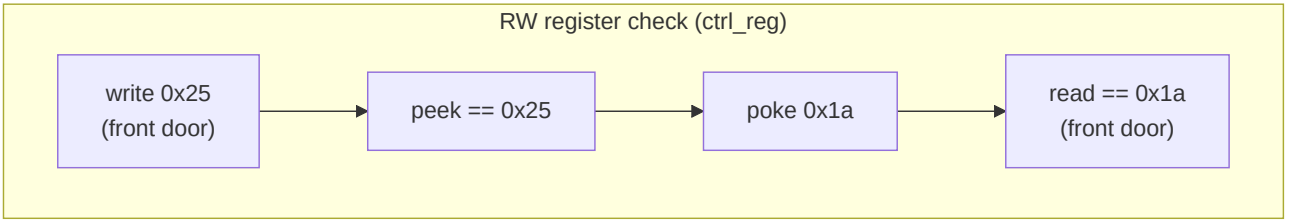
Objective

Write your own register stimulus: check that registers can be accessed as specified (RW vs RO, front door vs backdoor), then check that the enable bits and counters *behave*.

Concepts

`write` / `read` (front door) · `peek` / `poke` (backdoor) · `predict` · `set_check_on_read` · introspection (`get_registers`, `get_rights`)





Solution

```
//------------------------------------------------------------------------------
// router_test_lib.sv -- test library (Lab 11C: user-defined register stimulus)
//
// Register access API used here (all on uvm_reg):
//   write(status, value)  front door through the HBUS   (bus traffic)
//   read (status, value)  front door through the HBUS
//   poke (status, value)  backdoor: deposit into the RTL variable (no bus cycle)
//   peek (status, value)  backdoor: read the RTL variable
//   predict(value)        set the mirrored value without touching the DUT
//
// One class per file: the classes live in tests/<class>.sv and are included below.
//------------------------------------------------------------------------------

`include "tests/base_test.sv"
`include "tests/uvm_reset_test.sv"
`include "tests/uvm_mem_walk_test.sv"
`include "tests/reg_access_test.sv"
`include "tests/reg_function_test.sv"
`include "tests/reg_function_check_test.sv"
`include "tests/reg_introspection_test.sv"

//------------------------------------------------------------------------------
// base_test.sv -- base_test (test library (Lab 11C: user-defined register stimulus))
// Split out of router_test_lib.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// base_test
//------------------------------------------------------------------------------
class base_test extends uvm_test;

  router_tb m_tb;

  `uvm_component_utils(base_test)

  extern function new(string name, uvm_component parent);
  extern function void build_phase(uvm_phase phase);

  extern virtual function void configure_sequences();
  extern function void set_clock_and_channel_sequences();

  extern function void end_of_elaboration_phase(uvm_phase phase);
  extern task run_phase(uvm_phase phase);

  extern function void check_phase(uvm_phase phase);

endclass : base_test

//------------------------------------------------------------------------------
// base_test -- method implementations
//------------------------------------------------------------------------------

function base_test::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//------------------------------------------------------------------------------
function void base_test::build_phase(uvm_phase phase);
```

```
  super.build_phase(phase);
  `uvm_info(get_type_name(), "Executing the build phase of the test", UVM_HIGH)
  uvm_config_int::set(this, "*", "recording_detail", 1);
  configure_sequences();
  m_tb = router_tb::type_id::create("tb", this);
endfunction : build_phase

//------------------------------------------------------------------------------
function void base_test::configure_sequences();
  // nothing: derived tests decide what runs
endfunction : configure_sequences

//------------------------------------------------------------------------------
function void base_test::set_clock_and_channel_sequences();
  uvm_config_wrapper::set(this, "tb.clk_rst.agent.sequencer.run_phase",
                          "default_sequence", clk10_rst5_seq::get_type());
  uvm_config_wrapper::set(this, "tb.chan*.rx_agent.sequencer.run_phase",
                          "default_sequence", channel_rx_resp_seq::get_type());
endfunction : set_clock_and_channel_sequences

//------------------------------------------------------------------------------
function void base_test::end_of_elaboration_phase(uvm_phase phase);
  uvm_top.print_topology();
endfunction : end_of_elaboration_phase

//------------------------------------------------------------------------------
task base_test::run_phase(uvm_phase phase);
  uvm_objection obj = phase.get_objection();
  obj.set_drain_time(this, 200ns);
endtask : run_phase

//------------------------------------------------------------------------------
function void base_test::check_phase(uvm_phase phase);
  check_config_usage();
endfunction : check_phase

//------------------------------------------------------------------------------
// uvm_reset_test.sv -- uvm_reset_test (test library (Lab 11C: user-defined register stimulus))
// Split out of router_test_lib.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// uvm_reset_test (Lab 11B)
//------------------------------------------------------------------------------
class uvm_reset_test extends base_test;

  `uvm_component_utils(uvm_reset_test)

  extern function new(string name, uvm_component parent);
  extern function void configure_sequences();

  extern task run_phase(uvm_phase phase);

endclass : uvm_reset_test

//------------------------------------------------------------------------------
// uvm_reset_test -- method implementations
//------------------------------------------------------------------------------
```

```
function uvm_reset_test::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//------------------------------------------------------------------------------
function void uvm_reset_test::configure_sequences();
  set_clock_and_channel_sequences();
endfunction : configure_sequences

//------------------------------------------------------------------------------
task uvm_reset_test::run_phase(uvm_phase phase);
  uvm_reg_hw_reset_seq reset_seq;
  super.run_phase(phase);
  phase.raise_objection(this);
  reset_seq = uvm_reg_hw_reset_seq::type_id::create("reset_seq");
  reset_seq.model = m_tb.m_yapp_rm;
  reset_seq.start(null);
  phase.drop_objection(this);
endtask : run_phase

//------------------------------------------------------------------------------
// uvm_mem_walk_test.sv -- uvm_mem_walk_test (test library (Lab 11C: user-defined register
stimulus))
// Split out of router_test_lib.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// uvm_mem_walk_test (Lab 11B, optional)
//------------------------------------------------------------------------------
class uvm_mem_walk_test extends base_test;

  `uvm_component_utils(uvm_mem_walk_test)

  extern function new(string name, uvm_component parent);
  extern function void configure_sequences();

  extern task run_phase(uvm_phase phase);

endclass : uvm_mem_walk_test

//------------------------------------------------------------------------------
// uvm_mem_walk_test -- method implementations
//------------------------------------------------------------------------------

function uvm_mem_walk_test::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//------------------------------------------------------------------------------
function void uvm_mem_walk_test::configure_sequences();
  set_clock_and_channel_sequences();
endfunction : configure_sequences

//------------------------------------------------------------------------------
task uvm_mem_walk_test::run_phase(uvm_phase phase);
  uvm_mem_walk_seq walk_seq;
  super.run_phase(phase);
  phase.raise_objection(this);
  walk_seq = uvm_mem_walk_seq::type_id::create("walk_seq");
```

```
  walk_seq.model = m_tb.m_yapp_rm;
  walk_seq.start(null);
  phase.drop_objection(this);
endtask : run_phase

//------------------------------------------------------------------------------
// reg_access_test.sv -- reg_access_test (test library (Lab 11C: user-defined register stimulus))
// Split out of router_test_lib.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// reg_access_test (Lab 11C): front-door / backdoor access checks on one RW
// register (ctrl_reg) and one RO register (addr0_cnt_reg).
//
// Writing an RO register through the front door produces a real HBUS write
// cycle, which the DUT ignores; the model's mirror is not updated either
// because the field policy is RO.
//------------------------------------------------------------------------------
class reg_access_test extends base_test;

  // Convenience handle to the register block (path from the topology report)
  yapp_regs_c m_regs;

  `uvm_component_utils(reg_access_test)

  extern function new(string name, uvm_component parent);
  extern function void configure_sequences();

  extern function void connect_phase(uvm_phase phase);
  extern task run_phase(uvm_phase phase);

  // The stimulus of this test; derived tests replace it
  extern virtual task access_checks();

  // RW: front-door write -> peek, poke -> front-door read
  extern task check_rw_register(uvm_reg rg, uvm_reg_data_t wr_val, uvm_reg_data_t poke_val);

  // RO: poke -> front-door read, front-door write (ignored) -> peek unchanged
  extern task check_ro_register(uvm_reg rg, uvm_reg_data_t poke_val, uvm_reg_data_t wr_val);

endclass : reg_access_test

//------------------------------------------------------------------------------
// reg_access_test -- method implementations
//------------------------------------------------------------------------------

function reg_access_test::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//------------------------------------------------------------------------------
function void reg_access_test::configure_sequences();
  set_clock_and_channel_sequences();
endfunction : configure_sequences

//------------------------------------------------------------------------------
function void reg_access_test::connect_phase(uvm_phase phase);
  m_regs = m_tb.m_yapp_rm.router_yapp_regs;
endfunction : connect_phase
```

```
//------------------------------------------------------------------------------
task reg_access_test::run_phase(uvm_phase phase);
  super.run_phase(phase);                   // drain time
  phase.raise_objection(this);
  access_checks();
  phase.drop_objection(this);
endtask : run_phase

//------------------------------------------------------------------------------
task reg_access_test::access_checks();
  check_rw_register(m_regs.ctrl_reg,      8'h25, 8'h1a);
  check_ro_register(m_regs.addr0_cnt_reg, 8'h5a, 8'h33);
endtask : access_checks

//------------------------------------------------------------------------------
task reg_access_test::check_rw_register(uvm_reg rg, uvm_reg_data_t wr_val, uvm_reg_data_t
poke_val);
  uvm_status_e   status;
  uvm_reg_data_t val;
  `uvm_info("REG_ACCESS", $sformatf("--- RW register %s ---", rg.get_name()), UVM_NONE)

  rg.write(status, wr_val);
  `uvm_info("REG_ACCESS", $sformatf("front-door write 0x%02h", wr_val), UVM_NONE)
  rg.peek(status, val);
  `uvm_info("REG_ACCESS", $sformatf("peek            0x%02h", val), UVM_NONE)
  if (val != wr_val) begin
    `uvm_error("REG_ACCESS", $sformatf("%s: peek returned 0x%02h, expected 0x%02h",
rg.get_name(), val, wr_val))
  end

  rg.poke(status, poke_val);
  `uvm_info("REG_ACCESS", $sformatf("poke            0x%02h", poke_val), UVM_NONE)
  rg.read(status, val);
  `uvm_info("REG_ACCESS", $sformatf("front-door read 0x%02h", val), UVM_NONE)
  if (val != poke_val) begin
    `uvm_error("REG_ACCESS", $sformatf("%s: read returned 0x%02h, expected 0x%02h",
rg.get_name(), val, poke_val))
  end
endtask : check_rw_register

//------------------------------------------------------------------------------
task reg_access_test::check_ro_register(uvm_reg rg, uvm_reg_data_t poke_val, uvm_reg_data_t
wr_val);
  uvm_status_e   status;
  uvm_reg_data_t val;
  `uvm_info("REG_ACCESS", $sformatf("--- RO register %s ---", rg.get_name()), UVM_NONE)

  rg.poke(status, poke_val);
  `uvm_info("REG_ACCESS", $sformatf("poke            0x%02h", poke_val), UVM_NONE)
  rg.read(status, val);
  `uvm_info("REG_ACCESS", $sformatf("front-door read 0x%02h", val), UVM_NONE)
  if (val != poke_val) begin
    `uvm_error("REG_ACCESS", $sformatf("%s: read returned 0x%02h, expected 0x%02h",
rg.get_name(), val, poke_val))
  end

  rg.write(status, wr_val);
  `uvm_info("REG_ACCESS", $sformatf("front-door write 0x%02h (RO: must be ignored)", wr_val),
```

```
UVM_NONE)
  rg.peek(status, val);
  `uvm_info("REG_ACCESS", $sformatf("peek            0x%02h", val), UVM_NONE)
  if (val != poke_val) begin
    `uvm_error("REG_ACCESS", $sformatf("%s: RO register changed to 0x%02h after a write",
rg.get_name(), val))
  end
endtask : check_ro_register

//------------------------------------------------------------------------------
// reg_function_test.sv -- reg_function_test (test library (Lab 11C: user-defined register
stimulus))
// Split out of router_test_lib.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// reg_function_test (Lab 11C): do the enable bits and counters behave?
//   1. en_reg = router_en only; run yapp_012_seq; address counters stay 0
//   2. en_reg = 0xff; run yapp_012_seq twice; addr0/1/2 == 2, addr3 == 0,
//      parity_err_cnt == bad-parity packets seen by the monitor, oversized == 0
//------------------------------------------------------------------------------
class reg_function_test extends reg_access_test;

  yapp_tx_sequencer m_yapp_seqr;
  yapp_012_seq      m_seq;

  `uvm_component_utils(reg_function_test)

  extern function new(string name, uvm_component parent);
  extern function void build_phase(uvm_phase phase);

  extern function void connect_phase(uvm_phase phase);
  extern virtual task access_checks();

  // Front-door read and compare
  extern virtual task check_counter(uvm_reg rg, uvm_reg_data_t expected);

endclass : reg_function_test

//------------------------------------------------------------------------------
// reg_function_test -- method implementations
//------------------------------------------------------------------------------

function reg_function_test::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//------------------------------------------------------------------------------
function void reg_function_test::build_phase(uvm_phase phase);
  super.build_phase(phase);
  m_seq = yapp_012_seq::type_id::create("seq");
endfunction : build_phase

//------------------------------------------------------------------------------
function void reg_function_test::connect_phase(uvm_phase phase);
  super.connect_phase(phase);
  m_yapp_seqr = m_tb.m_yapp.m_agent.m_sequencer;      // hierarchical path, no config
endfunction : connect_phase
```

```
//------------------------------------------------------------------------------
task reg_function_test::access_checks();
  uvm_status_e   status;
  uvm_reg_data_t val;
  int bad_before, bad_after;

  // 1. router enabled, every counter disabled
  m_regs.en_reg.write(status, 8'h01);
  m_regs.en_reg.read(status, val);
  `uvm_info("REG_FUNC", $sformatf("en_reg reads 0x%02h", val), UVM_NONE)
  if (val != 8'h01) begin
    `uvm_error("REG_FUNC", $sformatf("en_reg reads 0x%02h, expected 0x01", val))
  end

  m_seq.start(m_yapp_seqr);                     // 3 packets, counters off
  check_counter(m_regs.addr0_cnt_reg, 0);
  check_counter(m_regs.addr1_cnt_reg, 0);
  check_counter(m_regs.addr2_cnt_reg, 0);
  check_counter(m_regs.addr3_cnt_reg, 0);

  // 2. every counter enabled
  bad_before = m_tb.m_yapp.m_agent.m_monitor.m_num_bad_parity;
  m_regs.en_reg.write(status, 8'hff);
  m_seq.start(m_yapp_seqr);
  m_seq.start(m_yapp_seqr);                     // 6 packets, 2 per address
  bad_after = m_tb.m_yapp.m_agent.m_monitor.m_num_bad_parity;

  check_counter(m_regs.addr0_cnt_reg, 2);
  check_counter(m_regs.addr1_cnt_reg, 2);
  check_counter(m_regs.addr2_cnt_reg, 2);
  check_counter(m_regs.addr3_cnt_reg, 0);
  check_counter(m_regs.parity_err_cnt_reg, bad_after - bad_before);
  check_counter(m_regs.oversized_pkt_cnt_reg, 0);   // maxpktsize is 63 (reset)
endtask : access_checks

//------------------------------------------------------------------------------
task reg_function_test::check_counter(uvm_reg rg, uvm_reg_data_t expected);
  uvm_status_e   status;
  uvm_reg_data_t val;
  rg.read(status, val);
  if (val != expected) begin
    `uvm_error("REG_FUNC", $sformatf("%s reads %0d, expected %0d", rg.get_name(), val, expected))
  end else begin
    `uvm_info("REG_FUNC", $sformatf("%s reads %0d as expected", rg.get_name(), val), UVM_NONE)
  end
endtask : check_counter

//------------------------------------------------------------------------------
// reg_function_check_test.sv -- reg_function_check_test (test library (Lab 11C: user-defined
register stimulus))
// Split out of router_test_lib.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// reg_function_check_test (Lab 11C, optional): automatic checking on read.
// With set_check_on_read(1) every read() compares the DUT value with the
// mirror. RO counters change without bus traffic, so the mirror must be
// loaded with predict() before each read.
//------------------------------------------------------------------------------
```

```
class reg_function_check_test extends reg_function_test;

  `uvm_component_utils(reg_function_check_test)

  extern function new(string name, uvm_component parent);
  extern task run_phase(uvm_phase phase);

  // Load the expected value, then let read() do the comparison
  extern virtual task check_counter(uvm_reg rg, uvm_reg_data_t expected);

endclass : reg_function_check_test

//------------------------------------------------------------------------------
// reg_function_check_test -- method implementations
//------------------------------------------------------------------------------

function reg_function_check_test::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//------------------------------------------------------------------------------
task reg_function_check_test::run_phase(uvm_phase phase);
  m_tb.m_yapp_rm.default_map.set_check_on_read(1);
  super.run_phase(phase);
endtask : run_phase

//------------------------------------------------------------------------------
task reg_function_check_test::check_counter(uvm_reg rg, uvm_reg_data_t expected);
  uvm_status_e   status;
  uvm_reg_data_t val;
  void'(rg.predict(expected));
  rg.read(status, val);                     // mismatch -> UVM_ERROR from the map
  `uvm_info("REG_FUNC", $sformatf("%s reads %0d (checked against predicted %0d)",
                                  rg.get_name(), val, expected), UVM_NONE)
endtask : check_counter

//------------------------------------------------------------------------------
// reg_introspection_test.sv -- reg_introspection_test (test library (Lab 11C: user-defined
register stimulus))
// Split out of router_test_lib.sv: one class per file.
//------------------------------------------------------------------------------

//------------------------------------------------------------------------------
// reg_introspection_test (Lab 11C, optional): let the model tell us which
// registers are RW and which are RO, then run the access checks on all of them.
//------------------------------------------------------------------------------
class reg_introspection_test extends reg_access_test;

  `uvm_component_utils(reg_introspection_test)

  extern function new(string name, uvm_component parent);
  extern virtual task access_checks();

endclass : reg_introspection_test

//------------------------------------------------------------------------------
// reg_introspection_test -- method implementations
//------------------------------------------------------------------------------
```

```
function reg_introspection_test::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

//------------------------------------------------------------------------------
task reg_introspection_test::access_checks();
  uvm_reg all_q[$];
  uvm_reg rw_q[$];
  uvm_reg ro_q[$];

  m_tb.m_yapp_rm.get_registers(all_q);          // every register in the model
  rw_q = all_q.find(r) with (r.get_rights() == "RW");
  ro_q = all_q.find(r) with (r.get_rights() == "RO");

  foreach (rw_q[i]) begin
    `uvm_info("REG_INTRO", $sformatf("RW register: %s @ 0x%04h", rw_q[i].get_name(),
rw_q[i].get_address()), UVM_NONE)
  end
  foreach (ro_q[i]) begin
    `uvm_info("REG_INTRO", $sformatf("RO register: %s @ 0x%04h", ro_q[i].get_name(),
ro_q[i].get_address()), UVM_NONE)
  end

  foreach (rw_q[i]) begin
    check_rw_register(rw_q[i], 8'h25, 8'h1a);
  end
  foreach (ro_q[i]) begin
    check_ro_register(ro_q[i], 8'h5a, 8'h33);
  end
endtask : access_checks
```

Test	What it does
`reg_access_test`	`check_rw_register(m_regs.ctrl_reg)` and `check_ro_register(m_regs.addr0_cnt_reg)` with every access logged at `UVM_NONE`
`reg_function_test`	`en_reg = 0x01` → `yapp_012_seq` → counters still 0; `en_reg = 0xff` → `yapp_012_seq` ×2 → `addr0/1/2 == 2`, `addr3 == 0`, parity counter == bad-parity packets seen by the monitor, oversized == 0
`reg_function_check_test` *(opt.)*	same, with `set_check_on_read(1)`; `check_counter` calls `predict(expected)` before `read()` so the automatic check passes

Test	What it does
`reg_introspection_test` *(opt.)*	asks the model for all registers, splits them into RW / RO queues with `get_rights()`, runs the access checks over every register

The test gets its register handles in `connect_phase` with hierarchical references — `m_regs = m_tb.m_yapp_rm.router_yapp_regs` (path from the topology report of Lab 11B) and `m_yapp_seqr = m_tb.m_yapp.m_agent.m_sequencer` — and starts the YAPP sequence explicitly with `seq.start(m_yapp_seqr)` inside its own objection.

Run

```
make run TEST=reg_access_test
make run TEST=reg_function_test
make run TEST=reg_function_check_test
make run TEST=reg_introspection_test
```

Expected:

- `reg_access_test`: eight `[REG_ACCESS]` lines; the RO write produces an HBUS `WRITE addr=0x1009` in the monitor log but the following `peek` still returns `0x5a`; `UVM_ERROR : 0`.
- `reg_function_test`: `addr*_cnt_reg reads 0 as expected` ×4, then `addr0_cnt_reg reads 2`, `addr1_cnt_reg reads 2`, `addr2_cnt_reg reads 2`, `addr3_cnt_reg reads 0`, `parity_err_cnt_reg reads N` (N = bad-parity packets among the last six), `oversized_pkt_cnt_reg reads 0`.
- `reg_function_check_test`: same values, 0 errors. Comment out the `predict()` call to see the check-on-read errors the lab talks about.
- `reg_introspection_test`: `RW register: ctrl_reg @ 0x1000`, `RW register: en_reg @ 0x1001`, seven `RO register:` lines, then the access checks for all nine registers, 0 errors.

See it run

The lines that matter from the real log of every test of this lab (Verilator, seed 1, from the last `make regress`; the paths and the packet tables are left out). Your own log must show the same lines -- `make check LAB=lab11c_rm_sim START=1` looks for them.

base_test

```
@0 reporter [RNTST] Running test base_test...
@0 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 report: 0 packets
collected
@0 uvm_test_top.tb.chan1.rx_agent.monitor [channel_rx_monitor] Channel 1 report: 0 packets
collected
@0 uvm_test_top.tb.hbus.monitor [hbus_monitor] HBUS report: 0 writes, 0 reads
--- Reference model report ---
--- Scoreboard report ---
  packets matched   : 0
  packets mismatched: 0
@0 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] YAPP driver report: 0 packets sent
@0 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP monitor report: 0 packets collected,
coverage 0.0%
@0 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 0 of 1 cycle,
0 of 2, 0 longer (gap coverage 0.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (1 more lines of the kinds above)
```

uvm_reset_test

```
@0 reporter [RNTST] Running test uvm_reset_test...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@85 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected READ  addr=0x1000 data=0x3f
@115 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected READ  addr=0x1001 data=0x01
@520 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 report: 0 packets
collected
@520 uvm_test_top.tb.chan1.rx_agent.monitor [channel_rx_monitor] Channel 1 report: 0 packets
collected
@520 uvm_test_top.tb.hbus.monitor [hbus_monitor] HBUS report: 0 writes, 9 reads
--- Reference model report ---
--- Scoreboard report ---
  packets matched   : 0
  packets mismatched: 0
@520 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] YAPP driver report: 0 packets sent
@520 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP monitor report: 0 packets
collected, coverage 0.0%
@520 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 0 of 1
cycle, 0 of 2, 0 longer (gap coverage 0.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (9 more lines of the kinds above)
```

uvm_mem_walk_test

```
@0 reporter [RNTST] Running test uvm_mem_walk_test...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@75 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1100 data=0xff
@95 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1101 data=0xfe
@125 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected READ  addr=0x1100 data=0xff
@195 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected READ  addr=0x1101 data=0xfe
@18150 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 report: 0 packets
collected
@18150 uvm_test_top.tb.chan1.rx_agent.monitor [channel_rx_monitor] Channel 1 report: 0 packets
collected
@18150 uvm_test_top.tb.hbus.monitor [hbus_monitor] HBUS report: 511 writes, 256 reads
--- Reference model report ---
--- Scoreboard report ---
  packets matched   : 0
  packets mismatched: 0
@18150 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] YAPP driver report: 0 packets sent
@18150 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP monitor report: 0 packets
collected, coverage 0.0%
@18150 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 0 of 1
cycle, 0 of 2, 0 longer (gap coverage 0.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (765 more lines of the kinds above)
```

reg_access_test

```
@0 reporter [RNTST] Running test reg_access_test...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@75 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1000 data=0x25
@75 uvm_test_top.tb.router_module.reference [router_reference] maxpktsize is now 37
@105 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected READ  addr=0x1000 data=0x1a
@135 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected READ  addr=0x1009 data=0x5a
@155 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1009 data=0x33
@350 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 report: 0 packets
collected
@350 uvm_test_top.tb.chan1.rx_agent.monitor [channel_rx_monitor] Channel 1 report: 0 packets
collected
@350 uvm_test_top.tb.hbus.monitor [hbus_monitor] HBUS report: 2 writes, 2 reads
--- Reference model report ---
--- Scoreboard report ---
  packets matched   : 0
  packets mismatched: 0
@350 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] YAPP driver report: 0 packets sent
@350 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP monitor report: 0 packets
collected, coverage 0.0%
@350 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 0 of 1
cycle, 0 of 2, 0 longer (gap coverage 0.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (2 more lines of the kinds above)
```

reg_function_test

```
@0 reporter [RNTST] Running test reg_function_test...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@75 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1001 data=0x01
@75 uvm_test_top.tb.router_module.reference [router_reference] router_en is now 1
@100 uvm_test_top.tb.yapp.agent.sequencer@@seq [yapp_012_seq] Executing yapp_012_seq sequence
@100 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h0
  length                      integral       6     42
  parity_type                 parity_type_e  32    GOOD_PARITY
@105 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected READ  addr=0x1001 data=0x01
@745 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral       2     'h0
  length        integral       6     42
  parity_type   parity_type_e  32    GOOD_PARITY
@750 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h1
  length                      integral       6     59
  parity_type                 parity_type_e  32    GOOD_PARITY
@875 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 collected packet:
  addr         integral               2     'h0
  length       integral               6     42
  parity_type  channel_parity_type_e  32    CHANNEL_GOOD_PARITY
... (38 lines left out)
@5400 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 1 of 1
cycle, 1 of 2, 6 longer (gap coverage 75.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (84 more lines of the kinds above)
```

reg_function_check_test

```
@0 reporter [RNTST] Running test reg_function_check_test...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@75 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1001 data=0x01
@75 uvm_test_top.tb.router_module.reference [router_reference] router_en is now 1
@100 uvm_test_top.tb.yapp.agent.sequencer@@seq [yapp_012_seq] Executing yapp_012_seq sequence
@100 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h0
  length                      integral       6     42
  parity_type                 parity_type_e  32    GOOD_PARITY
@105 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected READ  addr=0x1001 data=0x01
@745 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] Packet collected:
  addr          integral       2     'h0
  length        integral       6     42
  parity_type   parity_type_e  32    GOOD_PARITY
@750 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] Packet is
  addr                        integral       2     'h1
  length                      integral       6     59
  parity_type                 parity_type_e  32    GOOD_PARITY
@875 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 collected packet:
  addr         integral               2     'h0
  length       integral               6     42
  parity_type  channel_parity_type_e  32    CHANNEL_GOOD_PARITY
... (42 lines left out)
@5400 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 1 of 1
cycle, 1 of 2, 6 longer (gap coverage 75.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (80 more lines of the kinds above)
```

reg_introspection_test

```
@0 reporter [RNTST] Running test reg_introspection_test...
@0 uvm_test_top.tb.chan0.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.chan1.rx_agent.sequencer@@channel_rx_resp_seq [channel_rx_resp_seq] Executing
channel_rx_resp_seq (forever)
@0 uvm_test_top.tb.clk_rst.agent.sequencer@@clk10_rst5_seq [clk10_rst5_seq] Executing
clk10_rst5_seq
@75 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1000 data=0x25
@75 uvm_test_top.tb.router_module.reference [router_reference] maxpktsize is now 37
@105 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected READ  addr=0x1000 data=0x1a
@125 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected WRITE addr=0x1001 data=0x25
@125 uvm_test_top.tb.router_module.reference [router_reference] router_en is now 1
@155 uvm_test_top.tb.hbus.monitor [hbus_monitor] Collected READ  addr=0x1001 data=0x1a
@700 uvm_test_top.tb.chan0.rx_agent.monitor [channel_rx_monitor] Channel 0 report: 0 packets
collected
@700 uvm_test_top.tb.chan1.rx_agent.monitor [channel_rx_monitor] Channel 1 report: 0 packets
collected
@700 uvm_test_top.tb.hbus.monitor [hbus_monitor] HBUS report: 9 writes, 9 reads
--- Reference model report ---
--- Scoreboard report ---
  packets matched   : 0
  packets mismatched: 0
@700 uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] YAPP driver report: 0 packets sent
@700 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP monitor report: 0 packets
collected, coverage 0.0%
@700 uvm_test_top.tb.yapp.agent.monitor [yapp_tx_monitor] YAPP gaps: 0 back to back, 0 of 1
cycle, 0 of 2, 0 longer (gap coverage 0.0%)
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
... (16 more lines of the kinds above)
```

Try this

Small changes to make in `start/` once the lab works, each with a question; the answer unfolds.

After one packet with bad parity, read `parity_err_cnt_reg` with `read()` and then with `mirror()`. What does `mirror()` check that `read()` does not?

Checkpoint questions

What happens when you write to an RO register?

Why do the parity-error expectations use `tb.yapp.agent.monitor.num_bad_parity`?

Why does check-on-read fail for the counters without `predict()`?

Why do the reserved bits not get in the way here?

What changed since the previous lab

```
diff labs/lab11b_rm_integ/tb/router_test_lib.sv labs/lab11c_rm_sim/tb/router_test_lib.sv
```

Mentor's guide

Mentor's guide

One page per session of the course, for whoever teaches it: what the session is for, what to have open before it starts, how to spend the time, what to show live, the mistakes students make at that point and the questions to ask. The labs overview has the session plan and the dependency chain; every lab page has the objective, the steps, the expected result and the checkpoint questions with their answers; this guide is the layer above them.

Session	Labs	Page
1	1, 2	Data items, components, phases
2	3, 4	A UVC, the factory, configuration
3	5	Sequences
4	6	Interfaces and the DUT
5	7, 8	Integrating UVCs, the virtual sequencer
6	9A–9D	TLM, scoreboard, reference model
7	10	Functional coverage
8	11A–11C	The register model

Before the first session

- Every student opens the repository in a Codespace (or runs `bash scripts/setup_sim.sh` on her own Ubuntu / WSL2) and runs `make sim` in `test_install`. Verilator is the simulator of the course; Xcelium is the alternative for those who have it.
- The student kit (`make student-kit` → `build/StudentKit.zip`) holds the `start/` directory of every lab, the offline site and the project map, for a machine without internet.
- Students work in `labs/<lab>/start/` (Getting started), check with `make check LAB=<lab> START=1`, and compare with the finished lab next to it only after the check is green -- or when stuck for more than the

time you set.

What to have open in every session

- The project map on a second screen: the Environment view for "where are we", the Hierarchy view for a component, the Classes view for inheritance, the Test plan for "why this test". Every lab page starts with a deep link into the map at the right place.
- A terminal in the lab's `tb/` (the finished lab) for the live run, and the log of the previous regression (Regression results) when the simulation is slow.
- The glossary and the FAQ for the questions that are not about the lab.

Shape of a session

Three hours work well as *concept (30 min) → lab (90 min, in `start/`) → review (30 min) → checkpoint questions and the next lab's objective (30 min)*. The lab pages carry the concept diagrams; the review is the moment to open the finished lab and the map side by side and walk the code top-down. Every session page below says what to put in each slot.

Session 1 — Data items, components, phases

Labs: Lab 1, Lab 2

Goal

A packet class with constraints and the `do_*` methods, then an empty test / testbench pair that runs through the phases. By the end a student can explain what `run_test()` does and why a class in a package needs `uvm_object_utils`.

Before the session

- Run `make sim` in `labs/lab01_data/tb` and `labs/lab02_test/tb` once, so the compiled build is warm on your machine.
- Open the map at Classes → YAPP UVC (for `yapp_packet` and its subclass) and the Packet playground.
- Read the UVM big picture again: it is the picture you draw first.

Plan

1. Concept (30 min): the packet format on the DUT specification, then `uvm_sequence_item`, `rand` fields, constraints, `post_randomize` and the four `do_*` methods -- and why this repository writes them by hand instead of `` `uvm_field_* `` (Getting started).
2. Lab 1 (45 min) in `labs/lab01_data/start/`: the packet. `make check LAB=lab01_data START=1` until green.
3. Concept (20 min): components, the phases (Phases and objections), `run_test()` and `+UVM_TESTNAME`.
4. Lab 2 (40 min) in `labs/lab02_test/start/`: `router_tb` and `base_test`.
5. Review (30 min): the finished `yapp_packet.sv` on the map (Source column), the topology print of Lab 2, the checkpoint questions of both labs.
6. Next lab's objective (15 min): what a UVC is, the components of Lab 3 on the Environment view.

Show live

- `make sim` in Lab 1 with two seeds (`SEED=1`, `SEED=7`): the packets differ, the parity is always right -- `post_randomize` in action.
- The packet playground: build a packet by hand, flip a payload bit, watch the parity check fail.
- Lab 2 with `+UVM_VERBOSITY=UVM_HIGH` and then `UVM_LOW`: which lines are `uvm_info` and which are UVM's own.

Where students get stuck

- A `rand` field without a constraint on the array size: `m_payload` stays empty. The constraint `m_payload.size() == m_length` is the lesson.
- Parity computed before the payload is final (in `pre_randomize` or in the constraint): `post_randomize` is the right hook.
- In Lab 2: creating `router_tb` with `new()` works today and breaks in Lab 4 -- say it now, fix it in Lab 4.
- `+UVM_TESTNAME` misspelled runs the default test with a warning at the top of the log that nobody reads (FAQ).

Ask

- Why does the course keep the parity byte a plain field and compute it, rather than `rand` with a constraint? (A constraint cannot see the payload's final values in one pass.)
- What would happen with two `uvm_test` classes and no `+UVM_TESTNAME`?
- Where does the time go in a Lab 2 run that prints `$finish at 0`? (Nowhere: no component consumes time, the run phase ends at once.)

For next time

Read the Driver and Monitor guides; Lab 3 builds both.

Session 2 — A UVC, the factory, configuration

Labs: Lab 3, Lab 4

Goal

The YAPP agent (driver, sequencer, monitor) printing packets, then the same agent reconfigured from the test: a factory override that swaps the packet class and a passive agent through the configuration database.

Before the session

- Warm builds of `labs/lab03_uvc/tb` and `labs/lab04_factory/tb`.
- Map: Hierarchy → tb.yapp.agent for the agent, Classes view for `short_yapp_packet` under `yapp_packet`.
- Factory and configuration and Agent and env are the two pages the session leans on.

Plan

1. Concept (30 min): the agent as the unit of reuse; driver loop (`get_next_item` / `item_done`), sequencer, monitor; `is_active`; the default sequence set from the test.
2. Lab 3 (50 min) in `start/`: the five classes. `make check LAB=lab03_uvc START=1`.
3. Concept (20 min): `type_id::create` versus `new`, type overrides, `uvm_config_int::set/get`, `check_config_usage`.
4. Lab 4 (40 min): `short_yapp_packet`, `short_packet_test`, `set_config_test`.
5. Review (30 min): the topology of `set_config_test` (no driver, no sequencer) next to the Hierarchy view; the `UVM_PASSIVE` line; why `configure_sequences()` is a virtual function.
6. Checkpoint questions of both labs.

Show live

- Lab 3 `base_test`: five `Packet is` tables from the driver -- the monitor only says it is running (it has no interface yet).
- Lab 4 `short_packet_test` without the override line, then with it: the class name in the first line of every packet table changes and nothing else does.
- Lab 4 `set_config_test` with the default sequence left in `base_test::build_phase` (not in `configure_sequences`): the `configuration never used` warning at the end.

Where students get stuck

- `super.build_phase(phase)` after the override instead of before: the override does nothing because `router_tb` was already built.
- Include order in `yapp_pkg.sv`: a driver included before the sequencer it names (pitfall 1).
- `uvm_config_int::set` with the wrong instance path (`tb.yapp.agent` versus `tb.yapp_agent`): the warning at the end says which setting nobody read.
- `m_` on fields, never on arguments: the style check (`make style-check`) says `R5` and the line.

Ask

- What does the factory need from a class so that an override can replace it? (`uvm_object_utils` and creation through `type_id::create`.)
- Why is the configuration read in `build_phase` and not in `new()`? (The database is filled by the parent's `build_phase`, which runs after the child's `new()`.)
- Which of the two configuration mechanisms would you use for a knob a sequence needs at run time?

For next time

Read Sequencer and sequences; Lab 5 is only sequences, and the planned randomization failure is its best moment.

Session 3 — Sequences

Labs: Lab 5

Goal

The YAPP sequence library: sequences that create items, nest other sequences and raise objections -- and the planned constraint conflict that teaches how to read a randomization failure.

Before the session

- Warm build of `labs/lab05_seq/tb`. The Lab 4 version of `short_yapp_packet` (with `m_addr != 2`) at hand for the planned failure.
- Map: Classes → YAPP UVC with **Members** on, to show `body()` of every sequence and the `start()` arrows of the tests.
- Phases and objections: the objection section.

Plan

1. Concept (40 min): `uvm_sequence`, `body()`, `start_item` / `finish_item` written out (the four steps the `` `uvm_do `` macro hides), `randomize() with`, nested sequences through `start(sequencer, this)`, objections in `pre_body` / `post_body` and the `starting_phase == null` check.
2. Lab 5 (80 min) in `start/`: the nine sequences and the two tests. `make check LAB=lab05_seq START=1`; the `exhaustive_seq_test` check asks for every sequence by name.
3. The planned failure (20 min): put back `m_addr != 2` in `short_yapp_packet`, run `exhaustive_seq_test`, read the `UVM_ERROR ... randomize() failed` line, fix it, run again.
4. Review (30 min): `yapp_exhaustive_seq` on the map, the `Executing …` lines in order, the checkpoint questions.

Show live

- `exhaustive_seq_test`: one `Executing` line per sequence, in the order of the code -- count them with the class.
- `incr_payload_test` with `+UVM_VERBOSITY=UVM_FULL`: `Starting default sequence yapp_incr_payload_seq`, printed by the sequencer.
- The planned failure, then the GUI version if Xcelium is there (`make gui`, the constraint manager).

Where students get stuck

- `randomize()` before `start_item()` in a sequence whose item the test overrides: the item is created and randomized before the driver is ready; works here, teaches the wrong habit. The repository randomizes between `start_item` and `finish_item`.
- An objection raised in a sub-sequence: `starting_phase` is `null` there (pitfall 9).
- `req` declared but never created: `start_item(null)`. The `create` line is the first of the four steps.
- Forgetting `finish_item`: the driver's `get_next_item` returns, `item_done` is never called, the next sequence hangs (FAQ).

Ask

- What does `finish_item` wait for? (The driver's `item_done`.) What would `start_item` wait for with two sequences on one sequencer?
- Why does `yapp_111_seq` start `yapp_1_seq` three times instead of sending three packets itself?
- Where is the objection raised for a nested sequence, and why only once?

For next time

Read Lab 6's concept section before the session: interfaces are the step where the packets become waves.

Session 4 — Interfaces and the DUT

Labs: Lab 6

Goal

Packets flowing through the real router: `yapp_if`, the virtual interface in driver and monitor, `hw_top` with the DUT and `tb_top` publishing the interface, the drain time, and the first waves.

Before the session

- Warm build of `labs/lab06_vif/tb` **with waves** (`make sim TEST=yapp_012_test WAVES=1`) and a wave viewer that opens `.fst` (VaporView in Codespaces, Surfer, GTKWave).
- Map: Hierarchy → hw_top for the DUT pins, and the DUT specification for the input port protocol (`in_data_vld`, `in_suspend`).
- Logs, timelines and waves for how the wave pictures are read.

Plan

1. Concept (40 min): an `interface` as the bundle of wires, the virtual interface handle in a class, `uvm_config_db#(virtual yapp_if)` from `tb_top`, clocking: drive on the falling edge, sample on the rising edge; what `hw_top` and `tb_top` each hold; the drain time.
2. Lab 6 (80 min) in `start/`: the interface, the driver's `send_to_dut`, the monitor's `collect_packet`, `hw_top`, `tb_top`. `make check LAB=lab06_vif START=1` looks for the monitor's `Packet collected` lines.
3. Waves (30 min): the first packet on `in0` -- header, payload, parity, `in_data_vld`, and where `hw_top.dut.u_tracer.in_phase` labels them.
4. Review (30 min): the driver and monitor on the map with their interface arrows, the `YAPP driver report` / `monitor report` counts, the checkpoint questions.

Show live

- `yapp_012_test` with waves: three packets on the input, each on its own channel on the output; `data_vld_0/1/2`.
- Drain time 0 in `base_test`: the last packet is sent and never collected -- one count differs between the driver's and the monitor's report.
- `set_config_test` with the DUT: the agent is passive, nothing drives, the DUT stays idle, `0 packets collected`.

Where students get stuck

- The driver drives on the rising edge too: races with the DUT, bytes lost or doubled. Falling edge in the driver, rising edge in the DUT and the monitor.
- The monitor samples `in_data` when `in_data_vld` is low: the header read as a payload byte.
- `tb_top` publishes the interface under one name and the driver asks for another: the `uvm_fatal` `virtual interface must be set` at the start of the run.
- `hw_top` and `tb_top` both compiled as top modules: that is intended (two tops), students look for the mistake.

Ask

- Why is the interface compiled (listed in `run.f`) and not included in the package?
- What breaks if the monitor is given the driver's `vif` by assignment inside the agent instead of through the config_db? (Nothing today; the monitor can no longer be built passive without the driver.)
- Where does the DUT's `in_suspend` come from, and what must the driver do when it sees it?

For next time

Read Lab 7's table of the provided UVCs: next time three UVCs join at once and the `hdata_w` wire is the trap of the day.

Session 5 — Integrating UVCs, the virtual sequencer

Labs: Lab 7, Lab 8

Goal

The full environment: the HBUS, Channel and Clock & Reset UVCs configured and wired in `router_tb` and `hw_top`, then a multichannel sequence on a virtual sequencer that programs the router over HBUS and sends packets in one `body()`.

Before the session

- Warm builds of `labs/lab07_integ/tb` and `labs/lab08_mcseq/tb` (the first compile of the full environment takes a few minutes).
- Map: the Environment view (every UVC open), TLM / data flow for the arrows, and Hierarchy → tb.mcseqr.
- Virtual sequencer and the HBUS register map (the write of `maxpktsize` and `en_reg` is the first thing a multichannel sequence does).

Plan

1. Concept (30 min): reusing a UVC you did not write -- its interface, its configuration knobs (`num_masters`, `channel_id`), its default sequences; several interfaces in `hw_top`; the bidirectional `hdata` and why the DUT connects to `hdata_w`.
2. Lab 7 (60 min) in `start/`: `router_tb` with the four envs, `hw_top`, `tb_top`, `run.f`. `make check LAB=lab07_integ START=1`: the three channels must each collect a packet in `simple_test`.
3. Concept (20 min): the virtual sequencer, `` `uvm_declare_p_sequencer ``, a sequence that starts sub-sequences on `p_sequencer.hbus_seqr` and `p_sequencer.yapp_seqr`.
4. Lab 8 (40 min): `router_mcsequencer`, `router_simple_mcseq`, the test.
5. Review (30 min): `router_simple_mcseq` on the map with its `uses` arrows; the HBUS monitor lines `Collected WRITE addr=0x1000 data=0x14` and the read that returns 20; the checkpoint questions.

Show live

- Lab 7 `test_uvc_integration`: 88 packets, the HBUS programming at the start, the channel reports at the end.
- Lab 8 `router_simple_mcseq_test`: the order of the `Executing` lines -- HBUS small-packet sequence, then the read, then the YAPP sequence -- and `MAXPKTSIZE register reads 20`.

- The `hdata_w` wire in the waves during a read: `z` when nobody drives, the DUT's value in the second cycle.

Where students get stuck

- The DUT connected to `hbus0.hdata` (the logic variable) instead of `hbus0.hdata_w`: writes work, reads return `x` (Lab 7, the trap of the day).
- `num_masters` set after `super.build_phase`: two masters or none.
- `channel_id` not set per channel env: every monitor checks for address 0 and reports the other packets as errors.
- In Lab 8, the virtual sequence started on a real sequencer, or `p_sequencer` used without the macro: a `null` handle at run time.

Ask

- Why does the channel UVC's response sequence run `forever` and raise no objection? What ends the test, then?
- What does the virtual sequencer own? (Handles, no driver.) What would a test lose if the multichannel sequence wrote to the HBUS sequencer directly, by its hierarchical name?
- `num_slaves` does not exist: who is the slave on HBUS, and why does the UVC not need an agent for it?

For next time

Read Scoreboard and Reference model; session 6 turns the environment self-checking, in four labs.

Session 6 — TLM, scoreboard, reference model

Labs: Lab 9A, Lab 9B, Lab 9C and Lab 9D (optional)

Goal

Self-checking tests: analysis ports from the monitors into a scoreboard (9A), a reference model that predicts the drops so the scoreboard stops lying (9B), exports on the module UVC's env (9C) and an analysis-FIFO scoreboard as the alternative (9D).

Before the session

- Warm builds of the four `labs/lab09_*/tb`.
- Map: TLM / data flow with **Arrows** on -- the whole session is this picture; the **Lab 9D** button swaps the FIFO scoreboard in.
- TLM connections, and the test plan group *Drop rules* for what the reference model must predict.

Plan

1. Concept (30 min): `uvm_analysis_port` → `uvm_analysis_imp`, `write()`, `` `uvm_analysis_imp_decl `` for several imps in one component, `clone()` before storing a transaction; what a scoreboard compares and what it cannot know.
2. Lab 9A (40 min) in `start/`: the scoreboard. `make check LAB=lab09_sba START=1` expects the *planned* mismatch of `scoreboard_drop_test` (the test is listed with `expect: errors`).
3. Concept (15 min): the reference model as the DUT's rules at transaction level; the module UVC as the home of model + scoreboard.
4. Lab 9B (50 min): `router_reference`, `router_module_env`, the package, the connections in `router_tb`. The same `scoreboard_drop_test` now passes.
5. 9C / 9D (30 min, optional): exports on the env; the analysis FIFO and the blocking `get()`.
6. Review (15 min): the arrows on the map in the order a packet travels; the reports `packets matched / mismatched`; the checkpoint questions.

Show live

- 9A `scoreboard_drop_test`: the `MISMATCH` line and the report with mismatches -- the scoreboard is right that something differs and wrong about what.
- 9B the same test: `maxpktsize is now 20`, `router_en is now 1` from the reference model, then `0 mismatched`.

- The 9D variant on the map: the FIFO scoreboard replaces model + scoreboard; same arrows, different consumer.

Where students get stuck

- Storing the handle the monitor passed to `write()` instead of a `clone()`: the monitor reuses the object, the stored packet changes under the scoreboard.
- Two `uvm_analysis_imp` fields of the same type in one class: the `` `uvm_analysis_imp_decl(_suffix) `` macro and the `write_suffix` method.
- `connect_phase` connecting a port to an export of a component that is created later: order the envs, or connect one level up (Lab 9C).
- The reference model that predicts a drop for address 3 but not for `router_en == 0`: the test plan's drop rules, all four.

Ask

- What is the difference between a port and an export, and which end calls `write()`?
- Why is the scoreboard's expected queue per channel, not one for all?
- With the FIFO scoreboard (9D), what keeps the test alive while `get()` blocks, and what happens to a packet that was predicted and never arrives?

For next time

Read Functional coverage. Session 7 is one lab and a lot of looking at numbers; have the Coverage results page open.

Session 7 — Functional coverage

Labs: Lab 10

Goal

A covergroup on the monitor's packets, a subscriber that samples it, a sequence written to close it -- and reading a coverage report: bins, holes, crosses, and which tests the coverage needs.

Before the session

- Warm build of `labs/lab10_cov/tb`; `make coverage ONLY=lab10` after a run gives the merged bins on the terminal.
- Map: the Coverage view (one card per collector, a chip per coverpoint, the bins in the panel).
- Coverage results of the project, with the *Which tests the coverage needs* table, and the Bug hunt page: coverage says what was exercised, the bug hunt says what was *checked*.

Plan

1. Concept (40 min): coverpoints and bins, `illegal_bins`, crosses, `sample()` from a `uvm_subscriber`'s `write()`, coverage as a question to the stimulus; why `yapp_pkt_coverage` is a component of its own and not a covergroup inside the monitor.
2. Lab 10 (70 min) in `start/`: the collector and `yapp_coverage_seq`. The stub of the collector shadows the finished one through `-incdir ../sv` (see `TASKS.md`). `make check LAB=lab10_cov START=1` asks for `coverage 100.0%` in the monitor's report.
3. Reading the report (30 min): the coverage results page of the project -- a hole, a bin hit by one test only, the minimal set of tests -- then the bug hunt matrix next to it.
4. Review (30 min): the Coverage view on the map, the checkpoint questions.

Show live

- `coverage_test` before and after `yapp_coverage_seq` is complete: the percentage in the monitor's report.
- A bin added live (`bins max = {63}`), run again, the hole; which sequence closes it.
- The bug hunt page: pick one bug, show the test that catches it and the first error line; then `make sim TEST=... DEFINES=BUG_...` in `yapp_project/tb` with the waves.

Where students get stuck

- Sampling in the monitor's `run_phase` instead of in the subscriber's `write()`: the covergroup samples before the packet is complete.
- A covergroup constructed without `new()` in the component's constructor: nothing samples, 0% with no error.
- A cross that includes an illegal combination: 100% is impossible; `ignore_bins` or the right bins.
- Reading coverage as a verdict: 100% bins with no scoreboard proves nothing -- the session's last word, with the bug hunt as the evidence.

Ask

- What does a coverage hole tell you, and what does it not?
- Which test would you drop from the regression first, and why? (The *only this test* column.)
- A bug that survives every test: is it a coverage problem or a checking problem? (Look at the bug hunt page's survivors, if any.)

For next time

Read Register model and the register map. Session 8 is three short labs around one generated package.

Session 8 — The register model

Labs: Lab 11A, Lab 11B, Lab 11C

Goal

The register model: generated from the register description (11A), integrated with the adapter, the sequencer and the backdoor paths (11B), and used by tests that read, write, mirror and introspect (11C) -- the built-in sequences included.

Before the session

- Warm builds of `labs/lab11b_rm_integ/tb` and `labs/lab11c_rm_sim/tb`; for 11A the XML and the generated package side by side (no simulation).
- Map: Classes → **Register model** tab (the block, the registers, the memories), Hierarchy → tb.yapp_rm with the backdoor arrow to `u_regs`, and the register simulator on the specification page.
- Register model and the three lab pages.

Plan

1. Concept (30 min): `uvm_reg_field` / `uvm_reg` / `uvm_reg_block` / `uvm_reg_map`, front door through an adapter and a sequencer, backdoor through HDL paths, the mirror and prediction, the built-in sequences (`uvm_reg_hw_reset_seq`, `uvm_mem_walk_seq`).
2. Lab 11A (30 min): the XML and what the generator makes of it; `quicktest.sv` lints the package.
3. Lab 11B (50 min) in `start/`: `router_tb` builds the model, sets the adapter, sequencer and HDL root; the tests run the built-in sequences. `make check LAB=lab11b_rm_integ START=1` counts the HBUS accesses of the walk (511 writes, 256 reads).
4. Lab 11C (50 min): `reg_access_test`, `reg_function_test` and friends -- `write`, `read`, `peek`, `poke`, `mirror`, `predict`, and the introspection test.
5. Review (20 min): the `INJECT_ERROR` build of 11B (`make sim TEST=uvm_mem_walk_test DEFINES=INJECT_ERROR`): the walk finds the corrupted byte; the checkpoint questions; where the course goes from here (the project's test plan, the random tests, the bug hunt).

Show live

- `uvm_reset_test`: nine front-door reads, one per register, all equal to the reset values in the map.
- `uvm_mem_walk_test` with and without `INJECT_ERROR`: the one `UVM_ERROR` that names the address.

- `reg_function_test`: the counters read back through the model after six packets, and the `[TEST] STAGE` lines that tell the story.

Where students get stuck

- `set_hdl_path_root` with the wrong prefix: every backdoor access fails with `UVM_ERROR ... hdl path`; the path is `hw_top.dut.u_regs`.
- The register package imported after `router_tb` in `tb_top`: `yapp_router_regs_t` unknown (Lab 11B).
- `mirror()` on a counter the model cannot predict: the test must `predict()` first, or compare with `UVM_NO_CHECK`.
- The adapter's `bus2reg` not setting `status`: every access reports `UVM_NOT_OK`.

Ask

- When would you use the backdoor in a real test, and what can it never check? (The bus protocol.)
- What does `mirror()` compare, and where does the expected value come from after a front-door `write()`?
- The walk writes 511 times and reads 256 times: why not 256 and 256?

For next time

The course ends here; the project's test plan, its random tests and the bug hunt are where a student goes next -- the testbench as it is used, not as it is built.

Appendix

Files, classes, tests and sequences

Files and classes

File	Class / module
`yapp_packet.sv`	`yapp_packet`, `parity_type_e`
`short_yapp_packet.sv`	`short_yapp_packet`
`yapp_pkg.sv`	`yapp_pkg` (+ `yapp_vif_config`)
`yapp_tx_driver.sv`	`yapp_tx_driver`
`yapp_tx_sequencer.sv`	`yapp_tx_sequencer`
`yapp_tx_monitor.sv`	`yapp_tx_monitor`
`yapp_pkt_coverage.sv`, `yapp_gap_coverage.sv`	`yapp_pkt_coverage` (`yapp_pkt_cg`), `yapp_gap_covera` (`yapp_gap_cg`)
`yapp_tx_agent.sv`	`yapp_tx_agent`
`yapp_env.sv`	`yapp_env`
`yapp_tx_seqs.sv` → `seqs/<class>.sv`	`yapp_base_seq`, `yapp_5_packets`, … (one class per fi
`yapp_if.sv`	`yapp_if`
`top.sv` → `tb_top.sv`	`top` → `tb_top`

File	Class / module
`hw_top.sv`	`hw_top`
`clkgen.sv`	`clkgen`
`router_tb.sv`	`router_tb`
`router_test_lib.sv` → `tests/<class>.sv`	all tests, one class per file
`router_mcsequencer.sv`	`router_mcsequencer`
`router_mcseqs_lib.sv` → `mcseqs/<class>.sv`	`router_mcseq_base`, `router_simple_mcseq`; `router_random_mcseq`, `router_hbus_sweep_mcseq` (te
`tests/error_pulse_checker.sv`	`error_pulse_checker` (one `error` pulse per bad pac 1..10 cycles)
`yapp_router_tracer.sv`	`yapp_router_tracer` (+ `yapp_router_tracer_bind`): log lines and the per-cycle signal trace, bound into th router
`router_scoreboard.sv`	`router_scoreboard`
`packet_compare.sv`	`comp_equal()`, `comp_equal_uvm()`
`router_reference.sv`	`router_reference` (full model of the registers, count memories; checks every HBUS read)
`router_decision.sv`	`router_decision`, `router_reason_e` (what the refere decided about a packet, published on `decision_out`

File	Class / module
`router_coverage.sv`	`router_coverage` (`router_cfg_cg`: the configuration packet met)
`router_hbus_addr_coverage.sv`, `router_hbus_bits_coverage.sv`, `router_hbus_data_coverage.sv`, `router_hbus_counter_coverage.sv`	`router_hbus_*_coverage` (`hbus_addr_cg`, `hbus_bits_` `hbus_data_cg`, `hbus_counter_cg`)
`router_module_env.sv`	`router_module_env`
`router_module_pkg.sv`	`router_module_pkg`
`router_fifo_scoreboard.sv`	`router_fifo_scoreboard`
`yapp_router_reg_pkg.sv` → `reg/<class>.sv`	`yapp_router_regs_t`, `yapp_regs_c`, `*_c`
`hbus_*.sv`	`hbus_transaction`, `hbus_master_driver/sequencer/a` `hbus_monitor`, `hbus_env`, `hbus_reg_adapter`, `hbus_i`
`channel_*.sv`	`channel_packet`, `channel_rx_driver/sequencer/monitor/agent`, `channel_env`, `channel_if`
`clock_and_reset_*.sv`	`clock_and_reset_transaction/driver/sequencer/age` `clock_and_reset_if`
`yapp_router.sv`	`yapp_router` (top level, wiring only; instances `u_inp` `g_ch[0..2].u_ch`, `u_regs`, `u_error_timer`)
`yapp_input_fsm.sv`	`yapp_input_fsm`

File	Class / module
`yapp_output_channel.sv`	`yapp_output_channel` (contains `u_fifo`)
`yapp_fifo.sv`	`yapp_fifo`
`yapp_hbus_regs.sv`	`yapp_hbus_regs` (every register and memory, the HB slave; backdoor root `hw_top.dut.u_regs`)
`yapp_error_timer.sv`	`yapp_error_timer`
`yapp_router.f`	file list of the six RTL files, `-F ../rtl/yapp_router.f` every `run.f`

The table lists the labs' `tb/` directories; the final testbench with every test, the multichannel sequences and the register model is `yapp_project/tb/` (`tests/`, `mcseqs/`, `reg/`).

Tests

Every test class of the project and the labs, in course order (the test-plan tests last), with the purpose of its plan. The table is generated from the model by `make map`; the full plan of each test (stages, expected result, the items it verifies) is on the test plan page and in the **Test plan** view of the project map; its runs are in the regression results.

Test	Lab	Extends	Purpose
`base_test`	2	`uvm_test`	The skeleton e test extends: b router_tb, keep run alive 200 r after the last objection, repo unused setting

Test	Lab	Extends	Purpose
`set_config_test`	4	`base_test`	Lab 4: a passi agent through configuration database -- ar why the seque setting lives in virtual functior
`short_packet_test`	4	`base_test`	Lab 4: the fact override in act the sequence for yapp_pack and gets short_yapp_pa
`exhaustive_seq_test`	5	`base_test`	Lab 5: one sequence that every sequenc the library onc (sequences st sub-sequence
`incr_payload_test`	5	`base_test`	Lab 5: a seque that shapes th payload after randomisation
`yapp_012_test`	6	`base_test`	Lab 6: the first packets into th real DUT -- on channel, so th connection is

Test	Lab	Extends	Purpose
			to check in the waveform.
`simple_test`	7	`base_test`	Lab 7: every U talking to the D with the defau sequences onl
`test_uvc_integration`	7	`base_test`	Lab 7: 88 pack of every length address again router program for small packe
`router_simple_mcseq_test`	8	`base_test`	Lab 8: one multichannel sequence on t virtual sequen programs the r and sends the packets in ord
`scoreboard_drop_test`	9A	`router_simple_mcseq_test`	Lab 9A: the negative check full-size packe while maxpkts 20, so the rout drops some ar scoreboard mu notice.
`coverage_test`	10	`base_test`	Lab 10: stimulu written to close functional cov

Test	Lab	Extends	Purpose
			model of the Y packets (yapp_pkt_cg i yapp_pkt_cove fed by the mor
`uvm_mem_walk_test`	11B	`base_test`	Lab 11B: the bu memory seque walks yapp_m the RO yapp_pkt_mem skipped.
`uvm_reset_test`	11B	`base_test`	Lab 11B: the bu register seque that reads eve register after r
`reg_access_test`	11C	`base_test`	Lab 11C: hand- written front-d and backdoor checks on one and one RO register.
`reg_function_check_test`	11C	`reg_function_test`	Lab 11C (optio the same test the model che every read itse
`reg_function_test`	11C	`reg_access_test`	Lab 11C: do the enable bits an counters beha Packets in,

Test	Lab	Extends	Purpose
			counters read through the m
`reg_introspection_test`	11C	`reg_access_test`	Lab 11C (optio the model tells test which reg are RW and wh are RO, then e one is checke
`back_to_back_test`	test plan	`reg_function_test`	Test plan: the i port with the shortest gap - header in the c right after the byte -- and ga 1, 2 and 3 cycl
`backpressure_test`	test plan	`reg_function_test`	Test plan: the handshake un back-pressure slow receivers the 16-byte ch FIFOs, the rou stalls the input in_suspend an byte is lost.
`counter_wrap_test`	test plan	`reg_function_test`	Test plan: an 8 counter wraps 255 to 0 -- the specification i silent, the Dec

Test	Lab	Extends	Purpose
			of dut/spec.m the choice.
`hbus_protocol_test`	test plan	`reg_access_test`	Test plan: the port on the wi one-cycle writ two-cycle reac tri-state after t access, unma addresses -- v raw HBUS sequences, nc register model
`hbus_sweep_test`	test plan	`base_test`	Test plan: the side of the coverage -- ev address of the router read an written, check against the reference mod until the four H collectors reac 100 %.
`parity_error_test`	test plan	`reg_function_test`	Test plan: wha wrong parity b does -- the co counts it (whe enabled), erro pulses 1..10 cy later, and the

Test	Lab	Extends	Purpose
			packet is still delivered.
`pkt_mem_test`	test plan	`reg_function_test`	Test plan: the packet memor mem_size_reg the last packet byte by byte, e when the pack was dropped.
`random_backpressure_test`	test plan	`random_test`	Random tests: back-pressure random mix of channels -- slo and stalling receivers agai long packets s back to back.
`random_config_test`	test plan	`random_test`	Random tests: configuration s -- every maxpktsize ed the router off, mixed counter enables -- unc many short bu
`random_parity_test`	test plan	`random_test`	Random tests: parity where it happen -- forwarded, dro (address 3,

Test	Lab	Extends	Purpose
			oversized) and ignored (route -- and storms overlapping ba packets.
`random_test`	test plan	`base_test`	Random tests: seed decides t shape of the te and every resu checked automatically - expected valu written by han
`reg_bit_walk_test`	test plan	`reg_access_test`	Test plan: ever of every regist obeys its polic walking ones a zeros through RW registers, writes ignored the RO ones.
`router_disable_test`	test plan	`reg_function_test`	Test plan: rout = 0 makes the router deaf -- nothing reache channel, nothi counted, nothi stored; traffic resumes when enabled again.

Test	Lab	Extends	Purpose
`router_filter_test`	test plan	`reg_function_test`	Test plan: the rules around th maxpktsize lim and the counte behind them, v the counters enabled and th disabled.

Sequences

Every sequence class, by package, with the sequencer it runs on. Generated from the model by `make map`.

Sequence	Package	Lab	Runs on
`yapp_base_seq`	YAPP UVC	3	`yapp_packet` sequencer
`yapp_5_packets`	YAPP UVC	3	`yapp_packet` sequencer
`six_yapp_seq`	YAPP UVC	5	`yapp_packet` sequencer
`yapp_012_seq`	YAPP UVC	5	`yapp_packet` sequencer
`yapp_111_seq`	YAPP UVC	5	`yapp_packet` sequencer

Sequence	Package	Lab	Runs on
`yapp_1_seq`	YAPP UVC	5	`yapp_packet` sequencer
`yapp_exhaustive_seq`	YAPP UVC	5	`yapp_packet` sequencer
`yapp_incr_payload_seq`	YAPP UVC	5	`yapp_packet` sequencer
`yapp_repeat_addr_seq`	YAPP UVC	5	`yapp_packet` sequencer
`yapp_rnd_seq`	YAPP UVC	5	`yapp_packet` sequencer
`yapp_88_packets_seq`	YAPP UVC	7	`yapp_packet` sequencer
`yapp_coverage_seq`	YAPP UVC	10	`yapp_packet` sequencer
`yapp_boundary_seq`	YAPP UVC	test plan	`yapp_packet` sequencer
`yapp_gap_seq`	YAPP UVC	test plan	`yapp_packet` sequencer
`yapp_pkt_seq`	YAPP UVC	test plan	`yapp_packet` sequencer

Sequence	Package	Lab	Runs on
`yapp_random_burst_seq`	YAPP UVC	test plan	`yapp_packet` sequencer
`hbus_base_seq`	HBUS UVC	7	`hbus_transaction` sequencer
`hbus_large_packet_seq`	HBUS UVC	7	`hbus_transaction` sequencer
`hbus_read_max_pkt_seq`	HBUS UVC	7	`hbus_transaction` sequencer
`hbus_read_seq`	HBUS UVC	7	`hbus_transaction` sequencer
`hbus_set_default_regs_seq`	HBUS UVC	7	`hbus_transaction` sequencer
`hbus_small_packet_seq`	HBUS UVC	7	`hbus_transaction` sequencer
`hbus_write_seq`	HBUS UVC	7	`hbus_transaction` sequencer
`hbus_router_disable_seq`	HBUS UVC	11C	`hbus_transaction` sequencer
`hbus_router_enable_seq`	HBUS UVC	11C	`hbus_transaction` sequencer

Sequence	Package	Lab	Runs on
`channel_rx_base_seq`	Channel UVC	7	`channel_packet` sequencer
`channel_rx_fast_seq`	Channel UVC	7	`channel_packet` sequencer
`channel_rx_resp_seq`	Channel UVC	7	`channel_packet` sequencer
`channel_rx_random_seq`	Channel UVC	test plan	`channel_packet` sequencer
`channel_rx_slow_seq`	Channel UVC	test plan	`channel_packet` sequencer
`clock_and_reset_base_seq`	Clock & Reset UVC	7	`clock_and_reset_transaction` sequencer
`clk10_rst5_seq`	Clock & Reset UVC	7	`clock_and_reset_transaction` sequencer
`clk_rst_rand_seq`	Clock & Reset UVC	7	`clock_and_reset_transaction` sequencer

Sequence	Package	Lab	Runs on
`router_mcseq_base`	Testbench & tests	8	`uvm_sequence_item` sequencer
`router_simple_mcseq`	Testbench & tests	8	`uvm_sequence_item` sequencer
`router_hbus_sweep_mcseq`	Testbench & tests	test plan	`uvm_sequence_item` sequencer
`router_random_mcseq`	Testbench & tests	test plan	`uvm_sequence_item` sequencer

UVM concept → lab map

Concept	Lab	Where in the code
Data items, `do_print` / `do_copy` / `do_compare` / `do_pack`, constraints, `post_randomize`	1	`yapp_packet.sv`
Components, phases, `run_test`, `+UVM_TESTNAME`, verbosity, topology	2	`router_tb.sv`, `router_test_lib.sv`, `top.sv`
Driver / sequencer / monitor / agent / env, active vs passive	3	`yapp_tx_*.sv`, `yapp_env.sv`
Phase ordering (`start_of_simulation`, bottom-up)	3 opt.	every component of `labs/lab03_uvc/sv`
Factory `create`, type overrides	4	`short_packet_test`
Configuration (`uvm_config_int`, `uvm_config_wrapper`), `check_config_usage`	3, 4, 7	`base_test`, `set_config_test`, `router_tb`
Sequences, `start_item` / `randomize() with` / `finish_item`, nesting with `seq.start(m_sequencer, this)`	5	`yapp_tx_seqs.sv`, `seqs/`
Objections, drain time	5, 6, 8	`yapp_base_seq`, `base_test::run_phase`, `router_mcseq_base`

Concept	Lab	Where in the code
Randomization debug (SimVision / Verisium)	5	`exhaustive_seq_test` with the Lab 4 `short_yapp_packet`
Interfaces, virtual interfaces, `uvm_config_db`	6, 7	`yapp_if.sv`, `tb_top.sv`, driver/monitor `connect_phase`
Transaction recording (`begin_tr`, `recording_detail`)	4, 6	`base_test`, `yapp_tx_driver`, `yapp_tx_monitor`
Reusing UVCs, connecting several	7	`router_tb.sv`, `hw_top.sv`, `tb_top.sv`
Virtual / multichannel sequencer, `p_sequencer`	8	`router_mcsequencer.sv`, `router_mcseqs_lib.sv`
TLM analysis port / imp, `` `uvm_analysis_imp_decl ``	9A, 9B	`router_scoreboard.sv`, `router_reference.sv`
Scoreboard, custom compare, `uvm_comparer`	9A	`router_scoreboard.sv`, `packet_compare.sv`
Reference model, module UVC	9B	`router_reference.sv`, `router_module_env.sv`
TLM exports	9C	`router_module_env.sv`
TLM analysis FIFOs, `get_peek_export`	9D	`router_fifo_scoreboard.sv`
Functional coverage (covergroup, bins, cross)	10	`yapp_pkt_coverage.sv`, `yapp_coverage_seq`

Concept	Lab	Where in the code
RAL model structure (`uvm_reg`, `uvm_reg_block`, `uvm_mem`, maps)	11A	`yapp_router_reg_pkg.sv`
RAL integration: adapter, `set_sequencer`, auto-predict, built-in sequences	11B	`hbus_reg_adapter.sv`, `router_tb.sv`, `uvm_reset_test`
RAL access: `write` / `read` / `peek` / `poke`, `predict`, check-on-read, introspection	11C	`reg_*_test`

Pitfalls

The mistakes the course warns about, plus the ones that came up while building this reference environment.

Compile and include order

1. **Include order matters.** In a package and in `tb_top`, a class must be included before any file that uses it: packet → monitor → sequencer → sequences → driver → agent → env; `router_tb` before the tests; the multichannel sequencer before `router_tb`; the register package before `router_tb`.
2. **Interfaces are compiled, not included.** List `*_if.sv` in `run.f`.
3. `-timescale 1ns/1ns` in `run.f` avoids timescale mismatches between the RTL, the interfaces and the UVM library.

Configuration and factory

1. **Set configuration before building.** `is_active`, `channel_id`, `num_masters`, default sequences — all before the component that reads them is created (that is, before `super.build_phase()` in a test).
2. **Unused configuration warnings.** A passive agent makes a sequencer `default_sequence` unused (Lab 4). Only set what will be read.
3. `new()` **bypasses the factory.** Anything a test may want to override must be created with `type_id::create()`.
4. **Overrides after creation do nothing.** `set_type_override_by_type` goes before `super.build_phase(phase)`.

Sequences and objections

1. **Planned constraint conflict in Lab 5** (`short_yapp_packet` `m_addr != 2` vs `yapp_012_seq`). Fix it before Lab 6.
2. **Objections in sub-sequences.** `starting_phase` is `null` for a child sequence; always test for `null` before raising.
3. `forever` **+ objection = a test that never ends.** The channel response sequence runs forever and must not object.
4. **Missing** `item_done()` hangs the sequence; **missing drain time** loses the last packet (Lab 6).

Interfaces and timing

1. **Check** `uvm_config_db::get` **'s return value** and raise `` `uvm_error ``: a `null` virtual interface crashes later, far from the cause.

2. **Drive on the falling edge, sample on the rising edge.** Driving and sampling on the same edge races with non-blocking assignments.
3. **HBUS `hdata`.** Connect the DUT to the `hdata_w` wire, not to the `hdata` logic variable (multiple drivers otherwise).
4. **The parity byte has `in_data_vld` low.** Monitors must not wait for `in_data_vld` to collect it.
5. **Hold data while `in_suspend` is high**; a byte counts only at a rising edge where `in_suspend` is low.

TLM

1. **Hierarchical handles are not config strings.** `mcseqr.yapp_seqr = yapp.agent.sequencer` — no wildcards.
2. **Clone in the scoreboard.** Analysis imps pass handles; `clone()` before queueing. Analysis FIFOs never clone, so monitors must create a new object per transaction.
3. **`` `uvm_analysis_imp_decl `` goes outside the class**, once per suffix per package.
4. **Connect in `connect_phase`**, from the port to the imp/export.

Register model

1. **Backdoor access needs `-access +rwc`.**
2. **Model names must match RTL names** for HDL paths.
3. **Check-on-read with RO counters** needs `predict()` first.

UVM versions

1. **UVM 1.1d vs 1.2.** Use `` `YAPP_STARTING_PHASE `` from `common/uvm_version_compat.svh` instead of `starting_phase` / `get_starting_phase()` directly. The course recommends 1.1d for the training because of transaction-recording issues in 1.2.

xrun options used in the course

Option	Purpose
`-uvmhome CDNS-1.1d` / `CDNS-1.2`	which bundled UVM library to compile wit
`-f run.f`	read options and files from a command file
`-incdir <dir>`	search path for `` `include ``
`-timescale 1ns/1ns`	default timescale for files without one
`+UVM_TESTNAME=<test>`	select the test withou recompiling
`+UVM_VERBOSITY=UVM_LOW | UVM_MEDIUM | UVM_HIGH | UVM_FULL`	filter `` `uvm_info ``
`+SVSEED=random` / `+SVSEED=<n>`	random seed (printed in the log) / reproduc a run
`-gui`	SimVision
`-access +rwc` (`-access rwc`)	read/write/connectiv access to signals: waveforms, backdoo

Option	Purpose
`-debug_opts verisium_interactive`	Verisium Debug (separate license)
`-coverage U`	collect functional (covergroup) coverag
`-covoverwrite`	overwrite the previou coverage database
`-define INJECT_ERROR`	define a macro at compile time (the memory-test error)

The Makefiles

Every `labs/<lab>/tb/Makefile` includes `common/lab.mk`:

```
# Shared Makefile for every simulation directory. labs/<lab>/tb/Makefile does
#     include ../../../common/lab.mk
# and yapp_project/tb/Makefile does
#     include ../../common/lab.mk
#
#   make run  [TEST=<test>] [XRUN_OPTS="+SVSEED=random"]   batch simulation
#   make gui  [TEST=<test>]                                  SimVision, stop on constraint
failure
#   make lint                                                slang elaboration check (no
simulator)
#   make clean
#
# Free flow with Verilator (scripts/sim.py, install with scripts/setup_sim.sh):
#   make compile                                             build the simulation
(build/sim/<dir>/)
#   make sim  [TEST=<test>] [WAVES=1] [SEED=<n>|random]       run one test, PASS/FAIL at the end
#   make waves [TEST=<test>]                                 open the waves of the last WAVES=1
run
#   DEFINES="INJECT_ERROR" with compile / sim / waves: a separate build with these `defines
XRUN      ?= xrun
TEST      ?= base_test
XRUN_OPTS ?=
ROOT      := $(abspath $(dir $(lastword $(MAKEFILE_LIST)))/..)
LINT      := python3 $(ROOT)/scripts/lint.py
SIM       := python3 $(ROOT)/scripts/sim.py
WAVES     ?=
SEED      ?= 1
SIM_OPTS  ?=
DEFINES   ?=
SIM_DEFS  := $(foreach d,$(DEFINES),-D $(d))

.PHONY: run gui lint clean compile sim waves

run:
    $(XRUN) -f run.f +UVM_TESTNAME=$(TEST) $(XRUN_OPTS)

gui:
    $(XRUN) -f run.f +UVM_TESTNAME=$(TEST) -gui -access +rwc $(XRUN_OPTS)

lint:
    $(LINT) run.f

compile:
    $(SIM) compile . $(SIM_DEFS)

sim:
    $(SIM) run . -t $(TEST) --seed $(SEED) $(if $(WAVES),--waves) $(SIM_DEFS) $(SIM_OPTS)

waves:
    $(SIM) waves . -t $(TEST) $(SIM_DEFS)

clean:
    rm -rf xcelium.d INCA_libs xrun.log xrun.history *.shm cov_work .simvision *.key *.err
```

The root `Makefile` adds `lint`, `style`, `docs`, `serve`, `map`, `clean`, `run LAB=<lab> TEST=<test>` / `run-project TEST=<test>` (Xcelium) and, for Verilator, `sim`, `regress`, `sim-report` and `coverage` (Simulating with Verilator).

Other simulators

The code is standard SystemVerilog + UVM 1.1d/1.2 and does not use Cadence-specific constructs. To run elsewhere replace `-uvmhome` with the simulator's UVM option (`-ntb_opts uvm-1.2` for VCS, `-uvmversion 1.2` / `+incdir+$UVM_HOME/src $UVM_HOME/src/uvm_pkg.sv` for Questa), keep `-timescale`, and translate `-incdir` to `+incdir+`. Transaction recording and the GUI steps are tool specific.

Simulating with Verilator

The course targets Cadence Xcelium (`make run`). When no Xcelium licence is at hand, every lab and the complete project also run on Verilator, a free, open-source simulator, from the **same `run.f` files**. Verilator 5.052 runs the Accellera UVM 2020.3.1 source; the few places where it differs from Xcelium are handled in the code (see below).

	Xcelium	**Verilator**
Compile + run one test	`make run TEST=<test>`	`make sim TEST=<test>`
Waves	`make gui` (SimVision)	`make sim TEST=<test> WAVES=1`, then `make waves TEST=<test>`
Every lab and test	--	`make regress` at the root
Licence	Cadence	none (LGPL)

Getting a machine with Verilator

GitHub Codespaces (nothing to install)

On the repository page: **Code → Codespaces → Create codespace on main**. The first start builds the container from `.devcontainer/` (a few minutes); VS Code opens in the browser with Verilator, the UVM source, a SystemVerilog syntax extension and the **VaporView** waveform viewer installed.

The container asks for a 4-core machine (`hostRequirements` in `.devcontainer/devcontainer.json`), which halves the free monthly hours but also halves the compile time. Stop the codespace when you are done.

Linux / WSL2

```
bash scripts/setup_sim.sh        # or: make setup-sim
```

The script installs the build tools with `apt-get`, builds Verilator 5.052 into `/usr/local` (10-25 minutes, once), copies UVM's DPI code for Verilator next to it and fetches the Accellera UVM source. `PREFIX=$HOME/.local` installs without `sudo`. On Windows, install WSL2 with Ubuntu first (`wsl --install -d Ubuntu` in PowerShell) and run the script inside it.

Docker

`.devcontainer/Dockerfile` builds on the official `verilator/verilator:v5.052` image; VS Code's *Dev Containers: Reopen in Container* uses it locally.

Running

```
cd yapp_project/tb
make compile                                  # once; about 2.5 minutes with UVM
make sim TEST=reg_function_test               # PASS / FAIL at the end
make sim TEST=backpressure_test WAVES=1 SEED=7 # record waves, another seed
make sim TEST=uvm_mem_walk_test DEFINES=INJECT_ERROR   # a build with a `define (own directory)
make waves TEST=backpressure_test             # open the waves
```

`make sim` compiles first when a source changed; choosing another test does not recompile (the test is picked at run time with `+UVM_TESTNAME`). Everything goes to `build/sim/<directory>/`: `compile.log`, `logs/<test>_s<seed>.log`, and with `WAVES=1` also `waves/<test>_s<seed>.fst` and `trace/<test>_s<seed>.csv` (the per-cycle signal trace, see Logs, timelines and waves). The plusargs of `run.f` (`+UVM_VERBOSITY=...`) are passed on, plus `+SEED=<n>` for the banner of the log; more can be added with `SIM_OPTS="+UVM_VERBOSITY=UVM_HIGH"`.

A run **passes** when it ends normally with no `UVM_ERROR` and no `UVM_FATAL`.

Waves. `WAVES=1` records the whole design into an FST file. In Codespaces, open the file from the Explorer (VaporView opens `.fst`); on a desktop `make waves` starts `surfer` or `gtkwave` when one is installed. Verilator skips unpacked arrays of more than 32 elements by default, so `scripts/sim.py` passes `--trace-max-array 256`; without it `yapp_pkt_mem` (64) and `yapp_mem` (256) in `dut.u_regs` would be missing from the waves.

From the root: `make sim LAB=lab07_integ TEST=simple_test`, `make sim-project TEST=reg_function_test WAVES=1`.

The regression

`make regress` compiles every simulation directory listed in `scripts/regress.yaml` once and runs each of its tests with the seeds the file gives it (one per lab test; 4 per directed project test and 8 per random test, `{seeds: N, tests: [...]}` per bench or `{test: x, seeds: N}` per test; `SEEDS=3` gives every test 3 instead), `ONLY=lab09` for some directories, `WAVES=1` to record the waves and the signal trace of every run). The table goes to `build/sim/regress.md`; `make sim-report` then bundles the logs, timelines, waves and pictures of every run into `build/sim/report/` (Logs, timelines and waves).

The **sim** workflow of GitHub Actions (`.github/workflows/sim.yml`) runs the same regression on every push, one job per directory, in the official Verilator container; the table appears on the run's summary page and the logs of a failure are kept as an artifact.

What differs from Xcelium

Verilator compiles the design to C++. It is a two-state simulator (no `x` / `z`) and its constraint solver and coverage are younger than Xcelium's. These are the points that touch this code, and how the code deals with them; every change also runs unchanged on Xcelium.

Topic	Verilator 5.052	In the code
`payload.size() == length` under `randomize() with {...}`	the array is not resized: packets would go out without payload	`yapp_packet::post_rando` when the solver did not
`dist` together with an equality in `with {...}`	randomly unsatisfiable (the `dist` bucket is chosen first)	sequences that choose th `c_parity_dist` off
`'z` on a wire (two-state)	`hdata_w !== 8'bz` is always true	`hbus_protocol_test`, und enabled
Enum coverpoint without bins	binned by value range, not one bin	`parity_cp` lists its two bi

Topic	Verilator 5.052	In the code
	per value	
`ignore_bins ... binsof() intersect` in a cross	ignored (warning `COVERIGN`)	`yapp_pkt_cg`, under `` `ifd `` coverpoints that hold only
`get_inst_coverage()` of one coverpoint; `get_coverage()` of the type	not supported; returns 0	the collectors call `get_in` `router_coverage` counts
Coverage database	written only when compiled with a `--coverage*` flag	`sim.py` compiles with `--` and passes `+verilator+coverage+fil` `verilator_coverage` merg
`--public-flat-rw` on the whole design	does not compile with UVM	`common/verilator/public` (RAL backdoor)
A comment that starts with the word `verilator`	read as a Verilator directive	avoid it

Not used by this code, but known to be incomplete in Verilator: clocking blocks through virtual interfaces, `disable fork`, `process::kill()`. Two more found while writing the random library: a dynamic array cannot be assigned to a fixed-size array (compile error), and inside a class method `if (v inside {[-5:5]})` is false for a negative `v` (a range with a negative bound; compare with `>=` / `<=` instead).

What still needs Xcelium: four-state behaviour (`x` propagation, the real `'z` on HBUS), coverage numbers that match IMC, and the Cadence-specific steps (SimVision, IMC, `reg_verifier`). See Verification status.

Reading a simulation: logs, timelines and waves

Every run writes a log that tells its story in order: what the test is doing, what every component of the environment does with each packet, and what happens inside the router. `make sim-report` bundles the logs of a regression into one folder per run, with a timeline extracted from the log, the waves, and a few wave pictures of the moments worth looking at. This page explains the log lines, the report and how to get the waves into a viewer.

The shape of a log

The project runs at `+UVM_VERBOSITY=UVM_MEDIUM` (set in `run.f`): one line per event, no packet tables. `UVM_HIGH` adds the full `Packet is` / `Packet collected` tables of the course and the HBUS lines of the DUT tracer.

```
UVM_INFO tests/base_test.sv(94) @ 0: uvm_test_top [TEST] === backpressure_test  seed 1 ===
UVM_INFO tests/base_test.sv(99) @ 0: uvm_test_top [TEST] STAGE: 1. slow receivers
(channel_rx_slow_seq): 6 packets of 40..63 bytes fill the 16-byte FIFOs
UVM_INFO ...yapp_router_tracer.sv(148) @ 50: reporter [DUT] RESET released: router_en=1
maxpktsize=63
UVM_INFO ...yapp_tx_driver.sv(76) @ 50: uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] TX pkt
#1: addr=2 len=55 parity=good delay=20
UVM_INFO ...yapp_router_tracer.sv(213) @ 265: reporter [DUT] IN  pkt #1 header: addr=2 len=55 ->
channel 2
UVM_INFO ...yapp_router_tracer.sv(251) @ 425: reporter [DUT] SUSPEND asserted: FIFO 2 full, byte
0x49 held (pkt #1)
UVM_INFO ...yapp_router_tracer.sv(289) @ 565: reporter [DUT] OUT ch2 pkt #1 start: addr=2 len=55
(receiver waited 29 cycle(s))
UVM_INFO ...yapp_router_tracer.sv(257) @ 575: reporter [DUT] SUSPEND released after 15 cycle(s)
UVM_INFO ...yapp_router_tracer.sv(228) @ 975: reporter [DUT] IN  pkt #1 complete: addr=2 len=55
parity=ok forwarded, 57 bytes in 72 cycles (held 15)
UVM_INFO ...yapp_tx_driver.sv(80) @ 980: uvm_test_top.tb.yapp.agent.driver [yapp_tx_driver] TX
pkt #1 accepted by the DUT
UVM_INFO ...router_scoreboard.sv(121) @ 1345: uvm_test_top.tb.router_module.scoreboard
[router_scoreboard] ch2 pkt matched: addr=2 len=55 (1 matched so far)
UVM_INFO tests/base_test.sv(112) @ 6335: uvm_test_top [TEST] === backpressure_test  seed 1:
PASSED (0 UVM_ERROR, 0 UVM_FATAL) ===
```

Three kinds of lines carry the story:

Who	Id	Lines	Where it is written
The test	`[TEST]`	`=== <test> seed <n> ===` banner at the start, `STAGE: …` at every step of the test, `=== … PASSED/FAILED (…) ===` at the end	`base_test` (`stage` `start_of_simulati` `final_phase`); eve
The router	`[DUT]`	`RESET`, `IN pkt #n header / complete`, `SUSPEND asserted / released`, `OUT chX pkt #n start / done`, `ERROR pulse`, `CONFIG` ; at `UVM_HIGH` also `HBUS write / read` with the register name and `FIFO x full / has room`	`yapp_router_trace` **bound** into the DU (`yapp_project/tb/` it watches the port RTL is untouched
The environment	the component's type name, as in the course	`TX pkt #n: …` / `TX pkt #n accepted by the DUT` (driver), `RX pkt #n on the input: …` (monitor), `chX RX pkt #n: …` (channel monitor), `chX: packet consumed` (channel driver), `Executed WRITE/READ …` and `Collected …` (HBUS), `chX pkt matched: …` (scoreboard), `Packet forwarded / dropped …` (reference model), `Starting clock …` (clock and reset)	the UVCs under `ya`

The `[DUT]` lines are the ones that tell you what the hardware did, as opposed to what the environment sent or expected: a header that will be dropped says so (`-> DROP (illegal address 3)`), a held byte names the full FIFO, a packet leaving a channel says how long the receiver kept it waiting, an error pulse says how many cycles after the bad packet it came.

Every message with a number in it has a fixed format, so a script (or `grep`) can follow one packet from the driver (`TX pkt #3`) to the router (`IN pkt #3`), the channel (`OUT ch1 pkt #2`) and the scoreboard (`ch1 pkt matched`).

The report

```
make regress ONLY=yapp_project WAVES=1           # run, record waves and the signal trace
make sim-report ONLY=yapp_project                 # bundle: build/sim/report/ +
regression_report.zip
make sim-report ONLY=yapp_project OUT=yapp_project/regression   # refresh the committed report
```

The report of the project (17 directed tests with 4 seeds and 4 random tests with 8, 100 runs) is committed in `yapp_project/regression/`, so the logs, timelines, waves and pictures can be read on GitHub or in the ZIP without running anything; `index.md` is the entry point, `runs.json` the same table as data. The site shows it as Regression results: one page per test with its seeds, the links to every log, timeline and waveform, the stages and the pictures of its first seed; the project map shows the same under every test card. A committed report leaves out the files that only repeat the log or the waves (`all_logs.log`, `timeline.csv`, `signals.csv`), and the timeline of a very long run (`hbus_sweep_test`, 11 000 events) keeps only the test, DUT, HBUS and report events.

`scripts/sim_report.py` reads `build/sim/<bench>/logs/<test>_s<seed>.log` and writes `build/sim/report/`:

```
report/
  index.md                      every run: result, sim and wall time, packets in / forwarded /
dropped,
                                packets out per channel, suspends, errors, links; failures first
  runs.json                     the same per run, as data (the site pages and the map read it)
  all_logs.log                  every log in one file, each under a banner (local reports only)
  coverage.md                   the functional coverage merged over every run: holes,
                                every bin with its hits and tests, what each test adds
  coverage.json                 the same data (the site page and the project map read it)
  coverage_html/<bench>/        genhtml: the covergroup sources with a count per bin
                                (docs/coverage/<bench>/ for the committed report)
  yapp_project_tb/
    merged.dat, merged.info     the merged coverage database (verilator_coverage, lcov)
    backpressure_test_s1/
      backpressure_test_s1.log  the log, named after the test and the seed
      timeline.md               summary · stages · pictures · every event in order
      timeline.csv              the same events: time, severity, who, id, event (local reports
only)
      backpressure_test_s1.fst  the waves
      signals.csv               the per-cycle signal trace (see below; local reports only)
      wave_*.svg                the pictures
    backpressure_test_s2/
    ...
```

`timeline.md` starts with the verdict and a summary counted from the log (packets sent, forwarded, dropped and why, packets out per channel, scoreboard matches, back-pressure events, error pulses, HBUS accesses), lists the stages of the test with their times, shows the pictures, and ends with the full timeline: one row per event, `UVM_ERROR` rows marked.

`coverage.md` comes from the coverage database every run writes (`build/sim/<bench>/coverage/<test>_s<seed>.dat`); how to read it is in Functional coverage.

The pictures

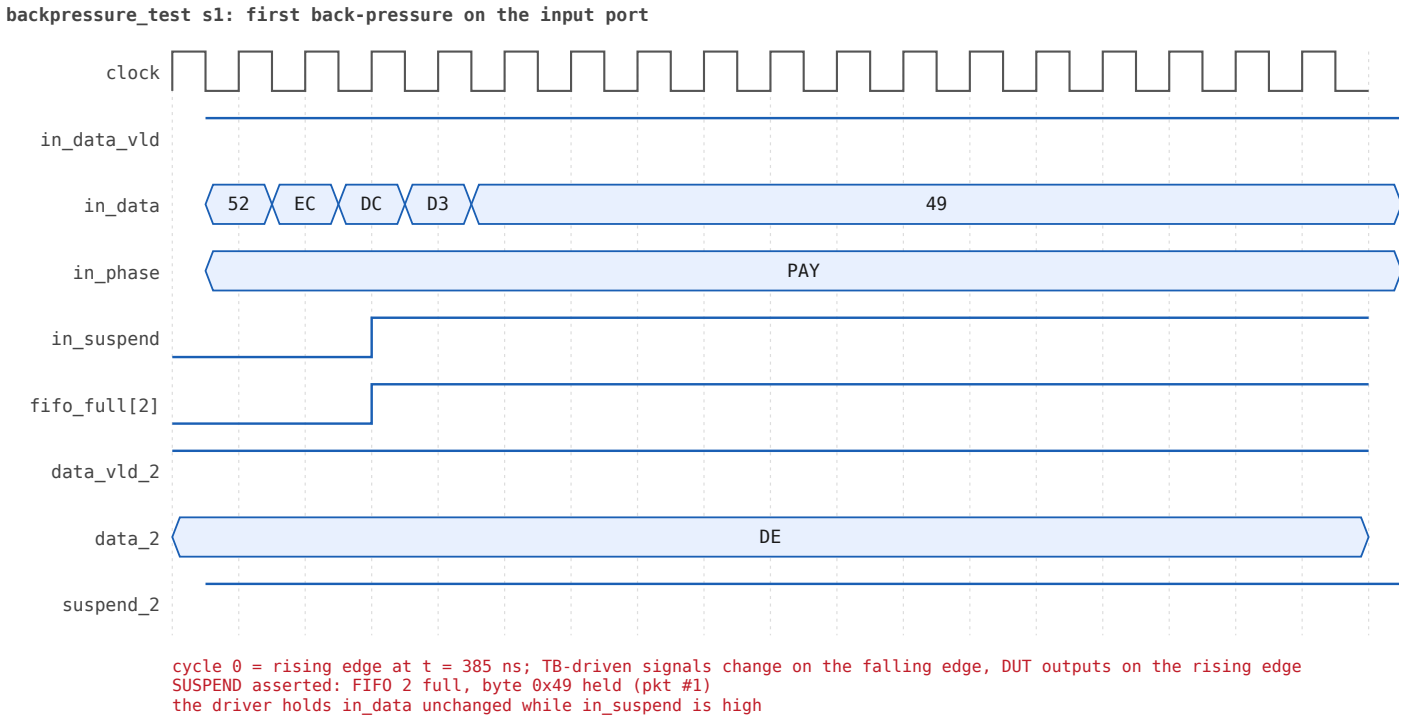
With `WAVES=1` the tracer also writes `trace/<test>_s<seed>.csv`: one row per clock cycle, sampled on the falling edge, with the input port, the three channels, the HBUS port, `error`, the live configuration and the state of the input FSM. `sim_report.py` finds the first occurrence of each interesting event in the log and draws a window of cycles around it with the same SVG drawer as the timing diagrams of the specification:

Picture	When it is drawn	What it shows
`wave_reset.svg`	always	reset, the first idle cycles, `router_en` and `maxpktsize` after reset
`wave_first_packet_in.svg`	a packet entered the router	header, payload and parity on `in_data` with their `in_phase` (HDR / PAY / PAR), `in_suspend`, the input FSM state, the target FIFO filling up
`wave_first_packet_out.svg`	a packet left on a channel	`data_x`, `data_vld_x`, the receiver's `suspend_x`
`wave_first_suspend.svg`	`in_suspend` rose	the held byte on `in_data`, the full FIFO, the channel draining it
`wave_first_error_pulse.svg`	`error` pulsed	the end of the bad-parity packet and the pulse 1..10

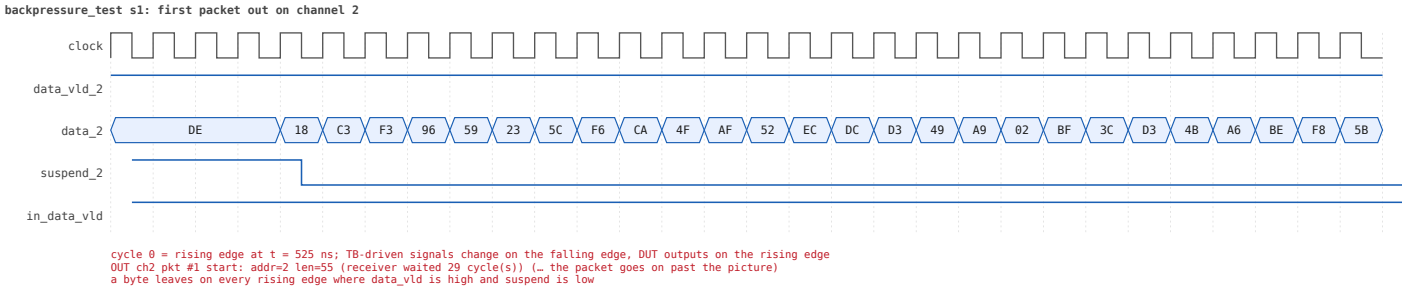
Picture	When it is drawn	What it shows cycles later
`wave_first_hbus_write.svg`, `wave_first_hbus_read.svg`	the host port was used	`hen`, `hwr_rd`, `haddr`, `hdata` for one write and one read

Testbench-driven signals are drawn changing on the falling edge, DUT outputs on the rising edge, exactly as the protocol works. A window is at most 30 cycles; a longer packet is cut and the note under the picture says so.

Below, the first packet of `backpressure_test` (seed 1): the receiver is slow, the FIFO of channel 2 fills up and the router holds the driver with `in_suspend`.



And the same packet leaving the router on channel 2:



The waves

`<test>_s<seed>.fst` is the complete design, every signal. Open it with

- **GTKWave** (Windows, Linux, macOS): `gtkwave backpressure_test_s1.fst`, then drag `hw_top.in0`, `hw_top.dut` and `hw_top.ch2` into the viewer. The `[DUT]` times of `timeline.md` are the rising edges to jump to.
- **Surfer** (`surfer file.fst`, also available as a web app), or the **VaporView** extension of VS Code (installed in the Codespace): open the file from the Explorer.
- From a lab directory, `make waves TEST=<test>` starts whichever viewer is installed.

`signals.csv` is the same information for the port signals in a form a spreadsheet or a script can read: one row per cycle.

A ready signal layout for Surfer

`yapp_project/tb/yapp_router.sucl` is a Surfer command file that opens the signals of the router in ten groups, one color per group, buses in hex, the FSM state and `in_phase` by name, plus the two memories of the spec, folded. Variable names are full paths (*Settings* → *Variable names* → *Global*):

Group	Signals	Scope
Clock, reset	`clock`, `reset`	`hw_top`
Input port	`in_data`, `in_data_vld`, `in_suspend`, `in_phase`	`hw_top.in0`; `in_phase` from `hw_top.dut.u_tracer`
Input FSM	`state`, `cur_addr`, `cur_len`, `byte_cnt`, `cur_drop`, `fifo_push`, `fifo_full`	`hw_top.dut.u_input_fsm`
FIFO fill	`ch0 count` … `ch2 count`: bytes waiting in each channel FIFO	`hw_top.dut.g_ch[c].u_ch.u_fifo.count`

Group	Signals	Scope
Channel 0 / 1 / 2	`data`, `data_vld`, `suspend`	`hw_top.ch0` … `ch2`
Error	`error`, `pkt_done`, `pulses owed` (`sched`: bit k = a pulse k cycles from now)	`hw_top.dut`, `u_error_timer`
HBUS	`hen`, `hwr_rd`, `haddr`, `hdata`	`hw_top.hbus0`
Config, counters	`router_en`, `maxpktsize`, `addr0..3_cnt_reg`, `parity_err_cnt_reg`, `oversized_pkt_cnt_reg`	`hw_top.dut`, `u_regs`
Packet memory (folded)	`[0]` … `[63]`: the last packet, header first (0x1010, RO)	`hw_top.dut.u_regs.yapp_pkt_mem`
Scratch memory (folded)	`[0]` … `[255]`: written and read over HBUS (0x1100, RW)	`hw_top.dut.u_regs.yapp_mem`

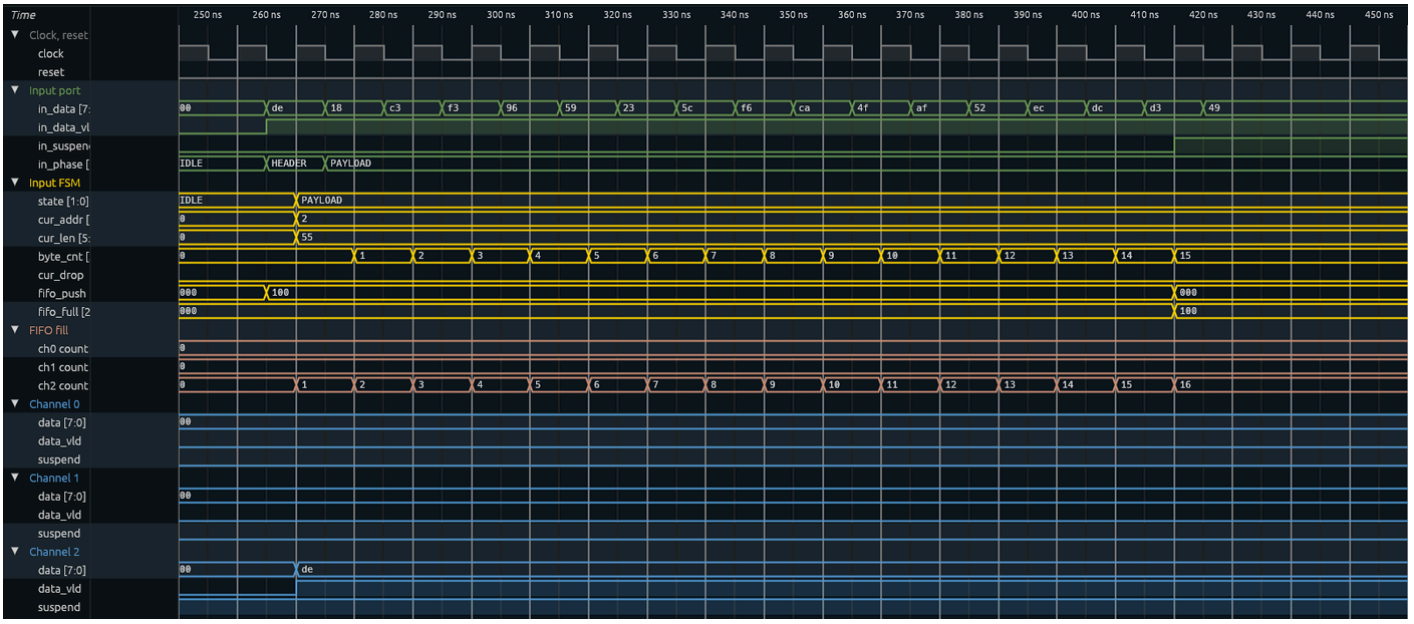
The memories are arrays, and Verilator leaves arrays of more than 32 elements out of the waves unless told otherwise: `scripts/sim.py` passes `--trace-max-array 256`, so `yapp_pkt_mem` (64 bytes) and `yapp_mem` (256 bytes) are in every `.fst` recorded since. In Surfer they are scopes under `u_regs`, one variable per byte; a `.fst` recorded before has no memories.

Four ways to use it:

- **One click.** The *Waves* column of `yapp_project/regression/index.md` (and the line under the result in every `timeline.md`) opens Surfer's web app with that run's `.fst` and the layout, both read from GitHub.
- **A local file in the web app or the desktop Surfer.** Open the `.fst`, then *File → Run command file…* and pick `yapp_router.sucl`. *File → Switch file…* then moves to another test and keeps the list.

- **Command line:** `surfer backpressure_test_s1.fst -c yapp_project/tb/yapp_router.sucl`.
- **With the view settings too:** `yapp_project/tb/yapp_router.surf.ron` is the same layout saved as a Surfer state, plus four settings no command can set: *Focus highlight → Background*, *Hierarchy → Tree*, *Show hierarchy icons* on, *Fill high values* off (and *Variable names → Global*). Open the `.fst`, then *File → Load state...*; the state keeps the waves that are open. The web app does not remember settings between visits, so this is the way to get them back in one step.

The first packet of `backpressure_test` (seed 1) in this layout: the header `de` (`HEADER`, one cycle) goes to channel 2, `ch2 count` climbs to 16, the FIFO is full and `in_suspend` holds byte `49`:



The layout is written by `scripts/surfer_layout.py` (`LAYOUT` at the top): edit it there and run `python3 scripts/surfer_layout.py` to rewrite the `.sucl` (`python3 scripts/surfer_layout.py --check <file.fst>` says whether every signal of the layout is in a recording: a signal that is missing does not just stay empty, the rows after it shift and the groups fall apart, so a renamed RTL signal must be renamed in `LAYOUT` too; `make sim-report` into the repository runs the same check). For the `.surf.ron`, run the new `.sucl` in Surfer, *File → Save state as...*, then `python3 scripts/surfer_layout.py --state <saved file>` (it puts the four settings of `STATE_SETTINGS` in). To keep your own arrangement after changing it in Surfer, use *File → Save state as...* and *File → Load state...* next time (open the `.fst` first; the state keeps the waves that are open and brings back the list).

A `.sucl` or `.surf.ron` file is never dragged into the Surfer window: a dropped file is read as waves and fails with `UnknownFileFormat`.

Which signals to look at

Following one packet from the driver to a channel (Surfer: click a scope in the tree, then the signals in the *Variables* list; *Format → Hex* on the buses):

Where	Signals	What they show
`hw_top.in0`	`clock`, `in_data`, `in_data_vld`, `in_suspend`	what the YAPP driver puts on the input port (it changes the inputs on the **falling** edge)
`hw_top.dut.u_tracer`	`in_phase`	what the byte now on `in_data` is: `IDLE`, `HEADER`, `PAYLOAD`, `PARITY`; changes with `in_data`, so `HEADER` lasts exactly the cycle of the header
`hw_top.dut.u_input_fsm`	`state`, `cur_addr`, `cur_len`, `cur_drop`, `fifo_push`, `fifo_full`	what the router decided, on the **rising** edge that sampled each byte
`hw_top.ch0` … `ch2`	`data`, `data_vld`, `suspend`	the packet leaving: the same bytes, in the same order, on the channel `addr` names

Why the FSM has no HEADER state. The FSM takes the header while it is in `IDLE`: on the rising edge where `in_data_vld` is high it latches `cur_addr` and `cur_len` from the header and moves to `PAYLOAD`. So `state` shows `PAYLOAD` half a cycle *before* the first payload byte is on the bus -- the testbench drives a byte on the falling edge and the router samples it on the next rising edge. A real HEADER state would cost a cycle per packet and change the protocol of the course; `in_phase` is the readable label, computed in the testbench (`yapp_router_tracer.sv`), with the RTL untouched.

Running one test by hand

```
cd yapp_project/tb
make sim TEST=parity_error_test SEED=3 WAVES=1          # log, waves, trace
make sim TEST=parity_error_test SIM_OPTS="+UVM_VERBOSITY=UVM_HIGH"   # with the packet tables
grep "\[DUT\]" ../../build/sim/yapp_project_tb/logs/parity_error_test_s3.log
```

With Xcelium the same `[TEST]`, `[DUT]` and component lines appear in `xrun.log` (`make run TEST=…`); the seed in the banner comes from `+SVSEED`, the signal trace from `+trace_csv=<file>`.

Verification status

Everything in this repository is **elaborated** with slang against the Accellera UVM source (`make lint`: 0 errors in every lab) and, since October 2026, **simulated** with Verilator 5.052 (`make regress`, and the **sim** workflow on every push): every lab and every test of `scripts/regress.yaml` passes -- see Simulating with Verilator. The course's own simulator, Cadence Xcelium, has still not run the code; the items below say what the Verilator runs proved and what only Xcelium can.

Confirmed by simulation (Verilator)

#	Item	What the runs show
1	**YAPP handshake under back-pressure** (`in_suspend`): driver holds, DUT accepts exactly once, monitor counts exactly the accepted bytes	`backpressure_test`: `in_suspend` rose 7 times, scoreboard 6 matched, 0 mismatched, nothing unexpected
2	**Channel handshake**: the receiver's `suspend` vs the FIFO pop	every scoreboard test matches all its packets (Labs 7-11, the project)
3	**HBUS read timing**: data in the second cycle	every register test reads back what it wrote (`reg_function_test`, `reg_bit_walk_test`, `hbus_protocol_test`)
5	`error` **pulse** 1..10 cycles after a bad-parity packet	`parity_error_test`: 12 bad packets, 12 pulses, 2..10 cycles after the packet
6	**Register model backdoor paths** (`hw_top.dut.u_regs.*`)	`reg_access_test` peek/poke through `uvm_hdl_*` (VPI)

#	Item	What the runs show
7	`uvm_reg_hw_reset_seq` **on counters**	`uvm_reset_test` : 0 errors
8	`uvm_mem_walk_seq`	`uvm_mem_walk_test` : 0 errors; with `INJECT_ERROR` exactly one, at `yapp_mem[42]` (both builds in the regression)
10	**Coverage closure** with `coverage_test`	`yapp_pkt_cg` at 100%; merged over the project's regression every covergroup is closed, 222 / 222 bins (coverage results; Verilator's coverage, see the differences below)

The first runs also found what slang could not: `set_parity()` left about half of the "bad parity" packets correct, and an explicit `base_test::build_phase()` call that Verilator turns into endless recursion -- both fixed. The random tests later found a third, in the DUT: a second bad-parity packet arriving before the first one's `error` pulse took that pulse away; `yapp_error_timer` now owes one pulse per packet (random tests).

Still needs Xcelium

#	Item	Why Verilator cannot settle it
3'	**HBUS tri-state release**: `hdata_w` is `z` between transfers	two-state simulator: `hbus_protocol_test` checks the output enables instead (`` `ifdef VERILATOR ``)
4	**Reset at time 0**: drivers wait for `reset === 0`	two-state: `reset` starts at 0, not `x` ; the tests pass, but the `x` window is not exercised

#	Item	Why Verilator cannot settle it
9	**UVM 1.1d compatibility** of library calls (`set_drain_time`, `uvm_config_wrapper`, `compare_field_int`, `find() with` on `uvm_reg` queues)	Verilator runs UVM 2020.3.1; compile with `-uvmhome CDNS-1.1d`
10'	**Coverage numbers in IMC**	Verilator's covergroups differ (see the differences); the cross is written another way under `` `ifdef VERILATOR ``
11	**Monitor race margins**: monitors sample on the rising edge while the DUT updates with non-blocking assignments	Verilator's scheduling is not Xcelium's; if values look one cycle off, add `#1step` or clocking blocks

Deliberately out of scope

- Cadence `reg_verifier` generation (Lab 11A) — replaced by a hand-written model.
- Verisium Debug steps (Lab 5) — tool specific.
- A slave agent in the HBUS UVC — the router is the only slave, so `hbus_env` has master agents and a monitor only.

How to report

Paste the `UVM Report Summary`, the component reports (scoreboard, monitors, HBUS) and the seed into an issue, or simply tell the mentor. With Verilator, `build/sim/<dir>/logs/<test>_s<seed>.log` holds the whole log and `make sim TEST=<test> SEED=<n> WAVES=1` replays the run with waves.